Tklib

Handbuchseiten (deutsch)

2026-10-27

Inhalt

apps **5**
bitmap-editor 5
diagram-viewer 5
shtmlview 7
autoscroll **8**
autoscroll 8
canvas **10**
canvas_drag 10
canvas_ecircle 12
canvas_epoints 17
canvas_epolyline 23
canvas_equad 29
canvas_erectangle 33
canvas_gradient 38
canvas_highlight 39
canvas_mvg 41
canvas_pdf 41
canvas_snap 42
canvas_sqmap 43
canvas_tags 45
canvas_trlines 46
canvas_zoom 48
chatwidget **49**
chatwidget 49
controlwidget **51**
controlwidget 51
crosshair **59**
crosshair 59
ctext **61**
ctext 61
cursor **65**
cursor 65
datefield **66**
datefield 66
diagrams **67**
diagram 68
getstring **112**
tk_getString 112
history **113**
tklib_history 113
ico **114**
ico 114
ipentry **117**
ipentry 117

khim **120**
khim 120
map **122**
area-display 122
area-file 123
area-map-display 124
area-store-fs 124
area-store-mem 125
area-table-display 126
box-display 126
box-entry 127
box-file 128
box-map-display 128
box-store-fs 129
box-store-mem 129
box-table-display 130
display 131
map_overview 132
mark 133
point-file 134
point-map-display 134
point-store-fs 135
point-store-mem 136
point-table-display 136
provider-osm 137
track-display 138
track-entry 139
track-file 139
track-map-display 140
track-store-fs 140
track-store-mem 141
track-table-display 142
menubar **142**
menubar 143
notifywindow **153**
notifywindow 153
ntext **153**
ntext 154
ntextBindings 157
ntextIndent 161
ntextWordBreak 165
persistentSelection **168**
persistentSelection 168
plotchart **172**
plotchart 172
shtmlview **241**

shtmlview-doctools 241
shtmlview-mkdoc 243
shtmlview 244
swaplist 252
swaplist 252
tkpiechart 253
canvaslabel 253
pie 256
pieboxlabeler 259
pieperipherallabeler 260
tooltip 262
tipstack 262
tooltip 263
widget 267
widget 267
widget_calendar 268
widget_dateentry 270
widget_toolbar 272
widgetPlus 273
widgetPlus 273
widgetl 279
widget_listentry 279
widget_listsimple 285
widgetv 289
widget_validator 289
Lizenz 291
Lizenzbedingungen license 291

apps

bitmap-editor

Stichwörter: bitmap-editor, bitmap, editor, image, pixel, xbm

Name

Bitmap-Editor – Editor für XBM-Bilder

Übersicht

bitmap-editor ?*xbmfile*?

Beschreibung

Die in diesem Dokument beschriebene Anwendung, **bitmap-editor**, ist ein einfacher Editor für XBM-basierte Bitmap-Bilder. Der von Keith Vetter entwickelte Originalcode ist unter http://wiki.tcl-lang.org/6298 zu finden.

Befehlszeile

bitmap-editor ?*xbmfile*?

Wird der Editor ohne Argument aufgerufen, wird die Editor-Benutzeroberfläche geöffnet und eine Standard-Bitmap zur Bearbeitung angezeigt. Wird er mit einem Argument aufgerufen, wird erwartet, dass es sich um den Pfad zu einer Bitmap-Datei im XBM-Format handelt, und die darin enthaltene Bitmap wird angezeigt.

path *xbmfile* (in/out)

Dieses Argument legt den Pfad zu einer Bitmap-Datei im XBM-Format fest, deren Inhalt von der Anwendung angezeigt und bearbeitet werden soll.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und die darin beschriebene Anwendung enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *bitmap* der Tcllib-SF-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen an der Anwendung oder an der Dokumentation.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *bitmap-editor* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Bildverarbeitung

Copyright


```
Copyright (c)
```


diagram-viewer

Stichwörter: diagram-viewer, canvas, conversion, diagram, vector

Name
diagram-viewer – Ein schlanker Diagramm-Prozessor

Übersicht
diagram-viewer show *inputfile...*
diagram-viewer convert -o *output format inputfile...*

Beschreibung
Die in diesem Dokument beschriebene Anwendung, **diagram-viewer**, ist ein schlanker Prozessor für tklib-Diagrammdateien

diagram-viewer basiert auf dem Paket **diagram**. Dort stehen Beispiele und die Sprachreferenz.

Anwendungsfälle
diagram-viewer wurde unter Berücksichtigung der folgenden beiden Anwendungsfälle entwickelt.

1. Prozess der Verarbeitung und Anzeige einer oder mehrerer Diagrammdateien.
2. Stapelkonvertierung einer oder mehrerer Diagrammdateien in Rasterbilder.

Befehlszeile
diagram-viewer show *inputfile*...
Dies ist die Aufrufform für den Anwendungsfall [1]. Die Anwendung öffnet eine grafische Benutzeroberfläche, auf der links die Liste der Eingabedateien angezeigt wird, sodass der Benutzer auswählen kann, welche davon auf der rechten Seite auf dem Canvas dargestellt werden sollen.

diagram-viewer convert -o *output format inputfile*...
Dies ist die Aufrufform für den Anwendungsfall [2]. Die Anwendung wandelt die Eingabedateien in ein Rasterbild des angegebenen *format* um.

path *output* (in)
Dieses Argument legt fest, wohin das erzeugte Bild geschrieben werden soll. Es kann sich um den Pfad zu einer Datei oder einem Verzeichnis handeln.

Wenn *output* nicht existiert, muss [file dirname $output] vorhanden sein und es muss sich um ein beschreibbares Verzeichnis handeln.

Bei mehreren Eingabedateien wird das generierte Bild in eine Datei in diesem Verzeichnis geschrieben, wobei der Name dieser Datei aus *inputfile* und *format* abgeleitet wird.

Bei einer einzelnen Eingabedatei wird das generierte Bild in diese Datei geschrieben.

(handle) *format* (in)
Dieses Argument legt das Bildformat fest, in das die Diagramme bei der Verarbeitung der Eingabe konvertiert werden sollen. Die Anwendung erkennt alle Formate, die vom Paket **Img** unterstützt werden, d. h. für die sie ein Paket **img::format** (zusammen mit **canvas::snap**) laden kann, sowie alle Formate, für die sie ein Paket **canvas::format** laden kann. Dies sind (derzeit) **mvg** und **pdf**.

path *inputfile* (in)

Dieses Argument legt den Pfad zur zu verarbeitenden Diagrammdatei fest. Die Datei muss vorhanden und lesbar sein sowie im *diagram*-Format vorliegen.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und die darin beschriebene Anwendung enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *diagram* der Tcllib-SF-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen an der Anwendung oder an der Dokumentation.

Fehler, Ideen, Rückmeldungen

Dieses Dokument sowie das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *diagram* des Tklib Trackers. Bitte teilen Sie uns auch Ihre Ideen für Verbesserungen mit, die Sie möglicherweise für eines der beiden Pakete und/oder die Dokumentation haben.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2010 Andreas Kupries <andreas_kupries@users.sourceforge.net>
```

shtmlview

Stichwörter: shtmlview, doctools, html, markdown, mkdoc, viewer

Name

shtmlview – Anwendung zur Anzeige von einfachem HTML und Markdown

Übersicht

shtmlview
shtmlview --help
shtmlview --version
shtmlview --render *text*
shtmlview --render -
shtmlview *filename*

Beschreibung

Die in diesem Dokument beschriebene Anwendung, **shtmlview**, ist ein einfacher Viewer für HTML-Dateien. Darüber hinaus werden Dokumente im *Markdown-*, *doctools-* oder *mkdoc*-Format unterstützt, wenn entsprechende Pakete zur Verfügung stehen. Siehe Abschnitt Über HTML hinaus.

Befehlszeile

shtmlview

siehe unten ...

shtmlview --help

Gibt Informationen zur Nutzung in die Standardfehlerausgabe aus.

shtmlview --version
: Gibt die Anwendungsversion auf der Standardausgabe aus.

shtmlview --render *text*
: Zeigt den eingegebenen Text in einem Anzeigefenster an. Der Text wird als Markdown behandelt, es sei denn, er beginnt mit einem erkannten HTML-Tag. In diesem Fall wird er als HTML interpretiert und entsprechend dargestellt.

shtmlview --render -
: Wie zuvor, mit dem Unterschied, dass der darzustellende Text aus der Standardeingabe gelesen wird.

shtmlview *filename*
: Liest die genannte Datei und zeigt sie in einem Anzeige-Widget an.

Über HTML hinaus

Dokumente in einem anderen Format als HTML erfordern die Installation verschiedener Pakete, damit die Anwendung diese anzeigen kann.

Markdown
- **Markdown**

doctools
- **shtmlview::doctools**
- **dtplite**

mkdoc
- **shtmlview::mkdoc**
- **mkdoc::mkdoc**

Fehler, Ideen, Rückmeldungen

Dieses Dokument und die darin beschriebene Anwendung enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *shtmlview* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen an der Anwendung oder an der Dokumentation.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *shtmlview* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Dokumentanzeige

Copyright


```
Copyright (c)
```


autoscroll

autoscroll

Stichwörter: autoscroll, scroll, scrollbar

Name

autoscroll – Sorgt dafür, dass eine Scrollbar je nach Bedarf automatisch ein- und ausgeblendet wird

Übersicht

package require Tcl
package require autoscroll ?**1.1**?
::autoscroll::autoscroll *scrollbar*
::autoscroll::unautoscroll *scrollbar*
::autoscroll::wrap
::autoscroll::unwrap

Beschreibung

Mit diesem Paket können Scrollbars je nach Größe und Inhalt des Widgets, das die Scrollbar verschiebt, nach Bedarf zugeordnet und wieder entfernt werden. Die Scrollbar muss entweder von pack oder grid verwaltet werden; andere Geometriemanager werden nicht unterstützt.

Bei der Verwaltung durch pack gehen alle Geometrieänderungen verloren, die im Elternwidget der Scrollbar zwischen dem Entfernen und dem erneuten Zuordnen vorgenommen werden. Es ist ein Fehler, eines der Geschwisterwidgets der Scrollbar zu löschen, während sie nicht zugeordnet ist. Bei der Verwaltung durch grid ersetzt die Scrollbar beim erneuten Zuordnen das, was inzwischen mit grid in derselben Zeile und Spalte platziert wurde.

Dieses Paket kann für jedes Widget verwendet werden, das einer Scrollbar ähnelt, sofern es den Unterbefehl **set** in derselben Weise wie eine Scrollbar unterstützt. Wenn der Unterbefehl **set** nicht verwendet wird, hat dieses Paket keine Wirkung.

::autoscroll::autoscroll *scrollbar*
: Sorgt dafür, dass die bereits vorhandene Scrollbar **scrollbar** je nach Bedarf ein- und ausgeblendet wird.

::autoscroll::unautoscroll *scrollbar*
: Gibt die angegebene Scrollbar in ihren ursprünglichen statischen Zustand zurück.

::autoscroll::wrap
: Stellt sicher, dass alle nach Ausführung dieses Befehls erstellten Scrollbars bei Bedarf automatisch zugeordnet und wieder entfernt werden.

::autoscroll::unwrap
: Deaktiviert das automatische Scrollen aller neuen Scrollbars. Hat keine Auswirkungen auf bereits vorhandene Scrollbars.

```
text .t -yscrollcommand ".scrolly set"
scrollbar .scrolly -orient v -command ".t yview"
pack .scrolly -side right -fill y
pack .t -side left -fill both -expand 1
::autoscroll::autoscroll .scrolly
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere

Probleme. Bitte melden Sie diese in der Kategorie *autoscroll* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas

canvas_drag

Stichwörter: canvas_drag, canvas, dragging

Name

canvas::drag – Verwalten des Ziehens von Canvas-Elementen oder Elementgruppen

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::drag ?**0.1**?
::canvas::drag on *canvas tagOrId cmd option...*
::canvas::drag off *canvas tagOrId*
::canvas::drag item *canvas tagOrId option...*
::canvas::drag group *canvas tagOrId cmd option...*
{*}cmd start *canvas item*
{*}cmd move *canvas clientdata dx dy*
{*}cmd done *canvas clientdata*

Beschreibung

Dieses Paket enthält Hilfsprogramme zum Einrichten und Entfernen des Ziehens von Elementen oder Elementgruppen auf einem Canvas, wobei alle komplexen Zusammenhänge hinsichtlich der Bindungen vor dem Benutzer verborgen bleiben.

API

::canvas::drag on *canvas tagOrId cmd option...*

Dieser Befehl initialisiert das Ziehen von Elementen im *canvas*-Widget, wobei die als Ziehgriffe verwendeten Elemente durch *tagOrId* identifiziert werden. Das Befehlspräfix *cmd*, das beim Start und während der Bewegung des Ziehvorgangs aufgerufen wird, ist für die Initialisierung und die eigentliche Ausführung des Ziehvorgangs zuständig.

Die Signatur des Befehlspräfixes wird später im Abschnitt Callback beim Ziehen beschrieben.

Ebenso werden die zulässigen Optionen und ihre Werte im Abschnitt Optionen beschrieben.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::drag off *canvas tagOrId*

Dieser Befehl hebt alle Drag-Operationen auf, die für die Elemente des Canvas *canvas* festgelegt wurden, die durch *tagOrId* identifiziert werden.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::drag item *canvas tagOrId option...*

Hierbei handelt es sich um einen Komfort-Befehl, der die Methode **on** (siehe oben) umschließt, um einzelne Elemente des *canvas*-Widgets, die durch *tagOrId* identifiziert werden, zu verschieben.

Dazu wird ein interner Standard-Callback verwendet.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::drag group *canvas tagOrId cmd option...*

Dies ist ein Komfort-Befehl, der die Methode **on** (siehe oben) umschließt, um einzelne Elemente des *canvas*-Widgets, die durch *tagOrId* identifiziert werden, zu verschieben.

Hierfür wird ein interner Standard-Callback verwendet. Die angegebene Rückruffunktion *cmd* hat dieselbe Signatur wie der Callback beim Ziehen, mit der Ausnahme, dass

1. Die Methode **move** wird nicht aufgerufen.
2. Das Ergebnis der Methode **start** *muss* ein "canvas"-Tag sein, der auf die gesamte Gruppe der zu verschiebenden Elemente verweist. Mit anderen Worten: Es muss die Umwandlung vom Ziehgriff (Element-ID) in die verschobene Gruppe (Tag) vornehmen.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Callback beim Ziehen

Die Drag-Rückruffunktion ist ein Befehlspräfix, das auf die folgenden zwei Arten aufgerufen wird:

{*}cmd start *canvas item*

Diese Funktion wird aufgerufen, wenn das Ziehen mithilfe des durch die *item*-ID des Canvas-Elements identifizierten Ziehgriffs initiiert wurde. Die Rückruffunktion muss nun alle für die jeweilige Art des Ziehvorgangs erforderlichen Schritte ausführen.

Das Ergebnis des Befehls kann beliebig sein. Es wird vom System als Client-Information gespeichert und unverändert an die Bewegungs-Rückruffunktion zur weiteren Verwendung übergeben. Auf diese Weise kann der Drag-Callback den benutzerdefinierten Zustand vom Start bis zur Bewegung beibehalten.

{*}cmd move *canvas clientdata dx dy*

Diese Form wird aufgerufen, wenn die Maus während eines Ziehvorgangs bewegt wird. Es wird mit den Client-Daten aus dem Start-Callback (oder dem vorherigen Bewegungs-Callback) sowie den Entfernungen aufgerufen, die die Maus in horizontaler und vertikaler Richtung zurückgelegt hat.

Das Ergebnis des Befehls kann beliebig sein. Es wird vom System als Client-Information gespeichert und unverändert an die nächste Rückruffunktion zur weiteren Verwendung übergeben. Auf diese Weise kann der Drag-Callback einen benutzerdefinierten Zustand von Bewegung zu Bewegung beibehalten.

{*}cmd done *canvas clientdata*

Diese Form wird aufgerufen, wenn der Ziehvorgang beendet ist. Es wird mit den Client-Daten aus der Rückruffunktion für die letzte Bewegung (oder dem Start-Callback, falls keine Bewegung stattgefunden hat) aufgerufen.

Das Ergebnis des Befehls wird ignoriert.

Optionen

Die Befehle zum Erstellen von Drag-Operationen (**on**, **item** und **group**) akzeptieren alle die folgenden Optionen zur Konfiguration des neuen Drag.

-event *spec*

Der Wert dieser Option legt fest, welche Maustaste zum Auslösen des Ziehvorgangs verwendet wird, sowie gegebenenfalls die Tastenkombination. Beispiele für Angaben:

Um einen Ziehvorgang durch Drücken der Maustaste 3 auf einem Ziehgriff auszulösen, verwenden Sie:

```
-event 3
```

Dies ist auch die Standardeinstellung, wenn keine Option festgelegt wird.

Um einen Ziehvorgang durch Drücken der rechten Maustaste auf einem Ziehgriff bei gedrückter Strg-Taste zu starten, verwenden Sie:

```
-event Control-2
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_ecircle

Stichwörter: canvas_ecircle, canvas, circle, editing

Name

canvas::edit::circle – Bearbeiten eines Kreises auf einem Canvas

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::circle ?**0.1**?
::canvas::edit circle *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName set *minx miny maxx maxy*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Beschreibung

Dieses Paket stellt eine Klasse bereit, deren Instanzen die Bearbeitung eines Kreises auf einem

Canvas übernehmen. Die Instanzen können hinsichtlich der Darstellung der Markierungen, die die Hauptpunkte des Kreises kennzeichnen, konfiguriert werden. Beachten Sie, dass die Instanzen den bearbeiteten Kreis nicht selbst speichern, sondern diese Aufgabe an ein konfigurierbares Objekt weiterleiten.

Interaktion

Der Kreis-Editor verwendet intern eine Instanz des Pakets **canvas::edit::points** und übernimmt alle damit verbundenen Optionen und Verhaltensweisen. Die Optionen zur Farbe werden zusätzlich auf das Kreis-Element dieses Editors angewendet.

Darüber hinaus bietet es die Option "**-circle-config**", mit der Sie das Kreisobjekt, das zur Darstellung des Kreises anhand seiner Hauptpunkte verwendet wird, individuell anpassen können. Beachten Sie, dass damit die vererbten Farboptionen nicht überschrieben werden können.

Während einer Instanz wird eine Zustandsmaschine ausgeführt, in der der Benutzer zwei Punkte legt; daraufhin vervollständigt das System den Kreis und meldet dies. Ab diesem Zeitpunkt verhindert das System das Hinzufügen weiterer Punkte. Das Ziehen der Punkte ist weiterhin möglich, wodurch eine Größenänderung und das Verschieben des Kreises ermöglicht werden. Wird einer der Hauptpunkte gelöscht, wird der gesamte Kreis gelöscht; daraufhin ist das System wieder bereit für die Erstellung eines neuen Kreises aus zwei Punkten.

API für Klassen

::canvas::edit circle *objectName canvas options*...

Dieser Befehl "class" erstellt und konfiguriert eine neue Instanz eines Kreiseditors mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas*-Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Die hier zulässigen Optionen und ihre Werte werden im Abschnitt Optionen erläutert.

Instanz-API

Die Instanzen der Kreis-Editoren stellen die folgende API bereit:

objectName destroy

Diese Methode löscht den Kreis-Editor und gibt alle seine internen Ressourcen frei.

Hinweis: Durch diesen Vorgang werden weder die vom Editor auf dem zugehörigen Canvas verwalteten Elemente noch der Canvas selbst gelöscht.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName enable

Diese Methode aktiviert die Bearbeitung des Kreises auf dem Canvas. Dies ist die Standardeinstellung nach der Erstellung der Instanz. Ein Aufruf wird ignoriert, wenn der Editor bereits aktiv ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die komplementäre Methode lautet **disable**. Die Abfragemethode für den aktuellen Status

lautet **active**.

objectName disable

Diese Methode deaktiviert die Bearbeitung des Kreises auf dem Canvas. Ein Aufruf wird ignoriert, wenn der Editor bereits deaktiviert ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode lautet **enable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName active

Diese Methode fragt den Status des Editors ab.

Das Ergebnis der Methode ist ein boolescher Wert: **true**, wenn der Editor aktiv ist, und andernfalls **false**, d. h. deaktiviert.

Die Methoden zum Ändern des Status lauten **enable** und **disable**.

objectName set *minx miny maxx maxy*

Diese Methode legt den Kreis programmgesteuert anhand der Koordinaten des Rahmens fest. Dabei wird ein zuvor gezeichneter (möglicherweise unvollständiger) Kreis überschrieben.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Beachten Sie, dass diese Methode die gesamte Reihe von Rückruffunktionen durchläuft, die beim Erstellen der Hauptpunkte aufgerufen werden, d. h. **-create-cmd** und, was noch wichtiger ist, **-data-cmd**.

Dies ist die Methode, mit der ein bereits vorhandener Kreis in eine Editor-Instanz geladen wird.

objectName clear

Diese Methode entfernt den Kreis programmgesteuert aus dem Editor und schafft so Platz für die Erstellung eines neuen Kreises.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft dieselben Rückruffunktionen, die auch dann aufgerufen werden, wenn der Benutzer einen Hauptpunkt interaktiv entfernt, d. h. **-data-cmd**.

Optionen

Der Befehl "class" akzeptiert die folgenden Optionen

-active-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn die Maus den Kreis oder einen der von der Instanz verwalteten Hauptpunkte berührt oder verlässt.

Wird kein Wert angegeben, wird standardmäßig ein leerer Befehl verwendet, der keine Aktion auslöst.

Die Signatur dieses Befehlspräfixes lautet

activeCmd *editorObj id*
Die *id* identifiziert das eingegebene Objekt.

Eine leere *id* bedeutet, dass das zuletzt eingegebene Objekt beibehalten wurde.

Der Wert **circle** gibt an, dass die Gliederung des Kreises eingegeben wurde.

Jeder andere Wert ist ein Code für einen der Hauptpunkte des Kreises, d. h. **center** und **edge**.

Das Ergebnis dieser Methode wird ignoriert.

-add-remove-point *eventspec*
Der Wert dieser Option ist eine Ereignisspezifikation (ohne Klammerwinkel), die festlegt, welches Ereignis das Hinzufügen bzw. Entfernen eines Punktes auslöst.

Diese Option kann nur bei der Erstellung festgelegt werden.

In der Standardeinstellung werden für das Hinzufügen und Entfernen von Punkten unterschiedliche Ereignisse verwendet, nämlich **ButtonPress-1** bzw. **ButtonPress-2**.

Bei Verwendung dieser Option wird für beide Vorgänge dasselbe Ereignis verwendet. Dies stellt kein Problem dar, da das Hinzufügen als canvas-globales Ereignis gebunden ist, während das Entfernen an die jeweiligen Canvas-Elemente gebunden ist.

-circle-config *list*
Der Wert dieser Option ist eine Liste von Optionen und deren Werten, die auf das Element "interner Kreis" angewendet werden sollen, sobald dieses nach Änderungen neu generiert wird. Hinweis: Diese Optionen können die durch die Hauptoptionen **-color** und **-hilit-color** festgelegte Farbe des Kreises *nicht* überschreiben.

Wird kein Wert angegeben, wird standardmäßig eine leere Liste verwendet.

-color *colorspec*
Der Wert dieser Option ist die Farbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **SkyBlue2**.

-create-cmd *command-prefix*
Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen neuen Hauptpunkt erstellen muss.

Obwohl diese Option auch nach der Erstellung festgelegt werden kann, wird empfohlen, diese Funktion ausschließlich dazu zu nutzen, benutzerdefinierte Verarbeitungsschritte einzufügen, die zum Zeitpunkt der Erstellung ausgeführt werden sollen und dabei den vorherigen Wert der Option speichern und aufrufen.

Wird kein Wert festgelegt, wird standardmäßig ein Befehl verwendet, der einen blau umrandeten Kreis mit einem Radius von 3 und dem Mittelpunkt an der Position des neuen

Punktes erstellt.

Die Signatur dieses Befehlspräfixes lautet
createCmd *canvas x y*
Das Ergebnis des Befehlspräfixes *muss* eine Liste der Canvas-Elemente sein, die zur Darstellung des Markers erstellt wurden. Beachten Sie hierbei, dass die visuelle Darstellung eines "Hauptpunkts" aus mehreren Canvas-Elementen in beliebiger Form bestehen kann.

Die zurückgegebene Liste der Elemente darf leer sein; dies wird als Hinweis darauf gewertet, dass die Rückruffunktion die Erstellung des Hauptpunkts abgelehnt hat.

-data-cmd *command-prefix*
Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn der Kreis in irgendeiner Weise bearbeitet wurde (erstellt, geändert, gelöscht). Auf diese Weise leitet der Editor die eigentliche Speicherung der Kreisdaten an ein externes Objekt weiter.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, ist der Standardwert eine leere Zeichenkette, und sie wird vom Editor ignoriert, d. h., nicht aufgerufen.

Die Signatur dieses Befehlspräfixes lautet
dataCmd *editorObj coordinates*
Diese Rückruffunktion wird aufgerufen, wenn der Kreis entweder interaktiv oder programmgesteuert geändert wurde. Informationen zum letzteren Fall finden Sie in der Instanzmethode **set**.

Das *editorObj* identifiziert die Instanz, die die Rückruffunktion aufruft, während *coordinates* eine Liste mit vier Elementen ist, die in dieser Reihenfolge die minimalen x- und y-Koordinaten sowie die maximalen x- und y-Koordinaten des Kreises enthält.

Das Ergebnis dieser Methode wird ignoriert.

-drag-point *eventspec*
Der Wert dieser Option ist eine Ereignisspezifikation (ohne eckige Klammern), die festlegt, welches Ereignis eine Drag-Aktion an Punkten auslöst.

Diese Option kann nur zur Erstellungszeit festgelegt werden.

Die Standard-Spezifikation lautet **ButtonPress-3**.

-hilit-color *colorspec*
Der Wert dieser Option ist die Hervorhebungsfarbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-highlight-cmd** festgelegt ist.

Der Standardwert lautet **red**.

-kind *name*
Der Wert dieser Option ist der Canvas-Elementtyp für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist. Es sind ausschließlich Elementtypen zulässig, die über einen Begrenzungsrahmen festgelegt werden.

Der Standardwert lautet **oval**.

-radius *int*
Der Wert dieser Option gibt den Radius für das Standardelement an, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **3**.

-tag *string*
Der Wert dieser Option ist der Name des Canvas-Tags, mit dem alle Elemente aller vom Editor verwalteten Eckpunkte identifiziert werden sollen.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, lautet der Standardwert **CIRCLE**.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_epoints

Stichwörter: canvas_epoints, canvas, editing, point cloud, points

Name

canvas::edit::points – Bearbeiten einer Punktwolke auf einem Canvas

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.3**?
::canvas::edit points *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd add *editorObj id x y*
dataCmd remove *editorObj id*
dataCmd move start *editorObj id*
dataCmd move delta *editorObj id x y dx dy*
dataCmd move done *editorObj id*

Beschreibung

Dieses Paket stellt eine Klasse bereit, deren Instanzen die Bearbeitung einer Ansammlung von Punktmarkierungen auf einem Canvas übernehmen. Die Instanzen können hinsichtlich der Darstellung der Markierungen konfiguriert werden. Beachten Sie, dass die Instanzen die bearbeiteten Punkte nicht selbst speichern, sondern diese Aufgabe an ein konfigurierbares Objekt weiterleiten.

Interaktion

Die Standard-Bindungen ermöglichen das Erstellen, Entfernen und Verschieben von Punkten (Markern) mithilfe der Maustasten 1, 2 und 3. Im Einzelnen:

1. Wenn Sie mit Taste 1 auf eine beliebige Stelle klicken, wird ein neuer Punkt hinzugefügt.
2. Wenn Sie mit Taste 2 auf einen Punkt klicken, wird dieser Punkt entfernt.
3. Durch Klicken auf einen Punkt mit Taste 3 wird ein Ziehvorgang gestartet, der mit dem Loslassen der Taste endet.

Mit der Option **-drag-point** können Sie die Taste ändern, mit der Ziehvorgänge ausgelöst werden. Hinweis: Der Wert der Option enthält nicht nur die Tastennummer, sondern auch ein Modifikatorpräfix, das durch einen Bindestrich von der Tastennummer getrennt ist ("**-**"). Die unterstützten Modifikatoren sind **Control**, **Shift** und **Alt**.

Die Option **-add-remove-point** kann verwendet werden, um die Taste zu ändern, mit der Punkte hinzugefügt oder entfernt werden. Dies funktioniert, da das Entfernen eine Bindung ist, während das Hinzufügen global für den Canvas gilt. Die Option erkennt für die Taste dieselbe Syntax wie **-drag-point**.

In der Standardkonfiguration ist das Canvas-Element, das einen Punkt darstellt, ein Kreis, der mithilfe der Darstellungsoptionen **-color**, **-hilit-color** und **-radius** konfiguriert wird. Ihre Standardwerte sind **Skyblue2**, **red** bzw. **3**.

Mit der Option **-kind** lässt sich das Canvas-Element leicht an beliebige Arten anpassen, wobei die Element-Optionen **-width**, **-outline**, **-fill** und **-activefill** akzeptiert.

Für eine umfassendere Anpassung ist es erforderlich, mithilfe der Option **-create-cmd** einen Erstellungs-Callback zu konfigurieren. Die Rückruffunktion nimmt den Canvas sowie die x- und y-Koordinaten in dieser Reihenfolge als Argumente entgegen und soll eine Liste der Canvas-Elemente zurückgeben, die den Punkt darstellen.

Beachten Sie den Plural. Mit der Rückruffunktion ist es möglich, einen Punkt mithilfe einer Gruppe von Elementen darzustellen.

Beachten Sie ferner, dass kein Punkt erstellt wird, wenn die Rückruffunktion eine leere Liste gibt. Mit anderen Worten: Die Rückruffunktion kann auch dazu verwendet werden, zu prüfen, ob ein Punkt an der angegebenen Position erwünscht ist oder nicht. Dies lässt sich mit dem Standardverhalten bei der Erstellung kombinieren, da die Option abgefragt werden kann und ihr Standardwert die Rückruffunktion für das Standardverhalten ist.

Um noch etwas bei der Rückruffunktion zu bleiben: Die Option **-active-cmd** legt eine Rückruffunktion fest, die immer dann aufgerufen wird, wenn sich der Mauszeiger über einem der erstellten Punkte befindet, d. h. wenn dieser aktiv ist. Dies kann zur Steuerung abgeleiteter Anzeigen genutzt werden, beispielsweise zum Hervorheben des Eintrags des Punktes in einer Tabelle oder einem anderen Widget.

Die letzte wichtige Option ist **-tag**. Ihr Wert ist der Name des Canvas-Tags, mit dem alle Elemente gekennzeichnet werden, die einem gehören, d. h. von diesem erstellt und verwaltet

werden. Der Standardwert lautet **POINT**.

Wenn mehrere Instanzen dieses Verhaltens an dieselbe "Canvas" angehängt werden, muss jede Instanz mit einem eindeutigen Tag konfiguriert werden, um zu verhindern, dass sie sich gegenseitig stören.

API für Klassen

::canvas::edit points *objectName canvas options*...

Dieser Befehl der Klasse erstellt und konfiguriert eine neue Instanz eines Punktwolken-Editors mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas*-Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Die hier zulässigen Optionen und ihre Werte werden im Abschnitt Optionen erläutert.

Instanz-API

Instanzen der Punktwolken-Editoren stellen die folgende API bereit:

objectName destroy

Diese Methode löscht den Punktwolken-Editor und gibt alle seine internen Ressourcen frei.

Beachten Sie, dass dieser Vorgang weder die Elemente der Punktmarkierungen, die der Editor auf dem zugehörigen Canvas verwaltet hat, noch den Canvas selbst löscht.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName enable

Diese Methode aktiviert die Bearbeitung der Punktwolke auf dem Canvas. Dies ist die Standardeinstellung nach der Erstellung der Instanz. Ein Aufruf wird ignoriert, wenn der Editor bereits aktiv ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode ist **disable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName disable

Diese Methode deaktiviert die Bearbeitung der Punktwolke auf dem Canvas. Ein Aufruf wird ignoriert, wenn der Editor bereits deaktiviert ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode ist **enable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName active

Diese Methode fragt den Status des Editors ab.

Das Ergebnis der Methode ist ein boolescher Wert: **true**, wenn der Editor aktiv ist, und

andernfalls **false**, d. h. deaktiviert.

Die Methoden zum Ändern des Status lauten **enable** und **disable**.

objectName add *x y*

Diese Methode erstellt programmgesteuert einen Punkt an der festgelegten Position.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft die gesamte Reihe von Rückruffunktionen, die aufgerufen werden, wenn der Benutzer interaktiv einen Punkt erstellt, d. h. **-create-cmd** und, was noch wichtiger ist, **-data-cmd**.

Dies ist die Methode, mit der bereits vorhandene Punkte in eine Editor-Instanz geladen werden können.

objectName clear

Diese Methode entfernt programmgesteuert alle Punkte aus dem Editor.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Beachten Sie, dass diese Methode die gleiche Rückruffunktion durchläuft, die auch dann aufgerufen wird, wenn der Benutzer einen Punkt interaktiv entfernt, d. h. **-data-cmd**.

Optionen

Der Befehl "class" akzeptiert die folgenden Optionen

-active-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn die Maus einen von der Instanz verwalteten Punkt erreicht oder verlässt.

Wird kein Wert angegeben, wird standardmäßig ein leerer Befehl verwendet, der keine Aktion ausführt.

Die Signatur dieses Befehlspräfixes lautet

activeCmd *editorObj id*

Die *id* identifiziert die Position innerhalb des Editors.

Eine leere *id* bedeutet, dass die zuletzt eingegebene Position verlassen wurde.

Das Ergebnis dieser Methode wird ignoriert.

-add-remove-point *eventspec*

Der Wert dieser Option ist eine Ereignisspezifikation (ohne Klammerwinkel), die festlegt, welches Ereignis das Hinzufügen bzw. Entfernen eines Punktes auslöst.

Diese Option kann nur bei der Erstellung festgelegt werden.

In der Standardeinstellung werden für das Hinzufügen und Entfernen von Punkten unterschiedliche Ereignisse verwendet, nämlich **ButtonPress-1** bzw. **ButtonPress-2**.

Bei Verwendung dieser Option wird für beide Vorgänge dasselbe Ereignis verwendet. Dies stellt kein Problem dar, da das Hinzufügen als canvas-globales Ereignis gebunden ist, während das Entfernen an die jeweiligen Canvas-Elemente gebunden ist.

-color *colorspec*

Der Wert dieser Option ist die Farbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **SkyBlue2**.

-create-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen neuen Punkt erstellen muss.

Obwohl diese Option auch nach der Erstellung festgelegt werden kann, wird empfohlen, diese Funktion ausschließlich dazu zu nutzen, benutzerdefinierte Verarbeitungsschritte einzufügen, die zum Zeitpunkt der Erstellung ausgeführt werden sollen und dabei den vorherigen Wert der Option speichern und aufrufen.

Wird kein Wert festgelegt, wird standardmäßig ein Befehl verwendet, der einen blauen Kreis mit schwarzem Rahmen und einem Radius von 3 erzeugt, dessen Mittelpunkt an der Position des neuen Punktes liegt.

Die Signatur dieses Befehlspräfixes lautet

createCmd *canvas x y*

Das Ergebnis des Befehlspräfixes *muss* eine Liste der Canvas-Elemente sein, die zur Darstellung des Markers erstellt wurden. Beachten Sie hierbei, dass die visuelle Darstellung eines "Punkts" aus mehreren Canvas-Elementen in beliebiger Form bestehen kann.

Die zurückgegebene Liste der Elemente darf leer sein; dies wird als Signal gewertet, dass die Rückruffunktion die Erstellung des Punkts abgelehnt hat.

-data-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn ein Punkt in irgendeiner Weise bearbeitet wurde. Auf diese Weise leitet der Editor die eigentliche Speicherung der Punktinformationen an ein externes Objekt weiter.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, ist der Standardwert eine leere Zeichenkette, und sie wird vom Editor ignoriert, d. h., nicht aufgerufen.

Die Signaturen dieses Befehlspräfixes lauten

dataCmd add *editorObj id x y*

Diese Rückruffunktion wird aufgerufen, wenn der Instanz ein neuer Punkt hinzugefügt wurde, sei es interaktiv oder programmgesteuert. Informationen zu letzterem finden Sie in der Instanzmethode **add**.

Die Variable *id* identifiziert den Punkt innerhalb des Editors und wird von den beiden anderen Rückruffunktionen verwendet, um festzulegen, welcher Punkt geändert werden soll.

Die letzten beiden Argumente *x* und *y* legen die Position des neuen Punkts in Canvas-Koordinaten fest.

Das Ergebnis dieser Methode wird ignoriert.

dataCmd remove *editorObj id*

Diese Rückruffunktion wird aufgerufen, wenn ein Punkt aus der Editor-Instanz entfernt wird.

Die *id* identifiziert den entfernten Punkt innerhalb des Editors.

Das Ergebnis dieser Methode wird ignoriert.

dataCmd move start *editorObj id*

Diese Rückruffunktion wird aufgerufen, wenn die Verschiebung eines Punktes in der Editor-Instanz begonnen hat.

Die *id* identifiziert den Punkt innerhalb des Editors, der verschoben werden soll.

Das Ergebnis dieser Methode wird ignoriert.

dataCmd move delta *editorObj id x y dx dy*

Diese Rückruffunktion wird aufgerufen, wenn der Punkt in der Editor-Instanz verschoben wurde.

Die Variable *id* identifiziert den verschobenen Punkt innerhalb des Editors, und die übrigen Argumente *x*, *y*, *dx* und *dy* geben die neue absolute Position des Punkts sowie die vollständigen Delta-Werte zur ursprünglichen Position an.

Zum Zeitpunkt des Aufrufs ist die Verschiebung vom System *noch nicht* bestätigt. Erst nachdem die Methode **move done** aufgerufen wurde und die letzte Position akzeptiert oder abgelehnt hat, aktualisiert der Editor seine internen Datenstrukturen, wobei er entweder die neue Position bestätigt oder die Verschiebung auf die ursprüngliche Position zurücknimmt.

Vor diesem Hintergrund sollten die hier bereitgestellten Positionsdaten bis dahin nur im temporären Speicher abgelegt werden.

Das Ergebnis dieser Methode wird ignoriert.

dataCmd move done *editorObj id*

Diese Rückruffunktion wird aufgerufen, wenn die Verschiebung eines Punktes in der Editor-Instanz abgeschlossen ist.

Die *id* identifiziert den verschobenen Punkt innerhalb des Editors.

Das Ergebnis dieser Methode muss ein boolescher Wert sein. Wenn die Methode den Wert **false** gibt, wird die Verschiebung abgelehnt und rückgängig gemacht.

-drag-point *eventspec*

Der Wert dieser Option ist eine Ereignisspezifikation (ohne eckige Klammern), die festlegt, welches Ereignis eine Drag-Aktion an Punkten auslöst.

Diese Option kann nur zur Erstellungszeit festgelegt werden.

Die Standard-Spezifikation lautet **ButtonPress-3**.

-hilit-color *colorspec*
Der Wert dieser Option ist die Hervorhebungsfarbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-highlight-cmd** festgelegt ist.

Der Standardwert lautet **red**.

-kind *name*
Der Wert dieser Option ist der Canvas-Elementtyp für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist. Es sind ausschließlich Elementtypen zulässig, die über einen Begrenzungsrahmen festgelegt werden.

Der Standardwert lautet **oval**.

-radius *int*
Der Wert dieser Option gibt den Radius für das Standardelement an, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **3**.

-tag *string*
Der Wert dieser Option ist der Name des Canvas-Tags, mit dem alle Elemente aller vom Editor verwalteten Punkte identifiziert werden sollen.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, lautet der Standardwert "**POINT**".

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_epolyline

Stichwörter: canvas_epolyline, canvas, editing, polyline

Name

canvas::edit::polyline – Bearbeiten einer Polylinie auf einem Canvas

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::polyline ?**0.2**?

::canvas::edit polyline *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Beschreibung

Dieses Paket stellt eine Klasse bereit, deren Instanzen die Bearbeitung einer einzelnen Polylinie (oder -gon) auf einem Canvas übernehmen. Instanzen können hinsichtlich der Darstellung der Markierungen, die die Eckpunkte der Linie kennzeichnen, konfiguriert werden; ebenso kann festgelegt werden, ob die Linie geschlossen sein soll oder nicht, und ob eine geschlossene Linie konvex sein muss oder nicht. Beachten Sie, dass Instanzen die bearbeitete Polylinie nicht selbst speichern, sondern diese Aufgabe an ein konfigurierbares Objekt weiterleiten.

Interaktion

Der Polylinien-Editor verwendet intern eine Instanz des Pakets **canvas::edit::points** und übernimmt alle damit verbundenen Optionen und Verhaltensweisen. Die Optionen zur Farbe werden zusätzlich auf das Linienelement dieses Editors angewendet.

Darüber hinaus bietet es zwei Optionen zur Konfiguration des Bearbeitungsmodus sowie eine Option zur Anpassung des Linien-/Polygon-Elements, das zum Zeichnen der Linie durch die Punkte verwendet wird.

Der Hauptbearbeitungsmodus wird durch die Option **-closed** gesteuert, die zwischen *Linie* (d. h. offen) und *Polygon* (d. h. geschlossen) unterscheidet. Die Standardeinstellung ist der Linien-/offene Modus.

Im Modus "Polygon/geschlossen" legt die Option **-convex** fest, ob der Editor die Konvexität des Polygons erzwingt oder nicht. Standardmäßig wird die Konvexität nicht erzwungen.

Die Option **-line-config** ermöglicht eine grundlegende Anpassung des Linien-/Polygon-Elements. Hinweis: Sie kann die vererbten Farboptionen nicht überschreiben.

API für Klassen

::canvas::edit polyline *objectName canvas options*...

Dieser Befehl, der Klassenbefehl, erstellt und konfiguriert eine neue Instanz eines Polylinien-Editors mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas*-Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Die hier zulässigen Optionen und ihre Werte werden im Abschnitt Optionen erläutert.

Instanz-API

Instanzen der Polylinien-Editoren stellen die folgende API bereit:

objectName destroy
Diese Methode löscht den Polylinien-Editor und gibt alle seine internen Ressourcen frei.

Hinweis: Durch diesen Vorgang werden weder die vom Editor auf dem zugehörigen Canvas verwalteten Elemente noch der Canvas selbst gelöscht.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName enable
Diese Methode aktiviert die Bearbeitung der Polylinie auf dem Canvas. Dies ist die Standardeinstellung nach der Erstellung der Instanz. Ein Aufruf wird ignoriert, wenn der Editor bereits aktiv ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die komplementäre Methode lautet **disable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName disable
Diese Methode deaktiviert die Bearbeitung der Polylinie auf dem Canvas. Ein Aufruf wird ignoriert, wenn der Editor bereits deaktiviert ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die komplementäre Methode lautet **enable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName active
Diese Methode fragt den Status des Editors ab.

Das Ergebnis der Methode ist ein boolescher Wert: **true**, wenn der Editor aktiv ist, und andernfalls **false**, d. h. deaktiviert.

Die Methoden zum Ändern des Status lauten **enable** und **disable**.

objectName add *x y*
Diese Methode fügt der Linie programmgesteuert einen Eckpunkt an der festgelegten Stelle hinzu.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Beachten Sie, dass diese Methode die gesamte Reihe von Rückruffunktionen durchläuft, die aufgerufen werden, wenn der Benutzer interaktiv einen Eckpunkt erstellt, d. h. **-create-cmd** und, was noch wichtiger ist, **-data-cmd**.

Es gibt jedoch einen wichtigen Unterschied: Der neue Scheitelpunkt wird stets am Ende der Linie hinzugefügt, während bei der interaktiven Erstellung Heuristiken zum Einsatz kommen, um ihn an einer geeigneten Stelle in die Linie einzufügen.

Dies ist die Methode, mit der die Scheitelpunkte einer bereits vorhandenen Linie in eine

Editor-Instanz geladen werden.

objectName clear

Diese Methode entfernt programmgesteuert alle Eckpunkte aus dem Editor, wodurch im Wesentlichen die ganze Linie gelöscht wird.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft dieselben Rückruffunktionen, die auch dann aufgerufen werden, wenn der Benutzer einen Eckpunkt interaktiv entfernt, d. h. **-data-cmd**.

Optionen

Der Befehl "class" akzeptiert die folgenden Optionen

-active-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn die Maus in die Zeile oder an einen von der Instanz verwalteten Punkt ein- bzw. aus diesem herausfährt.

Wird kein Wert angegeben, wird standardmäßig ein leerer Befehl verwendet, der keine Aktion ausführt.

Die Signatur dieses Befehlspräfixes lautet

activeCmd *editorObj id*

Die *id* identifiziert das eingegebene Objekt.

Eine leere *id* bedeutet, dass das zuletzt eingegebene Objekt beibehalten wurde.

Der Wert **line** gibt an, dass die Linie eingegeben wurde.

Ein numerischer Wert **N** bedeutet, dass der **N**. Scheitelpunkt der Linie eingegeben wurde, gezählt ab **0**.

Das Ergebnis dieser Methode wird ignoriert.

-add-remove-point *eventspec*

Der Wert dieser Option ist eine Ereignisspezifikation (ohne Klammerwinkel), die festlegt, welches Ereignis das Hinzufügen bzw. Entfernen eines Punktes auslöst.

Diese Option kann nur bei der Erstellung festgelegt werden.

In der Standardeinstellung werden für das Hinzufügen und Entfernen von Punkten unterschiedliche Ereignisse verwendet, nämlich **ButtonPress-1** bzw. **ButtonPress-2**.

Bei Verwendung dieser Option wird für beide Vorgänge dasselbe Ereignis verwendet. Dies stellt kein Problem dar, da das Hinzufügen als canvas-globales Ereignis gebunden ist, während das Entfernen an die jeweiligen Canvas-Elemente gebunden ist.

-closed *boolean*

Der Wert dieser Option ist ein boolescher Schalter, der angibt, ob die bearbeitete Polylinie geschlossen ist, d. h. eine Schleife bildet, oder nicht. *Hinweis*: Eine korrekte Schleife erfordert

mindestens drei Punkte. Bei weniger als drei Punkten wird eine offene Linie oder sogar nur ein Punkt angezeigt.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, gilt standardmäßig der Wert **false**.

-color *colorspec*

Der Wert dieser Option ist die Farbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **SkyBlue2**.

-convex *boolean*

Der Wert dieser Option ist ein boolescher Schalter, der angibt, ob die bearbeitete Polylinie streng konvex ist oder nicht. Diese Option ist nur dann von Bedeutung, wenn die Option **-closed** auf den Wert **true** eingestellt ist.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, gilt standardmäßig der Wert **false**.

-create-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen neuen Eckpunkt erstellen muss.

Obwohl diese Option auch nach der Erstellung festgelegt werden kann, wird empfohlen, diese Funktion ausschließlich dazu zu nutzen, benutzerdefinierte Verarbeitungsschritte einzufügen, die zum Zeitpunkt der Erstellung ausgeführt werden sollen und dabei den vorherigen Wert der Option speichern und aufrufen.

Wird kein Wert festgelegt, wird standardmäßig ein Befehl verwendet, der einen blauen Kreis mit schwarzem Rahmen und einem Radius von 3 erzeugt, dessen Mittelpunkt an der Position des neuen Punktes liegt.

Die Signatur dieses Befehlspräfixes lautet

createCmd *canvas x y*

Das Ergebnis des Befehlspräfixes *muss* eine Liste der Canvas-Elemente sein, die zur Darstellung des Markers erstellt wurden. Hinweis: Die visuelle Darstellung eines "Eckpunkts" kann aus mehreren Canvas-Elementen in beliebiger Form bestehen.

Die zurückgegebene Liste der Elemente darf leer sein; dies wird als Signal gewertet, dass die Rückruffunktion die Erstellung des Eckpunkts abgelehnt hat.

-data-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn die Linie in irgendeiner Weise bearbeitet wurde (Eckpunkt hinzugefügt, entfernt, verschoben). Auf diese Weise leitet der Editor die eigentliche Speicherung der Linieninformationen an ein externes Objekt weiter.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, ist der Standardwert eine leere Zeichenkette, und sie wird vom Editor ignoriert, d. h., nicht aufgerufen.

Die Signatur dieses Befehlspräfixes lautet
dataCmd *editorObj coordinates*
Diese Rückruffunktion wird aufgerufen, wenn die Linie entweder interaktiv oder programmgesteuert geändert wurde. Informationen zum letzteren Fall finden Sie in der Instanzmethode **add**.

Das *editorObj* identifiziert die Instanz, die die Rückruffunktion aufruft, während *coordinates* eine Liste von Scheitelpunktpositionen ist, wobei jede Position aus einem Paar von x- und y-Koordinaten besteht.

Das Ergebnis dieser Methode wird ignoriert.

-drag-point *eventspec*
Der Wert dieser Option ist eine Ereignisspezifikation (ohne eckige Klammern), die festlegt, welches Ereignis eine Drag-Aktion an Punkten auslöst.

Diese Option kann nur zur Erstellungszeit festgelegt werden.

Die Standard-Spezifikation lautet **ButtonPress-3**.

-hilit-color *colorspec*
Der Wert dieser Option ist die Hervorhebungsfarbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-highlight-cmd** festgelegt ist.

Der Standardwert lautet **red**.

-kind *name*
Der Wert dieser Option ist der Canvas-Elementtyp für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist. Es sind ausschließlich Elementtypen zulässig, die über einen Begrenzungsrahmen festgelegt werden.

Der Standardwert lautet **oval**.

-line-config *list*
Der Wert dieser Option ist eine Liste von Optionen und deren Werten, die auf das interne Linien-/Polygon-Element angewendet werden sollen, sobald dieses nach Änderungen neu generiert wird. Hinweis: Diese Optionen können die durch die Hauptoptionen **-color** und **-hilit-color** festgelegte Farbe der Linie/des Polygons *nicht* überschreiben.

Wird kein Wert angegeben, wird standardmäßig eine leere Liste verwendet.

-radius *int*
Der Wert dieser Option gibt den Radius für das Standardelement an, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **3**.

-tag *string*

Der Wert dieser Option ist der Name des Canvas-Tags, mit dem alle Elemente aller vom Editor verwalteten Eckpunkte identifiziert werden sollen.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, lautet der Standardwert **POLYLINE**.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_equad

Stichwörter: canvas_equad, canvas, concave, convex, editing, non-convex, quadrilateral

Name

canvas::edit::quadrilateral – Bearbeiten eines Vierecks auf einem Canvas

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::quadrilateral ?**0.2**?
::canvas::edit quadrilateral *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
createCmd *canvas x y*
highlightCmd on *canvas item*
highlightCmd off *canvas state*
dataCmd *editorObj coordinates*

Beschreibung

Dieses Paket stellt eine Klasse bereit, deren Instanzen die Bearbeitung eines Vierecks auf einem Canvas übernehmen. Die Instanzen lassen sich hinsichtlich der Darstellung (normal und hervorgehoben) der Eckpunktmarkierungen konfigurieren. Beachten Sie, dass die Instanzen das bearbeitete Viereck nicht selbst speichern, sondern diese Aufgabe an ein konfigurierbares Objekt weiterleiten.

API für Klassen

::canvas::edit quadrilateral *objectName canvas options*...

Dieser Befehl der Klasse erstellt und konfiguriert eine neue Instanz eines Viereck-Editors mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas*-Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Die hier zulässigen Optionen und ihre Werte werden im Abschnitt Optionen erläutert.

Instanz-API

Die Instanzen der Viereck-Editoren stellen die folgende API bereit:

objectName destroy

Diese Methode löscht den Viereck-Editor und gibt alle seine internen Ressourcen frei.

Beachten Sie, dass durch diesen Vorgang weder die vom Editor auf dem zugehörigen Canvas verwalteten Elemente noch der Canvas selbst gelöscht werden.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName enable

Diese Methode aktiviert die Bearbeitung des Vierecks auf dem Canvas. Dies ist die Standardeinstellung nach der Erstellung der Instanz. Ein Aufruf wird ignoriert, wenn der Editor bereits aktiv ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die komplementäre Methode lautet **disable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName disable

Diese Methode deaktiviert die Bearbeitung des Vierecks auf dem Canvas. Ein Aufruf wird ignoriert, wenn der Editor bereits deaktiviert ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode ist **enable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName active

Diese Methode fragt den Status des Editors ab.

Das Ergebnis der Methode ist ein boolescher Wert: **true**, wenn der Editor aktiv ist, und andernfalls **false**, d. h. deaktiviert.

Die Methoden zum Ändern des Status lauten **enable** und **disable**.

objectName add *x y*

Diese Methode fügt dem Viereck programmgesteuert einen Scheitelpunkt an der festgelegten Position hinzu.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft die gesamte Reihe von Rückruffunktionen, die aufgerufen werden, wenn der Benutzer interaktiv einen Scheitelpunkt erstellt, d. h. **-create-cmd** und,

was noch wichtiger ist, **-data-cmd**.

Dies ist die Methode, mit der die Eckpunkte eines bereits vorhandenen Vierecks in eine Editor-Instanz geladen werden.

Es gibt Hinweise darauf, dass höchstens 4 Eckpunkte angegeben werden können und mindestens 4 angegeben werden müssen, damit das Viereck vollständig ist.

objectName clear

Diese Methode hebt die Auswahl des Vierecks im Editor programmgesteuert auf.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft die gleiche Rückruffunktion, die auch dann aufgerufen wird, wenn der Benutzer einen Scheitelpunkt interaktiv entfernt, d. h. **-data-cmd**.

Optionen

Der Befehl "class" akzeptiert die folgenden Optionen

-convex *boolean*

Der Wert dieser Option ist ein boolescher Schalter. Wenn das Flag gesetzt ist, akzeptiert der Editor ausschließlich konvexe Vierecke und lehnt alle Operationen ab, die zu einer Verletzung der Konvexität führen würden.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, gilt standardmäßig der Wert **false**, d. h. es werden alle Vierecke akzeptiert.

-tag *string*

Der Wert dieser Option ist der Name des Canvas-Tags, mit dem alle Elemente aller vom Editor verwalteten Vierecke identifiziert werden sollen.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, lautet der Standardwert "**QUADRILATERAL**".

-create-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen neuen Scheitelpunkt erstellen muss.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird kein Wert angegeben, wird standardmäßig ein Befehl verwendet, der einen blauen Kreis mit schwarzem Rahmen und einem Radius von 3 erstellt, dessen Mittelpunkt an der Position des neuen Punktes liegt.

Die Signatur dieses Befehlspräfixes lautet

createCmd *canvas x y*

Das Ergebnis des Befehlspräfixes *muss* eine Liste der Canvas-Elemente sein, die es zur Darstellung des Markers erstellt hat. Beachten Sie hierbei, dass die visuelle Darstellung

eines "Eckpunkts" aus mehreren Canvas-Elementen in beliebiger Form bestehen kann.

-highlight-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen Eckpunkt hervorheben oder die Hervorhebung aufheben muss.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, wird standardmäßig ein Befehl verwendet, der das zu markierende Element rot einfärbt (und die Farbe beim Aufheben der Markierung wiederherstellt).

Die beiden Signaturen dieses Befehlspräfixes lauten

highlightCmd on *canvas item*

Diese Methode des Befehlspräfixes muss alle erforderlichen Schritte ausführen, um den Scheitelpunkt hervorzuheben, zu dem das *item* gehört (beachten Sie dabei den obigen Hinweis, dass ein Viereck aus mehreren Canvas-Elementen zusammengesetzt sein darf).

Das Ergebnis des Befehls kann beliebig sein und wird unverändert als Argument *state* an die Methode **off** übergeben.

highlightCmd off *canvas state*

Diese Methode wird aufgerufen, um die Hervorhebung eines durch den *state* beschriebenen Eckpunkts aufzuheben, wobei es sich um das unveränderte Ergebnis der Methode **on** des Befehlspräfixes handelt. Das Ergebnis dieser Methode wird ignoriert.

Beachten Sie, dass jegliche Interaktion zwischen dem Ziehen und der Hervorhebung eines Vierecks innerhalb des Editors abgewickelt wird und dass die Rückruffunktion hierfür keine Verantwortung trägt.

-data-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn das Viereck in irgendeiner Weise bearbeitet wurde. Auf diese Weise leitet der Editor die eigentliche Speicherung der Viereckdaten an ein externes Objekt weiter.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, ist der Standardwert eine leere Zeichenkette, und sie wird vom Editor ignoriert, d. h., nicht aufgerufen.

Die Signaturen dieses Befehlspräfixes lauten

dataCmd *editorObj coordinates*

Diese Rückruffunktion wird aufgerufen, wenn das Viereck entweder interaktiv oder programmgesteuert geändert wurde. Informationen zu letzterem finden Sie in der Instanzmethode **add**.

Das *editorObj* identifiziert die Instanz, die die Rückruffunktion aufruft, während *coordinates* eine Liste von Eckpunktpositionen ist, wobei jede Position aus einem Paar von x- und y-Koordinaten besteht.

Das Ergebnis dieser Methode wird ignoriert.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_erectangle

Stichwörter: canvas_erectangle, canvas, editing, rectangle

Name

canvas::edit::rectangle – Bearbeiten eines Rechtecks auf einem Canvas

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::rectangle ?**0.1**?
::canvas::edit rectangle *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName set *minx miny maxx maxy*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Beschreibung

Dieses Paket stellt eine Klasse bereit, deren Instanzen die Bearbeitung eines Rechtecks auf einem Canvas übernehmen. Die Instanzen können hinsichtlich der Darstellung der Markierungen, die die Ecken des Rechtecks kennzeichnen, konfiguriert werden. Hinweis: Die Instanzen speichern das bearbeitete Rechteck nicht selbst, sondern leiten diese Aufgabe an ein konfigurierbares Objekt weiter.

Interaktion

Der Rechteck-Editor verwendet intern eine Instanz des Pakets **canvas::edit::points** und übernimmt alle damit verbundenen Optionen und Verhaltensweisen. Die Optionen zur Farbe werden zusätzlich auf das Rechteck-Element dieses Editors angewendet.

Darüber hinaus bietet es die Option "**-rect-config**", mit der Sie das Rechteck-Element, das zur Darstellung des Rechtecks verwendet wird, anhand seiner Ecken anpassen können. Beachten Sie, dass damit die vererbten Farboptionen nicht überschrieben werden können.

Während einer Instanz wird eine Zustandsmaschine ausgeführt, in der der Benutzer zwei Punkte festlegt; daraufhin vervollständigt das System das Rechteck mit den fehlenden Ecken und meldet dies. Ab diesem Zeitpunkt verhindert das System das Hinzufügen weiterer Punkte. Das Ziehen der Punkte ist weiterhin möglich, wodurch eine Größenänderung und das Verschieben des Rechtecks ermöglicht werden. Das Löschen einer beliebigen Ecke löscht das gesamte

Rechteck; daraufhin ist das System wieder bereit für die Erstellung eines neuen Rechtecks aus zwei Punkten.

API für Klassen

::canvas::edit rectangle *objectName canvas options...*

Dieser Befehl, der "class"-Befehl, erstellt und konfiguriert eine neue Instanz eines Rechteck-Editors mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas* -Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Die hier zulässigen Optionen und ihre Werte werden im Abschnitt Optionen erläutert.

Instanz-API

Instanzen der Rechteck-Editoren stellen die folgende API bereit:

objectName destroy

Diese Methode löscht den Rechteck-Editor und gibt alle seine internen Ressourcen frei.

Beachten Sie, dass durch diesen Vorgang weder die vom Editor auf dem zugehörigen Canvas verwalteten Elemente noch der Canvas selbst gelöscht werden.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName enable

Diese Methode aktiviert die Bearbeitung des Rechtecks auf dem Canvas. Dies ist die Standardeinstellung nach der Instanzerstellung. Ein Aufruf wird ignoriert, wenn der Editor bereits aktiv ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode ist **disable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName disable

Diese Methode deaktiviert die Bearbeitung des Rechtecks auf dem Canvas. Ein Aufruf wird ignoriert, wenn der Editor bereits deaktiviert ist.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Die zugehörige Methode lautet **enable**. Die Abfragemethode für den aktuellen Status lautet **active**.

objectName active

Diese Methode fragt den Status des Editors ab.

Das Ergebnis der Methode ist ein boolescher Wert: **true**, wenn der Editor aktiv ist, und andernfalls **false**, d. h. deaktiviert.

Die Methoden zum Ändern des Status lauten **enable** und **disable**.

objectName set *minx miny maxx maxy*
Diese Methode legt das Rechteck programmgesteuert anhand der Rahmen-Koordinaten fest. Dabei wird ein zuvor vorhandenes (möglicherweise unvollständiges) Rechteck überschrieben.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Hinweis: Diese Methode durchläuft die gesamte Reihe von Rückruffunktionen, die beim Erstellen von Ecken aufgerufen werden, d. h. **-create-cmd** und, was noch wichtiger ist, **-data-cmd**.

Dies ist die Methode, mit der ein bereits vorhandenes Rechteck in eine Editor-Instanz geladen wird.

objectName clear
Diese Methode entfernt das Rechteck programmgesteuert aus dem Editor und schafft so Platz für die Erstellung eines neuen Rechtecks.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Beachten Sie, dass diese Methode dieselben Rückruffunktionen durchläuft, die auch dann aufgerufen werden, wenn der Benutzer interaktiv eine Ecke entfernt, d. h. **-data-cmd**.

Optionen

Der Befehl "class" akzeptiert die folgenden Optionen

-active-cmd *command-prefix*
Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn die Maus in das von der Instanz verwaltete Rechteck oder eine Ecke hinein- oder aus diesem herausbewegt wird.

Wird kein Wert angegeben, wird standardmäßig ein leerer Befehl verwendet, der keine Aktion ausführt.

Die Signatur dieses Befehlspräfixes lautet
activeCmd *editorObj id*
Die *id* identifiziert das eingegebene Objekt.

Eine leere *id* bedeutet, dass das zuletzt eingegebene Objekt beibehalten wurde.

Der Wert **rect** gibt an, dass die Gliederung des Rechtecks eingegeben wurde.

Jeder andere Wert ist ein Code für eine der Ecken des Rechtecks, d. h. **tl**, **tr**, **bl** und **br**.

Das Ergebnis dieser Methode wird ignoriert.

-add-remove-point *eventspec*
Der Wert dieser Option ist eine Ereignisspezifikation (ohne Klammerwinkel), die festlegt, welches Ereignis das Hinzufügen bzw. Entfernen eines Punktes auslöst.

Diese Option kann nur bei der Erstellung festgelegt werden.

In der Standardeinstellung werden für das Hinzufügen und Entfernen von Punkten unterschiedliche Ereignisse verwendet, nämlich **ButtonPress-1** bzw. **ButtonPress-2**.

Bei Verwendung dieser Option wird für beide Vorgänge dasselbe Ereignis verwendet. Dies stellt kein Problem dar, da das Hinzufügen als canvas-globales Ereignis gebunden ist, während das Entfernen an die jeweiligen Canvas-Elemente gebunden ist.

-color *colorspec*

Der Wert dieser Option ist die Farbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **SkyBlue2**.

-create-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn er einen neuen Eckpunkt erstellen muss.

Obwohl diese Option auch nach der Erstellung festgelegt werden kann, wird empfohlen, diese Funktion ausschließlich dazu zu nutzen, benutzerdefinierte Verarbeitungsschritte einzufügen, die zum Zeitpunkt der Erstellung ausgeführt werden sollen und dabei den vorherigen Wert der Option speichern und aufrufen.

Wird kein Wert festgelegt, wird standardmäßig ein Befehl verwendet, der einen blauen Kreis mit schwarzem Rahmen und einem Radius von 3 erzeugt, dessen Mittelpunkt an der Position des neuen Punktes liegt.

Die Signatur dieses Befehlspräfixes lautet

createCmd *canvas x y*

Das Ergebnis des Befehlspräfixes *muss* eine Liste der Canvas-Elemente sein, die zur Darstellung des Markers erstellt wurden. Beachten Sie hierbei, dass die visuelle Darstellung einer "Ecke" aus mehreren Canvas-Elementen in beliebiger Form bestehen kann.

Die zurückgegebene Liste der Elemente darf leer sein; dies wird als Hinweis darauf gewertet, dass die Rückruffunktion die Erstellung der Ecke abgelehnt hat.

-data-cmd *command-prefix*

Der Wert dieser Option ist ein Befehlspräfix, das der Editor aufruft, wenn das Rechteck in irgendeiner Weise bearbeitet wurde (erstellt, geändert, gelöscht). Auf diese Weise leitet der Editor die eigentliche Speicherung der Rechteckdaten an ein externes Objekt weiter.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, ist der Standardwert eine leere Zeichenkette, und sie wird vom Editor ignoriert, d. h., der Befehl wird nicht aufgerufen.

Die Signatur dieses Befehlspräfixes lautet

dataCmd *editorObj coordinates*

Diese Rückruffunktion wird aufgerufen, wenn das Rechteck entweder interaktiv oder programmgesteuert geändert wurde. Informationen zu letzterem finden Sie in der Instanzmethode **set**.

Das *editorObj* identifiziert die Instanz, die die Rückruffunktion aufruft, während *coordinates* eine Liste mit vier Elementen ist, die in dieser Reihenfolge die minimalen x- und y-Koordinaten sowie die maximalen x- und y-Koordinaten des Rechtecks enthält.

Das Ergebnis dieser Methode wird ignoriert.

-drag-point *eventspec*
Der Wert dieser Option ist eine Ereignisspezifikation (ohne eckige Klammern), die festlegt, welches Ereignis eine Drag-Aktion an Punkten auslöst.

Diese Option kann nur zur Erstellungszeit festgelegt werden.

Die Standard-Spezifikation lautet **ButtonPress-3**.

-hilit-color *colorspec*
Der Wert dieser Option ist die Hervorhebungsfarbe für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-highlight-cmd** festgelegt ist.

Der Standardwert lautet **red**.

-kind *name*
Der Wert dieser Option ist der Canvas-Elementtyp für das Standardelement, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist. Es sind ausschließlich Elementtypen zulässig, die über einen Begrenzungsrahmen festgelegt werden.

Der Standardwert lautet **oval**.

-radius *int*
Der Wert dieser Option gibt den Radius für das Standardelement an, das beim Hinzufügen eines Punktes erstellt wird, sofern kein **-create-cmd** angegeben ist.

Der Standardwert lautet **3**.

-rect-config *list*
Der Wert dieser Option ist eine Liste von Optionen und deren Werten, die auf das interne Rechteck-Element angewendet werden sollen, sobald dieses nach Änderungen neu generiert wird. Hinweis: Diese Optionen können die durch die Hauptoptionen **-color** und **-hilit-color** festgelegte Farbe des Rechtecks *nicht* überschreiben.

Wird kein Wert angegeben, wird standardmäßig eine leere Liste verwendet.

-tag *string*
Der Wert dieser Option ist der Name des Canvas-Tags, mit dem alle Elemente aller vom Editor verwalteten Eckpunkte identifiziert werden sollen.

Diese Option kann nur bei der Erstellung festgelegt werden.

Wird sie nicht angegeben, lautet der Standardwert **RECTANGLE**.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere

Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_gradient

Stichwörter: canvas_gradient, background, canvas, color gradient, gradients, image, widget background

Name

canvas::gradient – Canvas mit Farbverlaufshintergrund

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::gradient ?**0.2**?
::canvas::gradient *pathName* ?options?...

Beschreibung

Dieses Paket enthält einen Befehl zum Festlegen von Hintergrundverläufen auf Canvas.

NOTE! Dieses Paket reserviert das Canvas-Tag **canvas::gradient** für sich.

API

::canvas::gradient *pathName* ?options?...

Erstellt einen Hintergrundverlauf auf dem Canvas *pathName*. Ein bereits vorhandener Verlauf wird durch die neue Definition ersetzt.

Die verfügbaren Optionen werden weiter unten im Abschnitt Optionen beschrieben.

Optionen

-direction *direction*

Der Wert für diese Option legt die *direction* des Farbverlaufs fest. Wird kein Wert angegeben, verwendet das System standardmäßig "**x**", einen horizontalen Farbverlauf. Das Paket unterstützt fünf Richtungen:

x
für einen horizontalen Farbverlauf (Standard). Die Farbe ändert sich entlang der x-Achse, wobei die Startfarbe links liegt.

y
für einen vertikalen Farbverlauf. Die Farbe ändert sich entlang der y-Achse, wobei die Ausgangsfarbe oben liegt.

d1
für einen diagonalen Farbverlauf von links oben nach rechts unten. Die Farbe ändert sich entlang der Diagonale, wobei die Startfarbe links oben liegt.

d2
für einen diagonalen Farbverlauf von links unten nach rechts oben. Die Farbe ändert sich entlang der Diagonale, wobei die Startfarbe links unten liegt.

r
Für einen radialen Farbverlauf um den Mittelpunkt des Ansichtsbereichs: Die Farbe ändert

sich von der Mitte nach außen, wobei die Startfarbe in der Mitte liegt.

-color1 ***color***
Der Wert für diese Option legt die Startfarbe des Farbverlaufs fest. Wird kein Wert angegeben, wird standardmäßig "**red**" verwendet.

-color2 ***color***
Der Wert für diese Option legt die Endfarbe des Farbverlaufs fest. Wird kein Wert angegeben, wird standardmäßig "**green**" verwendet.

-type ***type***
Der Wert für diese Option legt fest, wie der Farbverlauf berechnet wird. Das Paket unterstützt derzeit nur eine Methode, die natürlich als Standard eingestellt ist:

linear
für eine lineare Interpolation zwischen der Start- und der Endfarbe (Standard).

Beispiel

```
package require canvas::gradient
canvas .c
canvas::gradient .c -direction x -color1 yellow -color2 blue
pack .c -fill both -expand 1
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) 2013 Jarek Lewandowski, MaxJarek
Copyright (c) Origin http://wiki.tcl-lang.org/6100
Copyright (c) Origin http://wiki.tcl-lang.org/37242
Copyright (c) Origin http://wiki.tcl-lang.org/9079
```

canvas_highlight

Stichwörter: canvas_highlight, canvas, enter callback, highlighting, leave callback

Name

canvas::highlight – Verwaltung der Hervorhebung von Canvas-Elementen oder Elementgruppen

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::highlight ?**0.1**?
::canvas::highlight on *canvas tagOrId cmd*
::canvas::highlight off *canvas tagOrId*
{*}cmd on *canvas item*
{*}cmd off *canvas clientdata*

Beschreibung

Dieses Paket enthält Hilfsprogramme für Befehle zum Ein- und Ausschalten von Hervorhebungen für Canvas-Elemente oder Elementgruppen, wobei letztere durch einen Tag gekennzeichnet sind.

API

::canvas::highlight on *canvas tagOrId cmd*

Dieser Befehl richtet eine allgemeine Hervorhebung ein, wobei die Elemente des Canvas *canvas*, die auf diese Weise hervorgehoben werden sollen, durch *tagOrId* identifiziert werden und das Präfix *cmd* die Umsetzung, d. h. die Art und Weise, wie die Hervorhebung durchgeführt werden soll, festlegt.

Die Signatur des Befehlspräfixes wird später im Abschnitt Callback für die Hervorhebung beschrieben.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Einschränkungen:

1. Wenn eine Hervorhebung aktiv ist, kann keine weitere Hervorhebung aktiviert werden. Das bedeutet, dass verschachtelte Hervorhebungen nicht möglich sind.
2. Das System kann abstürzen, wenn eine Hervorhebung innerhalb ihrer Rückruffunktion für die Hervorhebung entfernt wird.

::canvas::highlight off *canvas tagOrId*

Dieser Befehl entfernt alle Hervorhebungen, die für die Elemente des Canvas *canvas* festgelegt wurden, die durch *tagOrId* identifiziert werden.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Callback für die Hervorhebung

Der "Highlight"-Callback ist ein Befehlspräfix, das auf die folgenden zwei Arten aufgerufen wird:

{*}cmd on *canvas item*

Diese Funktion wird aufgerufen, wenn die Maus in eines der Elemente gelangt ist, für die die Hervorhebung eingerichtet wurde. Die Rückruffunktion muss nun alle notwendigen Neukonfigurationen vornehmen, um das Element (die Gruppe) hervorzuheben.

Das Ergebnis des Befehls kann beliebig sein. Es wird vom System als Client-Information gespeichert und unverändert an die Rückruffunktion zum Aufheben der Hervorhebung zur weiteren Verwendung übergeben. Auf diese Weise kann die Rückruffunktion zur Hervorhebung einen benutzerdefinierten Zustand vom Hervorheben bis zum Aufheben der Hervorhebung beibehalten.

Hinweis: Die Rückruffunktion muss weder den Zustand verwalten noch das Element (die Gruppe) tatsächlich neu konfigurieren. Im letzteren Fall dient die Rückruffunktion lediglich als einfache Benachrichtigung über das Betreten bzw. Verlassen des Bereichs.

{*}cmd off *canvas clientdata*

Diese Form wird aufgerufen, wenn die Maus (eines) der Elemente der derzeit aktiven Hervorhebung verlassen hat. Die Rückruffunktion muss nun alle erforderlichen Neukonfigurationen vornehmen, um die Hervorhebung des Elements (der Gruppe)

aufzuheben.

Das Ergebnis des Befehls muss ein boolescher Wert sein, dessen üblicher Wert **true** lautet. Durch die Rückgabe von **false** kann die Rückruffunktion das Aufheben der Hervorhebung verhindern.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_mvg

Stichwörter: canvas_mvg, canvas, graphics, imagemagick, magick vector graphics, mvg, print screen, serialization, vector graphics

Name

canvas::mvg – Von Canvas in das MVG-Vektorformat von ImageMagick

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::mvg ?**1**?
::canvas::mvg *pathName*

Beschreibung

Dieses Paket enthält einen Befehl, mit dem der Inhalt eines Canvas in das MVG-Vektorformat von ImageMagick konvertiert werden kann.

API

::canvas::mvg *pathName*

Gibt den Inhalt des Canvas *pathName* aus. Das Ergebnis ist eine Zeichenkette im MVG-Vektorformat von ImageMagick.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2010 Wolf-Dieter Busch (http://wiki.tcl-lang.org/15505)
Copyright (c) 2010 Documentation, Andreas Kupries
```


canvas_pdf

Stichwörter: canvas_pdf, canvas, graphics, imagemagick, pdf, portable document format, print screen, serialization, vector graphics

Name
canvas::pdf – Von Canvas zu PDF

Übersicht
package require Tcl 8.5
package require Tk 8.5
package require canvas::pdf ?**1**?
package require pdf4tcl
::canvas::pdf *pathName*

Beschreibung
Dieses Paket enthält einen Befehl, mit dem sich der Inhalt eines Canvas in das Vektorformat PDF von Adobe konvertieren lässt.

API
::canvas::pdf *pathName*
Gibt den Inhalt des Canvas *pathName* aus. Das Ergebnis ist eine Zeichenkette im Adobe-PDF-Format.

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) 2014 Andreas Kupries, Arjen Markus
Copyright (c) 2014 Documentation, Andreas Kupries
```


canvas_snap
Stichwörter: canvas_snap, canvas, image, photo, print screen, snapshot

Name
canvas::snap – Momentaufnahme der Canvas als Tk-Foto-Image

Übersicht
package require Tcl 8.5
package require Tk 8.5
package require canvas::snap ?**1.0.1**?
package require img::window
::canvas::snap *pathName*

Beschreibung
Dieses Paket enthält einen Befehl zum Erstellen von Momentaufnahmen beliebiger Canvas.

API
::canvas::snap *pathName*
Erstellt eine Momentaufnahme des Canvas *pathName*. Das Ergebnis ist der Name eines

neuen Tk-Foto-Images, das die Momentaufnahme enthält.

Hinweis: Dieser Befehl hat einen Nebeneffekt. Um zu vermeiden, dass weiße Rechtecke an den Stellen entstehen, an denen andere Fenster den Canvas überlappen könnten, zwingt dieser Befehl das Toplevel-Fenster, in dem sich der Canvas befindet, an die oberste Position der Stacking-Reihenfolge.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2004 George Petasis (http://wiki.tcl-lang.org/1404)
Copyright (c) 2010 Documentation, Andreas Kupries
```


canvas_sqmap

Stichwörter: canvas_sqmap, canvas, cell, grid, image, map, square map, tile

Name

canvas::sqmap – Canvas mit Kartenhintergrund aus quadratischen Kacheln

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require snit
package require uevent::onidle
package require cache::async
package require canvas::sqmap ?**0.3.1**?
::canvas::sqmap *pathName* ?*options*?
canvasName **image set** *cell image*
canvasName **image unset** *cell*
canvasName **flush**

Beschreibung

Dieses Paket stellt ein erweitertes Canvas-Widget für die Darstellung von Karten bereit, das auf einer Reihe quadratischer Bildkacheln basiert. Die Kacheln bilden den Hintergrund des Canvases, wobei alle anderen Canvas-Elemente stets vor ihnen angezeigt werden. Die Anzahl der angezeigten Kacheln, die Kachelgröße sowie die Quelle der anzuzeigenden Bilder sind konfigurierbar.

API

::canvas::sqmap *pathName* ?options?

Erstellt den Canvas *pathName* und konfiguriert ihn. Das neue Widget unterstützt alle Optionen und Methoden eines regulären Canvas sowie die im Folgenden beschriebenen Optionen und Methoden.

Das Ergebnis des Befehls ist *pathName*.

Optionen

-grid-cell-width
Der Wert für diese Option ist eine nicht-negative ganze Zahl. Er legt die Breite der Zellen fest, aus denen sich der Hintergrund zusammensetzt.

-grid-cell-height
Der Wert für diese Option ist eine nicht-negative ganze Zahl. Er legt die Höhe der Zellen fest, aus denen sich der Hintergrund zusammensetzt.

-grid-cell-command
Der Wert für diese Option ist ein Befehlspräfix. Er wird immer dann aufgerufen, wenn der Canvas das Bild für eine bestimmte Zelle des Hintergrunds benötigt, und zwar mit zwei zusätzlichen Argumenten: der ID der Zelle und einem Befehlspräfix, das aufgerufen werden soll, sobald das Bild bereitsteht oder feststeht, dass es nicht vorhanden ist.

Die ID der Zelle ist eine Liste mit zwei Elementen, die die Zeilen- und Spaltennummer der Zelle in dieser Reihenfolge enthält. Das resultierende Befehlspräfix (im folgenden Beispiel "$result" genannt) muss entweder mit zwei oder drei Argumenten aufgerufen werden, d. h.

```
    $result set   $cellid $image ; # image is known and ready
    $result unset $cellid        ; # image does not exist
```

Diese Option kann undefiniert bleiben, d. h., der Canvas kann auch ohne sie funktionieren. In diesem Fall werden in den Rasterzellen nur jene Bilder angezeigt, die explizit mit der Methode **image set** festgelegt wurden; siehe den nächsten Abschnitt. Alle anderen Rasterzellen bleiben einfach leer.

-viewport-command
Diese Option legt ein Befehlspräfix fest, das wenn der Ansichtsbereich des canvases geändert wird, aufgerufen wird, damit Benutzer jederzeit den Überblick darüber behalten können, an welcher Stelle im Bildlaufbereich sie sich gerade befinden. Dies kann beispielsweise dazu verwendet werden, abgeleitete Anzeigen zu steuern oder Elemente im Blickfeld zu halten, indem sie entsprechend der Bewegung des Ansichtsbereichs verschoben werden.

-image-on-load
Der Wert für diese Option ist ein Bild. Wird dieses Bild angegeben, wird es in einer Zelle angezeigt, während das eigentliche Bild für diese Zelle über die durch **-grid-cell-command** festgelegte Rückruffunktion geladen wird.

-image-on-unset
Der Wert für diese Option ist ein Bild. Wenn angegeben, wird das Bild in einer Zelle angezeigt, für die die durch **-grid-cell-command** angegebene Rückruffunktion gemeldet hat, dass kein tatsächliches Bild zur Anzeige vorhanden ist.

Methoden

canvasName* image set *cell image
Durch den Aufruf dieser Methode wird das *image* in die angegebene *cell* des Hintergrunds eingefügt. Die Zelle wird als Liste mit zwei Elementen angegeben, die in dieser Reihenfolge die Zeilen- und Spaltennummer enthält.

Hinweis: Ein Bild kann mehreren Zellen des Canvases zugeordnet und in diesen angezeigt werden.

canvasName* image unset *cell
Durch den Aufruf dieser Methode wird die angegebene *cell* des Hintergrunds als leer deklariert; ein in dieser Zelle angezeigtes Bild wird dabei verworfen. Die Zelle wird als Liste mit zwei Elementen angegeben, die in dieser Reihenfolge die Zeilen- und Spaltennummer enthält.

***canvasName* flush**
Durch den Aufruf dieser Methode wird das Canvas dazu gezwungen, die Bilder für alle Zellen vollständig neu zu laden. Verwenden Sie diese Methode nicht, wenn der Canvas ohne **-grid-cell-command** betrieben wird, da das Canvas in diesem Fall einfach alle Bilder vergisst, ohne sie neu laden zu können.

Besitz der Bilder

Beachten Sie, dass das Canvas *keinen* Besitzanspruch auf die Bilder erhebt, die es im Hintergrund anzeigt. Mit anderen Worten: Wenn wir sagen, dass der Canvas ein Bild "vergisst", bedeutet dies lediglich, dass die Zuordnung zwischen einer Rasterzelle und dem angezeigten Bild aufgehoben wird. Das Bild wird *nicht* gelöscht. Die Verwaltung des Lebenszyklus der vom Canvas angezeigten Bilder liegt in der Verantwortung des Canvas-Nutzers.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_tags

Stichwörter: canvas_tags, append tag, canvas, insert tag, remove tag, tags

Name

canvas::tag – Einfachere Verwaltung der Tags bei Canvas-Elementen oder Elementgruppen

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require canvas::tag ?**0.1**?
::canvas::tag append *canvas tagOrId tag...*
::canvas::tag prepend *canvas tagOrId tag...*
::canvas::tag append *canvas tagOrId index tag...*
::canvas::tag remove *canvas tagOrId tag...*
::canvas::tag match *canvas tagOrId pattern*

Beschreibung

Dieses Paket enthält Befehle, die die Verwaltung der mit Canvas-Elementen oder Elementgruppen verknüpften Tag-Listen erleichtern.

Das Problem bei der bestehenden Canvas-API besteht darin, dass eine Tag-Liste nur als Ganzes festgelegt werden kann, wodurch das Hinzufügen und Entfernen von Tags aus solchen Listen zu

relativ komplexen Vorgängen wird, bei denen die aktuelle Tag-Liste gelesen, geändert und anschließend die geänderte Liste wieder zurückgeschrieben werden muss.

Die in diesem Paket enthaltenen Befehle verbergen all diese Komplexität vor dem Benutzer.

API

::canvas::tag append *canvas tagOrId tag...*
Dieser Befehl fügt die Tags *tag*... zur Tag-Liste der Elemente hinzu, die durch *tagOrId* im *canvas*-Widget identifiziert werden. Die neuen Tags werden am Ende der Liste hinzugefügt.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::tag prepend *canvas tagOrId tag...*
Dieser Befehl fügt die Tags *tag*... zur Tag-Liste der Elemente hinzu, die durch *tagOrId* im *canvas*-Widget identifiziert werden. Die neuen Tags werden am Anfang der Liste eingefügt.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::tag append *canvas tagOrId index tag...*
Dieser Befehl fügt die Tags *tag*... zur Tag-Liste der Elemente hinzu, die durch *tagOrId* im *canvas*-Widget identifiziert werden. Die neuen Tags werden vor dem Element an Position *index* eingefügt.

index **0** bezieht sich auf den Anfang der Liste.

index **end** bezieht sich auf die Position nach dem Ende der Liste.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::tag remove *canvas tagOrId tag...*
Dieser Befehl entfernt die genannten Tags *tag*... aus der Tag-Liste der Elemente, die durch *tagOrId* im *canvas*-Widget identifiziert werden.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::canvas::tag match *canvas tagOrId pattern*
Dieser Befehl ermittelt alle Tags für die Elemente, die durch *tagOrId* im *canvas*-Widget identifiziert werden und dem Glob-Muster *pattern* entsprechen.

Das Ergebnis des Befehls ist eine Liste der übereinstimmenden Tags. Diese Liste kann leer sein.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_trlines

Stichwörter: canvas_trlines, canvas, crosshair, rubberband, tracking

Name
canvas::track::lines – Verwaltung einer Gruppe von Gummiband-Leinen

Übersicht
package require Tcl 8.5
package require Tk 8.5
package require canvas::tag ?**0.1**?
::canvas::track lines *objectName canvas*
objectName destroy
objectName start *current p...*
objectName move *current*
objectName done

Beschreibung
Dieses Paket enthält eine Klasse, die das Zeichnen einer Reihe von Halbkreuzlinien (Gummibandlinien) verwaltet.

API für Klassen
::canvas::track lines *objectName canvas*
Dieser Befehl ("Klasse") erstellt und konfiguriert eine neue Instanz des Trackers mit dem Namen *objectName*. Die Instanz wird mit dem angegebenen *canvas*-Widget verbunden.

Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Instanz-API
Instanzen dieser Klasse stellen die folgende API bereit:

objectName destroy
Diese Methode löscht die Instanz und gibt alle ihre internen Ressourcen frei.

Bei diesem Vorgang werden die Elemente, die die verfolgten Linien darstellen, gelöscht. Der zugehörige Canvas wird dabei nicht gelöscht.

Das Ergebnis der Methode ist eine leere Zeichenkette.

objectName start *current p...*
Diese Methode startet die Verfolgung einer Reihe von Linien, wobei jede Linie einem Punkt *p* zugeordnet ist, der den Zielendpunkt der jeweiligen Linie legt. Alle Linien haben *current* als gemeinsamen Endpunkt.

Beachten Sie, dass eine zuvor verfolgte Gruppe von Linien entfernt wird.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Jeder Punkt wird durch eine Liste mit zwei Elementen angegeben, die seine x- und y-Koordinaten in dieser Reihenfolge enthält.

objectName move *current*
Diese Methode aktualisiert den gemeinsamen Endpunkt aller Gummibänder und zeichnet

diese neu.

Das Ergebnis der Methode ist eine leere Zeichenkette.

Der Punkt wird durch eine Liste mit zwei Elementen festgelegt, die seine x- und y-Koordinaten in dieser Reihenfolge enthält.

objectName done
Diese Methode beendet die Verfolgung der aktuellen Zeilengruppe und entfernt diese aus dem Canvas.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

canvas_zoom

Stichwörter: canvas_zoom, zoom

Name

canvas::zoom – Zoom-Steuerung für canvas::sqmap

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require snit
package require uevent::onidle
package require canvas::zoom ?**0.2.1**?
::canvas::zoom *pathName* ?*options*?

Beschreibung

Dieses Paket enthält ein Widget, mit dem der Benutzer einer Kartenanzeige den Zoomfaktor steuern kann.

API

::canvas::zoom *pathName* ?options?
Erstellt das Zoom-Steuerelement *pathName* und konfiguriert es. Die vom neuen Widget unterstützten Methoden und Optionen werden in den folgenden Abschnitten beschrieben.

Das Ergebnis des Befehls ist *pathName*.

Optionen

-orient
Der Wert für diese Option lautet entweder **vertical** oder **horizontal** und legt die Ausrichtung der Hauptachse des Widgets fest. Der Standardwert ist **vertical**.

-levels
Der Wert für diese Option ist eine nicht-negative ganze Zahl. Er legt die Anzahl der zu unterstützenden Zoomstufen fest.

-variable
Der Wert für diese Option ist der Name einer globalen oder einer in einem Namespace definierten Variable, die mit dem Widget verknüpft ist. Änderungen der Zoomstufe, die am Widget vorgenommen werden, werden an diese Variable weitergegeben, und umgekehrt werden Änderungen an der Variablen in das Widget übernommen.

-command
Diese Option legt ein Befehlspräfix fest. Diese Rückruffunktion wird immer dann aufgerufen, wenn sich die Zoomstufe ändert. Sie wird mit zwei zusätzlichen Argumenten aufgerufen: dem Zoom-Widget und der neuen Zoomstufe, in dieser Reihenfolge.

Methoden
Das Widget unterstützt keine Methoden, die über die Standardmethoden (**configure**, **cget** usw.) hinausgehen.

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *canvas* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

chatwidget

chatwidget
Stichwörter: chatwidget, chat, composite widget, irc, mega-widget, widget

Name
chatwidget – Mehrteilige Ansicht für einen Chatraum oder einen IRC-Kanal

Übersicht
package require Tk 8.5
package require chatwidget ?**1.1.4**?
::chatwidget::chatwidget *path* ?*options*?
$widget topic *command args*
$widget name *nick args*
$widget message *text args*
$widget hook *command args*
$widget names *args*
$widget entry *args*
$widget chat *args*

Beschreibung
Ein zusammengesetztes Widget, das den Bau von Chat-Anwendungen vereinfacht. Es enthält Anzeigebereiche für Chat-Nachrichten, Benutzernamen und das Thema sowie einen Eingabebereich. Nachrichten werden je nick gefärbt, und die Vervollständigung der nicks läuft von selbst. Hooks lassen die Anzeige anpassen. Der Hauptbereich des Chats lässt sich teilen, für den Verlauf oder für die Suche.

Das Widget besteht aus mehreren Text-Widgets und Panedwindow-Widgets, sodass sich die

Größe jedes Teils der Anzeige einstellen lässt. Alle Text-Widgets sind über Durchreichbefehle erreichbar, wenn es genauer sein soll. Die Bereiche Thema und Namen lassen sich ausblenden.

Befehle

::chatwidget::chatwidget *path* ?*options*?

Erzeugt ein neues chatwidget mit der Tk-Fensterkennung *path*. Angegebene Optionen gehen derzeit direkt an das Text-Widget des Hauptchats.

Widget-Befehle

$widget topic *command args*

Das Chat-Widget kann eine Themenzeile anzeigen, zum Beispiel das Thema oder den Namen eines Mehrbenutzer-Chatraums oder eines IRC-Kanals.

show

Schaltet die Anzeige des Themas ein.

hide

Schaltet die Anzeige des Themas aus.

set *topic*

Setzt den Thementext auf *topic*.

$widget name *nick args*

Steuert die Namen und die Tags zu den Namen.

list ?*-full*?

Gibt eine Liste aller Benutzernamen aus der Namensansicht zurück. Ist ?-full? angegeben, ist die Liste eine Liste von Listen. Jede Unterliste besteht aus dem nick und danach den Optionen, die an diesem nick-Eintrag gesetzt sind. Damit lassen sich anwendungseigene Optionen prüfen, die **add** an einen nick hängt.

- **add** *nick* ?*options*?
- **delete** *nick*

$widget message *text args*

Fügt Nachrichten zur Anzeige hinzu. Die Optionen sind -nick, -time, -type, -mark -tags

$widget hook *command args*

Verwaltet Hooks. add (message, post, names_group, names_nick, chatstate), remove, run

$widget names *args*

Durchreichung an das Text-Widget der Namensanzeige. Alle Befehle stehen im Handbuch des Widgets **text**. chatwidget stellt zusätzlich **show** und **hide** bereit, mit denen sich die Anzeige dieses Teils steuern lässt.

$widget entry *args*

Durchreichung an das Text-Widget der Eingabe. Alle Befehle stehen im Handbuch des Widgets **text**.

$widget chat *args*

Durchreichung an das Text-Widget des Chats. Alle Befehle stehen im Handbuch des Widgets **text**.

Beispiel

```
chatwidget::chatwidget .chat
proc speak {w msg} {$w message $msg -nick user}
.chat hook add post [list speak .chat]
pack .chat -side top -fill both -expand 1
.chat topic show
.chat topic set "Chat widget demo"
.chat name add "admin" -group admin
.chat name add "user" -group users -color tomato
.chat message "Chatwidget ready" -type system
.chat message "Hello, user" -nick admin
.chat message "Hello, admin" -nick user
```

Ein umfangreicheres Beispiel steht im Code des Programms picoirc im Repository tclapps. Es verbindet das tcllib-Paket **picoirc** mit diesem Paket **chatwidget** zu einem einfachen IRC-Client.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *chatwidget* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

controlwidget

controlwidget

Stichwörter: controlwidget, controlling, displaying, numerical values, scale widget

Name

controlwidget – Sammlung von Widgets zur Anzeige und Steuerung numerischer Werte

Übersicht

package require Tcl ?**8.5**?
package require Tk ?**8.5**?
package require snit ?**2.0**?
package require controlwidget ?**0.1**?
::controlwidget::meter *w args*
::controlwidget::slider *w args*
::controlwidget::equalizerBar *w args*
::controlwidget::thermometer *w args*
::controlwidget::voltmeter *w args*
::controlwidget::tachometer *w args*
::controlwidget::rdial *w args*
$widget get
$widget set *value*
::controlwidget::led *w args*
::controlwidget::radioMatrix *w args*
$led on
$led off
$led setstate *state*
$matrix get

$matrix set *index*

Beschreibung

Das "controlwidget"-Paket hat den Fokus auf die Darstellung und interaktive Steuerung numerischer Werte. Es ahmt verschiedene Messgeräte und Bedienelemente nach, wie sie in Laborumgebungen, aber auch im Alltag zu finden sind: Spannungsmesser, Equalizer und Drehzahlmesser, um nur einige zu nennen. Das sind Alternativen zum Widget *scale*.

Hinweis: Das Paket ist bislang nicht umfassend geprüft. Andere Größen als die Vorgaben können die Darstellung unerwartet ändern. Auch die Optionen für Farben der einzelnen Teile, Schriften und Ähnliches sind nicht alle geprüft und können weitere Fehlerquellen sein.

Ein letzter Hinweis: Einige Teile wurden in keiner der Optionen berücksichtigt, insbesondere die Farben von Teilen, bei denen hellere und dunklere Farbtöne miteinander harmonieren müssen.

Arten von Widgets

Das Paket unterscheidet zwischen verschiedenen Arten von Widgets:

- Vertikale Anzeigen: Der Wert der Variable wird in eine vertikale Position umgewandelt, ähnlich wie bei den farbigen Balken einer Stereoanlage.
- Winkelanzeigen: Der Wert der Variable steht in Zusammenhang mit dem Winkel einer Nadel, ähnlich wie bei Drehzahlmessern.
- Interaktive Widgets: Die meisten Widgets ändern den Wert der Variablen, wenn die Maustaste auf der Nadel liegt und die Nadel nach oben und unten oder nach links und rechts geschoben wird.
- Nicht-interaktive Widgets: Einige Widgets, wie beispielsweise das Thermometer-Widget, lassen eine solche Interaktion nicht zu.
- Logische Widgets oder Auswahl-Widgets: Einige Widgets zeigen die Werte als on/off (oder true/false) oder als eine aus einer Menge diskreter Möglichkeiten.

Allen gemeinsam ist, dass sie sich an eine Variable binden lassen und dass eine Änderung der Variable die Anzeige ändert. Außerdem haben alle Widgets die Methoden **set** und **get**, um mit dem angezeigten Wert zu arbeiten, ob er in einer globalen Variablen liegt oder nicht.

Erscheinungsbild und Verhalten folgen einer oder mehreren Optionen, die sich beim Erzeugen und oft auch später setzen lassen. Alle Widgets bauen auf dem Paket *snit* auf, die Methoden **configure** und **cget** setzen und lesen diese Optionen.

Befehle

Derzeit enthält das Paket folgende Widgets vom Typ *vertikaler Messbalken*:

::controlwidget::meter *w args*

Erzeugt eine vertikale Anzeige, die aus einer Achse und einem beweglichen Pfeil besteht.

widget *w* (in)

Name des zu erstellenden Widgets.

arguments *args* (in)

Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name

Name der Variable, die dem Zähler zugeordnet werden soll

background color

Hintergrundfarbe für das Widget

height pixels
Höhe des Widgets

width pixels
Breite des Widgets

arrowthickness pixels
Dicke des angezeigten Pfeils in Pixeln

arrowcolor color
Farbe des angezeigten Pfeils

readonly boolean
Ob der Pfeil interaktiv verschoben werden kann oder nicht

::controlwidget::slider *w args*

Erzeugt ein Widget mit einem oder mehreren senkrechten Schiebereglern und einer Achse. Die Griffe lassen sich mit der Maus verschieben.

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name
Name der (Listen-)Variablen, die dem Widget zugeordnet werden soll

number integer
Anzahl der Schieberegler im Widget

background color
Hintergrundfarbe für das Widget

height pixels
Höhe des Widgets

width pixels
Breite des Widgets

sliderthickness pixels
Dicke der Schiebereglergriffe

sliderwidth color
Breite der Schiebereglergriffe

troughwidth color
Breite der Führungen, in denen die Gleitstücke gelagert sind

::controlwidget::equalizerBar *w args*

Erzeugt ein Widget, das einen oder mehrere vertikale Balken enthält, die denen ähneln, wie man sie bei grafischen HiFi-Equalizern findet. Hinweis: Es handelt sich um ein schreibgeschütztes Widget.

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)

Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name
Name der (Listen-)Variablen, die dem Widget zugeordnet werden soll

number integer
Anzahl der Balken im Widget

background color
Hintergrundfarbe für das Widget

height pixels
Höhe des Widgets

width pixels
Breite des Widgets

barwidth pixels
Breite der Stäbe

segments integer
Anzahl der Segmente, in die die Balken unterteilt sind

safecolor color
Farbe für Werte unterhalb der Warnschwelle

warningcolor color
Farbe für Werte, die über dem Wert für den Fehler liegen

warninglevel double
Schwellenwert; legt fest, ab welchem Wert sich die Farbe ändert

::controlwidget::thermometer *w args*

Erzeugt ein Thermometer-Widget (schreibgeschützt)

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name
Name der (Listen-)Variablen, die dem Widget zugeordnet werden soll

background color
Hintergrundfarbe für das Widget

height pixels
Höhe des Widgets

width pixels
Breite des Widgets

linethickness pixels
Dicke der Linie, die die "Temperatur" darstellt

linecolor color
Farbe der Linie

Diese Widgets verwenden alle eine vertikale Achse und verfügen über die folgenden

gemeinsamen Optionen zur Konfiguration der Achse:

axisformat string
Formatzeichenfolge zur Formatierung der Zahlen entlang der Achse

axisfont string
Name der Schriftart, die für die Zahlen verwendet werden soll

axiscolor color
Farbe der Achse (einschließlich der Zahlen)

from color
Niedrigerer Wert für die Achse

to color
Oberer Wert für die Achse

majorticks integer
Anzahl der zu zeichnenden Hauptmarkierungen (jede ist beschriftet)

minorticks integer
Anzahl der kleinen Skalenmarkierungen, die zwischen zwei großen Skalenmarkierungen eingezeichnet werden sollen

log boolean
Logarithmische Achse (true) oder lineare Achse (false).

Das Paket enthält das folgende Widget für Winkelanzeigen:

::controlwidget::voltmeter *w args*
Erzeugt ein Widget, das einem Voltmeter ähnelt.

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name
Name der Variable, die dem Zähler zugeordnet werden soll

value value
Wert für den Zähler (sofern er keiner Variable zugeordnet ist)

min value
Der Mindestwert für die Werte in der Anzeige

max value
Der Höchstwert für die Werte in der Anzeige

labels list
Die Bezeichnungen, die entlang der Skala angezeigt werden sollen. (Diese werden lediglich als Text betrachtet und stehen daher an sich in keinem Zusammenhang mit dem Minimal- und Maximalwert.)

title string
Text, der unterhalb des Zifferblatts angezeigt werden soll

width pixels
Die Breite des Widgets

height pixels

Die Höhe des Widgets

borderwidth pixels
Die Breite des Rahmens, der das gesamte Widget umgibt

relief string
Das Relief des Rahmens, der das gesamte Widget umgibt

highlightthickness pixels
Die Breite des Rahmens, der anzeigt, dass das Widget den Tastaturfokus hat

labelfont fontname
Schriftart der Beschriftungen auf der Skala

titlefont fontname
Schriftart des Titels unterhalb des Zifferblatts

dialcolor color
Farbe des Zifferblatts, auf dem sich der Zeiger befindet

needlecolor color
Farbe der Nadel

scalecolor color
Farbe der Skala (Bogen und Teilstriche)

labelcolor color
Farbe der Beschriftungen auf der Skala

titlecolor color
Farbe der Überschrift unterhalb des Zifferblatts

::controlwidget::tachometer *w args*
Erzeugt ein Widget, das einem Drehzahlmesser ähnelt.

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern. Zusätzlich zu den für das Voltmeter-Widget angegebenen:

variable name
Name der Variable, die dem Zähler zugeordnet werden soll

value value
Wert für den Zähler (sofern er keiner Variable zugeordnet ist)

dangerlevel value
Wert, ab dem die Werte in einer anderen Farbe angezeigt werden

dangercolor color
Farbe für die Werte oberhalb der "Gefahrenstufe"

dangerwidth pixels
Breite des farbigen Streifens, der die "Gefahrenstufe" anzeigt

pincolor color
Farbe für die Nadel und die Stecknadel

::controlwidget::rdial *w args*
Erzeugt einen Drehregler. Der Regler lässt sich ziehen, um den Wert zu ändern. Mit gedrückter

Taste Shift ändert sich der Wert langsam, mit gedrückter Taste Control schnell. Ebenso lässt sich der Regler mit dem Mausrad steuern. Shift und Control wirken dabei wie beim Mauszeiger. *Hinweis:* Shift und Control wirken unter Windows derzeit nicht.

widget *w* (in)
Name des zu erstellenden Widgets.

arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern:

variable name
Name der Variablen, die dem Drehregler zugeordnet werden soll

value value
Wert für das Dial-Element (sofern es keiner Variable zugeordnet ist)

orient string
Ob eine waagerechte Scheibe ("horizontal") oder eine senkrechte ("vertical") gebraucht wird.

step value
Erster Schritt zum Erhöhen oder Verringern des Wertes

bindwheel value
Schrittweite für den Wert über das Mausrad.

slow value
Faktor auf der Schrittweite, mit dem der Wert langsam steigt oder fällt.

fast value
Faktor auf der Schrittweite, mit dem der Wert schnell steigt oder fällt.

scale value
Skalenwert, der auf den tatsächlichen Wert angewandt wird.

callback command
Befehl, der beim Ändern des Wertes aufgerufen wird.

width pixels
Die Breite des Widgets

height pixels
Die Höhe des Widgets

background color
Die Hintergrundfarbe des Widgets (Farbe der Scheibe/des Zifferblatts)

foreground color
Die Vordergrundfarbe des Widgets (die Markierungslinien)

Alle diese Widgets verfügen über die folgenden Methoden:

$widget get
Gibt den aktuellen Wert oder die aktuellen Werte zurück, die das Widget anzeigt.

$widget set *value*
Setzt den oder die im Widget angezeigten Werte. Ist das Widget einer Variablen zugeordnet, wird diese Variable ebenfalls gesetzt.

value *double/list*
Neuer Wert oder neue Werte für das Widget.

Es stehen zwei weitere Widgets zur Verfügung, die zur Anzeige logischer Werte dienen:

::controlwidget::led *w args*
Erzeugt ein Widget, das wie eine LED aussieht.
widget *w* (in)
Name des zu erstellenden Widgets.
arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern. Zusätzlich zu den für das Voltmeter-Widget angegebenen:
variable name
Name der Variablen, die der LED zugeordnet werden soll
size pixels
Durchmesser des LED-Widgets
on color
Farbe, die für den "Ein"-Zustand verwendet werden soll
off color
Farbe, die für den "Aus"-Zustand verwendet werden soll

::controlwidget::radioMatrix *w args*
Erzeugt eine Matrix aus Radiobuttons, die sich wie ein einziges Widget verhält.
widget *w* (in)
Name des zu erstellenden Widgets.
arguments *args* (in)
Liste der Schlüssel-Wert-Paare, die das Erscheinungsbild und das Verhalten des Widgets steuern. Zusätzlich zu den für das Voltmeter-Widget angegebenen:
variable name
Name der Variable, die der Matrix zugeordnet werden soll
orient string
Anordnung der Radiobuttons (zuerst horizontal oder vertical)
rows integer
Anzahl der Zeilen in der Matrix
columns integer
Anzahl der Spalten in der Matrix
command list
Befehl der Radiobuttons. Wird aufgerufen, wenn der aktive Radiobutton wechselt.

Das LED-Widget verfügt über die folgenden öffentlichen Methoden:

$led on
Setzt den Zustand auf "on".
$led off
Setzt den Zustand auf "off".
$led setstate *state*
Setzt den Zustand nach dem Wert von "state".
state *boolean* (in)
Neuer Zustand für das LED-Widget

Das "radioMatrix"-Widget verfügt über zwei öffentliche Methoden:

$matrix get
Gibt den Index des gerade gewählten Radiobuttons zurück.

$matrix set *index*
Wählt den Radiobutton mit dem Index *index*.
index *integer*
Index des zu setzenden Radiobuttons.

Danksagungen

Der Code für die meisten dieser Widgets erschien zunächst im Wiki. In vielen Fällen hat Arjen Markus den Code lediglich ein wenig überarbeitet und einige davon auf *snit* umgestellt. Der ursprüngliche Code wurde von den folgenden Personen entwickelt:

- Senkrechtes Messgerät, LED-Anzeige, Radiobutton-Matrix: Ron Fox
- Drehregler: Gerhard Reithofer
- Spannungsmesser und Drehzahlmesser: Marco Maggi
- Code zum Bewegen der Nadel: ?

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *controlwidget* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2010 Ron Fox <rfox@...>
Copyright (c) 2010 Gerhard Reithofer <...@...>
Copyright (c) 2010 Marco Maggi <...@...>
Copyright (c) 2010 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


crosshair

crosshair

Stichwörter: crosshair, canvas, cross-hairs, location, tracking, viewport

Name

crosshair – Fadenkreuze für das Tk-Canvas

Übersicht

package require Tcl ?**8.4**?
package require Tk ?**8.4**?
package require crosshair ?**1.2.1**?
crosshair::crosshair *w* ?*arg...*?
crosshair::off *w*
crosshair::configure *w* ?*arg...*?
crosshair::track on *w cmdprefix*
crosshair::track off *w*

crosshair::bbox_add *w bbox*
crosshair::bbox_remove *token*

Beschreibung

Das Paket **crosshair** stellt Befehle bereit, mit denen sich Fadenkreuze auf Canvas-Widgets ein- und ausschalten und verfolgen lassen.

API

Die folgenden Befehle werden öffentlich bereitgestellt:

crosshair::crosshair *w ?arg...?*
Dieser Befehl schaltet die Anzeige eines Fadenkreuzpaares für das Canvas-Widget *w* ein. Die Fadenkreuze folgen dem Zeigegerät. Das Ergebnis des Befehls ist die leere Zeichenkette.

Alle Argumente nach dem Widget *w* gelten als Optionen eines Linien-Elements im Canvas *w*. Besonders wichtig sind **-fill** und **-dash**.

crosshair::off *w*
Dieser Befehl entfernt die Fadenkreuze aus dem Canvas-Widget *w*. Hatte das Widget keine Fadenkreuze, geschieht nichts. Das Ergebnis des Befehls ist die leere Zeichenkette.

crosshair::configure *w ?arg...?*
Dieser Befehl ändert das Aussehen der Fadenkreuze im Canvas-Widget *w*. Es ist ein Fehler, ihn für einen Canvas aufzurufen, der keine Fadenkreuze hat.

Alle Argumente nach dem Widget *w* gelten als Optionen eines Linien-Elements im Canvas *w*. Besonders wichtig sind **-fill** und **-dash**.

Das Ergebnis des Befehls sind die aktuellen Konfigurationseinstellungen.

crosshair::track on *w cmdprefix*
Dieser Befehl schaltet die Meldung der Position der Fadenkreuze im Canvas-Widget *w* ein. Es ist ein Fehler, diesen Befehl für einen Canvas zu verwenden, der keine Fadenkreuze hat. Das Ergebnis des Befehls ist die leere Zeichenkette.

Nach dem Aufruf wird das Befehlspräfix *cmdprefix* bei jeder Mausbewegung im Canvas mit 7 Argumenten aufgerufen. Der Reihe nach:
1. Das Widget *w*
2. Die x-Position der Fadenkreuze, in Pixeln.
3. Die y-Position der Fadenkreuze, in Pixeln.
4. Die x-Position der linken oberen Ecke des Ansichtsbereichs, in Pixeln.
5. Die y-Position der linken oberen Ecke des Ansichtsbereichs, in Pixeln.
6. Die x-Position der rechten unteren Ecke des Ansichtsbereichs, in Pixeln.
7. Die y-Position der rechten unteren Ecke des Ansichtsbereichs, in Pixeln.

Eine bereits vorhandene Rückruffunktion für *w* wird deaktiviert. Das heißt, pro Canvas-Widget mit Fadenkreuz ist nur eine Rückruffunktion möglich, die dessen Position meldet.

crosshair::track off *w*
Dieser Befehl schaltet die Meldung der Position der Fadenkreuze im Canvas-Widget *w* aus. Es ist ein Fehler, diesen Befehl für einen Canvas zu verwenden, der keine Fadenkreuze hat. Das Ergebnis des Befehls ist die leere Zeichenkette.

crosshair::bbox_add *w bbox*
Dieser Befehl fügt den Fadenkreuzen des Canvas *w* einen Begrenzungsrahmen hinzu. Die Fadenkreuze sind nur in diesem Bereich aktiv.

Das Ergebnis des Befehls ist ein Token, mit dem sich der Begrenzungsrahmen wieder entfernen lässt, siehe **crosshair::bbox_remove** unten.

Der Begrenzungsrahmen *bbox* ist eine Liste von 4 Werten, die linke untere und die rechte obere Ecke des Rahmens. Die Reihenfolge der Werte in der Liste ist:

```
llx lly urx ury
```

Hinweis: Dieser Befehl kann mehrfach verwendet werden, wobei bei jedem Aufruf ein weiterer Begrenzungsrahmen hinzugefügt wird. In diesem Fall entspricht der sichtbare Bereich der *Vereinigung* aller angegebenen Begrenzungsrahmen.

crosshair::bbox_remove *token*
Dieser Befehl entfernt den durch *token* bezeichneten Begrenzungsrahmen (ein Ergebnis von **crosshair::bbox_add**) aus den Fadenkreuzen des zugehörigen Canvas-Widgets.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tklib :: crosshair* der Tcllib-SF-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *crosshair* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2003 Kevin Kenny
Copyright (c) 2008 (docs) Andreas Kupries <andreas_kupries@users.sourceforge.net>
Copyright (c) 2013 Frank Gover, Andreas Kupries
```


ctext

ctext

Stichwörter: ctext, syntax highlighting, text, widget

Name

ctext – Ctext, ein Text-Widget mit Unterstützung für Hervorhebungen

Übersicht

package require Tk
package require ctext ?**3.3**?
ctext *pathName* ?*options*?
::ctext::addHighlightClass *pathName class color keywordlist*
::ctext::addHighlightClassWithOnlyCharStart *pathName class color char*

::ctext::addHighlightClassForSpecialChars *pathName class color charstring*
::ctext::addHighlightClassForRegexp *pathName class color pattern*
::ctext::clearHighlightClasses *pathName*
::ctext::getHighlightClasses *pathName*
::ctext::deleteHighlightClass *pathName class*
::ctext::enableComments *enable*
::ctext::disableComments *enable*
pathName **highlight** *startIndex endIndex*
pathName **fastdelete** *index1* ?*index2*?
pathName **fastinsert**
pathName **copy**
pathName **cut**
pathName **paste**
pathName **append**
pathName **configure** *option value* ?*...*?

Beschreibung

Das Paket **ctext** stellt das ctext-Widget bereit, ein erweitertes Text-Widget mit konfigurierbarer Syntaxhervorhebung und einigen zusätzlichen Befehlen.

Ctext überlädt das Text-Widget und stellt neue Befehle bereit, nämlich **highlight**, **copy**, **paste**, **cut**, **append** und **edit**. Außerdem gibt es mehrere Befehle, mit denen sich Klassen definieren lassen. Jede Klasse entspricht einem Tag im Widget.

Befehle

ctext *pathName* ?*options*?
Erstellt und konfiguriert ein ctext-Widget.

Hervorhebung

Die Hervorhebung steuern Tags des Text-Widgets. Diese Tags heißen Hervorhebungsklassen. *class* ist ein Tag-Name und lässt sich wie jedes Tag eines Text-Widgets konfigurieren. Vier Arten von Hervorhebungsklassen werden unterstützt. Die Methode **highlight** des Widgets verwendet alle Hervorhebungsklassen automatisch.

::ctext::addHighlightClass *pathName class color keywordlist*
Fügt dem ctext-Widget *pathName* die Hervorhebungsklasse *class* hinzu. Hervorgehoben wird mit der Farbe *color*. Alle Wörter in *keywordlist* werden hervorgehoben.

```
# highlight some tcl keywords
::ctext::addHighlightClass .t tclkeywords red [list set info interp uplevel
upvar]
```

::ctext::addHighlightClassWithOnlyCharStart *pathName class color char*
Fügt dem ctext-Widget *pathName* die Hervorhebungsklasse *class* hinzu. Hervorgehoben wird mit der Farbe *color*. Alle Wörter, die mit *char* beginnen, werden hervorgehoben.

```
::ctext::addHighlightClassWithOnlyCharStart .t vars blue \$
```

::ctext::addHighlightClassForSpecialChars *pathName class color charstring*
Fügt dem ctext-Widget *pathName* die Hervorhebungsklasse *class* hinzu. Hervorgehoben wird

mit der Farbe *color*. Alle Zeichen in *charstring* werden hervorgehoben.

::ctext::addHighlightClassForRegexp *pathName class color pattern*
Fügt dem ctext-Widget *pathName* die Hervorhebungsklasse *class* hinzu. Hervorgehoben wird mit der Farbe *color*. Alle Textteile, die zum regulären Ausdruck *pattern* passen, werden hervorgehoben.

::ctext::clearHighlightClasses *pathName*
Entfernt alle Hervorhebungsklassen aus dem Widget *pathName*.

::ctext::getHighlightClasses *pathName*
Listet alle Hervorhebungsklassen des Widgets *pathName* auf.

::ctext::deleteHighlightClass *pathName class*
Löscht die Hervorhebungsklasse *class* aus dem Widget *pathName*.

::ctext::enableComments *enable*
Schaltet die Hervorhebung von C-Kommentaren ein. *class* für Kommentare im C-Stil ist **_cComment**. Die Hervorhebung von C-Kommentaren ist standardmäßig aus.

::ctext::disableComments *enable*
Schaltet die Hervorhebung von C-Kommentaren aus.

Widget-Befehle

Jedes mit dem obigen Befehl erzeugte ctext-Widget unterstützt zusätzlich zu den Befehlen und Optionen des üblichen Text-Widgets die folgenden Befehle und Optionen.

pathName* highlight *startIndex endIndex
Hebt den Text zwischen *startIndex* und *endIndex* hervor.

***pathName* fastdelete *index1* ?*index2*?**
Löscht den Textbereich, ohne die Hervorhebung zu aktualisieren. Die Argumente stimmen mit dem Befehl *pathName* **delete** überein, der vom üblichen Text-Widget geerbt wird.

***pathName* fastinsert**
Fügt Text ein, ohne die Hervorhebung zu aktualisieren. Die Argumente stimmen mit dem Befehl *pathName* **insert** überein, der vom üblichen Text-Widget geerbt wird.

***pathName* copy**
Ruft **tk_textCopy** für die ctext-Instanz auf.

***pathName* cut**
Ruft **tk_textCut** für die ctext-Instanz auf.

***pathName* paste**
Ruft **tk_textPaste** für die ctext-Instanz auf.

***pathName* append**
Hängt die aktuelle Auswahl an die Zwischenablage an.

***pathName* configure *option value* ?...?**
Setzt die Optionen des ctext-Widgets. Auf jeden Optionsnamen folgt der neue Wert.

Widget-Optionen

-linemap
Erzeugt (-linemap 1) oder entfernt (-linemap 0) links neben dem Widget eine Zeilennummernliste. Voreingestellt wird die Linemap angezeigt.

-linemapfg

Ändert den Vordergrund der Linemap. Voreingestellt ist dieselbe Farbe wie beim Haupt-Text-Widget.

-linemapbg
Ändert den Hintergrund der Linemap. Voreingestellt ist dieselbe Farbe wie beim Haupt-Text-Widget.

-linemap_select_fg
Ändert den Vordergrund der ausgewählten Zeile. Voreingestellt ist schwarz.

-linemap_select_bg
Ändert den Hintergrund der ausgewählten Zeile. Voreingestellt ist gelb.

-linemap_mark_command
Ruft eine Prozedur oder einen Befehl auf. Argumente sind der *pathName* des ctext-Fensters, der *type* (**marked** oder **unmarked**) und die gewählte Zeilennummer. Der Prozedurprototyp lautet:

```
proc linemark_cmd {win type line}.
```

Siehe auch ctext_test_interactive.tcl

-highlight
Boolescher Wert. Legt fest, ob eingefügter oder gelöschter Text hervorgehoben wird. Voreingestellt ist 1.

-linemap_markable
Boolescher Wert. Legt fest, ob Zeilen in der Linemap mit der Maus markierbar sind. Voreingestellt ist 1.

Beispiel

```
    package require Tk
    package require ctext

    proc main {} {
    pack [frame .f] -fill both -expand 1
    pack [scrollbar .f.s -command {.f.t yview}] -side right -fill y

    pack [ctext .f.t -bg black -fg white -insertbackground yellow  -yscrollcommand
    {.f.s set}] -fill both -expand 1

    ctext::addHighlightClass .f.t widgets purple  [list ctext button label text frame
    toplevel  scrollbar checkbutton canvas listbox menu menubar menubutton
    radiobutton scale entry message tk_chooseDir tk_getSaveFile  tk_getOpenFile
    tk_chooseColor tk_optionMenu]

    ctext::addHighlightClass .f.t flags orange  [list -text -command -yscrollcommand
    -xscrollcommand -background -foreground -fg -bg  -highlightbackground -y -x
    -highlightcolor -relief -width  -height -wrap -font -fill -side -outline -style
    -insertwidth  -textvariable -activebackground -activeforeground
    -insertbackground  -anchor -orient -troughcolor -nonewline -expand -type
    -message  -title -offset -in -after -yscroll -xscroll -forward -regexp -count
    -exact -padx -ipadx -filetypes -all -from -to -label -value -variable  -regexp
    -backwards -forwards -bd -pady -ipady -state -row -column  -cursor
    -highlightcolors -linemap -menu -tearoff -displayof -cursor  -underline -tags
```

```
    -tag]

    ctext::addHighlightClass .f.t stackControl red  {proc uplevel namespace while for
    foreach if else}
    ctext::addHighlightClassWithOnlyCharStart .f.t vars mediumspringgreen "\$"
    ctext::addHighlightClass .f.t variable_funcs gold {set global variable unset}
    ctext::addHighlightClassForSpecialChars .f.t brackets green {[]{}}
    ctext::addHighlightClassForRegexp .f.t paths lightblue {\.[a-zA-Z0-9\_\-]+}
    ctext::addHighlightClassForRegexp .f.t comments khaki {#[^\n\r]*}
    .f.t fastinsert end [info body main]

    pack [frame .f1] -fill x

    .f.t highlight 1.0 end

    pack [button .f1.exit -text Exit -command exit] -side left

    pack [entry .e] -side bottom -fill x
    .e insert end "ctext::deleteHighlightClass .f.t "
    bind .e <Return> {eval [.e get]}
    }
    main
```

Weitere Beispiele liegen im Quellpaket von ctext.

Dank

Kevin Kenny, Neil Madden, Jeffrey Hobbs, Richard Suchenwirth, Johan Bengtsson, Mac Cody, Günther, Andreas Sievers und Michael Schlenker.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *ctext* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) George Peter Staplin <GeorgePS@XMission.com>
```


cursor

cursor

Stichwörter: cursor

Name

cursor – Prozeduren für Tk-Cursor

Übersicht

package require Tk
package require cursor ?**0.3.1**?
::cursor::propagate *widget cursor*

::cursor::restore *widget* ?*cursor*?
::cursor::display ?*parent*?

Beschreibung
Das Paket **cursor** stellt Befehle zur Verwaltung von Tk-Cursorn bereit.

Befehle
Folgende Befehle stehen zur Verfügung:

::cursor::propagate *widget cursor*
Legt den Cursor für das angegebene *widget* und alle seine Nachkommen auf *cursor* fest.

::cursor::restore *widget* ?*cursor*?
Stellt den ursprünglichen oder zuvor gesetzten Cursor für das angegebene *widget* und alle seine Nachkommen wieder her. Wird *cursor* angegeben, gilt er für jedes Widget, das keinen voreingestellten Cursor hatte (gesetzt durch einen früheren Aufruf von **::cursor::propagate**).

::cursor::display ?*parent*?
Es wird ein Dialogfeld mit einer listbox angezeigt, die alle Cursor-Namen enthält. Wenn Sie einen Cursor-Namen auswählen, wird dieser in diesem Dialogfeld angezeigt. Dies dient lediglich dazu, alle auf der Plattform verfügbaren Cursor anzuzeigen.

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *cursor* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) Jeffrey Hobbs <jeff@hobbs.org>
```


datefield

datefield
Stichwörter: datefield, clock, date, dateentry, entry, widget

Name
datefield – Tk-Widget "datefield"

Übersicht
package require Tk
package require datefield ?**0.3**?
::datefield::datefield *widgetpath* ?*options*?
-format

Beschreibung
Das Paket **datefield** stellt das Widget datefield bereit, ein erweitertes Entry-Widget zur Eingabe von Datumswerten. Für die eingebbaren Daten gibt es drei zulässige Formate:

1. Englische Form MM/DD/YYYY mit *-format* "*%m/%d/%Y*" (Vorgabe)

2. Deutsche Form DD.MM.YYYY mit *-format* "*%d.%m.%Y*"
3. ISO-Form YYYY-MM-DD mit *-format* "*%Y-%m-%d*"

Das Widget datefield ist tatsächlich nur ein Entry-Widget mit besonderen Bindungen. Alle Befehle und Optionen eines Entry-Widgets gelten hier ebenso.

Befehle

::datefield::datefield *widgetpath* ?*options*?
Erstellt und konfiguriert ein Datumsfeld-Widget.

Optionen

-format
Eines von "%m/%d/%Y" (englisch, Vorgabe, wenn die Option weggelassen wird), "%d.%m.%Y" (deutsch) oder "%Y-%m-%d" (ISO).

Die übrigen verfügbaren Optionen stehen in der Handbuchseite **entry**.

Beispiel

```
package require datefield

wm title . "Datefield example"
proc DayOfWeek {args} {
    set now [clock scan $::myDate]
    set ::myDate2 [clock format $now -format %A]
}
trace variable myDate w DayOfWeek

::datefield::datefield .df -textvariable myDate -format "%m/%d/%Y"
label .l1 -text "Enter a date:"   -anchor e
label .l2 -text "That date is a:" -anchor e
label .l3 -textvariable myDate2 -relief sunken -width 12

grid .l1 .df -sticky ew
grid .l2 .l3 -sticky ew
focus .df
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *datefield* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

Copyright

```
Copyright (c) Keith Vetter <keith@ebook.gemstar.com>
Copyright (c) Thomas Wunderlich <tcl.tk@blindenfreizeiten.de>
```

diagrams

diagram

Stichwörter: diagram, 2D geometry, arc, arrow, box, canvas, circle, diamond, drawing, drum, ellipse, image, interpolation, intersection, line, move, picture, plane geometry, plotting, point, raster image, spline, text, vector

Name

Diagramm – Erstellung eines Diagramms

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require diagram 1
::diagram *objectName canvas* ?*script*?
diagramObject **new direction** *name* ?*key value...*?
diagramObject **new element** *name attributes cmdprefix*
diagramObject **new alias** *name cmdprefix*
diagramObject **new command** *name arguments body*
diagramObject **new attribute** *name* ?*key value...*?
diagramObject **unknown attribute** *cmdprefix*
diagramObject **draw** *script*
arc *attr...*
arrow *attr...*
--> *attr...*
<--> *attr...*
<--> *attr...*
block *script attr...*
box *attr...*
circle *attr...*
O *attr...*
diamond *attr...*
<> *attr...*
drum *attr...*
ellipse *attr...*
line *attr...*
-- *attr...*
move *attr*
spline *attr...*
text *attr...*
west
w
left
l
south
s
down
bottom

bot
b
east
e
right
r
north
n
up
top
t
northwest
nw
up-left
upleft
leftup
northeast
ne
up-right
upright
rightup
southwest
sw
down-left
downleft
leftdown
southeast
se
down-right
downright
rightdown
number **cm**
number **mm**
number **inch**
number **pt**
number **number**
by *distance direction*
point1 **+** *point2*
point1 **-** *point2*
point **by** *distance direction*
point1 **|** *point2*
n **between** *poin1 point2*
intersect *elem1 elem2*
element **names** ?*pattern*?
element **corner**
element **corner1** *corner2...*

element ?*corner1*... ?*names* ?*pattern*? ?*]*?
nth ?*corner*?
nth **last** ?*corner*?
nth *shape* ?*corner*?
nth **last** *shape* ?*corner*?
last ?*corner*?
last *shape* ?*corner*?
1st
2nd
3rd

Beschreibung

Willkommen bei **diagram**, einem Paket zur einfachen Erstellung von Diagrammen (sic), d. h. 2D-Vektorgrafiken, die manchmal auch als *Bilder* bezeichnet werden. Hinweis: Dieses Paket ist kein Ersatz für die "Canvas" von **Tk**, sondern vielmehr eine darüber liegende Ebene, die die Nutzung vereinfacht. Mit anderen Worten: Durch die Nutzung des "Canvas" als zentrale Grafik-Engine abstrahiert **diagram** die Feinheiten der Verarbeitung von Koordinaten bei der Positionierung und Größenanpassung der gezeichneten Elemente, sodass sich der Benutzer stattdessen auf den Inhalt des Diagramms konzentrieren kann.

Dies ähnelt der PIC-Sprache von Brian Kernighan für troff, die als geistiger Vorläufer dieses Pakets gilt.

Dieses Dokument enthält eine Übersicht über die API- und Zeichnungsbefehle (Sprache). Es richtet sich an Nutzer des Pakets, die ihr Wissen auffrischen möchten. Einsteiger sollten zunächst das *Tutorial zur Diagrammsprache* lesen. Entwickler, die an den internen Strukturen des Pakets und seiner unterstützenden Pakete arbeiten möchten, sollten zunächst den Abschnitt **DIAGRAMMKLASSEN** lesen und anschließend die Kommentare im Quellcode der Pakete selbst.

Im weiteren Verlauf dieses Dokuments beschreiben wir zunächst die APIs der Diagrammklasse und ihrer Instanzen, gefolgt von der Sprachreferenz für die Zeichensprache selbst.

API

API für Klassen

Das Paket exportiert die hier beschriebene API.

::diagram *objectName canvas* ?*script*?

Der Befehl erstellt eine neue Instanz eines Diagramm-Controllers und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die neue Instanz wird mit dem angegebenen *canvas*-Objekt verbunden, das als Grafik-Engine für das Diagramm dient. In der Regel handelt es sich dabei um eine Instanz von Tks *canvas*; es kann jedoch jedes Objekt verwendet werden, das eine API-Kompatibilität mit Tks *canvas* hat.

Die API dieses Objektbefehls wird im folgenden Abschnitt "Objekt-API" beschrieben. Sie kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

Wenn das Argument *script* angegeben ist, wird die Methode **draw** darauf aufgerufen.

Objekt-API

Instanzen der Diagrammklasse unterstützen die folgenden Methoden:

***diagramObject* new direction *name* ?*key value*...?**

Diese Methode definiert eine neue benannte Richtung und deren Attribute. Letztere werden durch die *key*/*value*-Paare angegeben, die auf den *name* folgen.

Den Benutzern steht es weitgehend frei, beliebige Attribute mit einer beliebigen Bedeutung festzulegen. Eine Ausnahme bilden die Bezeichnungen *angle* und *opposite*. Diese sind spezifisch für das Diagramm-Paket und haben eine festgelegte Bedeutung.

angle

Dieses Attribut legt den Winkel der Richtung in Grad fest, wobei 0 nach Osten (nach rechts) und 90 nach Norden (nach oben) zeigt.

opposite

Dieses Attribut legt den Namen der Richtung fest, die als komplementär zu der genannten Richtung betrachtet werden soll.

diagramObject* new element *name attributes cmdprefix

Diese Methode definiert ein neues Grafikelement für die Zeichensprache. Das heißt, *name* wird zu einem neuen Befehl in der Sprache, und das angegebene Befehlspräfix (*cmdprefix*) wird aufgerufen, um die eigentliche Zeichnung auszuführen.

attributes legt die Menge der Attribute fest, für die Daten verfügbar sein müssen. Das heißt, das System führt für diese Attribute die Methode ...-callbacks aus. Weitere Informationen zu Attributdefinitionen finden Sie in der Methode **new attribute**.

Der Befehls-Präfix muss der folgenden Signatur entsprechen:

cmdprefix *canvas attributes*

Dabei ist *canvas* der Handle des Canvas-Widgets, auf dem gezeichnet werden soll, und *attributes* ein Dictionary, das die Attribute des Elements enthält, unabhängig davon, ob es sich um benutzerdefinierte oder um Standardwerte handelt.

Das Ergebnis des Befehls muss eine Liste sein, die mindestens zwei und höchstens vier Elemente enthält. Diese sind in der folgenden Reihenfolge:

1. Die Liste der Canvas-Elemente, aus denen das gezeichnete Element besteht.
2. Das Dictionary der benannten Positionen innerhalb der Elemente, seine *Ecken*.
3. Ein optionaler Modus, entweder "relativ" oder "absolut". Wird kein Wert gegeben, wird "relativ" angenommen. Bei einer relativen Elementposition werden die Attribute "with" und "at" verwendet, um die endgültige Position des neuen Elements zu bestimmen.
4. Ein optionaler Name für eine Ausrichtung. Wenn es sich nicht um eine leere Zeichenkette handelt, wird dieser Wert dem automatischen Layout-Modul als neue Ausrichtung übergeben.

diagramObject* new alias *name cmdprefix

Diese Methode definiert einen neuen Befehl für die Zeichensprache. Das heißt, *name* wird zu einem neuen Befehl in der Sprache, und bei Verwendung dieses neuen Befehls wird das angegebene Befehlspräfix (*cmdprefix*) aufgerufen. Alle an den Befehl übergebenen Argumente werden einfach an das Präfix weitergeleitet. Es gibt keine festgelegte Signatur.

Hinweis: Das Präfix wird im Kontext der Zeichensprache ausgeführt, wodurch die direkte

Verwendung aller vorhandenen Befehle möglich ist.

diagramObject* new command *name arguments body

Dies entspricht **new alias**, mit dem Unterschied, dass der neue Befehl als Prozedur im Kontext der Sprache definiert ist, mit einer regulären Argumentliste und einem regulären Prozedurkörper.

***diagramObject* new attribute *name* ?*key value*...?**

Diese Methode definiert ein neues benanntes Attribut, das von grafischen Elementen verwendet werden kann. Die Verarbeitung des Attributs durch den Prozessor wird durch die *key*/*value*-Paare festgelegt, die auf den *name* folgen.

Die zulässigen Schlüssel und ihre Bedeutungen lauten:

key

Der Wert dieses Schlüssels ist der Name des Schlüssels, unter dem der Wert des Attributs nach Abschluss der Attributverarbeitung in dem dem Zeichnungsbefehl übergebenen Attribut-Dictionary gespeichert werden soll.

Dieser Schlüssel ist optional. Wird er nicht angegeben, wird standardmäßig der Name des Attributs verwendet.

get

Der Wert dieses Schlüssels ist ein Befehlspräfix, das aufgerufen wird, um das bzw. die Argumente des Attributs über die Befehlszeile abzurufen.

Dieser Schlüssel ist optional. Wird er nicht angegeben, wird ein Standardwert verwendet, der das einzelne Wort nach dem Attributnamen als Wert des Attributs übernimmt.

Die Signatur des Befehlspräfixes lautet

cmdprefix *wordqueue*

Dabei ist *wordqueue* das Handle eines Warteschlangenobjekts, das der API der vom Paket **struct::queue** bereitgestellten Warteschlangen entspricht. Diese Warteschlange enthält den noch nicht verarbeiteten Teil der Attributdefinitionen, wobei pro Wort ein Eintrag vorhanden ist und der erste Eintrag den Namen des Attributs *nach* dem Wort enthält. Mit anderen Worten: Der Name des Attributs wurde bereits aus der Warteschlange entfernt.

Das Ergebnis des Befehls ist der Wert des Attributs, der möglicherweise aus der Warteschlange entnommen wurde – oder auch nicht.

transform

Der Wert dieses Schlüssels ist ein Befehlspräfix, das aufgerufen wird, um den abgerufenen Wert (siehe **get**) in seine endgültige Form umzuwandeln.

Dieser Schlüssel ist optional. Wird er nicht angegeben, erfolgt keine Transformation. Die Signatur des Befehlspräfixes lautet

cmdprefix *value*

Dabei ist *value* der zu transformierende Wert.

Das Ergebnis des Befehls ist der endgültige Wert des Attributs.

type

Der Wert dieses Schlüssels ist ein Befehlspräfix, das zur Validierung der Argumente des Attributs aufgerufen wird.

Dieser Schlüssel ist optional. Wird er nicht angegeben, findet keine Validierung statt.

Die Signatur des Befehlspräfixes entspricht der der Snit-Validierungstypen. Siehe die Dokumentation zum Paket **snit**.

merge

Der Wert dieses Schlüssels ist ein Befehlspräfix, das aufgerufen wird, um den transformierten und validierten Attributwert in das Dictionary der erfassten Attribute einzufügen.

Dieser Schlüssel ist optional. Wird er nicht angegeben, wird eine Standardzusammenführung gewählt, die auf den Daten für den Schlüssel **aggregate** basiert (siehe unten). Die Signatur des Befehlspräfixes lautet

cmdprefix *value dict*

Dabei ist *value* der einzufügende Wert und *dict* das bisher gesammelte Dictionary aus Attributen und Werten.

Das Ergebnis des Befehls ist das neue Dictionary.

aggregate

Der Wert dieses Schlüssels ist ein boolescher Schalter. Er wirkt sich genau dann aus, wenn der Schlüssel **merge** nicht angegeben wurde. Dieser Schlüssel ist optional. Wird er nicht angegeben, wird standardmäßig der Wert **False** verwendet.

Wenn der Schlüssel aktiv ist, bedeutet der Wert von **False**, dass der Wert des Attributs unter Verwendung des Werts des Schlüssels *key* (siehe oben) als Index in das Dictionary *eingetragen* wird, wobei jeder zuvor angegebene Wert *überschrieben* wird.

Wenn der Schlüssel aktiv ist, bedeutet der Wert von **True**, dass der Wert des Attributs unter Verwendung des Werts des Schlüssels *key* (siehe oben) als Index in das Dictionary *hinzugefügt* wird, wodurch zuvor angegebene Werte *erweitert* werden. Dies bedeutet, dass der endgültige Wert des Attributs nach dem Prozess eine Liste der gesammelten Werte sein wird.

default

Der Wert dieses Schlüssels ist ein Befehlspräfix, das aufgerufen wird, nachdem die Erfassung der Attribute abgeschlossen ist und dieses Attribut in der Liste der erforderlichen Attribute für das Zeichenelement enthalten ist (siehe Argument *attributes* der Methode **new element**).

Beachten Sie, dass die Verbindung über den Wert des Schlüssels *key* hergestellt wird, nicht über den Attributnamen an sich.

Beachten Sie ferner, dass dieses Befehlspräfix auch dann aufgerufen wird, wenn ein vom Benutzer angegebener Attributwert vorhanden ist. Dadurch kann der Befehl über das bloße Setzen von Standardwerten hinausgehen und auch abgeleitete Werte berechnen und speichern.

Dieser Schlüssel ist optional. Wenn ein Element dieses Attribut benötigt, *default* jedoch nicht angegeben ist, wird nichts unternommen.

Die Signatur des Befehlspräfixes lautet

cmdprefix init

Diese Methode wird ausgeführt, wenn das Attribut definiert wird. Ihre Aufgabe besteht darin, alle Elemente im Namespace für das Attribut zu initialisieren und die Standardverarbeitung durchzuführen.

Das Ergebnis dieser Methode wird ignoriert.

cmdprefix fill *varname*

Diese Methode wird ausgeführt, um Standardwerte oder abgeleitete Werte in das durch *varname* benannte AttributDictionary einzutragen. Diese Variable wird im aufrufenden Kontext gesucht.

Das Ergebnis dieser Methode wird ignoriert.

cmdprefix set *name value*

Diese Methode wird aufgerufen, um den aktuellen Wert eines Attributs in den Namespace der Sprache zu übertragen, damit dieser zum neuen Standardwert wird.

Das Ergebnis dieser Methode wird ignoriert.

linked

Dieser Schlüssel ist genau dann wirksam, wenn der Schlüssel **default** nicht angegeben ist. In diesem Fall stellt er eine Standardbehandlung für **default** bereit, indem er das Attribut mit einer Variable im Sprachkontext verknüpft.

Der Wert für diesen Schlüssel ist eine Liste mit zwei Elementen, die in dieser Reihenfolge den Namen der zu verknüpfenden Variablen und deren Anfangswert enthält.

diagramObject* unknown attribute *cmdprefix

Diese Methode registriert das Befehlspräfix bei dem Subsystem, das die Attribute für Elementbefehle verarbeitet, und weist es an, dieses Präfix aufzurufen, sobald es auf ein Attribut stößt, das es nicht eigenständig verarbeiten kann.

Es ist zulässig, mehr als eine Rückruffunktion zu registrieren; diese werden in der Reihenfolge ihrer Registrierung (d. h. von vorne nach hinten) aufgerufen, bis eine der Rückruffunktionen die aktuelle Eingabe akzeptiert. Das Befehlspräfix muss der folgenden Signatur entsprechen:

cmdprefix *wordqueue*

Dabei ist *wordqueue* das Handle eines Warteschlangen-Objekts, das der API der vom Paket **struct::queue** bereitgestellten Warteschlangen entspricht. Diese Warteschlange enthält den noch nicht verarbeiteten Teil der Attributdefinitionen, mit einem Eintrag pro Wort, wobei der erste Eintrag den Namen des Attributs enthält, das nicht verarbeitet werden konnte.

Das Ergebnis des Befehls muss ein boolescher Wert sein, wobei **True** signalisiert, dass diese Rückruffunktion das Attribut akzeptiert und verarbeitet hat und der neue Zustand von *wordqueue* den Punkt angibt, an dem die allgemeine Verarbeitung fortgesetzt werden

soll.

Angesichts der Signatur verfügt der Befehl im Wesentlichen über zwei Möglichkeiten, die von ihm erkannten Attribute zu verarbeiten (umzuschreiben):
1. Ersetzt das Attribut (und die Argumente) durch ein anderes Attribut und andere Argumente.
2. Weitere Wörter am Anfang setzen den allgemeinen Prozess wieder in Gang.

diagramObject* draw *script

Diese Methode führt das angegebene *script* im Kontext der Definitionen der Zeichensprache aus. Einzelheiten zu den verfügbaren Befehlen finden Sie im Abschnitt Sprachreferenz.

Hinweis: Das *script* gilt als *vertrauenswürdig*. Es wird im aktuellen Interpreter ausgeführt und hat Zugriff auf dessen gesamte Funktionen. Für die Ausführung nicht vertrauenswürdiger Diagrammskripte sollte dieser Interpreter ein sicherer sein.

Sprachreferenz

Elemente

In diesem Abschnitt werden die Befehle für die vordefinierten Zeichenelemente, auch "Formen" genannt, aufgeführt. Diese Befehle sind alle im Kontext der Sprache definiert. Alle Befehle dieses Abschnitts geben als Ergebnis den Handle des neu erstellten Elements zurück. Dieser Handle existiert auch als Befehl, mit dem die Eckpunkte des Elements (Namen, Werte) abgefragt werden können. Siehe Abschnitt Verschiedene Befehle. BILD: figure-02-basic-shapes

arc *attr...*

BILD: figure-02-arc Ein offenes Element mit den entsprechenden Eckpunkten, d. h. "Start", "Ende" und "Mitte". Hinweis: Es weist auch die Windrose geschlossener Elemente als Eckpunkte auf, wobei der Mittelpunkt des Kreises, zu dem der Bogen gehört, als Mittelpunkt der Windrose dient und die anderen Punkte auf dem Kreis, zu dem der Bogen gehört. Es unterstützt die Attribute

anchor *name*

siehe unten ...

ljust

siehe unten ...

rjust

siehe unten ...

above

siehe unten ...

below

BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist.

Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"

above
"Anker Süd"

below
"Anker Nord"

clockwise
siehe unten ...

cw
Legt die Richtung des Elements **arc** fest, in diesem Fall im Uhrzeigersinn. Das komplementäre Attribut lautet **counterclockwise**. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **clockwise** entnommen wird, deren Standardwert wiederum **false** ist, was einer Richtung gegen den Uhrzeigersinn entspricht.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

counterclockwise
siehe unten ...

ccw
Legt die Richtung des Elements **arc** fest, in diesem Fall gegen den Uhrzeigersinn. Das komplementäre Attribut lautet **clockwise**. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **clockwise** entnommen wird, deren Standardwert wiederum **false** ist, was einer Richtung gegen den Uhrzeigersinn entspricht.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

from *location*
Legt fest, welche Stelle das Element "**arc**" hat. Standardmäßig ist dies die aktuelle Position, wie sie vom Layout-System verwaltet wird.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist.

Die zulässigen Werte sind **left**, **right** und **center**.

radius *length*
Legt den Radius des Elements **arc** fest, genauer gesagt den Radius des Kreises, zu dem der dargestellte Bogen gehört. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **arcradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert

wiederum **Helvetica 12pt** ist.

to *location*
Legt die Position fest, an der das Element "**arc**" endet. Standardmäßig wird eine Position gewählt, an der ein 90-Grad-Bogen in der ausgewählten Richtung gezogen wird, beginnend bei "**from**".

arrow *attr...*
siehe unten ...

--> *attr...*
siehe unten ...

<--> *attr...*
siehe unten ...

<--> *attr...*
BILD: figure-02-arrow Ein Alias für das Element **line** (siehe unten), bei dem das Attribut **arrowhead** auf **->** voreingestellt ist, **<->** oder **<-** voreingestellt ist. Das Element **arrow** entspricht dem Element **-->**.

block *script attr...*
Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie einem "Mittelpunkt". Der Haupteffekt besteht in der Zusammenfassung aller durch das *script* erstellten Elemente zu einem einzigen Element. Dies bedeutet auch, dass die vom Skript erstellten Elemente zwar während der Ausführung des Skripts im Verlauf sichtbar sind, der Verlauf danach jedoch nur noch den Block selbst enthält und nicht mehr die Elemente, aus denen er besteht.

Das Skript hat Zugriff auf den aktuellen Zustand aller Variablen im Sprachkontext. Alle Änderungen an den Variablen werden nach der Ausführung des Blocks rückgängig gemacht. Allerdings werden auch alle im Skript festgelegten Variablen als Ecken der Elemente exportiert, sodass Benutzer ihre eigenen benannten Positionen im Block definieren können.

Was den Layout-Mechanismus betrifft, so werden alle durch das Skript vorgenommenen Änderungen rückgängig gemacht, sobald das Element fertiggestellt ist. Mit anderen Worten: Ein Block ist ein implizites **group**.

Blöcke verarbeiten alle Attribute und geben deren Einstellungen als Standardwerte an das Skript weiter, die während der Skriptausführung aktiv sind.

box *attr...*
BILD: figure-02-box Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie dem "Mittelpunkt". Es verwaltet die Attribute

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above

siehe unten ...

below

BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust

"Anker West"

rjust

"Anker Ost"

above

"Anker Süd"

below

"Anker Nord"

at *location*

Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

color *spec*

BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

fillcolor *spec*

BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

height *length*

siehe unten ...

ht *length*

Legt die Höhe des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxheight** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

justify left|center|right

Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist.

Die zulässigen Werte sind **left**, **right** und **center**.

slant *angle*
Legt den Winkel fest, um den das Element **box** geneigt ist, in Grad. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **slant** entnommen wird, deren Standardwert wiederum **90** ist, d. h. vertikal, ohne Neigung. 0 Grad entspricht einer Neigung direkt nach Osten, wobei die Spitze nach rechts zeigt. 90 Grad entsprechen einer Neigung nach Norden, wobei das Element senkrecht nach oben zeigt.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert,

wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

width *length*
siehe unten ...

wid *length*
Legt die Breite des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxwidth** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

circle *attr...*
siehe unten ...

O *attr...*
BILD: figure-02-circle Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie einem "Mittelpunkt". Es verfügt über folgende Attribute:

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above
siehe unten ...

below
BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"

above

"Anker Süd"

below
"Anker Nord"

at *location*
Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

diameter *length*
siehe unten ...

diam *length*
Legt den Durchmesser des Elements "**circle**" fest und stellt damit eine alternative Möglichkeit zur Angabe seines Radius ("**radius**") dar. Gilt genau dann, wenn der Radius nicht angegeben wurde. Das heißt, wenn sowohl Durchmesser als auch Radius angegeben sind, hat die Angabe des Radius Vorrang. Dieses Attribut hat keinen Standardwert, da die Standardwerte aus dem Radius übernommen werden.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

radius *length*
siehe unten ...

rad *length*
Legt den Radius des Elements **circle** fest. Wird dieser nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **circleradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus

der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

diamond *attr...*
siehe unten ...

<> attr...
BILD: figure-02-diamond Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie einem "Mittelpunkt". Es verwaltet die Attribute

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above
siehe unten ...

below
BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"

above
"Anker Süd"

below
"Anker Nord"

aspect *number*
Legt das Seitenverhältnis, d. h. das Verhältnis von Breite zu Höhe, des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn keine Angabe erfolgt, hängt zudem von den Angaben der Attribute **width** und **height** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe von **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Vorrang hat. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** festgelegt wird und eines der Attribute **width** oder **height**, dann wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt wird, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, die ihrerseits standardmäßig auf **2** gesetzt ist.

Wenn keines der Attribute **width** oder **height** festgelegt wird, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren

Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

at *location*
Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

height *length*
Legt die Höhe des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn dieser nicht angegeben ist, hängt auch von den Angaben der Attribute **aspect** und **width** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe für **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Angaben Vorrang haben. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** festgelegt wird und eines der Attribute **width** oder **height**, dann wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt wird, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, deren Standardwert wiederum **2** ist.

Wenn keines der Attribute **width** oder **height** festgelegt ist, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

width *length*
Legt die Breite des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn dieser nicht angegeben ist, hängt auch von den Angaben der Attribute **aspect** und **height** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe für **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Vorrang hat. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** sowie eines der Attribute **width** oder **height** angegeben sind, wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt ist, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, die ihrerseits standardmäßig auf **2** gesetzt ist.

Wenn keines der Attribute **width** oder **height** festgelegt ist, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

drum *attr*...
BILD: figure-02-drum Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie einem "Mittelpunkt". Es verwaltet die Attribute

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above
siehe unten ...

below
BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"

above
"Anker Süd"

below
"Anker Nord"

aspect *number*
Legt das Seitenverhältnis, d. h. das Verhältnis von Breite zu Höhe, der Ellipsen fest, die zum Zeichnen des oberen und unteren Randes des Elements "**drum**" verwendet werden. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **drumaspect** entnommen wird, deren Standardwert wiederum **0.35** ist.

at *location*
Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

height *length*
siehe unten ...

ht *length*
Legt die Höhe des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxheight** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*

BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

width *length*
siehe unten ...

wid *length*
Legt die Breite des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxwidth** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

ellipse *attr...*

BILD: figure-02-ellipse Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, und einem "Mittelpunkt". Es unterstützt die folgenden Attribute

anchor *name*

siehe unten ...

ljust

siehe unten ...

rjust

siehe unten ...

above

siehe unten ...

below

BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust

"Anker West"

rjust

"Anker Ost"

above

"Anker Süd"

below

"Anker Nord"

at *location*

Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

color *spec*

BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

fillcolor *spec*

BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

height *length*
siehe unten ...

ht *length*
Legt die Höhe des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxheight** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert

zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

width *length*
siehe unten ...

wid *length*
Legt die Breite des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxwidth** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

line *attr...*
siehe unten ...

-- *attr...*
BILD: figure-02-line Ein offenes Element mit den entsprechenden Eckpunkten, d. h. "start", "end" und "center". Es unterstützt die folgenden Attribute

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above
siehe unten ...

below
BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"
above
"Anker Süd"
below
"Anker Nord"

arrowhead *spec*
BILD: figure-19-style-arrowheads Legt fest, wo am Element **line** Pfeilspitzen gezeichnet werden sollen: am Anfang, am Ende, an beiden Enden oder gar nicht. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **arrowhead** übernommen wird, deren Standardwert **keine** ist. Die zulässigen Werte sind
none, -
Zeichnen Sie an keinem der beiden Enden der Linie Pfeilspitzen ein.
start, first, <-
Zeichnen Sie am Anfang der Linie eine Pfeilspitze, jedoch nicht am Ende.
end, last, ->
Zeichnen Sie am Ende der Linie eine Pfeilspitze, jedoch nicht am Anfang.
both, <->
Zeichnen Sie an beiden Enden der Linie Pfeilspitzen.

Hinweis: Die Werte "start", "end", "-", "->", "<-" und "<->" werden alle als Kurzformen für den Befehl **arrowhead** akzeptiert und als Argumente verwendet.

at *location*
Line-Elemente werden normalerweise absolut positioniert, wobei die über die Attribute **from**, **then** und **to** festgelegten Positionen verwendet werden. Wenn **at** festgelegt wird, werden diese Positionen ein letztes Mal verschoben, wobei die durch das Attribut **with** benannte Ecke der Linie an die durch dieses Attribut angegebene Position verschoben wird.

chop ?*length*?
Legt die Länge des Elements "**line**" fest, die am Anfang und/oder am Ende entfernt werden soll. Standardmäßig ist kein Wert festgelegt. Wenn einmal angegeben, gilt die Kürzung sowohl für den Anfang als auch für das Ende der Zeile. Wenn zweimal oder mehrmalig angegeben, werden die letzten beiden Angaben verwendet und in dieser Reihenfolge auf den Anfang und das Ende der Zeile angewendet. Wird das Attribut ohne explizite Längenangabe festgelegt, greift das System auf den Wert zurück, der aus der Sprachvariablen **circleradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was

"keine Füllung" bedeutet.

from *location*
Legt die Position fest, an der das Element "**line**" beginnt. Standardmäßig ist dies die aktuelle Position, wie sie vom Layout-System verwaltet wird.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

noturn
Legt fest, dass die Richtung des Elements "**line**" an dessen Ende nicht an die Layoutverwaltung weitergegeben wird. Wird dies nicht angegeben, wird die Richtung der Linie zur neuen Richtung im Layout.

smooth
Legt die Verwendung von Bézier-Splines für das Element **line** fest. Wird dies nicht angegeben, werden Linien exakt durch die angegebenen Wegpunkte gezogen, ohne glatte Kurven.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text

angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

then *location*
siehe unten ...

then (?*length*?)...
siehe unten ...

(?*length*?)...
Dieses Attribut legt einen Zwischenstandort fest, den das Element "**line**" durchlaufen muss. Es kann mehrfach angegeben werden, wobei jede Angabe der Reihe einen weiteren Standort hinzufügt, den die Linie durchlaufen wird. Diese Standorte werden in der Reihenfolge durchlaufen, in der sie angegeben wurden.

Der Standort kann explizit angegeben werden oder als Reihe von Wegbeschreibungen mit Entfernungsangaben. Im letzteren Fall werden für den Richtungsteil die Namen aller bekannten Richtungen akzeptiert. Wird für eine Richtung keine Entfernung angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **movelength** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Die gesamte Menge der Richtungs-Distanz-Paare wird als eine Abfolge von Verschiebungen behandelt, die addiert werden, um die endgültige Verschiebung zu ergeben, die den Zwischenpunkt (relativ zum vorhergehenden Punkt) festlegt.

Der zuletzt genannte Name der Richtung wird als zu befolgende Richtung an das Layout-System weitergegeben. Die Verwendung von **noturn** kann dieses Verhalten nicht außer Kraft setzen.

Schließlich dienen die Namen der registrierten Richtungen auch als Attributbefehle, denen implizit das Attribut **then** vorangestellt wird.

Wenn für die Linie weder ein Zwischen- noch ein Endpunkt festgelegt werden, greift das System auf einen Punkt zurück, der **movelength** Pixel vom Startpunkt entfernt liegt, und zwar in der aktuellen Richtung, wie sie vom Layout-System verwaltet wird.

to *location*
Legt fest, welche Stelle das Element "**line**" erreicht. Dieses Attribut hat keinen Standardwert. Der Standardwert wird durch das Attribut **then** geregelt, wodurch der

Eindruck entsteht, dass **to** einen Standardwert hat, wenn dieser nicht angegeben wird.

with *corner*

Line-Elemente werden normalerweise absolut positioniert, wobei die über die Attribute **from**, **then** und **to** festgelegten Positionen verwendet werden. Wenn **at** festgelegt wird, werden diese Positionen ein letztes Mal verschoben, wobei die durch das Attribut **with** benannte Ecke der Linie an die durch dieses Attribut angegebene Position verschoben wird. Dies bedeutet, dass *with* genau dann wirksam ist, wenn für die Linie zusätzlich das Attribut **at** festgelegt wurde.

move *attr*

Ein offenes Element mit den entsprechenden Ecken, d. h. "start", "end" und "center". Ein **move**-Element ist im Wesentlichen ein unsichtbares **line**. Zwar interessiert uns in erster Linie die Auswirkung, die es auf das Layout-System hat, doch handelt es sich um ein tatsächliches Element, d. h., es verfügt über dieselben Eckpunkte wie eine gewöhnliche Linie und wird auch im Verlauf angezeigt, sodass später auf alle Plätze verwiesen werden kann, die es berührt hat. Es unterstützt dieselben Attribute wie **line**-Elemente.

spline *attr*...

BILD: figure-02-spline Ein Alias für das Element **line** (siehe oben), bei dem das Attribut **smooth** voreingestellt ist.

text *attr*...

BILD: figure-02-text Ein geschlossenes Element mit den entsprechenden Ecken, d. h. den acht Himmelsrichtungen der Windrose, sowie dem "Mittelpunkt". Es unterstützt die folgenden Attribute

anchor *name*

siehe unten ...

ljust

siehe unten ...

rjust

siehe unten ...

above

siehe unten ...

below

BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust

"Anker West"

rjust

"Anker Ost"

above
"Anker Süd"

below
"Anker Nord"

at *location*
Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

height *length*
Legt die Höhe des Elements "**text**" fest. Standardmäßig entspricht diese der natürlichen Höhe des darin enthaltenen Textes.

justify left|center|right
Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

width *length*
Legt die Breite des Elements "**text**" fest. Standardmäßig entspricht diese der natürlichen Breite des darin enthaltenen Textes.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

Attribute

Nachstehend ist die Gesamtheit aller Attribute aufgeführt, die von allen Elementbefehlen

unterstützt werden. Auch wenn wir sie als Befehle bezeichnen und eine Syntax dafür angeben, stehen sie nicht wirklich als eigenständige Befehle zur Verfügung, sondern lediglich als Teil der Argumente eines Elementbefehls.

Hinweis: Einige der Attributnamen sind überladen, d. h. sie weisen mehrere unterschiedliche Definitionen auf. Bei der Verarbeitung der Attribute eines Elements wird die tatsächlich verwendete Definition anhand des Typs des Elements ausgewählt, für das der Prozess erfolgt.

Darüber hinaus gilt als Auffangklause, dass jedes Attribut, das nicht gemäß den nachstehenden Definitionen verarbeitet werden konnte, als Argument eines impliziten **text**-Attributs behandelt wird.

anchor *name*
siehe unten ...

ljust
siehe unten ...

rjust
siehe unten ...

above
siehe unten ...

below
BILD: figure-22-text-anchoring-3 Legt den Anker des Textes fest, der in der Mitte des Elements platziert werden soll, anhand seines Namens. Das heißt, dieses Attribut definiert die Position des Textes relativ zur Mitte des Elements. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **anchor** entnommen wird, deren Standardwert wiederum **center** ist. Zulässige Werte sind alle, die von Tk_GetAnchor akzeptiert werden. Die Befehle ohne Argumente sind allesamt Kurzformen, bei denen der Anker impliziert ist. Beachten Sie, dass sie sich nicht kombinieren lassen; es wird nur der letzte Befehl verwendet. Für kombinierte Ausrichtungen muss der Hauptattributbefehl **anchor** verwendet werden.

ljust
"Anker West"

rjust
"Anker Ost"

above
"Anker Süd"

below
"Anker Nord"

arrowhead *spec*
BILD: figure-19-style-arrowheads Legt fest, wo am Element **line** Pfeilspitzen gezeichnet werden sollen: am Anfang, am Ende, an beiden Enden oder gar nicht. Wenn kein Wert festgelegt wird, greift das System auf den Wert der Sprachvariablen **arrowhead** zurück, deren Standardwert **keine** ist. Die zulässigen Werte sind

none, -
Zeichnen Sie an keinem der beiden Enden der Linie Pfeilspitzen ein.

start, first, <-
Zeichnen Sie am Anfang der Linie eine Pfeilspitze, jedoch nicht am Ende.

end, last, ->
Zeichnen Sie am Ende der Linie eine Pfeilspitze, jedoch nicht am Anfang.

both, <->
Zeichnen Sie an beiden Enden der Linie Pfeilspitzen.

Hinweis: Die Werte "start", "end", "-", "->", "<-" und "<->" werden alle als Kurzformen für den Befehl **arrowhead** akzeptiert und als Argumente verwendet.

aspect *number*
Legt das Seitenverhältnis, d. h. das Verhältnis von Breite zu Höhe, des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn keine Angabe erfolgt, hängt zudem von den Angaben der Attribute **width** und **height** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe von **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Vorrang hat. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** festgelegt wird und eines der Attribute **width** oder **height**, dann wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt wird, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, die ihrerseits standardmäßig auf **2** gesetzt ist.

Wenn keines der Attribute **width** oder **height** festgelegt wird, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

aspect *number*
Legt das Seitenverhältnis, d. h. das Verhältnis von Breite zu Höhe, der Ellipsen fest, die zum Zeichnen des oberen und unteren Randes des Elements "**drum**" verwendet werden. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **drumaspect** entnommen wird, deren Standardwert wiederum **0.35** ist.

at *location*
Legt die Position der Ecke des Elements fest, die durch das Attribut **with** benannt ist. Standardmäßig wird die aktuelle Position verwendet, wie sie vom Layout-System verwaltet wird.

at *location*
Line-Elemente werden normalerweise absolut positioniert, wobei die über die Attribute **from**, **then** und **to** festgelegten Positionen verwendet werden. Wenn **at** festgelegt wird, werden diese Positionen ein letztes Mal verschoben, wobei die durch das Attribut **with** benannte Ecke der Linie an die durch dieses Attribut angegebene Position verschoben wird.

chop ?*length*?
Legt die Länge des Elements "**line**" fest, die am Anfang und/oder am Ende entfernt werden soll. Standardmäßig ist kein Wert festgelegt. Wenn einmal angegeben, gilt die Kürzung sowohl

für den Anfang als auch für das Ende der Zeile. Wenn zweimal oder mehrmalig angegeben, werden die letzten beiden Angaben verwendet und in dieser Reihenfolge auf den Anfang und das Ende der Zeile angewendet. Wird das Attribut ohne explizite Längenangabe festgelegt, greift das System auf den Wert zurück, der aus der Sprachvariablen **circleradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

clockwise
siehe unten ...

cw
Legt die Richtung des Elements **arc** fest, in diesem Fall im Uhrzeigersinn. Das komplementäre Attribut lautet **counterclockwise**. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **clockwise** entnommen wird, deren Standardwert wiederum **false** ist, was einer Richtung gegen den Uhrzeigersinn entspricht.

color *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der die Linien des Elements gezeichnet werden. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linecolor** entnommen wird, deren Standardwert wiederum **black** ist.

counterclockwise
siehe unten ...

ccw
Legt die Richtung des Elements **arc** fest, in diesem Fall gegen den Uhrzeigersinn. Das komplementäre Attribut lautet **clockwise**. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **clockwise** entnommen wird, deren Standardwert wiederum **false** ist, was einer Richtung gegen den Uhrzeigersinn entspricht.

diameter *length*
siehe unten ...

diam *length*
Legt den Durchmesser des Elements "**circle**" fest und stellt damit eine alternative Möglichkeit zur Angabe seines Radius ("**radius**") dar. Gilt genau dann, wenn der Radius nicht angegeben wurde. Das heißt, wenn sowohl Durchmesser als auch Radius angegeben sind, hat die Angabe des Radius Vorrang. Dieses Attribut hat keinen Standardwert, da die Standardwerte aus dem Radius übernommen werden.

fillcolor *spec*
BILD: figure-21-style-colors Legt die Farbe fest, mit der der Innenbereich des Elements gezeichnet wird. Wird keine Angabe gemacht, greift das System auf den Wert der Sprachvariablen **fillcolor** zurück, deren Standardwert die leere Zeichenkette ist, was "keine Füllung" bedeutet.

from *location*
Legt die Position fest, an der das Element "**line**" beginnt. Standardmäßig ist dies die aktuelle Position, wie sie vom Layout-System verwaltet wird.

from *location*
Legt die Position fest, an der das Element "**arc**" beginnt. Standardmäßig ist dies die aktuelle Position, wie sie vom Layout-System verwaltet wird.

height *length*
siehe unten ...

ht *length*

Legt die Höhe des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxheight** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

height *length*

Legt die Höhe des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn dieser nicht angegeben ist, hängt auch von den Angaben der Attribute **aspect** und **width** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe für **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Angaben Vorrang haben. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** festgelegt wird und eines der Attribute **width** oder **height**, dann wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt wird, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, deren Standardwert wiederum **2** ist.

Wenn keines der Attribute **width** oder **height** festgelegt ist, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

height *length*

Legt die Höhe des Elements "**text**" fest. Standardmäßig entspricht diese der natürlichen Höhe des darin enthaltenen Textes.

justify left|center|right

Legt fest, wie mehrzeiliger Text, der dem Element zugeordnet ist, innerhalb seines Rahmens positioniert wird. Der Wert wird ignoriert, wenn für das Element kein Text angegeben wurde. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **justify** entnommen wird, deren Standardwert wiederum **left** ist. Die zulässigen Werte sind **left**, **right** und **center**.

noturn

Legt fest, dass die Richtung des Elements "**line**" an dessen Ende nicht an die Layoutverwaltung weitergegeben wird. Wird dies nicht angegeben, wird die Richtung der Linie zur neuen Richtung im Layout.

radius *length*

siehe unten ...

rad *length*

Legt den Radius des Elements **circle** fest. Wird dieser nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **circleradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

radius *length*
Legt den Radius des Elements **arc** fest, genauer gesagt den Radius des Kreises, zu dem der dargestellte Bogen gehört. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **arcradius** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **1 cm** entspricht.

slant *angle*
Legt den Winkel fest, um den das Element **box** geneigt ist, in Grad. Wenn kein Wert festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **slant** entnommen wird, deren Standardwert wiederum **90** ist, d. h. vertikal, ohne Neigung. 0 Grad entspricht einer Neigung direkt nach Osten, wobei die Spitze nach rechts zeigt. 90 Grad entsprechen einer Neigung nach Norden, wobei das Element senkrecht nach oben zeigt.

smooth
Legt die Verwendung von Bézier-Splines für das Element **line** fest. Wird dies nicht angegeben, werden Linien exakt durch die angegebenen Wegpunkte gezogen, ohne glatte Kurven.

stroke *width*
BILD: figure-20-style-stroke Legt die Breite der für das Element gezeichneten Linien in Pixeln fest. Wenn dieser Wert nicht festgelegt wird, greift das System auf den Wert zurück, der aus der Sprachvariablen **linewidth** entnommen wird, deren Standardwert wiederum **1** ist.

style *spec*
BILD: figure-18-style-dash Legt den Stil fest, der zum Zeichnen der Linien des Elements verwendet wird. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **linestyle** entnommen wird, deren Standardwert wiederum **solid** (durchgezogene Linien) ist. Zulässige Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

Hinweis: Die Werte "solid", "dot(ted)", "dash(ed)", "dash-dot" und "dash-dot-dot" werden alle als Kurzformen für den Befehl **style** akzeptiert, wenn sie als Argument verwendet werden.

text *string*
Legt den Text fest, der dem Element zugeordnet werden soll. Standardmäßig ist kein Text angegeben. Bei mehrfacher Angabe entspricht der tatsächlich angezeigte Text der Verkettung der einzelnen Zeichenfolgen, die vertikal übereinandergestapelt werden, wobei die zuerst angegebene Zeichenfolge ganz oben steht.

textcolor *spec*
Legt die Farbe fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück,

der aus der Sprachvariablen **textcolor** entnommen wird, deren Standardwert wiederum **black** ist.

textfont *spec*
Legt die Schriftart fest, mit der der Text eines Elements dargestellt wird. Wird ignoriert, wenn kein Text vorhanden ist. Wird dieser Wert nicht angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **textfont** entnommen wird, deren Standardwert wiederum **Helvetica 12pt** ist.

then *location*
siehe unten ...

then (?*length*?)...
siehe unten ...

(?*length*?)...
Dieses Attribut legt einen Zwischenstandort fest, den das Element "**line**" durchlaufen muss. Es kann mehrfach angegeben werden, wobei jede Angabe der Reihe einen weiteren Standort hinzufügt, den die Linie durchlaufen wird. Diese Standorte werden in der Reihenfolge durchlaufen, in der sie angegeben wurden.

Der Standort kann explizit angegeben werden oder als Reihe von Wegbeschreibungen mit Entfernungsangaben. Im letzteren Fall werden für den Richtungsteil die Namen aller bekannten Richtungen akzeptiert. Wird für eine Richtung keine Entfernung angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **movelength** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Die gesamte Menge der Richtungs-Distanz-Paare wird als eine Abfolge von Verschiebungen behandelt, die addiert werden, um die endgültige Verschiebung zu ergeben, die den Zwischenpunkt (relativ zum vorhergehenden Punkt) festlegt.

Der zuletzt genannte Name der Richtung wird als zu befolgende Richtung an das Layout-System weitergegeben. Die Verwendung von **noturn** kann dieses Verhalten nicht außer Kraft setzen.

Schließlich dienen die Namen der registrierten Richtungen auch als Attributbefehle, denen implizit das Attribut **then** vorangestellt wird.

Wenn für die Linie weder ein Zwischen- noch ein Endpunkt festgelegt werden, greift das System auf einen Punkt zurück, der **movelength** Pixel vom Startpunkt entfernt liegt, und zwar in der aktuellen Richtung, wie sie vom Layout-System verwaltet wird.

to *location*
Legt fest, welche Stelle das Element "**line**" erreicht. Dieses Attribut hat keinen Standardwert. Der Standardwert wird durch das Attribut **then** geregelt, wodurch der Eindruck entsteht, dass **to** einen Standardwert hat, wenn dieser nicht angegeben wird.

to *location*
Legt die Position fest, an der das Element "**arc**" endet. Standardmäßig wird eine Position gewählt, an der ein 90-Grad-Bogen in der ausgewählten Richtung gezogen wird, beginnend bei "**from**".

width *length*

siehe unten ...

wid *length*
Legt die Breite des Elements fest. Wird kein Wert angegeben, greift das System auf den Wert zurück, der aus der Sprachvariablen **boxwidth** entnommen wird, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht.

width *length*
Legt die Breite des Elements **diamond** fest. Die Art und Weise, wie ein Standardwert berechnet wird, wenn dieser nicht angegeben ist, hängt auch von den Angaben der Attribute **aspect** und **height** ab, sofern vorhanden.

Wenn sowohl **width** als auch **height** festgelegt sind, wird jede Angabe für **aspect** ignoriert, da diese implizit auch in der Breite / Höhe definiert ist und diese Vorrang hat. Eine fehlende Angabe wird in diesem Fall ebenfalls ignoriert, d. h., es sind keine Standardwerte erforderlich.

Wenn **aspect** sowie eines der Attribute **width** oder **height** angegeben sind, wird das fehlende Attribut aus den beiden angegebenen Attributen berechnet. Auch in diesen Fällen sind keine Standardwerte erforderlich.

Wenn nur eines der Attribute **width** oder **height** festgelegt ist, verwendet das System für **aspect** einen Fallback, wobei der Wert aus der Sprachvariablen **diamondaspect** entnommen wird, die ihrerseits standardmäßig auf **2** gesetzt ist.

Wenn keines der Attribute **width** oder **height** festgelegt ist, verwendet das System einen Fallback für **width**, den Wert aus der Sprachvariablen **boxwidth**, deren Standardwert wiederum dem Pixeläquivalent von **2 cm** entspricht. Für das Seitenverhältnis wird entweder der vom Benutzer angegebene Wert oder der wie im vorigen Absatz beschriebene Standardwert verwendet.

width *length*
Legt die Breite des Elements "**text**" fest. Standardmäßig entspricht diese der natürlichen Breite des darin enthaltenen Textes.

with *corner*
Legt die Ecke des Elements fest, die an der durch das Attribut **at** angegebenen Position platziert werden soll. Standardmäßig wird die aktuelle Ecke verwendet, wie sie vom Layout-System verwaltet wird, es sei denn, der Wert für **at** wurde vom Benutzer festgelegt. In diesem Fall wird standardmäßig der Wert **center** verwendet.

with *corner*
Line-Elemente werden normalerweise absolut positioniert, wobei die über die Attribute **from**, **then** und **to** festgelegten Positionen verwendet werden. Wenn **at** festgelegt wird, werden diese Positionen ein letztes Mal verschoben, wobei die durch das Attribut **with** benannte Ecke der Linie an die durch dieses Attribut angegebene Position verschoben wird. Dies bedeutet, dass *with* genau dann wirksam ist, wenn für die Linie zusätzlich das Attribut **at** festgelegt wurde.

Ecken

Ecken sind Namen für Werte in Elementen, in der Regel Positionen.

- Die *geschlossenen* Elemente definieren die Ecken der Windrose, einschließlich des "Zentrums", sowie deren "Breite" und "Höhe".

BILD: figure-27-corners-closed

- Die Elemente "**block**" exportieren zusätzlich alle Variablen, die bei ihren Definitionen als Eckpunkte festgelegt wurden.
- Die *open*-Elemente hingegen definieren "start", "end" und "center". Die ersten beiden werden den Positionen zugeordnet, die ursprünglich über die Attribute **from** und **to** des Elements angegeben wurden.

BILD: figure-28-corners-open

- Der Mittelpunkt der Elemente **line** und **move** befindet sich auf halbem Weg zwischen der "Start"- und der "End"-Ecke, und zwar unabhängig von etwaigen Zwischenpunkten, die das Element aufweisen mag.
- Die Elemente **line** und **move** bezeichnen zudem alle ihre Positionen als Ecken, wobei sie Nummern als Namen verwenden, beginnend mit **1** (entspricht "start"), in der Reihenfolge der Abtastung.

BILD: figure-15-spline-1

- Der Mittelpunkt der Elemente von **arc** ist der Mittelpunkt des Kreises, zu dem der Bogen gehört.
- Die Elemente **arc** definieren zudem die Kompassrose geschlossener Elemente.

Benannte Richtungen

Die genannten Richtungen sind Befehle, die dem Layout-System mitteilen, in welche Richtung es beim Platzieren des nächsten Elements ohne explizite Positionsangabe vorgehen soll. Sie können auch als Argumente für das Attribut **then** sowie für den Befehl **by** bei relativen Punkten verwendet werden; die entsprechende Syntax finden Sie dort.

Der Diagrammkern definiert die Richtungen der Windrose sowie eine Reihe von Aliasen. Die vollständige Liste finden Sie weiter unten.

BILD: figure-27-corners-closed

Dies überschneidet sich mit den vordefinierten Ecken für geschlossene Elemente. Das Layout-System nutzt dies folgendermaßen: Wenn Sie sich in X-Richtung bewegen, entspricht der Name der entgegengesetzten Richtung dem Namen der Ecke, an der das neue Element an die aktuelle Position angefügt wird; existiert diese Ecke nicht, wird die winkelmäßig nächstgelegene tatsächliche Ecke verwendet.

- **west**
- **w**
- **left**
- **l**
- **south**
- **s**
- **down**
- **bottom**
- **bot**
- **b**
- **east**

- **e**
- **right**
- **r**
- **north**
- **n**
- **up**
- **top**
- **t**
- **northwest**
- **nw**
- **up-left**
- **upleft**
- **leftup**
- **northeast**
- **ne**
- **up-right**
- **upright**
- **rightup**
- **southwest**
- **sw**
- **down-left**
- **downleft**
- **leftdown**
- **southeast**
- **se**
- **down-right**
- **downright**
- **rightdown**

Verschiedene Befehle

***number* cm**
siehe unten ...

***number* mm**
siehe unten ...

***number* inch**
siehe unten ...

***number* pt**
Mit diesen Befehlen können Abstände und Koordinaten in metrischen und imperialen Einheiten angegeben werden; als Ergebnis gibt der Befehl den entsprechenden Abstand bzw. die entsprechende Koordinate in Pixeln zurück, da dies die Einheit ist, die intern für alle Berechnungen verwendet wird.

Die Umrechnungsfaktoren basieren auf dem Ergebnis von **tk scaling** und werden einmalig zum Zeitpunkt des Einladens des Pakets berechnet; spätere Änderungen des Faktors **tk scaling** haben keine Auswirkungen.

number number

BILD: figure-50-point-cons-absolute

Dieser Befehl nimmt die x- und y-Koordinaten eines Ortes entgegen und gibt den *absoluten* Punkt dafür zurück.

by *distance direction*

BILD: figure-51-point-cons-relative

Dieser Befehl nimmt *distance* und *direction* entgegen (Winkel in Grad oder registrierter Richtungsname) und gibt den *relativen* Punkt dafür zurück, d. h. das *Delta* oder die *Verschiebung*, die er darstellt.

Beachten Sie auch die (Un-)Ähnlichkeiten zu den Richtungsangaben für das Attribut **then** der Elemente **line** und **move**. Wo wir hier

```
by 50 east
```

Für das Attribut sagen wir

```
... then east 50 ...
```

oder einfach

```
... then east ...
```

point1 + point2

BILD: figure-48-point-vectoradd

Dieser Befehl interpretiert zwei Punkte als Vektoren und addiert sie. Wenn mindestens einer der Punkte *absolut* ist, ist auch das Ergebnis absolut. Das Ergebnis ist genau dann ein *relativer* Punkt, wenn beide Punkte *relativ* sind.

point1 - point2

BILD: figure-49-point-vectorsub

Dieser Befehl interpretiert zwei Punkte als Vektoren und subtrahiert den zweiten vom ersten. Ist mindestens einer der Punkte *absolut*, so ist auch das Ergebnis absolut. Das Ergebnis ist genau dann ein *relativer* Punkt, wenn beide Punkte *relativ* sind.

point* by *distance direction

Dieser Befehl ist eine bequemere oder zumindest kürzere Form von

```
    [$point + [by $distance $direction]]
```

point1* | *point2

BILD: figure-31-point-projection

Dieser Befehl berechnet die *Projektion* zweier Punkte, d. h., das Ergebnis ist der Punkt mit der x-Koordinate von *point1* und der y-Koordinate von *point2*.

n* between *poin1 point2

BILD: figure-29-point-interpolation-1

Dieser Befehl berechnet den Punkt, der sich zu *n**100 Prozent auf dem Weg zwischen *point1*

und *point2* befindet, und gibt ihn als Ergebnis zurück. Das bedeutet, dass für

***n* == 0**
Das Ergebnis lautet *point1*.

***n* == 1**
Das Ergebnis ist *point2*.

***n* == 0.5**
Das Ergebnis liegt auf halbem Weg zwischen den beiden Punkten.

usw. *Hinweis*: Für *n* sind Werte < 0 und > 1 zulässig

intersect *elem1 elem2*
BILD: figure-32-point-intersection

Dieser Befehl nimmt zwei *offene* Elemente entgegen, berechnet die Geraden, die durch deren "Start"- und "End"--corners verlaufen, und gibt den Schnittpunkt dieser beiden Geraden zurück. Der Befehl löst einen Fehler aus, wenn sich die Geraden nicht schneiden oder zusammenfallen.

***element* names ?*pattern*?**
Dieser Befehl gibt eine Liste zurück, die die Namen aller Ecken des *element* enthält. Wird ein Muster festgelegt, werden nur die Namen zurückgegeben, die diesem Muster entsprechen (über **string match**). Andernfalls werden alle Namen zurückgegeben (entspricht einem Standardmuster von *).

element corner
Dieser Befehl gibt den Wert für die *corner* des *element* zurück. Dies kann beliebig sein, einschließlich Punkten und Elementen.

element corner1 corner2...
Dies ist eine praktische Kurzform für

```
[[[$elem $corner1] $corner2] ...]
```

unter der Annahme, dass der Wert für

```
 [$elem $corner1]
```

.IP usw. ist wiederum ein Element.

***element* ?*corner1*... ?names ?*pattern*??]?**
Dies ist eine praktische Kurzform für

```
[[[$elem $corner1] ...] names ?pattern?]
```

unter der Annahme, dass der Wert für

```
 [$elem $corner1]
```

.IP usw. ist wiederum ein Element.

nth ?*corner*?
Dieser Befehl fragt den Diagrammverlauf nach dem **n**. erstellten Element ab, wobei er vom Anfang des Verlaufs (gezählt ab 1) aus sucht, und gibt dieses als Ergebnis zurück. Wenn die

corner festgelegt wird, wird stattdessen der Wert für diese Ecke zurückgegeben.

nth last ?*corner*?
Dieser Befehl fragt den Diagrammverlauf nach dem **n**. erstellten Element ab, wobei die Suche am Ende des Verlaufs beginnt, und gibt dieses als Ergebnis zurück. Wenn die *corner* festgelegt wird, gibt der Befehl stattdessen den Wert für diese Ecke zurück.

nth *shape* ?*corner*?
Dieser Befehl fragt den Diagrammverlauf nach dem **n**. erstellten Element der angegebenen *shape* ab, wobei die Suche am Anfang des Verlaufs (ab 1) beginnt, und gibt dieses als Ergebnis zurück. Wenn die *corner* festgelegt wird, wird stattdessen der Wert für diese Ecke zurückgegeben.

nth last *shape* ?*corner*?
Dieser Befehl fragt den Diagrammverlauf nach dem **n**. Element der angegebenen *shape* ab, wobei die Suche am Ende des Verlaufs beginnt, und gibt dieses als Ergebnis zurück. Wenn die *corner* festgelegt wird, gibt der Befehl stattdessen den Wert für diese Ecke zurück.

last ?*corner*?
siehe unten ...

last *shape* ?*corner*?
Die Convenience-Befehle erfordern eine Zuordnung zu "**1st last**" und "**1st last** *shape*".

1st
siehe unten ...

2nd
siehe unten ...

3rd
Aliase für **1th**, **2th** und **3th**, die der besseren Lesbarkeit dienen und überall dort verwendet werden können, wo **n** vorkommen kann.

Variablen

Der Sprachkontext enthält eine Reihe vordefinierter Variablen, die die Standardwerte für verschiedene Attribute enthalten. Diese Variablen, ihre Verwendungszwecke und ihre Werte lauten wie folgt:

anchor
Der Standardwert für das Attribut **anchor**. Initialisiert auf **center**. Die zulässigen Werte sind alle Werte, die von Tk_GetAnchor akzeptiert werden.

arcradius
Der Standardwert für das Attribut **radius** der Elemente **arc**. Wird auf den Pixelwert initialisiert, der dem Wert von **1 cm** entspricht.

arrowhead
Der Standardwert für das Attribut **arrowhead**. Initialisiert auf **none**. Die zulässigen Werte sind

none, -
Zeichnen Sie an keinem der beiden Enden der Linie Pfeilspitzen ein.

start, first, <-
Zeichnen Sie am Anfang der Linie eine Pfeilspitze, jedoch nicht am Ende.

end, last, ->

Zeichnen Sie am Ende der Linie eine Pfeilspitze, jedoch nicht am Anfang.

both, <->
Zeichnen Sie an beiden Enden der Linie Pfeilspitzen.

boxheight
Der Standardwert für das Attribut **height** der Elemente **box**, **diamond** und **ellipse**. Wird auf den Pixelwert initialisiert, der dem Wert von **2 cm** entspricht.

boxwidth
Der Standardwert für das Attribut **width** der Elemente **box**, **diamond** und **ellipse**. Wird auf den Pixelwert initialisiert, der dem Wert von **2 cm** entspricht.

clockwise
Der Standardwert für die Attribute **clockwise** und **counterclockwise** der Elemente **arc**. Initialisiert auf **False** für die Richtung gegen den Uhrzeigersinn.

circleradius
Der Standardwert für das Attribut **radius** der Elemente **circle** sowie der Standardwert für das Attribut **chop**, wenn diese ohne explizite Längenangabe angegeben werden. Wird auf den Pixelwert initialisiert, der dem Wert von **1 cm** entspricht.

drumaspect
Der Standardwert für das Attribut **aspect** der Elemente **drum**. Initialisiert auf **0.35**.

fillcolor
Der Standardwert für das Attribut **fillcolor** aller Elemente, die ausgefüllt werden können. Er wird auf die leere Zeichenkette initialisiert und signalisiert damit, dass das Element nicht ausgefüllt ist.

justify
Der Standardwert für das Attribut **justify**. Initialisiert auf **left**. Die zulässigen Werte sind **left**, **right** und **center**.

linecolor
Der Standardwert für das Attribut **color** aller Elemente, die Linien zeichnen müssen (alle außer **text**). Initialisiert auf **black**.

linestyle
Der Standardwert für das Attribut **style** aller Elemente, bei denen eine Linie gezeichnet werden muss. Initialisiert auf **solid**. Die zulässigen Werte sind alle Werte, die von Tk_GetDash akzeptiert werden, sowie zusätzlich alle unten aufgeführten Werte:

solid, leere Zeichenkette
Zeichnen Sie eine durchgezogene Linie.

dash, dashed, -
Zeichnen Sie eine gestrichelte Linie.

dot, dotted,.
Zeichnen Sie eine gepunktete Linie.

dash-dot, -.
Zeichnen Sie eine gestrichelte Linie

dash-dot-dot, -..
Zeichnen Sie eine gestrichelte Linie.

linewidth

Der Standardwert für das Attribut **stroke** aller Elemente, bei denen eine Linie gezeichnet werden muss. Initialisiert auf **1** (Pixel).

movelength
Der Standardwert für die Richtungsangabe von Zwischenstandorten durch das Attribut **then** der Elemente **line** und **move**. Wird auf den Pixelwert initialisiert, der dem Wert von **2 cm** entspricht.

slant
Der Standardwert für das Attribut **slant** der Elemente **box**. Initialisiert auf 90 Grad, d. h. senkrecht nach oben geneigt.

textcolor
Der Standardwert für das Attribut **textcolor** aller Elemente, die Text darstellen müssen. Initialisiert auf **black**.

textfont
Der Standardwert für das Attribut **textfont** aller Elemente, die Text darstellen müssen. Initialisiert auf **Helvetica 12pt**.

Diagrammklassen

Dieser Abschnitt richtet sich an Entwickler, die an den internen Abläufen des Diagramm-Pakets arbeiten möchten. Regelmäßige Nutzer von **diagram** können diesen Abschnitt überspringen, ohne etwas zu verpassen.

Die wichtigste Information, die Sie hier sehen, ist die nachstehende Abbildung, die die Klassenhierarchie im Implementierungsdiagramm darstellt.

BILD: figure-00-dependencies

Ganz unten, alle auf derselben Ebene, befinden sich die unterstützenden Pakete wie **snit** usw. Diese sind alle in Tcllib zu finden.

Darüber befindet sich die Gruppe der Diagrammklassen, die die verschiedenen Aspekte des Systems implementieren, d. h.:

diagram
Die Hauptklasse, die für den Benutzer sichtbar ist.

diagram::core
Die Kern-Engine, die selbst auf vier Hilfsklassen verteilt ist.

diagram::basic
Die Implementierung der Standards wie Rechteck, Kreis usw. basiert auf den Erweiterungen des Kerns.

diagram::element
Kernklasse zur Unterstützung, die Datenbank der erstellten Elemente. Sie verwaltet zudem den Verlauf, d. h. die Reihenfolge, in der die Elemente erstellt wurden.

diagram::attribute
Kernunterstützungsklasse, die generische Handhabung der Definition und Verarbeitung von Attributen.

diagram::direction
Kernunterstützungsklasse, die Datenbank der benannten Richtungen.

diagram::navigation
Kernunterstützungsklasse, der Status der Layout-Engine, d. h. aktuelle Position und Richtung,

sowie Operationen darauf.

diagram::point
Allgemeine Klasse zur Abwicklung verschiedener Vektoroperationen.

Literatur

Fehler, Ideen, Rückmeldungen
Dieses Dokument sowie das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *diagram* des Tklib Trackers. Bitte teilen Sie uns auch Ihre Ideen für Verbesserungen mit, die Sie möglicherweise für eines der beiden Pakete und/oder die Dokumentation haben.

Kategorie
Dokumentationswerkzeuge

getstring

tk_getString
Stichwörter: tk_getString, dialog, entry, string

Name
getstring – Ein Dialogfeld für Zeichenfolgen

Übersicht
package require Tcl 8.4
package require getstring ?**0.1**?
::getstring::tk_getString *pathName variable text* ?*options*?

Beschreibung
Dieses Paket stellt ein Dialogfeld bereit, das aus einem Entry sowie den Schaltflächen OK und Abbrechen besteht.

::getstring::tk_getString *pathName variable text* ?options?
Erstellt ein Dialogfeld, das den Benutzer mit *text* zur Eingabe einer Zeichenkette auffordert. Der Inhalt des Entry wird beim Schließen des Dialogs in die *variable* geschrieben. Der Befehl gibt einen booleschen Wert zurück, der angibt, ob der Benutzer OK gedrückt hat. Wenn -geometry nicht angegeben ist, wird der Dialog im übergeordneten Toplevel zentriert, außer das übergeordnete Fenster ist.; dann wird der Dialog auf dem Bildschirm zentriert. Optionen: -title -allowempty, ein boolesches Argument, das angibt, ob der Dialog eine leere Eingabe akzeptiert. -entryoptions gibt seine Argumente an das Entry-Widget weiter. Das ist nützlich für zusätzliche Prüfungen über die Validierungs-Hooks des Entry-Widgets. -geometry legt die Geometrie des Fensters fest.

Beispiel

```
package require getstring
namespace import getstring::*

if {[tk_getString .gs text "Feed me a string please:"]} {
    puts "user entered: $text"
}
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *getstring* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

history

tklib_history

Stichwörter: tklib_history, entry, history

Name

history – Verlauf für Entry-Widgets

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require history ?**0.3**?
::history::init *pathName* ?*length*?
::history::remove *pathName*
::history::add *pathName text*
::history::get *pathName*
::history::clear *pathName*
::history::configure *pathName option* ?*value*?

bell

Beschreibung

Dieses Paket stellt einen Verlauf für Entry-Widgets bereit. Der Verlauf ist über die Pfeiltasten nach oben und unten erreichbar.

::history::init *pathName* ?length?
: Merkt sich den Verlauf des genannten Entry-Widgets. Die optionale Länge legt fest, wie viele Verlaufseinträge behalten werden. Das lässt sich später mit **::history::configure** ändern. Verlaufseinträge müssen mit **::history::add** hinzugefügt werden, bevor sie sichtbar sind.

::history::remove *pathName*
: Vergisst alle Verlaufseinträge des Entry *pathName* und entfernt die Verlaufsbindungen.

::history::add *pathName text*
: Fügt Verlaufseinträge zu einem Entry hinzu, für das zuvor **::history::init** aufgerufen wurde. Der Befehl sollte aus dem Handler des Entry mit dem Inhalt des Entry aufgerufen werden, oder mit dem, was in den Verlauf soll.

::history::get *pathName*

Gibt eine Liste der Verlaufseinträge des Entry *pathName* zurück.

::history::clear *pathName*
Löscht die Verlaufsliste des genannten Entry.

::history::configure *pathName option* ?value?
Fragt Konfigurationsoptionen ab oder setzt sie. Erkannt werden *length* und *alert*. *length* bestimmt, wie viele Verlaufseinträge für das genannte Entry behalten werden. *alert* ist der Befehl, der läuft, wenn der Benutzer das Ende des Verlaufs erreicht. Die Vorgabe ist

bell

. **configure** verlangt ein Argument *pathName*, die Einstellung von alert ist aber global, und der Pfad wird ignoriert.

```
entry .e
bind .e <Return> [list ProcessEntry %W]
::history::init .e
pack .e

proc ProcessEntry {w} {
    set text [$w get]
    if {$text == ""} { return }
    ::history::add $w $text
    puts $text
    $w delete 0 end
}
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *entry::history* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

ico

ico

Stichwörter: ico, dll, entry, exe, icon

Name

ico – Windows-Symbole lesen und schreiben

Übersicht

package require Tcl 8.4
package require ico ?**1.1.3**?
::ico::icons *file* ?*option value...*?
::ico::iconMembers *file name* ?*option value...*?
::ico::getIcon *file name* ?*option value...*?
::ico::getIconByName *file name* ?*option value...*?
::ico::getFileIcon *file* ?*option value...*?
::ico::writeIcon *file name depth data* ?*option value...*?
::ico::copyIcon *file index file2 index2* ?*option value...*?
::ico::EXEtoICO *file* ?*dir*?

::ico::clearCache ?*file*?
::ico::transparentColor *image color*
::ico::Show *file* ?*option value...*?

Beschreibung

Dieses Paket bietet Funktionen zum Lesen und Schreiben von Windows-Symbolen aus ICO-, EXE-, DLL-, ICL- und BMP-Dateien. Im Sinne dieses Moduls ist ein Symbol eine visuelle Darstellung eines Objekts. Ein Symbol besteht aus einem oder mehreren Bildern, die in der Regel unterschiedliche Auflösungen und Farbtiefen aufweisen. Jedes Symbol und jedes Bild verfügt über eine Ressourcen-ID, bei der es sich um eine Textzeichenfolge oder einen positiven ganzzahligen Wert handeln kann. Die meisten Befehle verwenden diese ID, um festzulegen, auf welches Symbol oder Bild sie angewendet werden sollen.

API

::ico::icons *file* ?*option value*...?
Gibt eine Liste der Symbole in *file* zurück. Jedes Element ist der Name oder die numerische ID. Erkennt diese Optionen:
- **-type** fileFormat

::ico::iconMembers *file name* ?*option value*...?
Gibt eine Liste der Bilder zurück, aus denen das Symbol mit der ID *name* besteht. Jedes Element ist selbst eine Unterliste der Form {name width height bpp}. Erkennt diese Optionen:
- **-type** fileFormat

::ico::getIcon *file name* ?*option value*...?
Extrahiert das Symbol mit der ID *name* aus *file*. Voreingestellt ist **-format image** und der Befehl gibt den Namen eines Tk-Bildes mit dem Symbol zurück. Auflösung und Farbtiefe wählen ?-res?, ?-bpp? und ?-exact?. Ist -exact angegeben und gibt es keine genaue Entsprechung, entsteht ein Fehler. Mit **-image** lässt sich der Name des erzeugten Tk-Bildes angeben. Ist **-format colors**, kommt eine Liste von Farbnamen im Format #RRGGBB zurück. Jedes Listenelement ist eine waagerechte Zeile. Jede Zeile enthält die Farben aller Pixel dieser Zeile von links nach rechts. Ist **-format name**, kommt der Ressourcenname des gewählten Bildes zurück. Das ist nützlich für writeIcon oder getIconByName. Erkennt diese *option*.
- **-type** fileFormat
- **-format** value
- **-image** value
- **-res** value
- **-bpp** value
- **-exact** value

::ico::getIconByName *file name* ?*option value*...?
Extrahiert das Bild mit der ID *name* aus *file*. Dieser Name sollte der Name eines bestimmten Bildes sein, wie er von **::ico::iconMembers** gegeben wird, und nicht der Name eines Symbols, der von **::ico::icons** gegeben wird. Wenn in *file* keine passende Ressourcen-ID vorhanden ist, wird ein Fehler ausgelöst. Die folgenden Optionen werden unterstützt:
- **-type** fileFormat
- **-format** value

::ico::getFileIcon *file* ?*option value*...?
Dieser Befehl arbeitet nur unter Windows. Er liest die Windows-Registrierung, um das Anzeigesymbol für *file* zu bestimmen, wie es im Explorer oder einem ähnlichen Programm erschiene. *file* muss nicht existieren und kann auch eine Dateiendung mit führendem Punkt sein. Ist *file* ein Verzeichnis oder heißt es **Folder**, kommt das Ordnersymbol zurück. Argumente und Gebrauch sind dieselben wie bei **getIcon**:
- **-format** value
- **-image** value
- **-res** value
- **-bpp** value
- **-exact** value

::ico::writeIcon *file name depth data* ?*option value*...?
Schreibt ein Bild nach *file*. *name* ist der Ressourcenbezeichner des Bildes in *file*, das geschrieben wird. In eine EXE-, DLL- oder ICL-Datei lassen sich vorhandene Symbole nur mit einem Symbol derselben Abmessungen und Farbtiefe überschreiben. Diesen Dateitypen lassen sich keine Symbole hinzufügen.

Beim Schreiben nach BMP wird der Name ignoriert, weil dieser Typ nur ein Bild enthalten kann. Existiert die Datei schon, wird sie vollständig überschrieben.

Beim Schreiben nach ICO oder ICODATA wird ein Bild angehängt und der nächste Name in der Folge vergeben, wenn der angegebene Name nicht existiert. Der angegebene Name wird dann ignoriert. Bilder in ICO- und ICODATA-Dateien lassen sich mit anderen Abmessungen oder Farbtiefen überschreiben. Wer nicht jedes Bild ändert, aus dem ein Symbol besteht, bekommt beim Anzeigen merkwürdige Ergebnisse.

integer *depth* (in)
Dieses Argument muss **1**, **4**, **8**, **24** oder **32** sein. Hat *data* mehr Farben, als die Farbtiefe erlaubt, entsteht ein Fehler.

options *data* (in)
Dieses Argument ist entweder eine Farbliste in der Form von **::ico::getIcon -format colors** oder der Name eines Tk-Bildes.

Erkennt diese *option*.
- **-type** fileFormat

::ico::copyIcon *file index file2 index2* ?*option value*...?
Kopiert das Symbol bei *index* in *file* nach *index2* in *file2*.
- **-fromtype** fileFormat
- **-totype** fileFormat

::ico::EXEtoICO *file* ?dir?
Extrahiert alle Symbole aus der ausführbaren Datei *file* in ICO-Dateien in *dir*. ?dir? ist voreingestellt das Verzeichnis, in dem *file* liegt. Die Symboldateien heißen *file*-ID.ico, wobei ID der Ressourcenbezeichner des Symbols ist.
- **-type** fileFormat

::ico::clearCache ?file?
Der Befehl **::ico::getIconList** speichert Symbolversätze in EXE-, DLL-, ICL- und ICO-Dateien,

damit das Extrahieren schneller geht. Dieser Befehl leert den Speicher für die angegebene ?file? oder für alle Dateien.

::ico::transparentColor *image color*

Wenn *image* aus einem einzigen Wort besteht, wird davon ausgegangen, dass es sich um den Namen eines Tk-Bildes handelt. Alle Pixel im *image*, die der Farbe *color* entsprechen, werden mit Transparenz gesetzt. Alternativ kann *image* auch eine Farbliste sein; in diesem Fall wird eine modifizierte Liste zurückgegeben.

::ico::Show *file* ?*option value*...?

Befehl auf Anwendungsebene. Zeigt ein Fenster mit allen Symbolen in *file* und ihrem Namen.
- **-type** fileFormat
- **-parent** pathName

Beispiel

```
button .explore -image [::ico::getIcon explorer.exe 0 -name explore -res 16
-bpp 8]

set i [lsearch -inline [::ico::iconMembers tclkit.exe 0] {* 32 32 8}]
set colorlist [::ico::getIconByName tclkit.exe [lindex $i 0] -format colors]
```

Einschränkungen

Anderen Dateitypen als ICO lassen sich keine Symbole hinzufügen oder entnehmen. Symbole in diesen Dateien lassen sich nur durch Symbole derselben Abmessungen und Farbtiefe ersetzen.

Symbole mit 8bpp oder weniger müssen Schwarz in der Palette haben. Enthält das Symbol kein Schwarz, muss eine Farbe frei bleiben, damit writeIcon sie aufnimmt.

Derzeit gibt es keine Möglichkeit, Alphakanal-Informationen aus 32-bpp-Symbolen auszulesen.

Tk-Bilder haben keinen Alphakanal. Ein echtes 32bpp-Symbol lässt sich deshalb nur aus einer Farbliste schreiben. Ein 32bpp-Symbol aus einem Tk-Bild zu schreiben ist dasselbe wie ein 24bpp-Symbol.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *ico* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

ipentry

ipentry

Stichwörter: ipentry, entry, ip address, network

Name

ipentry – Ein Widget zur Eingabe von IP-Adressen

Übersicht

package require Tcl 8.4
package require Tk 8.4

package require ipentry ?**0.3.2**?
::ipentry::ipentry *pathName* ?*option value...*?
::ipentry::ipentry6 *pathName* ?*option value...*?
pathName **complete**
pathName **get**
pathName **insert** *iplist*
pathName **icursor** *index*
pathName **configure** *option value...*
pathName **cget** *option*

Beschreibung

Dieses Paket enthält ein Widget zur Eingabe einer IP-Adresse.

::ipentry::ipentry *pathName* ?*option value*...?
Erzeugt ein neues IPv4-ipentry-Widget und konfiguriert es mit den angegebenen Optionen und ihren Werten.

::ipentry::ipentry6 *pathName* ?*option value*...?
Erzeugt ein neues ipentry-Widget zur Eingabe einer IPv6-Adresse. Alle Optionen sind dieselben wie beim IPv4-Widget.

Jedes so erzeugte Widget unterstützt die folgenden Methoden:

***pathName* complete**
Gibt einen booleschen Wert zurück. true bedeutet, dass das Entry eine vollständige IP-Adresse enthält, also jedes Feld einen Wert hat. In manchen Fällen ist eine IPv6-Adresse auch gültig, wenn Felder fehlen. Das ist gesondert zu prüfen.

***pathName* get**
Gibt den Inhalt des Entry als Liste aus 4 oder 8 Elementen zurück.

pathName* insert *iplist
IPv4 übernimmt eine Liste von 4 Elementen und fügt der Reihe nach je eines in jedes Quad des Entry ein. Alle Werte der Liste müssen leer oder ganze Zahlen sein. Werte außerhalb des Bereichs 0 bis 255 werden in den Bereich verschoben. IPv6 übernimmt eine Liste von 8 Elementen und fügt der Reihe nach je eines in jedes Quad des Entry ein. Alle Werte der Liste müssen leer oder 1 bis 4 Hexadezimalziffern sein.

pathName* icursor *index
Setzt die Position des Einfügecursors des Widgets. Für ipentry gelten nur ganze Zahlen von 0 bis 15, für ipentry6 von 0 bis 31. Das Setzen des icursor wirkt nur, wenn das Widget bereits den Eingabefokus hat.

***pathName* configure *option value*...**
Ändert die Konfiguration des Widgets. Zu den Optionen und ihrer Bedeutung siehe den Abschnitt **WIDGET-OPTIONEN**.

pathName* cget *option
Gibt zur angegebenen Option die aktuelle Konfiguration des Widgets zurück. Zu den Optionen und ihrer Bedeutung siehe den Abschnitt **WIDGET-OPTIONEN**.

Widget-Optionen

-textvariable (textvariable/Variable)
Der Name einer Variablen, die den Wert der IP-Adresse enthält. Der Wert muss eine

Zeichenkette in der Form NNN.NNN.NNN.NNN für IPv4 oder HHHH:HHHH:HHHH:HHHH:HHHH:HHHH:HHHH:HHHH für IPv6 sein, wobei H für eine Hexadezimalziffer steht. Die Variable wird so angepasst, dass sie eine gültige IP-Adresse darstellt, wenn dies nicht bereits der Fall ist.

-state (state/State)
Gibt einen von drei Zuständen des Entry an: **normal**, **disabled** oder **readonly**.

-font (font/Font)
siehe unten ...

-bd (borderWidth/BorderWidth)
siehe unten ...

-fg (foreground/Foreground)
siehe unten ...

-bg (background/Background)
siehe unten ...

-relief (relief/Relief)
siehe unten ...

-highlightthickness (highlightThickness/HighlightThickness)
siehe unten ...

-highlightcolor (highlightColor/HighlightColor)
siehe unten ...

-highlightbackground (highlightBackground/HighlightBackground)
siehe unten ...

-selectbackground (selectBackground/Background)
siehe unten ...

-selectforeground (selectForeground/Foreground)
siehe unten ...

-selectborderwidth (selectBorderWidth/BorderWidth)
siehe unten ...

-disabledbackground (disabledBackground/DisabledBackground)
siehe unten ...

-disabledforeground (disabledForeground/DisabledForeground)
siehe unten ...

-readonlybackground (readonlyBackground/ReadonlyBackground)
siehe unten ...

-insertbackground (insertBackground/Background)
Standard-Optionen für Widgets. Eine Beschreibung ihrer Bedeutungen und Werte finden Sie unter **options**.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *ipentry* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

khim

khim

Stichwörter: khim, character, i18n, input, international, method

Name

khim – Bietet Bindungen zur Eingabe internationaler Zeichen auf einer Tastatur, die diese nicht unterstützt

Übersicht

package require Tcl
package require khim ?**1.0.3**?
::khim::getOptions *path*
::khim::getConfig
::khim::setConfig *version enabled compose map*
::khim::showHelp *path*

Beschreibung

Dieses Paket stellt Tastenbindungen bereit, mit denen sich beliebige Zeichen auf einer Tastatur eingeben lassen, die sie nicht anbietet. Dafür wird eine selten benutzte Taste zur Compose-Taste bestimmt. Diese Taste ist eine andere als eine Taste, die bereits so beschriftet ist, und ist oft Pause, F12 oder L2. Die Compose-Taste und danach eine Folge aus zwei Tasten fügt ein Zeichen in ein Widget ein. Zweimal hintereinander angeschlagen öffnet die Compose-Taste einen Dialog mit einer Unicode-Zeichentabelle, über den sich beliebige Zeichen einfügen lassen.

Der weitaus größte Teil des Pakets steckt in einem einzigen Bindtag, **KHIM**. Eine Anwendung lässt ein Text- oder Entry-Widget beliebige Zeichen aufnehmen, indem sie diesen Bindtag vor den Bindtag **Text** oder **Entry** des Widgets setzt:

```
text .t -width 80 -height 24
bindtags .t {.t KHIM Text . all}
```

Der Tag **KHIM** muss vor der Klassenbindung **Text** oder **Entry** stehen, sonst arbeitet das Paket nicht richtig.

Prozeduren

Neben den Befehlen zum Bindtag KHIM exportiert das Paket die folgenden Befehle:

::khim::getOptions *path*
: Öffnet einen modalen Toplevel-Dialog mit dem Pfadnamen *path*, der nach den KHIM-Optionen fragt. Die Tastenfolgen für die Compose-Taste lassen sich neu festlegen, die Taste für die Compose-Funktion wechseln, und die KHIM-Tastenbindungen ganz ein- oder ausschalten.

::khim::getConfig
: Gibt ein Tcl-Skript zurück, das die aktuelle KHIM-Konfiguration wiederherstellt: den Zustand ein oder aus, die gewählte Compose-Taste und die Tastenfolgen, die sich zusammensetzen lassen. Das Skript ist dazu gedacht, in einer Konfigurationsdatei zu stehen und beim nächsten Start derselben Anwendung zu gelten:

```
# Save KHIM configuration
set f [open ~/.khimrc w]
puts $f [::khim::getConfig]
close $f

# Restore KHIM configuration
source ~/.khimrc
```

::khim::setConfig *version enabled compose map*
Stellt eine früher gespeicherte Konfiguration wieder her. Kaum eine Anwendung ruft diesen Befehl anders auf, als das von **::khim::getConfig** gelieferte Skript auszuwerten.

::khim::showHelp *path*
Zeigt einen Toplevel-Dialog mit der Hilfe zu KHIM für den Benutzer. Der Dialog hat den Pfadnamen *path*.

Lokalisierung

Wer KHIM in einer nicht englischsprachigen Locale anbieten will, legt eine **.msg**-Datei mit dem passenden Text an. Der Katalog braucht die folgenden Meldungen:

Apply
Text der Schaltfläche Apply im Dialog, der die KHIM-Optionen setzt.

Cancel
Text der Schaltfläche Cancel in mehreren Dialogen.

Change
Text der Schaltfläche Change. Sie ändert die Bindung eines Paares zusammengesetzter Zeichen, legt sie an oder ersetzt sie.

Character
Text der Beschriftung des Entry-Widgets, das das Zeichen aus einer Compose-Folge aufnimmt.

{Compose Key}
Fenstertitel des Dialogs, der zum Anschlagen der Taste auffordert, die die Compose-Taste wird.

{Compose key:}
Beschriftung der Komponente, die im Dialog der KHIM-Optionen die gewählte Compose-Taste zeigt.

{Composed sequence must be two characters long}
Fehlermeldung, wenn eine Compose-Folge kürzer oder länger als zwei Zeichen definiert wird.

Delete
Text einer Schaltfläche, die eine Compose-Folge löscht.

Help...
Text einer Schaltfläche, die den KHIM-Hilfedialog zeigt.

HELPTEXT
Vollständiger Hilfetext für den Benutzer. Die englische Fassung steht in *en.msg*.

{Input key sequence}
Text der Beschriftung des Entry-Widgets, das nach einer Folge aus zwei Zeichen für die Compose-Taste fragt.

{Insert Character}
Fenstertitel des Dialogs, der eine Unicode-Zeichentabelle zeigt und nach dem einzufügenden

Zeichen fragt.

{Key sequences}
Text einer Beschriftung über einer Listbox der derzeit gebundenen Compose-Folgen.

{KHIM Controls}
Fenstertitel des Dialogs, der nach den KHIM-Einstellungen fragt.

{KHIM Help}
Fenstertitel des Fensters, das den Hilfetext zu KHIM zeigt.

OK
Beschriftung der Schaltfläche OK in mehreren Dialogen.

Select code page:
Beschriftung einer Spinbox, die nach der Nummer einer Unicode-Codepage fragt.

SELECT COMPOSE KEY
Eine Meldung in kurzen Zeilen, die zum Drücken der Taste auffordert, die in KHIM die Compose-Taste wird.

Unicode...
Text einer Schaltfläche, die die Zeichentabelle öffnet, um das Zeichen zu wählen, an das eine Compose-Folge gebunden wird.

{Use KHIM}
Text eines Checkbuttons, der fragt, ob KHIM die Compose-Tastenfolgen verwalten soll.

Danksagungen

KHIM geht auf die Tastenbindungen zurück, die Brent Welch für den Editor sedit im Mail-Programm exmh entwickelt hat. Der Code von KHIM ist vollständig von dem von sedit getrennt.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *khim* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

map

area-display

Stichwörter: area-display, area display, tabular, map

Name

map::area::display – Widget zur Anzeige einer einzelnen Geo-/Gebietsdefinition

Übersicht

package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil

package require snit
package require tablelist
package require map::area::display ?**0.1**?

Beschreibung
Dieses Paket enthält ein Widget zur Anzeige der Informationen einer einzelnen *geo/area* -Definition.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

area-file
Stichwörter: area-file, file, geo/area, map

Name
map::area::file – Lesen und Schreiben von tklib-Geo-/Area-Dateien

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::file ?**0.1**?

Beschreibung
Dieses Paket stellt Lese- und Schreibfunktionen für Tklib-Dateien vom Typ *geo/area* bereit.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

area-map-display

Stichwörter: area-map-display, addon, area display, map display, map

Name

map::area::map-display – Map Action Engine – Ebene zur Darstellung von Gebietsdefinitionen

Übersicht

package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::map-display ?**0.1**?

Beschreibung

Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Dieser Zusatz zeigt eine Menge von *geo/area*-Definitionen. Nur die sichtbaren Gebiete belegen Canvas-Ressourcen (Elemente).

API

Beispiel

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Kartenanzeige und Hilfsmittel

area-store-fs

Stichwörter: area-store-fs, filesystem store, geo/area, map, store, filesystem

Name

map::area::store::fs – Dateisystembasierte Speicherung von Geo-/Flächendefinitionen

Übersicht

package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::file
package require map::area::store::fs ?**0.1**?

Beschreibung
Dieses Paket stellt einen Speicher bereit, der *geo/area*-Definitionen aus einem Verzeichnis im Dateisystem lädt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

area-store-mem
Stichwörter: area-store-mem, center, geo/area, diameter, geo/area pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name
map::area::store::memory – In-Memory-Speicher für Geo-/Flächendefinitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::store::memory ?**0.1**?

Beschreibung
Dieses Paket bietet einen In-Memory-Speicher für *geo/area*-Definitionen mit einer zwischengespeicherten Berechnung der erweiterten Attribute.

Die Basisdaten werden aus einem Backing-Store abgerufen. Ein map::area::store::fs oder ein damit API-kompatibles Produkt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

area-table-display

Stichwörter: area-table-display, area display, tabular, map

Name

map::area::table-display – Widget zur Anzeige einer Tabelle von geo/area-Definitionen

Übersicht

package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::area::table-display ?**0.1**?

Beschreibung

Dieses Paket stellt ein Widget bereit, das die Informationen vieler *geo/area*-Definitionen in einer Tabelle anzeigt.

API

Beispiel

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Kartenanzeige und Hilfsmittel

box-display

Stichwörter: box-display, box display, tabular, map

Name

map::box::display – Widget zur Anzeige einer einzelnen Box-Definition

Übersicht

package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit

package require tablelist
package require map::box::display ?**0.1**?

Beschreibung
Dieses Paket stellt ein Widget zur Verfügung, mit dem die Informationen einer einzelnen *geo/box* -Definition angezeigt werden können.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

box-entry
Stichwörter: box-entry, addon, box entry, map display, map

Name
map::box::entry – Map Action Engine – Box-Eintrag

Übersicht
package require Tcl 8.6-
package require canvas::edit::rectangle
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::entry ?**0.1**?

Beschreibung
Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Mit diesem Zusatz lässt sich ein begrenzendes *geo/box* eingeben.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

box-file

Stichwörter: box-file, file, geo/box, map

Name

map::box::file – Lesen und Schreiben von tklib-Geo-/Box-Dateien

Übersicht

package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::file ?**0.1**?

Beschreibung

Dieses Paket stellt Lese- und Schreibfunktionen für Tklib-Dateien vom Typ *geo/box* bereit.

API

Beispiel

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Kartenanzeige und Hilfsmittel

box-map-display

Stichwörter: box-map-display, addon, box display, map display, map

Name

map::box::map-display – Map Action Engine – Ebene zur Anzeige von Box-Definitionen

Übersicht

package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::rectangle
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::map-display ?**0.1**?

Beschreibung

Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Dieser Zusatz zeigt eine Menge von *geo/box*-Definitionen. Nur die sichtbaren Boxen belegen

Canvas-Ressourcen (Elemente).

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

box-store-fs
Stichwörter: box-store-fs, filesystem store, geobox, map, store, filesystem

Name
map::box::store::fs – Dateisystembasierte Speicherung von Geo-/Box-Definitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::store::fs ?**0.1**?

Beschreibung
Dieses Paket stellt einen Speicher bereit, der *geo/box*-Definitionen aus einem Verzeichnis im Dateisystem lädt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

box-store-mem
Stichwörter: box-store-mem, center, geo/box, diameter, geo/box pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name
map::box::store::memory – In-Memory-Speicher für Geo-/Box-Definitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::store::memory ?**0.1**?

Beschreibung
Dieses Paket stellt einen In-Memory-Speicher für *geo/box*-Definitionen bereit, wobei die Berechnung der erweiterten Attribute zwischengespeichert wird.

Die Basisdaten werden aus einem Backing-Store abgerufen. Ein map::box::store::fs oder ein damit API-kompatibles Produkt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

box-table-display
Stichwörter: box-table-display, box display, tabular, map

Name
map::box::table-display – Widget zur Anzeige einer Tabelle mit Box-Definitionen

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::box::table-display ?**0.1**?

Beschreibung

Dieses Paket stellt ein Widget bereit, das die Informationen zahlreicher *geo/box*-Definitionen in einer Tabelle anzeigt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

display
Stichwörter: display, map, map display

Name
map::display – Widget zur Kartenanzeige

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::sqmap
package require canvas::zoom
package require crosshair
package require debug
package require debug::caller
package require snit
package require widget::scrolledwindow
package require map::display ?**0.1**?

Beschreibung
Dieses Paket stellt ein Karten-Widget mit dem Grundverhalten Ziehen, Zentrieren und Zoom +/− bereit. Die Kacheln kommen von einem einstellbaren Anbieter wie map::provider::osm oder von jedem dazu API-kompatiblen Anbieter. An das Widget lassen sich Bausteine anbinden, die weiteres Verhalten liefern, etwa die Anzeige von geo/*-Ressourcen oder deren Bearbeitung.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

map_overview
Stichwörter: map_overview, map

Name
map_overview – Einleitung zu den Paketen im Modul "Map"

Beschreibung
Speicher
siehe unten ...

Datei
Definitionen einfacher Dateiformate für verschiedene geo/*-Ressourcen und das Lesen und Schreiben dieser Dateien.
- map::area::file
- map::box::file
- map::point::file
- map::track::file

Festplatte
Speicher auf der Platte für verschiedene geo/*-Ressourcen. Es sind einfache Beispiele zum Einstieg und Vorführungen der API. Im ursprünglichen Projekt wurden diese Pakete aus einem Speicher mit **sqlite** herausgelöst. Dieser Speicher ist jedoch zu stark mit jenem Projekt verflochten, um herausgelöst zu werden.
- map::area::store::fs
- map::box::store::fs
- map::point::store::fs
- map::track::store::fs

Hauptspeicher
Speicher im Hauptspeicher. Die Speicher halten geo/*-Daten vor und ergänzen abgeleitete Attribute. Die Brücke zwischen den Speichern auf der Platte und den Widgets und Verhalten.
- map::area::store::memory
- map::box::store::memory
- map::point::store::memory
- map::track::store::memory

Widgets
Zur Darstellung von Karten, einzelnen Geo-Ressourcen und Tabellen vieler Geo-Ressourcen derselben Art.

Karte
map::display

Tabellen von Geo-Ressourcen
- map::area::table-display
- map::box::table-display

- map::point::table-display
- map::track::table-display

Anzeige von Geo-Ressourcen

- map::area::display
- map::box::display
- map::track::display

Verhalten

Bausteine, die sich an map::display anbinden lassen und der angezeigten Karte eigenes Verhalten geben. Die beiden Klassen zeigen Overlays für bestimmte Geo-Ressourcen und bearbeiten oder erfassen einzelne Geo-Ressourcen.

Anzeige

- map::area::map-display
- map::box::map-display
- map::point::map-display
- map::track::map-display

Bearbeitung

- map::box::entry
- map::mark
- map::track::entry

Unterstützung

map::provider::osm

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Kartenanzeige und Hilfsmittel

mark

Stichwörter: mark, location marking, map, mark location

Name

map::mark – Map Action Engine – Einen Punkt markieren

Übersicht

package require Tcl 8.6-
package require debug
package require debug::caller
package require snit
package require map::mark ?**0.1**?

Beschreibung
Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Mit diesem Zusatz lassen sich Orte markieren, also *geo/point*s. Die markierten Orte werden über eine Rückruffunktion gemeldet.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

point-file
Stichwörter: point-file, file, geo/point, map

Name
map::point::file – Lesen und Schreiben von tklib-Geo-/Point-Dateien

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::file ?**0.1**?

Beschreibung
Dieses Paket stellt Lese- und Schreibfunktionen für Tklib-Dateien vom Typ *geo/point* bereit.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

point-map-display
Stichwörter: point-map-display, addon, point display, map display, map

Name
map::point::map-display – Map Action Engine – Ebene zur Darstellung von Punktdefinitionen

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::points
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::map-display ?**0.1**?

Beschreibung
Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Dieser Zusatz zeigt eine Menge von *geo/point*-Definitionen. Nur die sichtbaren Punkte belegen Canvas-Ressourcen (Elemente).

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

point-store-fs
Stichwörter: point-store-fs, filesystem store, geo/point, map, store, filesystem

Name
map::point::store::fs – Dateisystembasierte Speicherung von Geo-/Punkt-Definitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::store::fs ?**0.1**?

Beschreibung
Dieses Paket stellt einen Speicher bereit, der *geo/point*-Definitionen aus einem Verzeichnis im Dateisystem lädt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

point-store-mem
Stichwörter: point-store-mem, center, geo/point, diameter, geo/point pixels, zoom, memory store , perimeter length, length, perimeter, map, pixels, store, memory

Name
map::point::store::memory – In-Memory-Speicher für Geo-/Punkt-Definitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::store::memory ?**0.1**?

Beschreibung
Dieses Paket stellt einen In-Memory-Speicher für *geo/point*-Definitionen bereit, der eine zwischengespeicherte Berechnung erweiterter Attribute ermöglicht.

Die Basisdaten werden aus einem Backing-Store abgerufen. Ein map::point::store::fs oder ein damit API-kompatibles Produkt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

point-table-display
Stichwörter: point-table-display, map, point display, tabular

Name
map::point::table-display – Widget zur Anzeige einer Tabelle mit Punktdefinitionen

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require .map::point::table-display ?**0.1**?

Beschreibung
Dieses Paket stellt ein Widget bereit, das die Informationen zahlreicher *geo/point*-Definitionen in einer Tabelle anzeigt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

provider-osm
Stichwörter: provider-osm, map, mapnik, openstreetmap, provider, tiles, tile provider

Name
map::provider::osm – Kachel-Anbieter, der OpenStreetMap Mapnik als Quelle nutzt

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require map::slippy::cache
package require map::slippy::fetcher
package require snit
package require map::provider::osm ?**0.1**?

Beschreibung
Dieses Paket enthält einen Kachel-Provider, der für die Verwendung mit map::display geeignet

ist. Es nutzt OpenStreetMap Mapnik als Quelle und speichert die abgerufenen Kacheln im lokalen Dateisystem zwischen.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-display
Stichwörter: track-display, map, tabular, track display

Name
map::track::display – Widget zur Anzeige einer einzelnen Track-Definition

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::track::display ?**0.1**?

Beschreibung
Dieses Paket enthält ein Widget zur Anzeige der Informationen einer einzelnen *geo/track* -Definition.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-entry

Stichwörter: track-entry, addon, track entry, map display, map

Name

map::track::entry – Map Action Engine – Track-Eingabe

Übersicht

package require Tcl 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require snit
package require map::slippy 0.8
package require map::track::entry ?**0.1**?

Beschreibung

Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Mit diesem Zusatz lässt sich eine offene Folge von Punkten eingeben, also ein *geo/track*.

API

Beispiel

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Kartenanzeige und Hilfsmittel

track-file

Stichwörter: track-file, file, geo/track, map

Name

map::track::file – Lesen und Schreiben von tklib-Geo-/Track-Dateien

Übersicht

package require Tcl 8.6-
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::file ?**0.1**?

Beschreibung

Dieses Paket stellt Lese- und Schreibfunktionen für *geo/track*-Dateien von Tklib bereit.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-map-display
Stichwörter: track-map-display, addon, track display, map display, map

Name
map::track::map-display – Map Action Engine – Ebene zur Anzeige von Track-Definitionen

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::map-display ?**0.1**?

Beschreibung
Dieses Paket enthält einen Zusatz zu map::display, der das Verhalten der Karte erweitert. Dieser Zusatz zeigt eine Menge von *geo/track*-Definitionen. Nur die sichtbaren Tracks belegen Canvas-Ressourcen (Elemente).

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-store-fs
Stichwörter: track-store-fs, filesystem store, geo/track, map, store, filesystem

Name
map::track::store::fs – Dateisystembasierte Speicherung von Geo-/Track-Definitionen

Übersicht
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::store::fs ?**0.1**?

Beschreibung
Dieses Paket stellt einen Speicher bereit, der *geo/track*-Definitionen aus einem Verzeichnis im Dateisystem lädt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-store-mem
Stichwörter: track-store-mem, center, geo/track, diameter, geo/track pixels, zoom, memory store , perimeter length, length, perimeter, map, pixels, store, memory

Name
map::track::store::memory – In-Memory-Speicher für Geo-/Track-Definitionen

Übersicht
package require Tcl 8.6-
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::store::memory ?**0.1**?

Beschreibung
Dieses Paket stellt einen In-Memory-Speicher für *geo/track*-Definitionen bereit, mit einer zwischengespeicherten Berechnung der erweiterten Attribute.

Die Basisdaten werden aus einem Backing-Store abgerufen. Ein map::track::store::fs oder ein

damit API-kompatibles Produkt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

track-table-display
Stichwörter: track-table-display, map, tabular, track display

Name
map::track::table-display – Widget zur Anzeige einer Tabelle von Track-Definitionen

Übersicht
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::track::table-display ?**0.1**?

Beschreibung
Dieses Paket stellt ein Widget bereit, das die Informationen zahlreicher *geo/track*-Definitionen in einer Tabelle anzeigt.

API

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *map* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie
Kartenanzeige und Hilfsmittel

menubar

menubar

Stichwörter: menubar

Name

menubar – Erstellt eine Instanz der Klasse *menubar*.

Übersicht

package require Tcl 8.6
package require Tk 8.6
package require menubar ?**0.5.1**?
menubar new ?*options*?
mBarInst **define** *body*
mBarInst **install** *pathName body*
mBarInst **menu.configure** *option tag-settings* ?*option tag-settings* ...?
mBarInst **menu.namespace** *tag namespace*
mBarInst **menu.hide** *tag*
mBarInst **menu.show** *tag*
mBarInst **tag.add** *tag value*
mBarInst **tag.configure** *pathName tag* ?*option value ...option value*?
mBarInst **tag.cget** *pathName tag* ?option?
mBarInst **group.add** *tag label* ?*cmd*? ?*accel*? ?*sequence*? ?state?
mBarInst **group.delete** *tag label*
mBarInst **group.move** *direction tag label*
mBarInst **group.configure** *tag label* ?*option value ...option value*?
mBarInst **group.serialize** *tag*
mBarInst **group.deserialize** *tag stream*
mBarInst **notebook.addTabStore** *pathname*
mBarInst **notebook.deleteTabStore** *pathname*
mBarInst **notebook.setTabValue** *pathname tag*
mBarInst **notebook.restoreTabValues** *pathname*

Beschreibung

menubar new ?options?

siehe unten ...

Erzeugt eine Instanz der Klasse menubar und gibt sie zurück. Die Klasse menubar kapselt Definition, Installation und das dynamische Verhalten einer Menüleiste. Sie hängt von keinem Rahmen ab und lässt sich mit oder ohne Rahmen verwenden, zum Beispiel Bwidget, IWidget oder Snit. Anders als andere Tk-Widget-Befehle hat der Befehl menubar kein Argument *pathName*, weil Menüleisten der Fenstermanager verwaltet, also wm, und nicht die Anwendung.

Optionen

Dem Befehl *menubar new* können die folgenden Optionen übergeben werden.

Diese Optionen werden vom Tk-Menü-Befehl übernommen; ihre Wirkung ist plattformabhängig.

- -activebackground
- -activeborderwidth

- -activeforeground
- -background
- -borderwidth
- -cursor
- -disabledforeground
- -font
- -foreground
- -relief

Einleitung

Eine Instanz der Klasse "menubar" stellt Methoden bereit, mit denen eine Beschreibung der Menüleiste erstellt, Menüeinträge konfiguriert und die Menüleiste in Toplevel-Fenstern eingebunden werden können.

Eine Menüleiste lässt sich als Baumstruktur aus verschachtelten Menüs vorstellen. Benutzer definieren eine Menüleiste mithilfe einer Sprache, die zu einer für Menschen lesbaren Beschreibung der Menüleiste führt. Die Beschreibung der Menüleiste wird anschließend von einer Instanz der Menüleisten-Klasse kompiliert, woraufhin sie in einem oder mehreren Toplevel-Fenstern installiert werden kann.

Die Menüleisten-Klasse bietet zahlreiche einzigartige Funktionen, die in anderen Tcl/Tk-Menüleisten-Implementierungen nicht zu finden sind. Dazu gehören unter anderem:

- Ein Tagging-System, das den Zugriff auf Menüeinträge in der Menüstruktur vereinfacht.
- Unterstützung für benutzerdefinierte Tags, die vom Kontext des Toplevel-Fensters abhängen.
- Eine vereinfachte und einheitliche Schnittstelle für alle Rückrufbefehle.
- Namespace-Unterstützung für alle Rückrufbefehle, sodass sie sich leicht in Namespaces gruppieren lassen.
- Unterstützung für das Ausblenden und Einblenden von Menüs in der Menüleiste.
- Eine vereinfachte Methode zum Erstellen von Radiobutton-Gruppen.
- Automatische Verwaltung der Zustandsvariablen von Checkbuttons und Radiobuttons.
- Steuerung des Gültigkeitsbereichs der Zustandsvariablen von Checkbuttons und Radiobuttons.
- Verwaltung der Abreißmenüs, die sicherstellt, dass nur ein Abreißmenü erstellt wird.
- Unterstützung für die dynamische Menü-Erweiterung zur Vereinfachung der Erstellung von Menüs mit zuletzt verwendeten Dokumenten.
- Unterstützung für das Speichern und Wiederherstellen dynamischer Menüerweiterungen.

Begriffsbestimmungen

MENUBAR

Die sichtbare Menüleiste in einem Toplevel-Fenster ist eine waagerechte Gruppe kaskadierender Tk-Menüs.

MENU

Ein Menü ist eine geordnete Liste von Einträgen und wird senkrecht dargestellt. Sichtbar wird es erst, wenn eine Aktion erfolgt, normalerweise ein Ereignis. Ein Menü kann beliebig viele Kindmenüs enthalten, die als kaskadierende Menüs dargestellt werden. Kaskadierende Menüs erscheinen bei der Aktivierung neben dem Elternmenü.

MENU ENTRY

Ein Menü enthält eine geordnete Liste von Einträgen. Menüeinträge haben einen Typ. Die

Klasse menubar unterstützt diese sechs Eintragstypen: *Command*, *Checkbutton*, *Radiobutton*, *Separator*, *Group* und *Menu*.

ENTRY LABEL

Jeder Menüeintrag verfügt über eine sichtbare Zeichenfolge, die als Eintragsbezeichnung bezeichnet wird.

TAG

Ein Tag ist ein Name, der normalerweise verwendet wird, um auf einen Eintrag in einem Menübaum zu verweisen. Ein Tag-Name ist eine alphanumerische Zeichenfolge, die das Unterstrichzeichen enthalten darf. Menübaum-Tags werden für alle Knoten und Endelemente in einem Menübaum definiert. Dies bietet eine flache Abstraktion des Baums und vereinfacht die Referenzierung von Elementen in Menüleisten-Methoden. Ohne diese Abstraktion wäre es notwendig, auf Menüelemente über einen Pfad zu verweisen, der sich zur Laufzeit ändern könnte. Die Menubar-Klasse verfügt zudem über eine Methode, mit der ein benutzerdefiniertes Tag erstellt werden kann. Benutzerdefinierte Tags speichern Werte, die sich je nach dem aktuell aktiven Toplevel-Fenster ändern. Benutzerdefinierte Tags können verwendet werden, um Widget-Pfadnamen zu speichern, die von Rückruffunktionen verwendet werden, sodass die Ausgabe an das entsprechende Toplevel-Fenster weitergeleitet werden kann.

Methoden

mBarInst define *body*

Wandelt *body* in einen Baum aus Menüeinträgen um, die das visuelle Layout der Menüleiste definieren. Das Argument *body* beschreibt das Layout anhand der folgenden Syntax, deren Elemente im Folgenden erläutert werden.

body == definitions

```
definitions    ::= { <ignore> | <definition> | <definition> <definitions> }
ignore         ::= { <nl> | <white-space> <nl> | # <comment> <nl> }
definition     ::= { <command> | <checkbutton> | <radiobutton> | <separator> |
    <group> | <menu> }
command        ::= <label> C <tag> <nl>
checkbutton    ::= <label> X<scope> { <tag> | <tag>+ } <nl>
radiobutton    ::= <label> R<scope> { <tag> | <tag>+ } <nl>
separator      ::= <stext> S <tag> <nl>
group          ::= <dummy> G <tag> <nl>
menu           ::= <label> { M:<tag> | M:<tag>+ } <nl> <definitions>
stext          ::= '--' | <label>
scope          ::= '' | '@' | '='
```

C - Command

Der Eintrag vom Typ C ist der häufigste Eintragstyp. Dieser Eintrag führt bei seiner Ausführung einen Befehl aus.

X - Checkbutton

Ein Eintrag vom Typ X verhält sich weitgehend wie ein Tk-Checkbutton. Aufgerufen wechselt er zwischen ausgewählt und nicht ausgewählt. Der Wert eines Checkbuttons ist ein boolescher Wert, also 1 oder 0. Voreingestellt sind alle Checkbuttons nicht ausgewählt. Soll der Checkbutton anfangs ausgewählt sein, steht hinter dem Tag-Namen ein Plus (+). Den Gültigkeitsbereich beschreibt der Abschnitt **STEUERUNG DES**

GÜLTIGKEITSBEREICHS.

R - Radiobutton
Ein Menüeintrag vom Typ R verhält sich weitgehend wie ein Tk-Radiobutton. Jeder Radiobutton-Eintrag gehört zu einer Radiobutton-Gruppe, die das Verhalten der Radiobuttons in der Gruppe steuert. Alle Radiobuttons einer Gruppe haben denselben Tag-Namen. Im Beispiel unten haben Red, Green und Blue denselben Tag und gehören damit zur selben Gruppe. Ein Plus (+) hinter dem Tag-Namen macht den Eintrag zum anfangs ausgewählten. Den Gültigkeitsbereich beschreibt der Abschnitt **STEUERUNG DES GÜLTIGKEITSBEREICHS**.

S - Separator
Ein Menüeintrag vom Typ "S" ist ein Eintrag, der entweder als horizontale Trennlinie oder als Beschriftung angezeigt wird. Trennzeichen sind keine aktiven Elemente eines Menüs und lösen beim Aufruf kein bestimmtes Verhalten aus. Ist zwei Bindestriche, also '--', erscheint das Trennzeichen als waagerechte Linie. Sonst erscheint als fette Beschriftung zwischen doppelten Bindestrichen, zum Beispiel "-- --", mit dem Hintergrund lightgray.

G - Command Group
Der Menüeintrag vom Typ "G" kennzeichnet eine Stelle im Menübaum, an der Einträge dynamisch hinzugefügt und entfernt werden können. Eine Menüerweiterung kann nur am Ende eines Menüs erfolgen, daher müssen Einträge vom Typ "G" der letzte Eintrag in einem Menü sein. Ein Eintrag vom Typ "G" wird als Trennlinie dargestellt. Die Unterbefehle *group.* dienen zur Bearbeitung von Befehlsgruppeneinträgen.

M - Menu
Ein Eintrag vom Typ "M" wird sowohl zur Definition von Menüs in der Menüleiste als auch von kaskadierenden Menüs verwendet. Menüeinträge sind die komplexesten der sechs Menütypen. Ein Menüeintrag besteht aus drei Listenelementen. Das erste Element der Liste ist seine Bezeichnung. Das zweite Element der Liste ist eine zusammengesetzte Zeichenkette, bestehend aus einer Typkennung (M), gefolgt von einem optionalen Tag (das mit einem Trennzeichen ":" beginnt) und schließlich einem optionalen Plus (+), das ein Abreißmenü kennzeichnet. Das letzte Element der Liste ist ein LISTENWERT.

mBarInst* install *pathName body
Die Methode *install* setzt die mit *define* erzeugte Menüleiste in das Toplevel-Fenster *pathName*. Das Argument *body* enthält ein Tcl-Skript, das die eingesetzte Menüleiste initialisiert. Normalerweise ruft das Skript dazu verschiedene Methoden von menubar auf. Der Initialisierungscode läuft nur einmal, beim Einsetzen der Menüleiste. Der Namespace, in dem *install* ausgeführt wird, wird der voreingestellte Namespace der Rückrufbefehle. Einzelheiten stehen unten bei *menu.namespace*.

Methoden – menu.xxx

mBarInst* menu.configure *option tag-settings ?option tag-settings ...?
Konfiguriert die Tags einer Menüleiste und gibt eine leere Zeichenkette zurück. Diese Methode bietet eine bequeme Möglichkeit, eine größere Anzahl von Tags zu konfigurieren, ohne dass die *tag.configure*-Methode mit ihrem hohen Schreibaufwand verwendet werden muss.

option
Option kann jeden der Werte annehmen, die von der Methode *tag.configure* akzeptiert werden.

tag-settings

Das Argument *tag-settings* ist eine Zeichenkette, die unter Verwendung der folgenden Syntax in eine Liste von Tag-Wert-Paaren umgewandelt wird.

Syntax für *tag-settings*.

```
tag-settings ::= { <ignore> | <value> | <value> <tag-settings> }
ignore       ::= { <nl> | <white-space> <nl> | # <comment> <nl> }
value        ::= <tag> <option-value> <nl>
```

mBarInst* menu.namespace *tag namespace

Ändert den Namespace eines Teilbaums der Menüleiste ab dem Eintrag *tag*. Der neue Wert ist *namespace*. Jeder Eintrag im Baum hat einen Namespace für seine Rückrufprozedur. Voreingestellt ist der Namespace, in dem *install* ausgeführt wurde. Mit dieser Methode lässt sich der Namespace der Rückrufe in einem Teilbaum ändern. Sie gilt nur in einem *install* -Skript.

mBarInst* menu.hide *tag

Entfernt (verbirgt) einen Eintrag der Menüleiste. Beim Definieren des Baums sind alle Einträge sichtbar. Mit dieser Methode lässt sich ein Eintrag verbergen. Die Methoden *hide* können in einem *install*-Skript stehen, sodass ein Menü beim Start der Anwendung zunächst verborgen ist. *tag* ist der Tag-Name des zu verbergenden Menüs.

mBarInst* menu.show *tag

Zeigt einen verborgenen Eintrag der Menüleiste wieder. Beim Definieren des Baums sind alle Einträge sichtbar. Ein mit menu.hide verborgener Eintrag lässt sich mit show wieder zeigen. *tag* ist der Tag-Name des zu zeigenden Menüs.

Methoden – tag.xxx

mBarInst* tag.add *tag value

Fügt einen benutzerdefinierten Wert *tag* hinzu. *tag.add* ergänzt die für eine Menüleiste definierten Tags um ein Tag-Wert-Paar. Benutzerdefinierte Tags unterscheiden sich von den Tags aus *define*. *tag.add* gilt nur in einem *install*-Skript, und der Wert hängt am Toplevel, in dem die Menüleiste eingesetzt ist. Das Tag ist damit kontextabhängig: Rückrufcode, der den Tag-Wert abfragt, erhält den Wert des Fensters, das den Rückruf ausgeführt hat.

mBarInst* tag.configure *pathName tag ?option value ...option value?

Konfiguriert zum *pathName* eines Toplevel-Fensters und einem *tag* den zugehörigen Menüeintrag und gibt die leere Zeichenkette zurück.

Standardoptionen

Diese Optionen sind dieselben wie die der Menüeinträge in der Tk-Dokumentation zu *menu* .

- **-activebackground**
- **-activeforeground**
- **-background**
- **-bitmap**
- **-columnbreak**
- **-compound**
- **-font**

- **-foreground**
- **-hidemargin**
- **-image**
- **-indicatoron**
- **-label**
- **-selectcolor**
- **-selectimage**
- **-state**

Klassenspezifische Optionen

siehe unten ...

-bind {uline accel sequence}

Der Wert der Option *-bind* ist eine Liste mit drei Elementen, deren Werte wie folgt lauten.

uline

Ein ganzzahliger Index für ein Zeichen, das im Eintrag unterstrichen werden soll. Dieser Wert hat dieselbe Funktion wie die Tk-Menüoption *-underline*. Ist dieser Wert eine leere Zeichenkette, wird keine Unterstreichung vorgenommen.

accel

Eine Zeichenkette, die rechts neben dem Menüeintrag angezeigt werden soll. Die Zeichenkette beschreibt in der Regel eine Tastenkombination, die eingegeben werden kann, um dieselbe Funktion wie der Menüeintrag aufzurufen. Dieser Wert hat dieselbe Funktion wie die Tk-Option *menu* -accelerator. Wenn dieser Wert eine leere Zeichenkette ist, wird keine Tastenkombination angezeigt.

sequence

Eine Bindsequenz, die den zum Eintrag gehörenden Befehl auslöst.

-command cmdprefix

Der Wert der Option *-command* ist ein Befehlspräfix, das beim Aufruf des Menüeintrags ausgewertet wird. Voreingestellt wird der Rückruf in dem Namespace ausgewertet, in dem *install* ausgeführt wurde. Weitere Werte werden an *cmdprefix* angehängt und so als Argumente an den Rückrufbefehl übergeben. Diese zusätzlichen Argumente stehen in der folgenden Liste.

command entry

1) Der Pfadname des Toplevel-Fensters, das die Rückruffunktion ausgelöst hat.

checkbutton entry

1) Der Pfadname des Toplevel-Fensters, das den Rückruf ausgelöst hat.
1. Der Tag-Name des Checkbuttons.
2. Der neue Wert des Checkbuttons.

radiobutton entry

1) Der Pfadname des Toplevel-Fensters, das den Rückruf ausgelöst hat.
1. Der Tag-Name des Radiobuttons.
2. Die Beschriftung der ausgewählten Schaltfläche.

group entry

1) Der Pfadname des Toplevel-Fensters, das die Rückruffunktion ausgelöst hat.

mBarInst* tag.cget *pathName tag ?option?
Gibt den Wert der durch *option* angegebenen Konfigurationsoption oder den Wert eines benutzerdefinierten Tags zurück. Das Argument "option" kann eine beliebige Option sein, die von der Methode *tag.configure* für den Typ *tag* akzeptiert wird. Benutzerdefinierte Tags werden ohne einen *option*-Wert abgefragt.

Methoden – group.xxx

mBarInst* group.add *tag label ?cmd? ?accel? ?sequence? ?state?
Fügt der Gruppe mit dem Tag-Namen *tag* einen Befehl hinzu. Die Methode hängt einen neuen Befehlseintrag ans Ende einer Befehlsgruppe. Die Reihenfolge der Argumente ist fest, Argumente rechts dürfen entfallen. Die Argumente bedeuten:

tag *(string)*
Der Tag-Name der Befehlsgruppe.

label *(string)*
Die angezeigte Beschriftung des Menüeintrags.

cmd *(string)*
Ein Befehlspräfix für den Rückrufbefehl.

accel *(string)*
Eine Accelerator-Zeichenkette, die neben der Eintragsbeschriftung erscheint.

sequence *(string)*
Eine Bindsequenz, die an den Rückrufbefehl gebunden wird.

state *(enum)*
Setzt den Zustand des Befehls. Einer von normal, disabled, active.

mBarInst* group.delete *tag label
Löscht einen Befehl aus der Gruppe mit dem Tag-Namen *tag*. Die Methode löscht den Befehl *label* aus einer Befehlsgruppe.

mBarInst* group.move *direction tag label
Ändert die Position eines Eintrags in der Gruppe mit dem Tag-Namen *tag*. *direction* ist die Richtung, up oder down, in die der Eintrag wandert. Der Eintrag heißt *label*.

mBarInst* group.configure *tag label ?option value ...option value?
Konfiguriert die Optionen eines Eintrags in der Befehlsgruppe mit dem Tag-Namen *tag*. Die Methode entspricht *tag.configure*, gilt aber für Einträge einer Befehlsgruppe. Die Optionen eines Befehlseintrags stehen oben bei *tag.configure*.

mBarInst* group.serialize *tag
Gibt eine Zeichenfolgen-Serialisierung der Einträge in einer Befehlsgruppe zurück. Das Argument *tag* ist der Tag-Name der Gruppe, die serialisiert werden soll. Die resultierende Serialisierung ist eine Liste, die drei Elemente enthält: (1) den Namen des Tags der Gruppe, (2) ein Dictionary mit Optionen auf Gruppenebene und (3) eine Liste aus null oder mehr ähnlichen Dreierlisten, die die Einträge in der Gruppe beschreiben.

mBarInst* group.deserialize *tag stream
Ersetzt den Inhalt der Gruppe *tag* durch die Befehle in der Serialisierung *stream*. Der bisherige Inhalt der Gruppe geht verloren.

Methoden – notebook.xxx

mBarInst* notebook.addTabStore *pathname

Diese Methode gehört in Code, der ein neues Notebook-Register erzeugt. Der Aufruf legt einen Zustandsspeicher für das neue Register an. Der Pfadname des Notebook-Registers ist das Argument.

mBarInst* notebook.deleteTabStore *pathname

Gibt den Zustandsspeicher eines Notebook-Registers frei. Der Pfadname des Registers ist das Argument.

mBarInst* notebook.setTabValue *pathname tag

Diese Methode gehört in den Rückruf von Checkbuttons oder Radiobuttons der Menüleiste mit Gültigkeitsbereich eines Notebook-Registers. Der Aufruf schiebt den zum Tag gehörenden Wert in den Speicher des durch den Pfadnamen bezeichneten Registers.

mBarInst* notebook.restoreTabValues *pathname

Diese Methode gehört in ein Bindskript, das das Ereignis <> eines Notebooks auslöst.

Steuerung des Gültigkeitsbereichs

Standardmäßig sieht eine Menüleisten-Instanz in allen installierten Toplevel-Fenstern gleich aus. Werden an einer Instanz einer Menüleiste Änderungen vorgenommen, werden alle anderen Instanzen sofort aktualisiert. Das bedeutet, dass der interne Zustand aller Menüeinträge für die Instanzen synchronisiert wird. Dieses Verhalten wird als globale Steuerung des Menüleisten-Zustands bezeichnet.

Die Klasse menubar erlaubt eine feinere Steuerung des Gültigkeitsbereichs bei Checkbuttons und Radiobuttons. Der Gültigkeitsbereich dieser Eintragstypen ändert sich durch ein Zusatzzeichen am Typzeichen. Zwei Zusatzzeichen werden unterstützt, wie die Tabelle zeigt.

```
''  ::= global scope (no character)
'@' ::= local scope modifier
'=' ::= notebook tab scope modifier
```

Wenn das Zeichen für den lokalen Geltungsbereich (@) zur Definition einer Schaltfläche hinzugefügt wird, erhält die Schaltfläche für jedes installierte Toplevel-Fenster eine neue Variable. Dies bewirkt, dass der Zustand der Schaltfläche auf das jeweilige Fenster beschränkt ist (d. h. lokaler Geltungsbereich). Ein Beispiel ist eine Statusleiste, die ein Checkbutton ein- und ausschaltet. Der Entwickler möchte dem Benutzer möglicherweise ermöglichen, die Sichtbarkeit der Statusleiste für jedes Fenster einzeln zu steuern. In diesem Fall würde dem Selektor der Statusleiste ein lokaler Modifikator hinzugefügt, sodass die Rückruffunktion einen entsprechenden Wert basierend auf dem aktuellen Toplevel-Fenster erhält.

Das Zeichen für den Gültigkeitsbereich eines Notebook-Registers (=) wirkt ähnlich wie das Zeichen für den lokalen Gültigkeitsbereich, nur dass auch die Auswahl eines Notebook-Registers den Zustand einer Schaltfläche führen kann. Der Zusatz schaltet diese Steuerung ein. Die Unterbefehle notebook.xxxx müssen die Zustandswerte dann führen, wenn Register hinzukommen, gelöscht oder ausgewählt werden.

Beispiel

```
package require Tcl
package require Tk
package require menubar

set tout [text .t -width 25 -height 12]
pack ${tout} -expand 1 -fill both
set mbar [menubar new \
    -borderwidth 4 \
    -relief groove  \
    -foreground black \
    -background tan \
    ]
${mbar} define {
    File M:file {
        Exit                 C      exit
    }
    Edit M:items+ {
    #   Label               Type    Tag Name(s)
    #   -----------------   ----    ---------
        "Cut"               C       cut
        "Copy"              C       copy
        "Paste"             C       paste
        --                  S       s2
        "Options" M:opts {
            "CheckList" M:chx+ {
                Coffee      X       coffee+
                Donut       X       donut
                Eggs        X       eggs
                }
            "RadioButtons" M:btn+ {
                "Red"       R       color
                "Green"     R       color+
                "Blue"      R       color
                }
        }
    }
    Help M:help {
        About               C       about
    }
}
${mbar} install . {
    ${mbar} tag.add tout ${tout}
    ${mbar} menu.configure -command {
        # file menu
        exit            {Exit}
        # Item menu
        cut             {CB Edit cut}
        copy            {CB Edit copy}
        paste           {CB Edit paste}
        # boolean menu
        coffee          {CB CheckButton}
        donut           {CB CheckButton}
        eggs            {CB CheckButton}
```

```
        # radio menu
        color           {CB RadioButton}
        # Help menu
        about           {CB About}
    } -bind {
        exit        {1 Cntl+Q  Control-Key-q}
        cut         {2 Cntl+X  Control-Key-x}
        copy        {0 Cntl+C  Control-Key-c}
        paste       {0 Cntl+V  Control-Key-v}
        coffee      {0 Cntl+A  Control-Key-a}
        donut       {0 Cntl+B  Control-Key-b}
        eggs        {0 Cntl+C  Control-Key-c}
        about       0
    } -background {
        exit red
    } -foreground {
        exit white
    }
}
proc pout { txt } {
    global mbar
    set tout [${mbar} tag.cget . tout]
    ${tout} insert end "${txt}\n"
}
proc Exit { args } {
    puts "Goodbye"
    exit
}
proc CB { args } {
    set alist [lassign ${args} cmd]
    pout "${cmd}: [join ${alist} {, }]"
}
wm minsize . 300 300
wm geometry . +4+4
wm protocol . WM_DELETE_WINDOW exit
wm title . "Example"
wm focusmodel . active
pout "Example started ..."
```

Einschränkungen

Diese Implementierung nutzt TclOO und erfordert daher Version 8.6. Der Code wurde unter Windows (Vista), Linux und OSX (10.4) getestet.

Literatur

Siehe auch

1. menu
2. Ein Befehl, der Menüleistenobjekte erzeugt

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *menubar* der Tklib-Tracker. Bitte melden Sie

dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2009 Tom Krehbiel <krehbiel.tom@gmail.com> All rights reserved.
```


notifywindow

notifywindow

Stichwörter: notifywindow, alert, notification, tk

Name

notifywindow – Stellt ein unauffälliges Fenster für Warnmeldungen und Benachrichtigungen aus Tk-Anwendungen bereit

Übersicht

package require tk
package require notifywindow ?**1.0.2**?
::notifywindow::notifywindow *msg img*
::notifywindow::demo

Beschreibung

Dieses Paket stellt ein unauffälliges, diskretes Fenster für Warnmeldungen und Benachrichtigungen aus Tk-Anwendungen bereit, das kurz in der oberen rechten Ecke des Bildschirms erscheint. In Anlehnung an den "Growl"-Rahmen unter Mac OS X wird das Fenster auf Plattformen, die solche Effekte unterstützen, ein- und ausgeblendet, während es auf anderen Plattformen einfach erscheint und wieder verschwindet.

::notifywindow::notifywindow *msg img*
: Zeigt das Toplevel-Fenster mit der Zeichenfolge *msg* an, die nicht länger als 40 Zeichen sein sollte, und einem optionalen Bild, das 16x16 Pixel groß sein sollte.

::notifywindow::demo
: Einfaches Beispiel, das veranschaulicht, wie das Paket funktioniert.

Danksagungen

Obwohl **notifywindow** vom Growl-Paket für Mac OS X inspiriert wurde, weist es keinerlei gemeinsame Code-Teile mit diesem Projekt auf.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *notifywindow* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2015 Kevin Walzer/WordTech Communications LLC <kw@codebykevin.com>
```


ntext

ntext

Stichwörter: ntext, bindtags, re_syntax, regexp, text

Name

ntext – Alternative Bindungen für das Text-Widget

Übersicht

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?
::ntext::new_textCopy *pathName*
::ntext::new_textCut *pathName*
::ntext::new_textPaste *pathName*
::ntext::syncIndentColor *pathName*
::ntext::syncTabColor *pathName*

Beschreibung

Das **ntext**-Paket dient dazu, das Verhalten des Text-Widgets stärker an das anderer Textbearbeitungsprogramme anzupassen. Dadurch eignet sich das Text-Widget besser für die Implementierung eines Texteditors und verhält sich auf eine Weise, die den meisten Benutzern vertrauter ist.

Das Paket stellt ein Bindungs-Tag namens *Ntext* bereit, das von Text-Widgets anstelle des standardmäßigen Bindungs-Tags *Text* verwendet werden kann.

Die Funktionen und Variablen des Pakets **ntext** sind vollständig im Namespace **::ntext** enthalten; der übrige Code befindet sich im Bindungs-Tag *Ntext*. **ntext** enthält keine Exporte in den globalen oder anderen Namespaces und keine neuen Widget-Befehle. Es verwendet modifizierte Kopien des **Tk**-Codes, wobei der ursprüngliche Code und das Bindungs-Tag *Text* unverändert bleiben.

Die Unterschiede zwischen dem Bindungs-Tag *Ntext* und dem Standard-Bindungs-Tag *Text* lassen sich in drei Kategorien einteilen:

- Einige *Text*-Bindungen verhalten sich anders als in den meisten Textbearbeitungsprogrammen. *Ntext* gibt diesen Bindungen ein vertrauteres Verhalten. Einzelheiten stehen in *ntextBindings*.
- Wird eine logische Zeile mit führendem Leerraum im Wortumbruch auf mehrere Anzeigezeilen umbrochen, beginnen die umbrochenen Anzeigezeilen weiter links als die erste Anzeigezeile. Das Layout kann dadurch unordentlich und schwer lesbar werden. *Ntext* kann die umbrochenen Zeilen so einrücken, dass sie zum führenden Leerraum der ersten Anzeigezeile passen (diese Möglichkeit ist vorgegeben ausgeschaltet). Einzelheiten stehen in *ntextIndent*.
- Wenn der Benutzer durch den Text navigiert oder Text auswählt, muss **Tcl**/**Tk** gelegentlich Wortgrenzen erkennen. *Ntext* bietet verbesserte Regeln für die Erkennung von Wortgrenzen. Einzelheiten finden Sie unter *ntextWordBreak*.

Im weiteren Verlauf dieser Seite werden die grundlegende Verwendung und Konfiguration aller drei Aspekte von *Ntext* beschrieben. Ausführlichere Informationen zu den verschiedenen Funktionen von *Ntext* finden Sie auf den Seiten *ntextBindings*, *ntextIndent* und *ntextWordBreak*.

Im Abschnitt Beispiel erfahren Sie, wie Sie das Bindungs-Tag *Ntext* anstelle des Bindungs-Tags *Text* anwenden können.

Befehle

::ntext::new_textCopy *pathName*
Ersatz für ::tk_textCopy.

::ntext::new_textCut *pathName*
Ersatz für ::tk_textCut, der zudem die *Ntext*-Einrückung beibehält (siehe *ntextIndent*).

::ntext::new_textPaste *pathName*
Ersatz für ::tk_textPaste, der zudem die *Ntext*-Einrückung beibehält (siehe *ntextIndent*).

::ntext::syncIndentColor *pathName*
Befehl zum Anwenden des aktuellen Werts der Variablen **::ntext::indentColor** auf vorhandene Zeilen in einem Text-Widget. Dieser Befehl ist nützlich, wenn ein Text-Widget erstellt wurde, Text in das Widget eingefügt wurde *und anschließend* der Wert von **::ntext::indentColor** geändert wird (siehe *ntextIndent*).

::ntext::syncTabColor *pathName*
Befehl zum Anwenden der aktuellen Werte der Variablen **::ntext::tabColor** und **::ntext::tabSelColor** auf Tabulatorzeichen in einem Text-Widget. Dieser Befehl ist nützlich, wenn ein Text-Widget erstellt wurde, Text in das Widget eingefügt wurde *und anschließend* der Wert von **::ntext::tabColor** oder **::ntext::tabSelColor** geändert wird.

Konfigurationsoptionen

Ntext bietet Alternativen zu einer Reihe von Verhaltensweisen des klassischen Bindungs-Tags *Text*. Sofern Optionen vorhanden sind, ist das *Ntext*-Verhalten (mit Ausnahme der Einrückung der Anzeigezeilen) standardmäßig aktiviert.

Das Verhalten von *Ntext* lässt sich anwendungsweit konfigurieren, indem die Werte einer Reihe von Namespace-Variablen festgelegt werden:

::ntext::classicAnchor

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., der Anker ist fest
- **1** – wählt das klassische *Text*-Verhalten aus, d. h., der Anker ist variabel
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::classicExtras

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., mehrere herkömmliche *Text*-Bindungen werden deaktiviert
- **1** – wählt das klassische *Text*-Verhalten, das heißt, alle *Text*-Bindungen sind aktiviert
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::classicMouseSelect

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., der Anker für Mausauswahlvorgänge wird durch die Tastaturnavigation verschoben
- **1** – wählt das klassische *Text*-Verhalten
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::classicParagraphs

- **0** – (Vorgabe) Unter macOS Aqua verhalten sich bestimmte Tastatur-Bindungen wie in der Mac-Anwendung TextEdit. Die Bindungen betreffen das vertikale Verschieben des Bildschirms und lauten <?Shift-?Option-(Up|Down)>.
- **1** – Unter macOS Aqua verhalten sich bestimmte Bindungen wie klassisches *Text* und

ignorieren die Konventionen von Aqua. Die Bindungen betreffen das vertikale Verschieben des Bildschirms und lauten <?Shift-?Option-(Up|Down)>.
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::classicSelection

- **0** - (Standardwert unter macOS Aqua) wählt ein Mac-ähnliches Verhalten aus, d. h., wenn ein Navigationstastenbefehl eine Auswahl aufhebt, springt die Einfügemarke zunächst an das Ende der Auswahl, das durch die Navigationsrichtung des Tastenbefehls bestimmt wird, und erst danach wird der Tastenbefehl ausgeführt.
- **1** – (Standardwert, außer unter macOS Aqua) wählt ein PC-ähnliches Verhalten aus (entspricht dem klassischen *Text*), d. h., wenn ein Navigationstastenbefehl eine Auswahl aufhebt, wird die Einfügemarke nicht verschoben, bevor der Tastenbefehl ausgeführt wird.
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::classicWordBreak

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h. plattformunabhängig, zwei Klassen von Wortzeichen und eine Klasse von Nicht-Wortzeichen.
- **1** – wählt das klassische *Text*-Verhalten, d. h. plattformabhängig, mit einer Klasse für Wortzeichen und einer Klasse für Nicht-Wortzeichen
- Nach der Änderung dieses Werts sollten die Übereinstimmungsmuster neu berechnet werden. Weitere Informationen sowie erweiterte Konfigurationsoptionen finden Sie unter *ntextWordBreak*.

::ntext::classicWrap

- **0** – wählt das *Ntext*-Verhalten, das heißt, Anzeigezeilen von Text-Widgets im Modus *-wrap word* werden so eingerückt, dass sie zum anfänglichen Leerraum der ersten Anzeigezeile einer logischen Zeile passen. Enthält das Widget beim Setzen dieses Werts schon Text, kann ein Funktionsaufruf nötig sein. Die genaue Verwendung der Einrückung von *Ntext* steht in *ntextIndent*.
- **1** – (Vorgabe) wählt das klassische *Text*-Verhalten, das heißt keine Einrückung
- Weitere Informationen finden Sie unter *ntextIndent*

::ntext::indentColor

- **{}** – Wenn der Wert eine leere Zeichenkette ist, so hat der Einzug der umgebrochenen Anzeigezeilen dieselbe Farbe wie der Hintergrund des Widgets.
- *color* – eine gültige **Tk**-Farbe, die für den Einzug von umgebrochenen Anzeigezeilen verwendet werden soll (Standardwert *#d9d9d9*).

  Die Farbgebung wird über die Tag-Option *-lmargincolor* des Text-Widgets implementiert, die ab **Tk** 8.6.6 verfügbar ist. Die Einrückungsfarbe von *Ntext* hat in früheren Versionen von **Tk** keine Auswirkung.

  Der Wert von **::ntext::indentColor** wird häufig beim Start festgelegt. Wenn der Wert geändert wird, während bereits Text-Widgets vorhanden sind und Text enthalten, können diese Widgets durch Aufruf des Befehls **::ntext::syncIndentColor** aktualisiert werden.
- Weitere Informationen finden Sie unter *ntextIndent*

::ntext::overwrite

- **0** – (Ausgangswert) Der über die Tastatur eingegebene Text wird in das Widget eingefügt
- **1** – Über die Tastatur eingegebener Text überschreibt den bereits im Widget vorhandenen Text

- Der Wert wird mit der Taste *Insert* umgeschaltet, außer unter macOS Aqua, wo es diese Taste nicht gibt.
- Weitere Informationen finden Sie unter *ntextBindings*

::ntext::tabColor

- **{}** – Wenn der Wert eine leere Zeichenkette ist, haben die Tabulatorzeichen dieselbe Farbe wie der Hintergrund des Widgets.
- *color* – eine gültige **Tk**-Farbe für die Tabulatorabstände (Vorgabe *#ffffaa*).

Der Wert von **::ntext::tabColor** wird oft beim Start gesetzt. Wird er geändert, während Text-Widgets schon Text enthalten, lassen sich diese Widgets mit **::ntext::syncTabColor** aktualisieren.

::ntext::tabSelColor

- **{}** – Ist der Wert die leere Zeichenkette, ist die Farbe der Tabulatorabstände in der Auswahl vorgegeben dieselbe wie die der nicht ausgewählten Tabulatoren (siehe **::ntext::tabColor**) und *nicht* die Farbe *-selectbackground* des Widgets.
- *color* – eine gültige **Tk**-Farbe für die Tabulatorabstände in der Auswahl (Vorgabe *#418bd4*).

Der Wert von **::ntext::tabSelColor** wird oft beim Start gesetzt. Wird er geändert, während Text-Widgets schon Text enthalten, lassen sich diese Widgets mit **::ntext::syncTabColor** aktualisieren.

::ntext::useBlockCursor

- **0** – Der Blockcursor wird nicht verwendet. Ist das Widget im Modus *overwrite* (siehe **::ntext::overwrite**), ändert stattdessen der gewöhnliche Cursor seine Farbe.
- **1** – Der Blockcursor wird verwendet, wenn das Widget im Modus *overwrite* ist (siehe **::ntext::overwrite**).
- Der Standardwert hängt von der Version von **Tk** ab. In Versionen von **Tk**, die älter sind als **8.5.12**, wies die Größenanpassung des Blockcursors einen Fehler auf, und **::ntext::useBlockCursor** wird standardmäßig auf **0** gesetzt. Ab **8.5.12** ist der Fehler behoben, und **::ntext::useBlockCursor** wird standardmäßig auf **1** gesetzt.
- Weitere Informationen finden Sie unter *ntextBindings*

Bugs

Diese Version von **ntext** soll mit allen Releases von **Tk** 8.5, 8.6 und 8.7 kompatibel sein und mit den Zweigen *core-8-5-branch*, *core-8-6-branch* und *trunk* im Quellcode-Repository von **Tk**. Jede Inkompatibilität mit einer dieser Versionen, bei jedem Fenstersystem von **Tk**, ist als Fehler zu melden. Bitte melden Sie das in der Kategorie *ntext* der Tklib-Tracker.

Beispiel

So erstellen Sie ein Text-Widget .t und verwenden die *Ntext*-Bindungen:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
```

Weitere Informationen finden Sie unter **bindtags**.

ntextBindings

Stichwörter: ntextBindings, bindtags, text

Name

ntextBindings – Alternative Bindungen für das Text-Widget

Übersicht

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?

Beschreibung

Das Paket **ntext** stellt ein Bindungs-Tag namens *Ntext* zur Verfügung, das von Text-Widgets anstelle des standardmäßigen Bindungs-Tags *Text* verwendet werden kann.

Das Bindungs-Tag *Text* bietet etwa hundert Bindungen für das Text-Widget (die genaue Anzahl ist plattformabhängig). Einige davon verhalten sich anders als die meisten modernen Textverarbeitungsprogramme. *Ntext* zielt darauf ab, ein vertrauteres Verhalten zu bieten.

Merkmale der *Ntext*-Bindungen, die sich von den Standard-*Text*-Bindungen unterscheiden:

- Wenn Sie nahe dem Ende einer (logischen) Zeile klicken, springt der Cursor an das Ende dieser Zeile *(nicht an den Anfang der nächsten Zeile)*. Befindet sich das Widget im Modus *-wrap word*, gilt dieselbe Regel für die Anzeigezeilen.
- Durch einen Doppelklick oder das Ziehen nahe dem Ende einer (logischen) Zeile werden Zeichen ab dem Ende dieser Zeile markiert bzw. ausgewählt *(nicht die nächste Zeile oder der Bereich am Zeilenende, in dem sich keine Zeichen befinden)*. Wenn das Widget im Modus *-wrap word* ist, gilt dieselbe Regel für die Anzeige von Zeilen.
- Auf anderen Fenstersystemen als macOS Aqua führt die *Home*-Taste die Funktion "Smart Home" aus (durch wiederholtes Drücken der Taste wird der Cursor zunächst an den Anfang der Anzeigezeile, dann zum ersten Nicht-Leerzeichen am Anfang der logischen Zeile bewegt und schließlich zwischen dem Anfang und dem Ende der führenden Leerzeichen umgeschaltet); die *End*-Taste implementiert "Smart End" (das ähnlich wie "Smart Home" funktioniert). Unter macOS Aqua wird diese Funktionalität durch die Operationen und bereitgestellt.
- Wenn eine Auswahl markiert ist, überschreibt eine Operation "Paste" (zum Beispiel) die Auswahl (wie es bei den meisten Editoren der Fall ist), und zwar auf allen Plattformen.
- Bei anderen Fenstersystemen als macOS Aqua schaltet die Taste *Insert* zwischen den Modi Insert und Overwrite für die Tastatureingabe um. *(Das Bindungs-Tag Text benutzt Insert dagegen, um die primäre Auswahl einzufügen. Das geht auch mit einem Klick der mittleren Maustaste.)*
- Mit der *Escape*-Taste wird die Auswahl aufgehoben.
- Im macOS-Aqua-Fenstersystem hält sich *Ntext* an die Aqua-Konventionen für die Tastaturnavigation. Bestimmte Bindungen für das vertikale Scrollen können so eingestellt werden, dass sie sich genauso verhalten wie bei den Windows- und X11-Fenstersystemen, indem Sie **::ntext::classicParagraphs** auf **1** setzen (siehe Abschnitt Konfigurationsoptionen).
- Im macOS-Aqua-Fenstersystem hält sich *Ntext* an die Aqua-Konventionen für die Platzierung der Einfügemarke, wenn eine Auswahl durch die Tastaturnavigation aufgehoben wird. Dieses Verhalten kann deaktiviert werden, indem **::ntext::classicSelection** auf **1** gesetzt wird (siehe Abschnitt Konfigurationsoptionen).
- Wenn Sie auswählen, erfolgt die Auswahl an der vorherigen Position des Einfügecursors. *(Im*

Tag "Text binding" kann der Anker der Auswahl die Position des letzten Mausklicks sein.)
- -Operationen haben keinen Einfluss auf den Anker der Auswahl. *(Im Text-Bindungs-Tag ist dies jedoch der Fall.)*
- Standardmäßig bietet das Bindungs-Tag *Ntext* einige der durch das Bindungs-Tag *Text* bereitgestellten Strg-Tastenkombinationen nicht an. Moderne Tastaturen bieten Alternativen, wie beispielsweise die Cursortasten zur Navigation; moderne Anwendungen nutzen die Strg-Tasten-Bindungen häufig für andere Zwecke (z. B. für "Drucken").

In den letzten drei Fällen ist das Verhalten von *Text* oft nützlich, daher bietet Ihnen *Ntext* die Option, es beizubehalten, indem Sie die im Namespace ::ntext definierten Variablen auf 1 setzen (anstelle ihres Standardwerts 0). Eine ausführlichere Erläuterung dieser Funktionen:

- Wenn Sie an Position A mit der Maus klicken, anschließend den Cursor mithilfe der Tastatur zu B bewegen, dann die Taste Shift gedrückt halten und an Position C mit der Maus klicken, ergibt das Bindungs-Tag *Text* eine Auswahl von A bis C; gibt *Ntext* eine Auswahl von B bis C an. Wenn Sie möchten, dass sich *Ntext* in dieser Hinsicht wie *Text* verhält, setzen Sie **::ntext::classicMouseSelect** auf 1.
- Das Bindungs-Tag *Text* ermöglicht es, dass aufeinanderfolgende -Ereignisse beide Enden der Auswahl verändern, indem der Anker der Auswahl an das Ende der Auswahl verschoben wird, das am weitesten vom Mausklick entfernt ist. Im Gegensatz dazu fixiert das Bindungs-Tag *Ntext* den Anker, und mehrere Ereignisse Shift-Button-1 können nur das nicht verankerte Ende der Auswahl verschieben. Soll sich *Ntext* darin wie *Text* verhalten, setzt man **::ntext::classicAnchor** auf 1. Bei *Text* und bei *Ntext* verändert die Tastaturnavigation bei gedrückter Taste Shift die Auswahl und hält den Auswahlanker fest.
- Die folgenden "zusätzlichen" *Text*-Bindungen sind standardmäßig deaktiviert, können jedoch in *Ntext* aktiviert werden, indem Sie **::ntext::classicExtras** auf 1 setzen:,,,,,,,,,,,,.

Konfigurationsoptionen

Ntext bietet Alternativen zu einer Reihe von Verhaltensweisen des klassischen *Text* -Bindungs-Tags. Sofern eine Option vorhanden ist, ist das *Ntext*-Verhalten standardmäßig aktiviert, mit Ausnahme der Einrückung von Zeilen, die auf einer separaten Seite unter *ntextIndent* erläutert wird.

Das Verhalten von *Ntext* lässt sich anwendungsweit konfigurieren, indem die Werte einer Reihe von Namespace-Variablen festgelegt werden:

::ntext::classicAnchor

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., der Anker ist fest
- **1** – wählt das klassische *Text*-Verhalten aus, d. h., der Ankerpunkt ist variabel

::ntext::classicExtras

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., mehrere "zusätzliche" *Text*-Bindungen werden deaktiviert
- **1** – wählt das klassische *Text*-Verhalten aus, d. h., die "zusätzlichen" *Text*-Bindungen werden aktiviert

::ntext::classicMouseSelect

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h., der Anker für Mausauswahlvorgänge wird durch die Tastaturnavigation verschoben
- **1** – wählt das klassische *Text*-Verhalten

::ntext::classicParagraphs

- **0** – (Vorgabe) Unter macOS Aqua verhalten sich bestimmte Tastatur-Bindungen wie in der Mac-Anwendung TextEdit. Die Bindungen betreffen das vertikale Verschieben des Bildschirms und lauten <?Shift-?Option-(Up|Down)>.
- **1** – Unter macOS Aqua verhalten sich bestimmte Bindungen so, wie sie es unter den Windows- und X11-Fenstersystemen sowie im klassischen *Text* tun, wobei die Konventionen von Aqua außer Acht gelassen werden. Die Bindungen betreffen das vertikale Scrollen des Bildschirms und lauten <?Shift-?Option-(Up|Down)>.

::ntext::classicSelection

- **0** - (Standardwert unter macOS Aqua) wählt ein Mac-ähnliches Verhalten aus, d. h., wenn ein Navigationstastenbefehl eine Auswahl aufhebt, springt die Einfügemarke zunächst an das Ende der Auswahl, das durch die Navigationsrichtung des Tastenbefehls bestimmt wird, und erst danach wird der Tastenbefehl ausgeführt.
- **1** – (Standardwert, außer unter macOS Aqua) wählt ein PC-ähnliches Verhalten aus (entspricht dem klassischen *Text*), d. h., wenn ein Navigationstastenanschlag eine Auswahl aufhebt, wird die Einfügemarke nicht verschoben, bevor der Tastenanschlag ausgeführt wird.

::ntext::overwrite

- **0** – (Ausgangswert) Der über die Tastatur eingegebene Text wird in das Widget eingefügt
- **1** – Über die Tastatur eingegebener Text überschreibt den bereits im Widget vorhandenen Text
- Der Wert wird mit der Taste *Insert* umgeschaltet.

::ntext::useBlockCursor

- **0** – Der Blockcursor wird nicht verwendet. Ist das Widget im Modus *overwrite* (siehe **::ntext::overwrite**), ändert stattdessen der gewöhnliche Cursor seine Farbe.
- **1** – Der Blockcursor wird verwendet, wenn das Widget im Modus *overwrite* ist (siehe **::ntext::overwrite**).
- Der Standardwert hängt von der Version von **Tk** ab. In Versionen von **Tk**, die älter sind als **8.5.12**, wies die Größenanpassung des Blockcursors einen Fehler auf, und **::ntext::useBlockCursor** wird standardmäßig auf **0** gesetzt. Ab **8.5.12** ist der Fehler behoben, und **::ntext::useBlockCursor** wird standardmäßig auf **1** gesetzt.

Interne Abläufe

Um unabhängig von der Version von **Tk** (8.5 oder 8.6) zu bleiben, definiert **ntext** eigene virtuelle Ereignisse. Diese neuen virtuellen Ereignisse werden ausschließlich im Tag für Bindungen *Ntext* verwendet und beeinträchtigen daher nicht die in anderem Code verwendeten realen oder virtuellen Ereignisse.

Zu diesen Ereignissen gehören <>, <>, <>, die jeweils anstelle von <>, <> und <> verwendet werden.

Die Definitionen der virtuellen Ereignisse <> (usw.) entsprechen den Definitionen der <> (usw.) in **Tk** 8.6.

Bugs

Diese Version von **ntext** soll mit allen Releases von **Tk** 8.5, 8.6 und 8.7 kompatibel sein und mit den Zweigen *core-8-5-branch*, *core-8-6-branch* und *trunk* im Quellcode-Repository von **Tk**. Jede Inkompatibilität mit einer dieser Versionen, bei jedem Fenstersystem von **Tk**, ist als Fehler zu

melden. Bitte melden Sie das in der Kategorie *ntext* der Tklib-Tracker.

Beispiel

Um *Ntext* zu verwenden, dabei aber die Funktion des variablen Ankers des klassischen *Text* beizubehalten:

```
package require ntext
text .t
set ::ntext::classicAnchor 1
bindtags .t {.t Ntext . all}
```

ntextIndent

Stichwörter: ntextIndent, bindtags, re_syntax, regexp, text

Name

ntextIndent – Einrückung für das Text-Widget

Übersicht

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?
::ntext::new_textCopy *pathName*
::ntext::new_textCut *pathName*
::ntext::new_textPaste *pathName*
::ntext::syncIndentColor *pathName*

Beschreibung

Das Paket **ntext** stellt ein Bindungs-Tag namens *Ntext* zur Verfügung, das von Text-Widgets anstelle des standardmäßigen Bindungs-Tags *Text* verwendet werden kann.

Das Text-Widget von Tk kann so konfiguriert werden, dass Textzeilen, die länger sind als die Breite des Textbereichs, umgebrochen werden – eine Funktion, die man aus Texteditoren und Textverarbeitungsprogrammen kennt. Eine vollständige Textzeile (abgegrenzt durch Zeilenumbrüche oder durch den Anfang bzw. das Ende des Dokuments) wird als "logische Zeile" bezeichnet. Wenn eine logische Zeile auf mehrere Zeilen des Anzeigebereichs umgebrochen wird, werden diese Fragmente der logischen Zeile als "Anzeigzeilen" bezeichnet.

Beginnt eine logische Zeile mit Leerraum, beginnen die umbrochenen Anzeigezeilen weiter links als die erste Anzeigezeile. Das Layout kann dadurch unordentlich und schwer lesbar werden. Das Bindungs-Tag *Ntext* kann ein Text-Widget im Modus *-wrap word* so einrichten, dass die Anzeigezeilen außer der ersten automatisch zum anfänglichen Leerraum der ersten Anzeigezeile eingerückt werden.

Diese Einrückung steht Text-Widgets nur im *-wrap word*-Modus zur Verfügung.

Befehle

::ntext::new_textCopy *pathName*
Ersatz für ::tk_textCopy.

::ntext::new_textCut *pathName*
Ersatz für ::tk_textCut, der auch die *Ntext*-Einrückung beibehält.

::ntext::new_textPaste *pathName*
Ersatz für ::tk_textPaste, der auch die *Ntext*-Einrückung beibehält.

::ntext::syncIndentColor *pathName*
Befehl zum Anwenden des aktuellen Werts der Variablen **::ntext::indentColor** auf bereits vorhandene Zeilen in einem Text-Widget. Dieser Befehl ist nützlich, wenn ein Text-Widget erstellt wurde, Text in das Widget eingefügt wurde *und anschließend* der Wert von **::ntext::indentColor** geändert wird.

Konfigurationsoptionen

Das Verhalten von *Ntext* lässt sich anwendungsweit konfigurieren, indem die Werte einer Reihe von Namespace-Variablen festgelegt werden:

::ntext::classicWrap

- **0** – wählt das *Ntext*-Verhalten, das heißt, die Anzeigezeilen werden so eingerückt, dass sie zum anfänglichen Leerraum der ersten Anzeigezeile einer logischen Zeile passen.

  Es sind keine weiteren Maßnahmen erforderlich, sofern diese Option sowie die Option *-wrap* des Text-Widgets festgelegt sind, bevor Text in das Widget eingegeben wird, und sofern die Texteingabe und -bearbeitung ausschließlich über Maus und Tastatur erfolgt. Wenn der Text hingegen durch ein Skript bearbeitet wird oder wenn die Option *-wrap* des Text-Widgets oder der Wert von **::ntext::classicWrap** geändert werden, während das Widget Text enthält, sind Aufrufe der *ntext*-Funktionen erforderlich, um die Einrückung anzupassen. Detaillierte Anweisungen finden Sie im Abschnitt Einrückung von Anzeigezeilen.
- **1** – (Vorgabe) wählt das klassische *Text*-Verhalten, das heißt keine Einrückung.

::ntext::indentColor

- **{}** – Wenn der Wert eine leere Zeichenkette ist, so hat der Einzug der umgebrochenen Anzeigezeilen dieselbe Farbe wie der Hintergrund des Widgets.
- *color* – eine gültige Tk-Farbe, die für die Einrückung von umgebrochenen Anzeigezeilen verwendet werden soll (Standardwert *#d9d9d9*).

  Die Farbgebung wird über die Tag-Option *-lmargincolor* des Text-Widgets implementiert, die ab **Tk** 8.6.6 verfügbar ist. Die Einrückungsfarbe von *Ntext* hat in früheren Versionen von **Tk** keine Wirkung.

  Der Wert von **::ntext::indentColor** wird häufig beim Start festgelegt. Wenn der Wert geändert wird, während bereits Text-Widgets vorhanden sind und Text enthalten, können diese Widgets durch Aufruf des Befehls **::ntext::syncIndentColor** aktualisiert werden.

Erweiterte Verwendung

::ntext::newWrapRegexp

- Der Wert ist ein reguläres Muster, das das Zeichen einer logischen Zeile festlegt, an dem alle Anzeigezeilen außer der ersten ausgerichtet werden. Der Standardwert **[^[:space:]]** gewährleistet die Ausrichtung am ersten Zeichen, das kein Leerzeichen ist.

Einrückung von Anzeigezeilen

So verwenden Sie die Einrückung der Anzeigezeile von *Ntext*:

1. Setzen Sie die Variable **::ntext::classicWrap** auf **0** (der Standardwert lautet **1**). Dadurch werden Bindungen aktiviert, die die Einrückung beibehalten, wenn der Benutzer den Inhalt

des Widgets über Tastatur und Maus ändert. Wenn das Widget bereits Text enthält, rufen Sie **::ntext::wrapIndent** auf, um die Einrückung zu initialisieren.

Weitere Anweisungen gelten, wenn das Programm den Inhalt des Widgets, die Zeilenumbruch-Konfiguration oder die Einrückungskonfiguration ändert.

2. Das Programm kann den Textinhalt ändern, z. B. mit dem Befehl ".text insert". Eine solche Änderung löst keine Bindung aus; daher sollte das Programm nach dem Einfügen von Text explizit die Funktion **::ntext::wrapIndent** aufrufen.
3. Eine automatische Einrückung erfolgt nur, wenn sich das Widget im Modus *-wrap word* befindet. Wenn das Programm in den Modus *-wrap word* wechselt oder zurückkehrt, während das Widget nicht leer ist, sollte es die Funktion **::ntext::wrapIndent** aufrufen, um den Text des Widgets zu formatieren.
4. Wenn Einrückungen verwendet werden und diese anschließend durch die Einstellung von **::ntext::classicWrap** auf **1** deaktiviert werden, rufen Sie bitte **::ntext::wrapIndent** auf, um die Einrückungen zu entfernen.

Funktionen

::ntext::wrapIndent *textWidget* ?index1? ?index2?

- Passen Sie den Einzug eines Text-Widgets an. Im Folgenden werden verschiedene Fälle erläutert.

::ntext::wrapIndent *textWidget*

- Passen Sie den Einzug des gesamten Textes im Text-Widget *textWidget* an.

::ntext::wrapIndent *textWidget index1*

- Passen Sie den Einzug einer einzelnen logischen Zeile eines Text-Widgets an – der Zeile von *textWidget*, die den Index *index1* enthält.

::ntext::wrapIndent *textWidget index1 index2*

- Passen Sie den Einzug eines Bereichs logischer Zeilen eines Text-Widgets an – also der Zeilen von *textWidget*, die die Indizes *index1* bis *index2* enthalten.

Verwendung

- **::ntext::wrapIndent** sollte nur aufgerufen werden, wenn das Skript den Inhalt oder die Anzeigeeigenschaften des Widgets ändert. Wenn der Inhalt des Widgets über die Tastatur oder die Maus geändert wurde, ist es nicht erforderlich, dass das Skript **::ntext::wrapIndent** aufruft, da die entsprechenden Aufrufe automatisch durch die *Ntext*-Bindungen erfolgen.
- Das Skript sollte normalerweise **::ntext::wrapIndent** aufrufen, wenn es beispielsweise eine der folgenden Einstellungen ändert, während das Widget nicht leer ist: den Wert von **::ntext::classicWrap**, oder den Status *-wrap* des Widgets, oder den Tabulatorabstand des Widgets, oder die Schriftgröße, oder den Inhalt des Widgets.
- Ein Aufruf in der Form **::ntext::wrapIndent** *textWidget* ist stets ausreichend; sollten jedoch nur bestimmte Zeilen geändert werden müssen, ist es effizienter, diese Zeilen mithilfe der optionalen Argumente ?index1? und ?index2? anzugeben.
- Ist das Widget im Modus *-wrap word* und **::ntext::classicWrap** auf **0** gesetzt, rückt **::ntext::wrapIndent** die logischen Zeilen in dem Bereich ein, den die Argumente der Funktion angeben.
- In den anderen Fällen, also im Modus *-wrap char* oder *-wrap none*, oder wenn **::ntext::classicWrap** auf **1** steht, entfernt **::ntext::wrapIndent** die Einrückung der logischen

Zeilen in diesem Bereich.

Bugs

Diese Version von **ntext** soll mit allen Releases von **Tk** 8.5, 8.6 und 8.7 kompatibel sein und mit den Zweigen *core-8-5-branch*, *core-8-6-branch* und *trunk* im Quellcode-Repository von **Tk**. Jede Inkompatibilität mit einer dieser Versionen, bei jedem Fenstersystem von **Tk**, ist als Fehler zu melden. Bitte melden Sie das in der Kategorie *ntext* der Tklib-Tracker.

Beispiele

So aktivieren Sie die Einrückung von *Ntext* und verwenden diese im Widget .t unter Verwendung der Standardfarbe für Einrückungen *#d9d9d9*:

```
package require ntext
set ::ntext::classicWrap 0
text .t -wrap word
bindtags .t {.t Ntext . all}
```

So aktivieren Sie die Einrückung von *Ntext* und verwenden diese in der Widget-Datei .t ohne farbige Einrückungen:

```
package require ntext
set ::ntext::classicWrap 0
set ::ntext::indentColor {}
text .t -wrap word
bindtags .t {.t Ntext . all}
```

So aktivieren Sie die Einrückung von *Ntext* und verwenden diese in der Widget-Datei .t, wobei die Einrückungen die Farbe Schwarz haben:

```
package require ntext
set ::ntext::classicWrap 0
set ::ntext::indentColor black
text .t -wrap word
bindtags .t {.t Ntext . all}
```

Um die Einrückung von *Ntext* später zu deaktivieren:

```
set ::ntext::classicWrap 1
::ntext::wrapIndent .t
```

So können Sie die Einrückung von *Ntext* später wieder aktivieren:

```
set ::ntext::classicWrap 0
::ntext::wrapIndent .t 1.0 end
```

So fügen Sie Text in das Widget ein:

```
set foo [.t index end]
.t insert end {This line was added by the script, not the keyboard!}
::ntext::wrapIndent .t $foo end
```

So ändern Sie die Farbe der Einrückung, wenn ein Widget vom Typ ".t" bereits Text enthält:

```
set ::ntext::indentColor black
::ntext::syncIndentColor .t
```

Um in den Modus *-wrap char* zu wechseln:

```
.t configure -wrap char
::ntext::wrapIndent .t
```

ntextWordBreak

Stichwörter: ntextWordBreak, bindtags, re_syntax, regexp, text

Name

ntextWordBreak – Erkennung von Wortgrenzen in ntext für das Text-Widget

Übersicht

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?

Beschreibung

Das Paket **ntext** stellt ein Bindungs-Tag namens *Ntext* zur Verfügung, das von Text-Widgets anstelle des standardmäßigen Bindungs-Tags *Text* verwendet werden kann.

Die Navigation und Auswahl in einem Text-Widget erfordern die Erkennung von Wörtern und deren Grenzen. Die von Tcl/Tk über das Bindungs-Tag *Text* bereitgestellten Funktionen zur Wortgrenzenerkennung sind eingeschränkt, da sie nur eine Klasse von "Wort"-Zeichen und eine Klasse von "Nicht-Wort"-Zeichen definieren. Das Bindungs-Tag *Ntext* verwendet allgemeinere Regeln für die Wortgrenzenerkennung, die *zwei* Klassen von "Wort"-Zeichen und eine Klasse von "Nicht-Wort"-Zeichen definieren.

Konfigurationsoptionen

Das Verhalten von *Ntext* lässt sich anwendungsweit konfigurieren, indem die Werte einer Reihe von Namespace-Variablen festgelegt werden. Eine davon ist für die Erkennung von Wortgrenzen relevant:

::ntext::classicWordBreak

- **0** – (Standardwert) wählt das *Ntext*-Verhalten aus, d. h. plattformunabhängig, zwei Klassen von Wortzeichen und eine Klasse von Nicht-Wortzeichen.
- **1** – wählt das klassische *Text*-Verhalten, das heißt plattformabhängig, eine Klasse von Wortzeichen und eine Klasse von Nicht-Wortzeichen
- Nach der Änderung dieses Werts sollten die Regexp-Muster zum Abgleich neu berechnet werden. Weitere Informationen sowie erweiterte Konfigurationsoptionen finden Sie unter Funktionen.

Fortgeschrittene Nutzung

Variablen (fortgeschrittene Anwendung)

::ntext::tcl_match_wordBreakAfter

::ntext::tcl_match_wordBreakBefore

::ntext::tcl_match_endOfWord

::ntext::tcl_match_startOfNextWord

::ntext::tcl_match_startOfPreviousWord

Diese Variablen enthalten die regulären Muster, die von *Ntext* zur Suche nach Wortgrenzen verwendet werden. Werden sie geändert, wirkt sich dies sofort auf nachfolgende Suchvorgänge aus. In vielen Fällen ist es nicht erforderlich, die Werte dieser Variablen direkt zu ändern: Rufen Sie stattdessen eine der Funktionen **::ntext::initializeMatchPatterns** oder **::ntext::createMatchPatterns** auf.

Im Bindungs-Tag *Text* können Sie die Suchregeln ändern, indem Sie die Werte der globalen Variablen **tcl_wordchars** und **tcl_nonwordchars** anpassen. Die entsprechende Operation im *Ntext*-Bindungs-Tag besteht darin, **::ntext::createMatchPatterns** mit den entsprechenden Argumenten aufzurufen.

Funktionen (fortgeschrittene Anwendung)

Wenn sich eine einfache Suche mit regulären Ausdrücken als unzureichend erweist, können die folgenden Funktionen (analog zu den Funktionen **tcl_wordBreakAfter** usw. des Tcl/Tk-Kerns) vom Entwickler ersetzt werden:

ntext::new_wordBreakAfter

ntext::new_wordBreakBefore

ntext::new_endOfWord

ntext::new_startOfNextWord

ntext::new_startOfPreviousWord

Funktionen

Jede Funktion berechnet die fünf Regexp-Suchmuster, die die Suche nach Wortgrenzen definieren. Diese Werte werden in den oben aufgeführten Namespace-Variablen gespeichert.

::ntext::initializeMatchPatterns

- Diese Funktion wird bei der ersten Verwendung von *Ntext* aufgerufen und muss nur dann erneut aufgerufen werden, wenn das Skript den Wert von **::ntext::classicWordBreak** oder **::tcl_platform(platform)** ändert. Die Funktion wird ohne Argumente aufgerufen. Sie ist nützlich, wenn die gewünschten Suchmuster die Standardmuster für das Tag *Ntext* oder *Text* sind und somit implizit allein durch die Werte von **::ntext::classicWordBreak** und **::tcl_platform(platform)** festgelegt werden.

::ntext::createMatchPatterns *new_nonwordchars new_word1chars* ?new_word2chars?

- Diese Funktion ist in einem breiteren Anwendungsspektrum einsetzbar als **::ntext::initializeMatchPatterns**. Sie berechnet die Regexp-Suchmuster für jeden Fall, bei dem eine Klasse von "Nicht-Wort"-Zeichen und eine oder zwei Klassen von "Wort"-Zeichen vorliegen.

  Jedes Argument sollte ein regulärer Ausdruck sein, der eine Klasse von Zeichen definiert. Ein Argument ist in der Regel ein Klammerausdruck, kann aber alternativ auch eine Kurzschreibweise für eine Klasse von Zeichen oder ein einzelnes Zeichen sein. Das dritte Argument kann weggelassen oder als leere Zeichenkette angegeben werden; in diesem Fall wird es nicht verwendet.

  Das erste Argument wird als Klasse der Nicht-Wort-Zeichen interpretiert; das zweite Argument (und das dritte, falls vorhanden) sind Klassen von Wortzeichen. Die Klassen sollten alle

möglichen Zeichen enthalten und schließen sich normalerweise gegenseitig aus: Oft ist es zweckmäßig, eine Klasse als Negation der beiden anderen zu definieren.

Abgleich von Wortgrenzen

Die Auswahl der Wortgrenzen ist ein heikles Thema, da Text dazu dient, eine Vielzahl unterschiedlicher Informationsarten darzustellen, und es keine einfachen Regeln gibt, die für alle Datentypen oder für alle Zwecke geeignet sind.

Ntext versucht, die im klassischen *Text* verfügbaren Funktionen zu verbessern, indem es Möglichkeiten für komplexere Definitionen von Wörtern bietet (mit drei statt zwei Klassen von Zeichen).

Was ist ein Wort? Warum zwei Klassen von Wörtern?

Wenn die modifizierten Cursortasten und zur Navigation durch ein *Ntext*-Widget verwendet werden, wird der Cursor an den Anfang eines Wortes platziert. Ein Wort ist definiert als eine Folge von einem oder mehreren Zeichen, die ausschließlich aus einer der beiden definierten "Wort"-Klassen stammen; ihm kann ein Zeichen aus der anderen "Wort"-Klasse oder aus der "Nicht-Wort"-Klasse vorangestellt sein.

Durch einen Doppelklick mit der ersten Maustaste wird ein Wort im Text ausgewählt, wobei ein "Wort" in diesem Fall entweder der oben genannten Definition entsprechen kann oder alternativ aus einer Folge von einem oder mehreren Zeichen aus der Klasse "Nicht-Wörter" bestehen kann.

Traditionell hat Tcl nur eine Wortklasse und eine Nicht-Wortklasse definiert: Unter Windows ist die Nicht-Wortklasse Leerraum, sodass Buchstaben, Ziffern und Satzzeichen derselben Klasse angehören. Auf anderen Plattformen gehören Satzzeichen zusammen mit dem Leerraum zu den "Nicht-Wort"-Zeichen. In beiden Fällen sind die Navigation und die Auswahl von Text unnötig grob granuliert und führen manchmal zu wenig hilfreichen Ergebnissen.

Die Verwendung von drei Zeichenklassen könnte die Auswahl zu detailliert gestalten; in diesem Fall ist die Taste *Shift* gedrückt halten und ein anderes Wort doppelklicken jedoch eine hervorragende Möglichkeit, einen längeren Textabschnitt auszuwählen (eine nützliche Bindung, die Tcl/Tk schon seit langem bietet, die jedoch in anderen Systemen fehlt).

Neben den Standardeinstellungen ermöglicht *Ntext* dem Entwickler, eigene Klassen von Zeichen zu definieren, auf die klassischen *Text*-Definitionen zurückzugreifen oder eigene reguläre Ausdrücke für die Muster des Abgleichs festzulegen.

Bugs

Diese Version von **ntext** soll mit allen Releases von **Tk** 8.5, 8.6 und 8.7 kompatibel sein und mit den Zweigen *core-8-5-branch*, *core-8-6-branch* und *trunk* im Quellcode-Repository von **Tk**. Jede Inkompatibilität mit einer dieser Versionen, bei jedem Fenstersystem von **Tk**, ist als Fehler zu melden. Bitte melden Sie das in der Kategorie *ntext* der Tklib-Tracker.

Beispiel

So verwenden Sie *Ntext* unter Anwendung der üblichen Regeln zur Wortgrenzenerkennung von Tcl/Tk:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
set ::ntext::classicWordBreak 1
::ntext::initializeMatchPatterns
```

Weitere Informationen finden Sie unter "bindtags".

So definieren Sie einen anderen Satz von Regeln zur Erkennung von Wortgrenzen:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
::ntext::createMatchPatterns  {[[:space:][:cntrl:]]} {[[:punct:]]}
    {[^[:punct:][:space:][:cntrl:]]}
```

Weitere Informationen finden Sie unter "regexp" und "re_syntax".

persistentSelection

persistentSelection

Stichwörter: persistentSelection

Name

persistentSelection – Erweiterte PRIMARY-Auswahl

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require persistentSelection ?**1.0**?
persistentSelection::fixText
persistentSelection::getClassicPrimarySel ?*displayOf*? ?*withOthers*?
persistentSelection::getPrimarySel ?*displayOf*?
persistentSelection::report *type pathName*

Beschreibung

Nehmen wir ein **Tk**-Widget, das seine Option **-exportselection** auf den booleschen Wert **true** setzt und dadurch seine Auswahl exportiert. Wenn die Auswahl im Widget aufgehoben wird, hebt **Tk** standardmäßig auch die Auswahl **PRIMARY** auf. In dieser Situation schlägt ein Versuch, die Auswahl **PRIMARY** auszulesen – beispielsweise durch ein **<>**-Ereignis – fehl.

Das Paket "**persistentSelection**" bietet ein komfortableres Verhalten. Es behält – wie bei der Auswahl "**PRIMARY**" – den letzten nicht leeren Wert bei, der von einem der von ihm überwachten "**Tk**"-Widgets gesetzt wurde, solange die Auswahl "**PRIMARY**" nicht von einem anderen Prozess oder von einem Widget beansprucht wird, das es nicht überwacht.

Das Paket "**persistentSelection**" funktioniert, indem es eine Bindung an das Ereignis "**<>**" der betreffenden Widgets herstellt und diese Bindung nutzt, um jede Änderung der Auswahl im Widget zu überwachen. Seine Befehle sind auf den Namespace **::persistentSelection** beschränkt.

Befehle

persistentSelection::fixText

Einmal aufrufen, um Probleme mit **text**-Widgets zu beheben, die den voreingestellten Bindtag **Text** benutzen und nicht **Ntext**. Die Erklärung steht unter Text-Widgets.

Der Befehl passt einige **Text**-Bindungen so an, dass **<>**-Ereignisse generiert werden; und ordnet die Aufrufe des Befehls **tag** für das **text**-Widget im privaten Befehl **::tk::TextSelectTo** neu an, damit dieser Befehl besser mit **persistentSelection** zusammenarbeitet.

Diese Anpassungen sind nur bei älteren Versionen von **Tk** erforderlich. Der Befehl **persistentSelection::fixText** hat bei ausreichend aktuellen Versionen keine Auswirkungen. Er kann bei jeder Version von **Tk** bedenkenlos aufgerufen werden.

persistentSelection::getClassicPrimarySel ?*displayOf*? ?*withOthers*?

- *displayOf* – ein **Tk**-Fensterpfad auf dem abzufragenden Display (optional, voreingestellt {**.**})
- *withOthers* – Gibt an, ob die Auswahl "**PRIMARY**" zurückgegeben werden soll, wenn sie einem anderen Prozess oder Interpreter gehört (optional, boolescher Wert, Standardwert: **false**)

Ein Befehl, der die klassische (nicht persistente) Form der Auswahl "**PRIMARY**" gibt. Wenn die Auswahl nicht existiert oder ausschließlich durch "**persistentSelection**" aufrechterhalten wird, gibt der Befehl die leere Zeichenkette zurück.

persistentSelection::getPrimarySel ?*displayOf*?

- *displayOf* – ein **Tk**-Fenster auf dem abzufragenden Display (optional, voreingestellt {**.**})

Ein Befehl, der den Wert der Auswahl "**PRIMARY**" gibt. Wenn die Auswahl nicht existiert, gibt der Befehl die leere Zeichenkette zurück. Die üblichen "**Tk**"-Befehle sind zwar durchaus gültig, lösen jedoch einen Fehler aus, wenn die Auswahl nicht existiert.

persistentSelection::report *type pathName*

- *type* – (Wert **text** oder **entry**) Einordnung des Widgets, dessen Auswahl sich geändert hat
- *pathName* – der **Tk**-Fensterpfad des Widgets, dessen Auswahl sich geändert hat

Damit das Paket **persistentSelection** ein Widget überwacht, muss dieser Befehl an das Ereignis **<>** dieses Widgets gebunden werden. Der Befehl erfasst Änderungen an der Auswahl des Widgets und nutzt diese Informationen, um auf jedem Display eine nicht leere **PRIMARY** -Auswahl zu halten.

Verwendung

Anforderungen an das Skript

Für jedes Widget, das von **persistentSelection** überwacht werden soll, muss der Befehl **persistentSelection::report** in einem der Bindungs-Tags des Widgets an das Ereignis **<>** gebunden werden.

Beispielsweise möchte der Entwickler möglicherweise **persistentSelection** mit jedem **text**- und **entryPlus**-Widget in einer Anwendung verwenden: Dies lässt sich erreichen, indem die Bindung "**<>**" zu den Bindungen "**Text**" und "**EntryPlus**" hinzugefügt wird. Siehe Beispiele.

Wenn **persistentSelection** nicht für alle Widgets einer bestimmten Klasse erforderlich ist, kann

die Bindung den Bindtags der einzelnen Widgets hinzugefügt werden. Siehe Beispiele.

Anforderungen an Widgets

Damit ein Widget von **persistentSelection** überwacht werden kann, muss es drei Bedingungen erfüllen:

- Es muss über die Option "**-exportselection**" verfügen, die es ermöglicht, die aktuelle Auswahl in die Auswahl "**PRIMARY**" zu exportieren.
- Es muss bei jeder Änderung der Auswahl ein Ereignis vom Typ "**<>**" auslösen.
- Die Bindungen zu anderen Ereignissen sollten angemessen sein – insbesondere in Versionen von **Tk**, die älter sind als **8.6.6**, sollte ein GUI-Ereignis, das die Auswahl von einem nicht leeren Wert auf einen anderen erweitert oder einschränkt, keinen Zwischenzustand mit einer leeren Auswahl verwenden, weil das **persistentSelection** verwirrt.

Die **Tk**- und **Ttk**-Widgets, die ihre Auswahl exportieren können, sind **text**, **entry**, **listbox**, **spinbox**, **ttk::entry**, **ttk::spinbox** und **ttk::combobox**.

Text-Widgets

In Versionen von **Tk**, die älter sind als **8.6.9**, löst das **text**-Widget in einigen "Sonderfällen", in denen sich seine Auswahl ändert, keine **<>**-Ereignisse aus. Diese Auslassungen lassen sich durch Änderungen an den **Text**-Bindungen beheben, um die zweite Bedingung der Anforderungen an Widgets zu erfüllen.

Zudem verarbeiten Versionen von **Tk**, die älter sind als **8.6.6**, Ereignisse der Auswahl etwas anders als neuere Versionen, und in Kombination mit den **Text**-Bindungen verwirrt das **persistentSelection**. Wenn ein Upgrade auf die aktuelle Version von **Tcl/Tk** nicht möglich ist, lässt sich dieses Problem durch eine geringfügige Anpassung der Widget-Bindungen beheben, um die dritte Anforderung der Anforderungen an Widgets zu erfüllen.

Entweder sollte das Skript den Befehl "**persistentSelection::fixText**" aufrufen, um die **Text**-Bindungen und die von diesen aufgerufenen Befehle anzupassen; oder das Widget kann anstelle der standardmäßigen Bindung **Text** das Bindungs-Tag **Ntext** (für **ntext** Version 1.0 oder höher) verwenden.

In beiden Fällen soll das Argument *type* für **persistentSelection::report** den Wert **text** haben.

Die Widgets entry, spinbox, ttk::entry, ttk::spinbox und ttk::combobox

Die Widgets **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox** und **ttk::combobox** erzeugen kein **<>**-Ereignis, wenn sich ihre Auswahl ändert, und müssen daher angepasst werden.

Das Paket **widgetPlus** stellt jeweils die **snit**-Wrapper **widgetPlus::entryPlus**, **widgetPlus::spinboxPlus**, **widgetPlus::ttkEntryPlus**, **widgetPlus::ttkSpinboxPlus** und **widgetPlus::ttkComboboxPlus** bereit. Jedes **widgetPlus**-Widget generiert die erforderlichen **<>**-Ereignisse.

In all diesen Fällen sollte das an den Befehl **persistentSelection::report** übergebene Argument *type* den Wert **entry** haben. Dieses Argument legt fest, wie **persistentSelection** die Auswahl des Widgets überprüft, und die Befehle, die dies tun, sind für alle diese Widgets identisch.

Listbox-Widgets

Für das Widget "**listbox**", bei dem die Auswahl mehrerer Elemente möglich ist, wurde kein

entsprechender Wrapper erstellt.

Noch zu tun

- Auf einem System mit mehreren Displays prüfen.

Bugs

Diese Version von **persistentSelection** bietet Kompatibilität mit allen Ausgaben von **Tk** 8.5- und 8.6 sowie mit den Zweigen *core-8-5-branch*, *core-8-6-branch*, *revised_text* und *trunk* im Quellcode-Repository von **Tk**. Jede Unverträglichkeit mit einer dieser Versionen, für jedes Fenstersystem von **Tk**, ist als Fehler zu melden. Bitte melden Sie solche Fehler in der Kategorie *persistentSelection* der Tklib-Tracker.

Beispiele

In jedem Beispiel wird "**persistentSelection**" verwendet, um den letzten nicht leeren Wert der Auswahl in bestimmten Widgets beizubehalten. Jedes Beispiel umfasst zudem die Erstellung von Beispiel-Widgets.

Alle Widgets **entryPlus** überwachen.

```
package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
bind EntryPlus <<Selection>> {::persistentSelection::report entry %W}
```

Alle **text**-Widgets überwachen, die die voreingestellten Bindungen **Text** benutzen.

```
text .t
pack .t

package require persistentSelection
::persistentSelection::fixText
bind Text <<Selection>> {::persistentSelection::report text %W}
```

Alle **text**-Widgets mit den voreingestellten Bindungen **Text** und alle Widgets **entryPlus** überwachen.

```
text .t
pack .t

package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
::persistentSelection::fixText
bind Text <<Selection>> {::persistentSelection::report text %W}
bind EntryPlus <<Selection>> {::persistentSelection::report entry %W}
```

Alle **text**-Widgets überwachen, die Bindungen **Ntext** benutzen.

```
text .t
pack .t

package require ntext
bindtags .t {.t Ntext . all}

package require persistentSelection
bind Ntext <<Selection>> {::persistentSelection::report text %W}
```

Ein einzelnes Widget **entryPlus .ep** überwachen.

```
package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
bind .ep <<Selection>> {::persistentSelection::report entry %W}
```

Ein einzelnes **text**-Widget **.t** überwachen.

```
text .t
pack .t

package require persistentSelection
bind .t <<Selection>> {::persistentSelection::report text %W}
```

plotchart

plotchart

Stichwörter: plotchart

Name

Plotchart – Einfaches Paket zum Zeichnen und Erstellen von Diagrammen

Übersicht

package require Tcl ?**8.5**?
package require Tk ?**8.5**?
package require Plotchart ?**2.6.2**?
::Plotchart::createXYPlot *w xaxis yaxis args*
::Plotchart::createStripchart *w xaxis yaxis args*
::Plotchart::createTXPlot *w timeaxis xaxis args*
::Plotchart::createXLogYPlot *w xaxis yaxis args*
::Plotchart::createLogXYPlot *w xaxis yaxis args*
::Plotchart::createLogXLogYPlot *w xaxis yaxis args*
::Plotchart::createPolarPlot *w radius_data args*
::Plotchart::createWindrose *w radius_data sectors*
::Plotchart::createIsometricPlot *w xaxis yaxis stepsize*
::Plotchart::createHistogram *w xaxis yaxis args*
::Plotchart::create3DPlot *w xaxis yaxis zaxis args*
::Plotchart::create3DRibbonPlot *w yaxis zaxis*

::Plotchart::createPiechart *w*
::Plotchart::createSpiralPie *w*
::Plotchart::createRadialchart *w names scale style*
::Plotchart::createBarchart *w xlabels yaxis noseries args*
::Plotchart::createHorizontalBarchart *w xaxis ylabel noseries*
::Plotchart::create3DBarchart *w yaxis nobars*
::Plotchart::create3DRibbonChart *w names yaxis zaxis*
::Plotchart::createBoxplot *w xdata ydata orientation*
::Plotchart::createTimechart *w time_begin time_end args*
::Plotchart::createGanttchart *w time_begin time_end args*
::Plotchart::createRightAxis *w_or_plot yaxis*
::Plotchart::createTableChart *w columns* ?*widths*?
::Plotchart::createTernaryDiagram *w args*
::Plotchart::createNormalPlot *w xscale args*
::Plotchart::createStatusTimeline *w xaxis ylabel args*
::Plotchart::createDendrogram *w args*
$anyplot title *text position*
$anyplot subtitle *text*
$anyplot canvas
$anyplot saveplot *filename args*
$anyplot xtext *text*
$anyplot ytext *text*
$anyplot vtext *text*
$anyplot xsubtext *text*
$anyplot ysubtext *text*
$anyplot vsubtext *text*
$anyplot xconfig -option *value ...*
$anyplot yconfig -option *value ...*
$anyplot background *part colour_or_image dir* ?*brightness*?
$anyplot xticklines *colour* ?*dash*?
$anyplot yticklines *colour* ?*dash*?
$anyplot legend *series text* ?*spacing*?
$anyplot removefromlegend *series*
$anyplot legendconfig -option *value ...*
$anyplot balloon *x y text dir*
$anyplot balloonconfig *args*
$anyplot plaintext *x y text dir*
$anyplot plaintextconfig *args*
$anyplot object *itemtype series args*
$anyplot deletedata
$xyplot plot *series xcrd ycrd*
$normalplot plot *series mean stdev data*
$normalplot diagonal
$xyplot plotlist *series xlist ylist every*
$xyplot region *series xlist ylist*
$xyplot minmax *series xcrd ymin ymax*

$histogram plotcumulative *series xcrd ycrd*
$xyplot trend *series xcrd ycrd*
$xyplot rchart *series xcrd ycrd*
$xyplot interval *series xcrd ymin ymax* ?*ycentr*?
$xyplot box-and-whiskers *series xcrd ycrd*
$xyplot violin *series xcrd ycrd*
$xyplot vector *series xcrd ycrd ucmp vcmp*
$xyplot vectorconfig *series* **-option** *value ...*
$xyplot dot *series xcrd ycrd value*
$xyplot dotconfig *series* **-option** *value ...*
$xyplot contourlines *xcrd ycrd values* ?*classes*?
$xyplot contourlinesfunctionvalues *xvec yvec valuesmat* ?*classes*?
$xyplot contourfill *xcrd ycrd values* ?*classes*?
$xyplot contourbox *xcrd ycrd values* ?*classes*?
$xyplot colorMap *colours*
$xyplot legendisolines *values classes*
$xyplot legendshades *values classes*
$xyplot grid *xcrd ycrd*
$xyplot xband *ymin ymax*
$xyplot yband *xmin xmax*
$xyplot labeldot *x y text orient*
$polarplot plot *series radius angle*
$windrose plot *data colour*
$plot3d plotfunc *function*
$plot3d plotfuncont *function contours*
$plot3d gridsize *nxcells nycells*
$plot3d plotdata *data*
$plot3d interpolatedata *data contours*
$plot3d colour *fill border*
$plot3d ribbon *yzpairs*
$plot3d plot *yzpairs*
$xyplot dataconfig *series* **-option** *value ...*
$pie plot *data*
$pie colours *colour1 colour2 ...*
$pie explode *segment*
$radial plot *data colour thickness*
$pie colours *colour1 colour2 ...*
$barchart plot *series ydata colour* ?*dir*? ?*brightness*?
$barchart config -option *value ...*
$barchart plot *series xdata colour* ?*dir*? ?*brightness*?
$barchart config -option *value ...*
$barchart plot *label yvalue colour*
$barchart config -option *value ...*
$ribbon line *xypairs colour*
$ribbon area *xypairs colour*
$boxplot plot *series label values*

$boxplot violin *series label values*
$timechart period *text time_begin time_end colour*
$timechart addperiod *time_begin time_end colour*
$timechart milestone *text time colour*
$timechart addmilestone *time colour*
$timechart vertline *text time colour*
$timechart hscroll *scrollbar*
$timechart vscroll *scrollbar*
$ganttchart task *text time_begin time_end completed*
$ganttchart milestone *text time colour*
$ganttchart vertline *text time*
$ganttchart connect *from to*
$ganttchart summary *text args*
$ganttchart color *keyword newcolor*
$ganttchart font *keyword newfont*
$ganttchart hscroll *scrollbar*
$ganttchart vscroll *scrollbar*
$isoplot plot rectangle *x1 y1 x2 y2 colour*
$isoplot plot filled-rectangle *x1 y1 x2 y2 colour*
$isoplot plot circle *xc yc radius colour*
$isoplot plot filled-circle *xc yc radius colour*
$table row *items*
$table separator
$table formatcommand *procname*
$table cellconfigure *args*
$ternary plot *series xcrd ycrd zcrd text dir*
$ternary line *series coords*
$ternary fill *series coords*
$ternary text *xtext ytext ztext*
$ternary ticklines *colour*
$timeline plot *series item start stop color*
$timeline vertline *text time args*
$dendrogram plot *series data*
::Plotchart::viewPort *w pxmin pymin pxmax pymax*
::Plotchart::worldCoordinates *w xmin ymin xmax ymax*
::Plotchart::world3DCoordinates *w xmin ymin zmin xmax ymax zmax*
::Plotchart::coordsToPixel *w x y*
::Plotchart::coords3DToPixel *w x y z*
::Plotchart::polarCoordinates *w radmax*
::Plotchart::polarToPixel *w rad phi*
::Plotchart::pixelToCoords *w x y*
::Plotchart::pixelToIndex *w x y*
::Plotchart::determineScale *xmin xmax inverted*
::Plotchart::determineScaleFromList *values inverted*
::Plotchart::plotconfig *charttype component property value*
::Plotchart::plotstyle *subcmd style args*

::Plotchart::eraseplot *anyplot*
::Plotchart::createTargetDiagram *w limits scale*
$target plot *series xvalues yvalues*
::Plotchart::createPerformanceProfile *w max*
$performance plot *series_and_data_pairs*
::Plotchart::createTaylorDiagram *w radius_data args*
$taylor plot *series stdev corr*
::Plotchart::createHeatmap *w rowlabels columnlabels args*
$heatmap plot row *label data*
$heatmap plot column *label data*
$heatmap plot cell *rowlabel columnlabel value*
$heatmap scale values *min max*
$heatmap scale colours *mincolour maxcolour*
::Plotchart::createCircleplot *w labels args*
$circleplot connect *label1 label2 colour width*
$circleplot modify *label args*
::Plotchart::plotmethod *charttype methodname plotproc*
::Plotchart::plotpack *w dir args*
$anyplot bindplot *event command args*
$anyplot bindlast *series event command*

Beschreibung

Plotchart ist ein reines Tcl-Paket, das auf die einfache Erstellung von xy-Diagrammen, Balkendiagrammen und anderen gängigen Arten grafischer Darstellungen ausgerichtet ist. Der Fokus liegt dabei eher auf der Benutzerfreundlichkeit als auf der Flexibilität. Die Prozeduren zur Erstellung eines Diagramms nutzen das gesamte Canvas-Fenster aus, wodurch das Layout des Diagramms vollständig automatisch erfolgt.

Dadurch entsteht ein xy-Diagramm mit beispielsweise zehn Zeilen Code:

```
    package require Plotchart

    canvas .c -background white -width 400 -height 200
    pack   .c -fill both

    #
    # Create the plot with its x- and y-axes
    #
    set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0 20.0}]

    foreach {x y} {0.0 32.0 10.0 50.0 25.0 60.0 78.0 11.0 } {
        $s plot series1 $x $y
    }

    $s title "Data series"
```

Ein Nachteil des Pakets könnte darin bestehen, dass es keine Datenverwaltung übernimmt. Wird der Canvas mit der Darstellung in der Größe geändert, muss die ganze Darstellung neu gezeichnet werden. Der Vorteil besteht jedoch darin, dass es eine Reihe von Darstellungs- und Diagrammtypen bietet:

- XY-Diagramme wie das oben gezeigte mit einer beliebigen Anzahl von Datenreihen.
- Stripcharts sind eine Art von XY-Diagrammen, bei denen die horizontale Achse automatisch angepasst wird. Das Ergebnis ist eine Art gleitendes Fenster über der Datenreihe.
- Polardiagramme, bei denen die Koordinaten polar statt kartesisch sind.
- Histogramme zur Darstellung statistischer Informationen.
- Balkendiagramme, Kreisdiagramme, Gantt-Diagramme, Zeitdiagramme.
- Isometrische Darstellungen, bei denen der Maßstab der Koordinaten in beiden Richtungen stets gleich ist, d. h., ein Kreis in Weltkoordinaten erscheint auf dem Bildschirm als Kreis.

  Diese Darstellungen lassen sich vergrößern, verkleinern und verschieben (*Hinweis:* am besten ohne eingezeichnete Achsen, die Routinen für Zoom und Verschieben unterscheiden die Achsen nicht), mit den Maustasten und der Taste Control sowie mit den Pfeiltasten und der Taste Control.
- Kreisdiagramme mit automatischer Skalierung zur Darstellung der Anteile.
- Balkendiagramme, entweder mit vertikalen oder horizontalen Balken, gestapelten Balken oder nebeneinander angeordneten Balken.
- Zeitdiagramme, in denen Balken einen Zeitraum anzeigen und Meilensteine oder andere wichtige Zeitpunkte durch Dreiecke dargestellt werden.
- 3D-Diagramme (sowohl zur Darstellung von Oberflächen als auch von 3D-Balken)

Mit Version 1.5 wurde ein neuer Befehl eingeführt: "plotconfig", mit dem sich die Darstellungsoptionen für bestimmte Arten von Diagrammen und Grafiken konfigurieren lassen (vgl. Konfigurationsoptionen und weitere Befehle) Mit Version 1.8.3 wurden mehrere neue Funktionen eingeführt, die eine größere Interaktivität ermöglichen (vgl. Interaktive Nutzung). Mit Version 2.0 ist es nun möglich, mehrere Diagramme oder Grafiken auf einem Canvas anzuordnen.

Befehle zur Erstellung von Diagrammen

Das Diagramm entsteht mit einem einzigen Befehl und wird danach mit Daten gefüllt:

::Plotchart::createXYPlot *w xaxis yaxis args*

Erzeugt ein neues xy-Diagramm (Konfigurationstyp: xyplot).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält. Für eine umgekehrte Achse, bei der der Maximalwert auf der linken Seite steht, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält. Für eine umgekehrte Achse, bei der der Maximalwert unten erscheint, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *args* (in)

Null oder mehr Optionen, die das Erscheinungsbild der Grafik beeinflussen:

- *-xlabels {labels}:* Benutzerdefinierte Beschriftungen für die x-Achse. Wenn die Beschriftungen numerisch sind, werden sie entsprechend der angegebenen Skalierung

positioniert; andernfalls werden sie in gleichen Abständen positioniert, basierend auf der Anzahl der Beschriftungen. Hinweis: Dies funktioniert nur, wenn die Schrittweite des Arguments "xaxis" eine leere Zeichenkette ist.
- *-ylabels {labels}:* Ebenso benutzerdefinierte Beschriftungen für die y-Achse.
- *-box {measures}:* Siehe Anordnung mehrerer Diagramme auf einem Canvas
- *-axesbox {measures}:* Siehe Anordnung mehrerer Diagramme auf einem Canvas
- *-timeformat format:* Siehe Verwendung von Datums- und Zeitangaben
- *-gmt 0/1:* Siehe Verwendung von Datums- und Zeitangaben
- *-axestozero:* Zeichnen Sie die Achsen am Ursprung (1) oder an den Rändern des Diagrammbereichs (0, Standard). Diese Option ist nur für XY-Diagramme implementiert.
- *-isometric:* Passen Sie die Skalierung der Achsen so an, dass ein Quadrat in den Koordinaten als Quadrat auf dem Bildschirm erscheint (1), oder verwenden Sie die vorgegebenen Extremwerte (0, Standard). Diese Option ist nur für XY-Diagramme implementiert.

::Plotchart::createStripchart *w xaxis yaxis args*

Erzeugt ein neues Streifendiagramm (Konfigurationstyp: "stripchart"). Der einzige Unterschied zu einem gewöhnlichen XY-Diagramm besteht darin, dass die x-Achse automatisch angepasst wird, wenn die x-Koordinate eines neuen Punktes den Maximalwert überschreitet.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält. Hinweis: Eine invertierte x-Achse wird bei dieser Art von Diagramm *nicht* unterstützt.

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält. Für eine umgekehrte Achse, bei der der Maximalwert unten erscheint, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *args* (in)

Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createTXPlot *w timeaxis xaxis args*

Erzeugt ein neues Zeit-x-Diagramm (Konfigurationstyp: txplot). Die horizontale Achse stellt das Datum und die Uhrzeit der Daten dar, die vertikale Achse die Werte selbst.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *timeaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge das anzuzeigende Mindest- und Höchstdatum bzw. die Mindest- und Höchstuhrzeit sowie die Schrittweite (*in Tagen*) für die Zeitachse enthält. Hinweis: Eine umgekehrte Zeitachse wird *nicht* unterstützt.

list *xaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die vertikale Achse enthält. Für eine umgekehrte Achse, bei der der

Maximalwert unten erscheint, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createXLogYPlot *w xaxis yaxis args*
Erzeugt ein neues xy-Diagramm, bei dem die y-Achse logarithmisch skaliert ist (Konfigurationstyp: xlogyplot).

Die Daten sollten wie bei einer linearen Skalierung angegeben werden, da die logarithmische Transformation intern vorgenommen wird.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält. Für eine umgekehrte Achse, bei der der Maximalwert auf der linken Seite steht, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *yaxis* (in)
Eine Liste mit zwei Elementen, die den Minimal- und den Maximalwert für die y-Achse in dieser Reihenfolge enthält. Hinweis: Eine invertierte logarithmische Achse wird *nicht* unterstützt.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createLogXYPlot *w xaxis yaxis args*
Erzeugt ein neues xy-Diagramm, bei dem die x-Achse logarithmisch skaliert ist (Konfigurationstyp: logxyplot).

Die Daten sollten wie bei einer linearen Skalierung angegeben werden, da die logarithmische Transformation intern vorgenommen wird.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit zwei Elementen, die den Minimal- und den Maximalwert für die x-Achse in dieser Reihenfolge enthält. Hinweis: Eine invertierte logarithmische Achse wird *nicht* unterstützt.

list *yaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält. Für eine umgekehrte Achse, bei der der Maximalwert auf der linken Seite erscheint, dafür: Maximalwert, Minimalwert und eine *negative* Schrittweite.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createLogXLogYPlot *w xaxis yaxis args*
Erzeugt ein neues xy-Diagramm, bei dem sowohl die x-Achse als auch die y-Achse

logarithmisch skaliert sind (Konfigurationstyp: logxlogyplot).

Die Daten sollten wie bei einer linearen Skalierung angegeben werden, da die logarithmische Transformation intern vorgenommen wird.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit zwei Elementen, die den Minimal- und den Maximalwert für die x-Achse in dieser Reihenfolge enthält. Hinweis: Eine invertierte logarithmische Achse wird *nicht* unterstützt.

list *yaxis* (in)
Eine Liste mit zwei Elementen, die den Minimal- und den Maximalwert für die y-Achse in dieser Reihenfolge enthält. Hinweis: Eine invertierte logarithmische Achse wird *nicht* unterstützt.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createPolarPlot *w radius_data args*
Erzeugt ein neues Polardiagramm (Konfigurationstyp: polarplot).

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *radius_data* (in)
Eine Liste mit zwei Elementen, die den maximalen Radius und die Schrittweite für die Radialachse in dieser Reihenfolge enthält.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::createWindrose *w radius_data sectors*
Erzeugt ein neues Windrosendiagramm. Das Diagramm besteht aus konzentrischen Kreisen, die durch das Argument *radius_data* definiert werden, sowie aus einer Anzahl von Sektoren (die durch das Argument *sectors* vorgegeben wird). Die Sektoren werden nach der Konvention "nautical" gezeichnet, d. h.: Der erste befindet sich auf der positiven y-Achse, der zweite rechts davon und so weiter im Uhrzeigersinn.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem das Diagramm angezeigt werden soll

list *radius_data* (in)
Eine Liste mit zwei Elementen, wobei das erste Element den maximalen Radius und das zweite Element den für die Kreise zu verwendenden Schritt angibt.

int *sectors*
Anzahl der zu verwendenden Sektoren (Standardwert: 16).

::Plotchart::createIsometricPlot *w xaxis yaxis stepsize*
Erzeugt eine neue isometrische Darstellung, bei der die vertikalen und horizontalen Koordinaten so skaliert sind, dass ein Kreis tatsächlich als Kreis erscheint (Konfigurationstyp: isometric).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit zwei Elementen, die in dieser Reihenfolge den Minimalwert und den Maximalwert für die x-Achse enthalten.

list *yaxis* (in)
Eine Liste mit zwei Elementen, die in dieser Reihenfolge den Minimalwert und den Maximalwert für die y-Achse enthalten.

float|noaxes *stepsize* (in)
Entweder die von beiden Achsen verwendete Schrittweite oder das Schlüsselwort **noaxes**, um der Darstellung mitzuteilen, dass sie den gesamten Bereich des Widgets nutzen und keine der Achsen zeichnen soll.

::Plotchart::createHistogram *w xaxis yaxis args*
Erzeugt ein neues Histogramm (Konfigurationstyp: histogram).

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält.

list *yaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

list *args* (in)
Null oder mehr Optionen – weitere Informationen stehen beim XY-Diagramm.

::Plotchart::create3DPlot *w xaxis yaxis zaxis args*
Erzeugt eine neue 3D-Grafik.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält.

list *yaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

list *zaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die z-Achse enthält.

list *args* (in)
Optionale Liste von Optionen (Schlüsselwörter-Werte-Paare). Derzeit unterstützt: -xlabels zum Festlegen der Beschriftungen für die x-Achse.

::Plotchart::create3DRibbonPlot *w yaxis zaxis*
Erzeugt ein neues 3D-Banddiagramm. Es handelt sich dabei um eine Vereinfachung des vollständigen 3D-Diagramms, bei der lediglich ein Band gezeichnet wird (die x-Achse wird

weggelassen).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

list *zaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die z-Achse enthält.

::Plotchart::createPiechart *w*

Erzeugt ein neues Kreisdiagramm (Konfigurationstyp: piechart).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

::Plotchart::createSpiralPie *w*

Erzeugt ein neues "spiral pie" (Konfigurationstyp: spiralpie), eine Variante des gewöhnlichen Kreisdiagramms. Der Wert dient zur Skalierung des Radius und nicht des Winkels. Standardmäßig werden die Daten sortiert.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

::Plotchart::createRadialchart *w names scale style*

Erzeugt ein neues Radialdiagramm (die Daten werden als Linie dargestellt, die die Speichen des Diagramms verbindet) (Konfigurationstyp: radialchart).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *names* (in)

Namen für die Speichen.

float *scale* (in)

Wert zur Bestimmung der Position der Daten entlang der Speichen.

string *style* (in)

Darstellungsstil des Diagramms (optional). Eine der folgenden Optionen:
- *lines* – Standardeinstellung: Die Daten werden als unabhängige Polylinien dargestellt.
- *cumulative* – Zeichnen Sie die Daten als Polylinien, wobei die Daten kumuliert werden.
- *filled* – Zeichnen Sie die Daten als gefüllte Polygone an den Stellen, an denen die Daten kumuliert sind.

::Plotchart::createBarchart *w xlabels yaxis noseries args*

Erzeugt ein neues Balkendiagramm mit vertikalen Balken (Konfigurationstyp: vertbars). Auf der horizontalen Achse werden die im Argument *xlabels* enthaltenen Beschriftungen angezeigt. Die durch *noseries* angegebene Anzahl der Reihen bestimmt sowohl die Breite der Balken als auch die Art und Weise, wie die Reihen gezeichnet werden.

Ist das Schlüsselwort **stacked** angegeben, werden die Reihen übereinander gestapelt gezeichnet. Andernfalls wird jede gezeichnete Reihe nach rechts versetzt dargestellt.

Die Anzahl der Reihen bestimmt die Breite der Balken, sodass Platz für diese Anzahl an Balken vorhanden ist. Steht statt einer ganzen Zahl wie 2 eine Gleitkommazahl wie 2.2, wird ein kleiner Abstand zwischen den Balkengruppen dargestellt – die Breite hängt dabei vom Bruchteil ab.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xlabels* (in)
Liste der Beschriftungen für die x-Achse. Ihre Länge bestimmt zudem die Anzahl der Balken, die pro Reihe dargestellt werden.

list *yaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

int|stacked *noseries* (in)
Die Anzahl der Datenreihen, die dargestellt werden sollen. Dies muss eine ganze Zahl größer als Null sein (wenn **stacked** nicht verwendet wird).

string *args* (in)
Ein oder mehrere Schlüssel-Wert-Paare, die die Optionen definieren. Speziell für Balkendiagramme: Der Wert "**-xlabelangle**" gibt den Winkel an, in dem die Beschriftungen gezeichnet werden sollen. Der Standardwert ist 0 – horizontaler Text. Diese Option ist besonders nützlich, wenn Sie relativ lange Beschriftungen haben.

::Plotchart::createHorizontalBarchart *w xaxis ylabel noseries*

Erzeugt ein neues Balkendiagramm mit horizontalen Balken (Konfigurationstyp: "horizbars"). Auf der vertikalen Achse werden die im Argument *ylabels* enthaltenen Beschriftungen angezeigt. Die durch *noseries* angegebene Anzahl der Reihen bestimmt sowohl die Breite der Balken als auch die Art und Weise, wie die Reihen gezeichnet werden.

Ist das Schlüsselwort **stacked** angegeben, werden die Reihen von links nach rechts gestapelt gezeichnet. Andernfalls wird jede gezeichnete Reihe nach oben versetzt dargestellt.

widget *w* (in)
Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xaxis* (in)
Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält.

list *ylabels* (in)
Liste der Beschriftungen für die y-Achse. Ihre Länge bestimmt zudem die Anzahl der Balken, die pro Reihe dargestellt werden.

int|stacked *noseries* (in)
Die Anzahl der Datenreihen, die dargestellt werden sollen. Dies muss eine ganze Zahl größer als Null sein (wenn **stacked** nicht verwendet wird).

::Plotchart::create3DBarchart *w yaxis nobars*

Erzeugt ein neues Balkendiagramm mit vertikalen 3D-Balken (Konfigurationstyp: 3dbars). Auf der horizontalen Achse werden die Beschriftungen der einzelnen Balken angezeigt. Die durch *nobars* angegebene Anzahl der Balken bestimmt deren Position und Breite. Die Farben können pro Balken variiert werden. (Dieser Diagrammtyp wurde von der Wiki-Seite zu 3D-Balken von

Richard Suchenwirth inspiriert.)

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

int *nobars* (in)

Die Anzahl der Balken, die dargestellt werden sollen.

::Plotchart::create3DRibbonChart *w names yaxis zaxis*

Erzeugt ein neues "ribbon chart" (Konfigurationstyp: 3dribbon). Hierbei handelt es sich um ein Diagramm, bei dem die Datenreihen als Bänder in einem dreidimensionalen Achsenkoordinatensystem dargestellt werden. Entlang der x-Achse (die "in den Bildschirm hinein" verläuft) werden die Namen aufgetragen, wobei jeder Name für eine einzelne Reihe steht. Der erste Plot-Befehl zeichnet die am weitesten entfernte Reihe, der zweite die davor liegende Reihe und so weiter.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

widget *w* (in)

Namen der Reihen, als Beschriftungen entlang der x-Achse dargestellt

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse (horizontal dargestellt!) enthält.

list *zaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die z-Achse (vertikal dargestellt) enthält.

int *nobars* (in)

Die Anzahl der Balken, die dargestellt werden sollen.

::Plotchart::createBoxplot *w xdata ydata orientation*

Erzeugt ein neues Boxplot-Diagramm mit horizontalen oder vertikalen Boxen (Box-and-Whiskers) (Konfigurationstyp: boxplot). Je nach Ausrichtung wird die x- oder y-Achse mit Beschriftungen dargestellt. Die Boxen werden auf der Grundlage der Rohdaten gezeichnet (siehe den Unterbefehl "plot" für diesen Diagrammtyp).

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *xdata* (in)

Hierbei handelt es sich entweder um eine Liste mit drei Elementen, die in dieser Reihenfolge (bei horizontaler Ausrichtung) den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält, oder um eine Liste von Beschriftungen für die x-Achse (bei vertikaler Ausrichtung). Die Länge der Beschriftungsliste bestimmt zudem die Anzahl der Felder, die gezeichnet werden können. Die Beschriftungen werden auch im Unterbefehl "plot" verwendet.

list *ydata* (in)

Hierbei handelt es sich entweder um eine Liste mit drei Elementen, die in dieser Reihenfolge (bei vertikaler Ausrichtung) den Minimalwert, den Maximalwert und die

Schrittweite für die y-Achse enthält, oder um eine Liste von Beschriftungen für die y-Achse (bei horizontaler Ausrichtung). Die Länge der Beschriftungsliste bestimmt zudem die Anzahl der Felder, die gezeichnet werden können. Die Beschriftungen werden auch im Unterbefehl "plot" verwendet.

string *orientation* (in)

Sind "horizontal" oder "vertical" angegeben, bestimmen sie die Ausrichtung der Boxen. Dieser optionale Wert (Standard: horizontal) bestimmt zudem die Interpretation der Argumente "xdata" und "ydata".

::Plotchart::createTimechart *w time_begin time_end args*

Erzeugt ein neues Zeitdiagramm (Konfigurationstyp: timechart). Die Zeitachse (= x-Achse) reicht von *time_begin* bis *time_end*, und der vertikale Abstand richtet sich nach der Anzahl der darzustellenden Elemente.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

string *time_begin* (in)

Die Startzeit in einem Format, das vom Befehl **clock scan** erkannt wird (z. B. "1. Januar 2004").

string *time_end* (in)

Die Endzeit in einem Format, das vom Befehl **clock scan** erkannt wird (z. B. "1. Januar 2004").

arguments *args* (in)

Die übrigen Argumente können wie folgt lauten:

- Die erwartete/maximale Anzahl von Elementen. Diese bestimmt den vertikalen Abstand. (Wenn angegeben, muss dies das erste Argument nach "time_end" sein.)
- Das Schlüsselwort -barheight und die Anzahl der Pixel pro Balken. Dies ist eine alternative Methode zur Bestimmung des vertikalen Abstands.
- Das Schlüsselwort -ylabelwidth und die Anzahl der Pixel, die für die Beschriftungen auf der y-Achse reserviert werden sollen.

::Plotchart::createGanttchart *w time_begin time_end args*

Erzeugt ein neues Gantt-Diagramm (Konfigurationstyp: ganttchart). Die Zeitachse (= x-Achse) reicht von *time_begin* bis *time_end*, und der vertikale Abstand richtet sich nach der Anzahl der darzustellenden Elemente. Über die entsprechenden Befehle können Sie anschließend Aufgaben und Verbindungen zwischen den Aufgaben hinzufügen.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

string *time_begin* (in)

Die Startzeit in einem Format, das vom Befehl **clock scan** erkannt wird (z. B. "1. Januar 2004").

string *time_end* (in)

Die Endzeit in einem Format, das vom Befehl **clock scan** erkannt wird (z. B. "1. Januar 2004").

arguments *args* (in)

Die übrigen Argumente können wie folgt lauten:

- Die erwartete bzw. maximale Anzahl von Elementen. Diese bestimmt den vertikalen Abstand. (Wird dieser Wert auf diese Weise angegeben, muss er das erste Argument nach "time_end" sein.)
- Die erwartete/maximale Breite des beschreibenden Textes (ungefähr in Zeichen; für den tatsächlich für den Text reservierten Platz wird davon ausgegangen, dass ein Zeichen etwa zehn Pixel breit ist). Der Standardwert beträgt 20. (Wird dieser Wert auf diese Weise angegeben, muss er das zweite Argument nach "time_end" sein.)
- Das Schlüsselwort -barheight und die Anzahl der Pixel pro Balken. Dies ist eine alternative Methode zur Bestimmung des vertikalen Abstands.
- Das Schlüsselwort -ylabelwidth und die Anzahl der Pixel, die für die Beschriftungen auf der y-Achse reserviert werden sollen.

::Plotchart::createRightAxis *w_or_plot yaxis*

Erzeugt einen Plot-Befehl, der anstelle der linken Achse die rechte Achse verwendet (Konfigurationstyp: von dem vorhandenen Plot übernommen). Das Canvas-Widget muss bereits einen gewöhnlichen Plot enthalten, da die horizontale Achse und andere Eigenschaften wiederverwendet werden. Verwenden Sie vorzugsweise den Plot-Befehl, da bei mehreren Plots in einem Canvas (auch beim Neudefinieren eines bestehenden Plots!) möglicherweise die falsche Geometrie verwendet wird.

Um Daten unter Verwendung der rechten Achse zu plotten, verwenden Sie diesen neuen Befehl; um Daten unter Verwendung der Achse *left* zu plotten, verwenden Sie den ursprünglichen Plot-Befehl.

widget *w_or_plot* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll, oder vorzugsweise der Plot-Befehl für die Grafik mit der linken Achse.

list *yaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die y-Achse enthält.

::Plotchart::createTableChart *w columns* ?widths?

Erzeugt einen Befehl zum Zeichnen einer Tabelle. Sie können verschiedene Befehle verwenden, um die einzelnen Zeilen der Tabelle zu zeichnen, die Anzahl der Spalten ist jedoch festgelegt. (Beispiele finden Sie unter Tabellendiagramme)

widget *w* (in)

Name des Canvas-Widgets, in dem die Tabelle angezeigt werden soll.

list *columns* (in)

Die Überschriften der Spalten in der Tabelle. Die Anzahl der Elemente bestimmt die Anzahl der Spalten.

list *widths* (in)

Wird ein Wert angegeben, so handelt es sich entweder um einen einzelnen Wert – die Breite in Pixeln für alle Spalten – oder um die Breite der jeweiligen Spalten. Wird kein Wert angegeben, erstreckt sich die Tabelle über die gesamte Breite des canvases (abzüglich der Außenabstände).

::Plotchart::createTernaryDiagram *w args*

Erzeugt einen Befehl zum Zeichnen eines ternären Diagramms (Konfigurationstyp: ternary).

Sie können einzelne (beschriftete) Datenpunkte im Diagramm sowie Linien und ausgefüllte Polygone zeichnen.

widget *w* (in)

Name des Canvas-Widgets, in dem das ternäre Diagramm angezeigt werden soll.

list *args* (in)

Null oder mehr Optionen-Wert-Paare zur Steuerung der Position und der Darstellung des Diagramms. Zusätzlich zu den Positionierungsoptionen (-box und -axesbox) unterstützt das Diagramm: *-fractions ja/nein*, um die Zahlen 0 bis 1 anstelle von 0 bis 100 % anzuzeigen, sowie *-steps Zahl*, um die Anzahl der Beschriftungen entlang der drei Seiten zu beeinflussen.

::Plotchart::createNormalPlot *w xscale args*

Erzeugt einen Befehl zum Zeichnen eines Normalverteilungsdiagramms – nützlich, um zu untersuchen, ob ein Datensatz normalverteilt ist oder nicht. In diesem Fall liegen die Daten auf oder nahe der Diagonalen. Es handelt sich somit um eine spezielle Prozedur zur Diagrammerstellung.

Die Einzelheiten der Prozedur wurden aus der Funktion *qqnorm* des "R"-Pakets *stats* übernommen und sind auf Wikipedia beschrieben.

Da die Implementierung dieses Diagrammtyps auf dem Paket *math::statistics* basiert, ist sie nur verfügbar, wenn dieses Paket geladen werden kann.

widget *w* (in)

Name des Canvas-Widgets, in dem die normale Darstellung angezeigt werden soll.

list *xscale* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält. Daraus wird die Skalierung der y-Achse bestimmt. *Wichtig:* Die Skalierung ist in Bezug auf die normierten Daten anzugeben, d. h.: 0 steht für den Mittelwert der Daten, 1 für eine Standardabweichung vom Mittelwert usw.

list *args* (in)

Null oder mehr Paare mit Optionen zur Steuerung der Position und der Darstellung des Diagramms – weitere Einzelheiten stehen beim XY-Diagramm.

::Plotchart::createStatusTimeline *w xaxis ylabel args*

Erzeugt einen Befehl zum Zeichnen einer sogenannten Status-Zeitleiste. Das Layout ähnelt einem horizontalen Balkendiagramm, jedoch werden die Balken in verschiedenen Farben dargestellt, wobei jede Farbe den Status des Elements im Zeitverlauf (auf der horizontalen Achse) repräsentiert.

widget *w* (in)

Name des Canvas-Widgets, in dem das ternäre Diagramm angezeigt werden soll.

list *xaxis* (in)

Eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die x-Achse enthält.

list *ylabels* (in)

Liste der Beschriftungen für die y-Achse. Ihre Länge bestimmt zudem die Anzahl der Balken, die pro Reihe dargestellt werden.

list *args* (in)

Null oder mehr Paare mit Werten für Optionen zur Anpassung der Position und der Darstellung des Diagramms. Zusätzlich zu den Positionierungsoptionen (-box und -axesbox) unterstützt das Diagramm: *-xaxis ja/nein*, um die Anzeige der numerischen Beschriftungen ein- oder auszuschalten.

::Plotchart::createDendrogram *w args*

Erzeugt ein Dendrogramm, um hierarchisch organisierte Daten darzustellen. Die Beschriftungen, die die Endpunkte der Hierarchie kennzeichnen, können entweder alle auf derselben Ebene oder an den Endpunkten der jeweiligen Zweige eingezeichnet werden. Dies wird über die Option "-extend" gesteuert.

Die Daten sind als verschachtelte Listen organisiert, beispielsweise:

```
    \$dendrogram plot data { {"Label1"  "Label2"} {F {A B} {C {D E}}} }
```

Unterlisten mit mehr als einem Element gelten als Verzweigung und werden entsprechend dargestellt. Unterlisten mit nur einem Element gelten als Blatt und werden als Text dargestellt. Da sich eine Zeichenfolge mit zwei oder mehr Wörtern nicht von einer Liste mit zwei oder mehr Elementen unterscheiden lässt, wurde für diesen Fall eine Regelung getroffen: Wenn das erste Element "LEAF" ist, wird davon ausgegangen, dass die Liste ausschließlich aus Blättern besteht, und alle Elemente – selbst wenn es sich tatsächlich um verschachtelte Listen handelt – werden als Text dargestellt:

```
    \$dendrogram plot data { {LEAF "Label 1"  "Label 2"} {F {A B} {C {D E}}} }
```

widget *w* (in)

Name des Canvas-Widgets, in dem das Dendrogramm angezeigt werden soll.

list *args* (in)

Null oder mehr Paare mit Optionen-Werten:

- Die Option "-direction" kann einen der folgenden Werte annehmen: left-right, right-left, top-bottom, bottom-top, wobei "top-bottom" der Vorgabewert ist. Der Baum wird entsprechend dieser Option gezeichnet; die Beschriftungen erscheinen entweder horizontal oder vertikal am Ende des Baums.
- Die Option "-extend" mit dem Wert 1 (Standard) oder 0 (keine Erweiterung), um anzugeben, dass alle Beschriftungen auf derselben Ebene erscheinen sollen.

Diagrammmethoden

Jeder der im letzten Abschnitt erläuterte Befehl zur Erstellung gibt den Namen eines neuen Objektbefehls zurück, mit dem sich die Grafik oder das Diagramm bearbeiten lässt. Welche Unterbefehle für einen Diagrammbefehl verfügbar sind, hängt vom Typ des Diagramms ab.

Allgemeine Unterbefehle für alle Diagrammtypen. \$anyplot ist der vom Erstellungsbefehl zurückgegebene Befehl:

$anyplot title *text position*

Legt den Titel des gesamten Diagramms fest.

string *text* (in)

Der Text des zu erstellenden Titels.

string *position* (in)

Die Position des Titels. Die Standardposition ist "center", Sie können jedoch alternativ auch "left" oder "right" verwenden. Sie können mehrere Titel mit unterschiedlichen Positionen verwenden.

$anyplot subtitle *text*

Legt die Unterüberschrift für das gesamte Diagramm fest.

string *text* (in)

Der Text des zu erstellenden Untertitels.

$anyplot canvas

Gibt den Namen der Canvas zurück (bzw. den Alias, falls Sie innerhalb eines Canvas mehrere Diagramme verwenden). Verwenden Sie diesen Wert für die Transformationen der Koordinaten.

$anyplot saveplot *filename args*

Zeichnet die Grafik mithilfe von PostScript in eine Datei.

string *filename* (in)

Enthält den Pfadnamen der Datei, in die die Grafik geschrieben wird.

list *args* (in)

Wenn die Standard-PostScript-Ausgabe verwendet wird, kann die Option -plotregion angegeben werden, um die gesamte Grafik (Wert: bbox) zu speichern, unabhängig davon, was im Fenster sichtbar ist. Standardmäßig (Wert: window) wird nur der sichtbare Teil der Grafik ausgegeben.

Optional lässt sich -format "some picture format" setzen, um die Grafik in einer anderen Datei als einer PostScript-Datei zu speichern. Dies setzt jedoch voraus, dass das Img-Paket den eigentlichen Auftrag übernimmt.

Hinweis: Da das Fenster, in dem sich die Grafik befindet, vollständig sichtbar sein muss, bevor Img sie erfolgreich erfassen kann, wird es zunächst in den Vordergrund geholt. Auf einigen Systemen, beispielsweise unter Linux mit KDE, erfolgt das Heranholen eines Fensters nicht automatisch, sondern das Fenster in der Taskleiste braucht einen Klick. Ähnliches gilt für Windows XP.

Unter bestimmten Umständen scheint es zu Problemen zu kommen; daher wartet die Prozedur, anstatt auf die Sichtbarkeit des Fensters zu warten, einfach zwei Sekunden. Das ist zwar nicht ideal, scheint aber besser zu funktionieren.

$anyplot xtext *text*

Legt den Titel der (horizontalen) x-Achse für jene Diagramme fest, die über eine gerade x-Achse verfügen.

string *text* (in)

Der Text der zu zeichnenden Beschriftung auf der x-Achse.

$anyplot ytext *text*

Legt den Titel der (horizontalen) y-Achse für jene Diagramme fest, die über eine gerade y-Achse verfügen.

string *text* (in)

Der Text der zu zeichnenden Beschriftung auf der y-Achse.

$anyplot vtext *text*
Zeichnet eine Beschriftung *vertical* an die y-Achse. Hinweis: Hierfür ist Tk 8.6 oder höher erforderlich; bei älteren Versionen hat dies keine Auswirkung.
string *text* (in)
Der Text soll an der y-Achse ausgerichtet werden

$anyplot xsubtext *text*
Gibt den Untertext der (horizontalen) x-Achse für jene Diagramme an, die über eine gerade x-Achse verfügen. Dieser Text wird unterhalb des Haupttextes eingezeichnet.

Da hierfür die Positionierung des Haupttextes und die Festlegung der Außenabstände erforderlich sind, müssen Sie die Option "usesubtext" für die untere Achse über den Befehl "plotstyle" festlegen. Die relevanten Optionen lauten: usesubtext, subtextcolor und subtextfont.
string *text* (in)
Der sekundäre Text der zu zeichnenden Beschriftung auf der x-Achse.

$anyplot ysubtext *text*
Legt den Untertext der (vertikalen) y-Achse für jene Diagramme fest, die eine gerade y-Achse aufweisen. Dieser Text wird unterhalb des Haupttextes eingezeichnet, und zwar sowohl für die linke als auch für die rechte Achse.

Da hierfür die Positionierung des Haupttextes und die Festlegung der Außenabstände erforderlich sind, müssen Sie die Option "usesubtext" für die linke oder rechte Achse über den Befehl "plotstyle" festlegen. Die entsprechenden Optionen lauten: "usesubtext", "subtextcolor" und "subtextfont".
string *text* (in)
Der zu zeichnende sekundäre Text der Beschriftung auf der y-Achse.

$anyplot vsubtext *text*
Legt den Untertext der (vertikalen) y-Achse für jene Diagramme fest, die eine gerade y-Achse aufweisen. Dieser Text wird *rechts* vom Haupttext eingezeichnet, sowohl für die linke als auch für die rechte Achse.

Da hierfür die Positionierung des Haupttextes und die Festlegung der Außenabstände erforderlich sind, müssen Sie die Option "usesubtext" für die linke oder rechte Achse über den Befehl "plotstyle" festlegen. Die entsprechenden Optionen lauten: usevsubtext, vsubtextcolor und vsubtextfont. (Beachten Sie den Hinweis "v", um diese Option von dem Text am oberen Rand einer vertikalen Achse zu unterscheiden, der über *$anyplot ytext* oder *$anyplot ysubtext* gezeichnet wird.)
string *text* (in)
Der sekundäre (vertikale) Text der zu zeichnenden y-Achsenbeschriftung.

$anyplot xconfig -option *value* ...
Legt einen oder mehrere Konfigurationsparameter für die x-Achse fest. Folgende Optionen werden unterstützt:
format fmt

Das Format für die Zahlen entlang der Achse.

ticklength length
Die Länge der Skalenstriche (in Pixeln).

ticklines boolean
Ob Gitterlinien gezeichnet werden sollen (**true**) oder nicht (**false**).

scale scale_data
Neue Skalierungsdaten für die Achse, d. h. eine Liste mit drei Elementen, die in dieser Reihenfolge den Minimalwert, den Maximalwert und die Schrittweite für die Achse enthält.

Achtung: Wenn Sie diese Option aktivieren, werden alle Daten aus der Darstellung gelöscht.

$anyplot yconfig -option *value* ...
Legt einen oder mehrere Konfigurationsparameter für die y-Achse fest. Diese Methode akzeptiert dieselben Optionen und Werte wie die Methode **xconfig**.

$anyplot background *part colour_or_image dir* ?brightness?
Den Hintergrund eines Teils des Diagramms festlegen

string *part*
Welcher Teil des Diagramms: "axes" für den Achsenbereich und "plot" für den inneren Teil. Die Interpretation hängt von der Art des Diagramms ab. Zwei weitere Möglichkeiten sind:
- *image*, wobei in diesem Fall ein vordefiniertes Bild als Hintergrund der Grafik geladen wird.
- *gradient*, wobei der Hintergrund in verschiedenen Schattierungen der angegebenen Farbe dargestellt wird. Das Argument "dir" legt die Richtung fest, in der die Farbe heller wird.

string *colour_or_image*
Farbe für diesen Teil oder der Name des Bildes, wenn "part" gleich "image" ist

string *dir*
Richtung des Farbverlaufs. Eine von top-down, bottom-up, left-right oder right-left.

string *brightness*
Ob die Farbe heller (bright) oder dunkler (dark) wird. Voreinstellung ist bright.

$anyplot xticklines *colour* ?dash?
Zeichnet an jedem Skalenstrich eine vertikale Gitterlinie ein

string *colour*
Farbe der Linien. Durch Festlegen einer leeren Farbe ("") werden diese wieder entfernt. Die Voreinstellung ist "black".

string *dash*
Optionales Argument, um das Strichmuster für die Linien festzulegen. Der Standardwert lautet "lines". Mögliche Werte: lines, dots1, dots2, dots3, dots4, dots5. Der tatsächliche Effekt hängt von der Plattform ab.

$anyplot yticklines *colour* ?dash?
Zeichnet an jedem Skalenstrich eine horizontale Gitterlinie ein

string *colour*
Farbe der Linien. Durch Festlegen einer leeren Farbe ("") werden diese wieder entfernt. Die

Voreinstellung ist "black".

string *dash*
Optionales Argument, um das Strichmuster für die Linien festzulegen. Der Standardwert lautet "lines". Mögliche Werte: lines, dots1, dots2, dots3, dots4, dots5. Der tatsächliche Effekt hängt von der Plattform ab.

$anyplot legend *series text* ?spacing?
Fügt der Legende einen Eintrag hinzu. Die Reihe bestimmt, welches grafische Symbol verwendet werden soll. (Als Nebeneffekt wird die Legende tatsächlich gezeichnet.)

string *series*
Name der Datenreihe. Dieser bestimmt die Farbe der Linie und das Symbol (falls vorhanden), das gezeichnet wird.

string *text*
Text, der neben der Linie/dem Symbol eingefügt werden soll.

integer *spacing*
Optionales Argument, um den vertikalen Abstand zwischen den Einträgen festzulegen (in Pixeln). (Beachten Sie, dass dieser Abstand später wiederverwendet wird.)

$anyplot removefromlegend *series*
Entfernt einen Eintrag für eine Reihe aus der Legende und zeichnet ihn erneut.

string *series*
Name der zu löschenden Datenreihe.

$anyplot legendconfig -option *value* ...
Legt eine oder mehrere Optionen für die Legende fest. Die Legende wird als Rechteck dargestellt, in dem sich Text und Grafiken befinden.

background colour
Farbe des Hintergrunds (Voreinstellung white). Die leere Zeichenkette ergibt eine durchsichtige Legende.

border colour
Farbe des Rahmens (Voreinstellung black). Die leere Zeichenkette lässt den Rahmen weg.

canvas c
Zeichnet die Legende in einem separaten Canvas-Widget. Dadurch haben Sie die Möglichkeit, die Legende außerhalb der eigentlichen Grafik zu positionieren.

font font
Legt die Schriftart fest, die zum Zeichnen des Textes neben dem Symbol verwendet werden soll.

legendtype type
Überschreibt den für den Diagrammtyp vorgegebenen Legendentyp. Möglich sind rectangle oder line.

position corner
Position der Legende. Möglich sind top-left, top-right, bottom-left oder bottom-right. Voreinstellung ist top-right.

spacing pixels
Senkrechter Abstand zwischen den Legendeneinträgen, in Pixeln. bottom-left oder bottom-right. Voreinstellung ist 10 Pixel.

order string
Legt die Reihenfolge der Legendenelemente fest – "normal" ist die Reihenfolge, in der die Einträge definiert sind, "reverse" genau das Gegenteil. (Standard: normal)

$anyplot balloon *x y text dir*
Fügt dem Diagramm Sprechblasentext hinzu (außer bei 3D-Diagrammen). Der Pfeil zeigt auf die angegebenen x- und y-Koordinaten. Bei xy-Diagrammen und ähnlichen Darstellungen beziehen sich die Koordinaten direkt auf die Achsen; bei vertikalen Balkendiagrammen wird die x-Koordinate als Anzahl der Balken minus 1 gemessen, bei horizontalen Balkendiagrammen gilt Ähnliches.

float *x*
X-Koordinate des Punktes, auf den der Pfeil des Ballons zeigen wird.

float *y*
Y-Koordinate des Punktes, auf den der Pfeil des Ballons zeigen wird.

string *text*
Text, der in die Sprechblase eingefügt werden soll.

string *dir*
Pfeilrichtung, eine der folgenden: Norden, Nordosten, Osten, Südosten, Süden, Südwesten, Westen oder Nordwesten.

$anyplot balloonconfig *args*
Konfiguriert den Sprechblasentext für die Grafik. Die neuen Einstellungen werden für den nächsten Sprechblasentext verwendet.

font fontname
Für den Text zu verwendende Schriftart

justify left|center|right
Möglichkeit zur Ausrichtung von mehrzeiligem Text

textcolour colour
Farbe für den Text (Synonym: textcolor)

background colour
Farbe für den Ballon

outline colour
Farbe der Gliederung des Ballons

margin value
Außenabstand um den Text herum (in Pixeln)

rimwidth value
Breite der Gliederung des Ballons (in Pixeln)

arrowsize value
Längenfaktor für den Pfeil (in Pixeln)

$anyplot plaintext *x y text dir*
Fügt dem Diagramm einfachen Text hinzu (mit Ausnahme von 3D-Diagrammen). Der Text wird an den angegebenen x- und y-Koordinaten positioniert. Bei xy-Diagrammen und ähnlichen Darstellungen beziehen sich die Koordinaten direkt auf die Achsen; bei vertikalen Balkendiagrammen wird die x-Koordinate als Anzahl der Balken minus 1 gemessen, bei

horizontalen Balkendiagrammen gilt Ähnliches.

float *x*
X-Koordinate der Textposition

float *y*
Y-Koordinate der Textposition

string *text*
Zu zeichnender Text.

string *dir*
Anker für den Text in eine der folgenden Richtungen: Norden, Nordosten, Osten, Südosten, Süden, Südwesten, Westen oder Nordwesten.

$anyplot plaintextconfig *args*
Konfiguriert die Klartext-Anmerkung für die Grafik. Die neuen Einstellungen werden für den nächsten Klartext verwendet.

font fontname
Für den Text zu verwendende Schriftart

justify left|center|right
Möglichkeit zur Ausrichtung von mehrzeiligem Text

textcolour colour
Farbe für den Text (Synonym: textcolor)

$anyplot object *itemtype series args*
Zeichnet ein Canvas-Element in das Diagramm ein, dessen Koordinaten anhand des Koordinatensystems des Diagramms skaliert werden. Neben den üblichen Canvas-Typen gibt es circle, dot und cross.

Hinweis: Derzeit implementiert für xy-Diagramme, (vertikale und horizontale) Balkendiagramme sowie Kreisdiagramme.

Hinweis: Um einen Eintrag in der Legende für das Objekt hinzuzufügen, können Sie den Unterbefehl *dataconfig* mit dem Typ "rectangle" verwenden. Dadurch wird ein Rechteck angezeigt.

string *itemtype* (in)
Name eines Standard-Canvas-Elements oder "circle", "dot" oder "cross"

string *series* (in)
Die Datenreihe, zu der sie gehört und die zur Festlegung der Standard-Zeichen-Optionen verwendet wird

list *args* (in)
Liste der Koordinaten und Zeichenoptionen

$anyplot deletedata
Entfernt die Linien, Symbole und sonstigen grafischen Objekte, die mit den eigentlichen Daten in Verbindung stehen, aus dem Diagramm.

Hinweis: Derzeit nur für XY-Diagramme implementiert.

Hinweis: Die vorhandenen Optionen für Datenreihen und den Legenden-Eintrag bleiben

unverändert.

Hinweis: Derzeit treten Nebenwirkungen auf, wenn der Canvas mehr als ein Diagramm enthält.

Hinweis: Die Befehle **xconfig** und **yconfig** sind derzeit nur für XY-Diagramme implementiert, und nur die Option **-format** hat eine Wirkung.

Für *XY-Diagramme*, *Streifendiagramme*, *Histogramme* und *Zeit-X-Diagramme*:

$xyplot plot *series xcrd ycrd*

Fügt dem Diagramm einen Datenpunkt hinzu.

string *series* (in)
Name der Datenreihe, zu der der neue Datenpunkt gehört.

float *xcrd* (in)
X-Koordinate des neuen Punktes. (Bei Zeit-x-Diagrammen muss es sich um ein gültiges Datum/eine gültige Uhrzeit handeln, die mit dem Befehl *clock scan* ausgelesen werden kann.)

float *ycrd* (in)
Y-Koordinate des neuen Punktes.

Für *normale Handlungen*:

$normalplot plot *series mean stdev data*

Stellt den Datensatz unter Verwendung des angegebenen Mittelwerts und der Standardabweichung grafisch dar.

Da Sie den Mittelwert und die Standardabweichung separat angeben, kann die Darstellung für mehrere Datenreihen oder zur Ergänzung einer bestehenden Datenreihe verwendet werden.

string *series* (in)
Name der Datenreihe – dient zur Festlegung der Darstellung

float *mean* (in)
Angenommener Mittelwert des Datensatzes.

float *stdev* (in)
Angenommene Standardabweichung des Datensatzes.

list *data* (in)
Verzeichnis der Daten, aus denen sich der Datensatz zusammensetzt

$normalplot diagonal

Zeichnet eine diagonale Linie ein, die den idealen, normalverteilten Datensatz darstellt.

Für *xy-, x-log-y-, log-x-y- und log-x-log-y-Diagramme* steht der zusätzliche Befehl *plotlist* zur Verfügung, der sich besonders für die Darstellung großer Datenmengen eignet:

$xyplot plotlist *series xlist ylist every*

Stellt eine Datenreihe als Ganzes dar. Wenn Symbole verlangt werden, zeichnen Sie diese nur für jeden N-ten Datenpunkt ein.

string *series* (in)
Name der Datenreihe, zu der die neue Zeile gehört.

float *xlist* (in)
Liste der X-Koordinaten für die Datenreihe.

float *ycrd* (in)
Liste der Y-Koordinaten für die Datenreihe.

int *every* (in)
Optionales Argument, das angibt, wie oft ein Symbol (falls vorhanden) gezeichnet werden soll. Wird es weggelassen, wird eine einfache Heuristik verwendet: N = √(Anzahl der Datenpunkte).

Weitere Befehle für *xy-, x-log-y-, log-x-y- und log-x-log-y-Diagramme* sind *region* und *minmax* zum Zeichnen ausgefüllter Polygone:

$xyplot region *series xlist ylist*
Zeichnet ein ausgefülltes Polygon (Bereich). Die Konfiguration der Serie wirkt sich wie folgt auf das Polygon aus: *-fillcolour* wird zum Ausfüllen des Polygons verwendet, *-colour* wird für die Umrandung verwendet (setzen Sie den Wert auf {}, wenn keine Umrandung erforderlich ist), und *-width* bestimmt die Breite der Umrandung.

string *series* (in)
Name der Datenreihe, zu der die neue Region gehört.

float *xlist* (in)
Liste der X-Koordinaten für den Bereich.

float *ycrd* (in)
Liste der Y-Koordinaten für den Bereich.

$xyplot minmax *series xcrd ymin ymax*
Zeichnet einen ausgefüllten Streifen, der ein Minimum und ein Maximum darstellt. Die Konfiguration der Reihe wirkt sich wie folgt auf das Polygon aus: *-fillcolour* wird zum Ausfüllen des Polygons verwendet, *-colour* wird für die Begrenzung verwendet (setzen Sie den Wert auf {}, wenn keine Begrenzung erforderlich ist), und *-width* bestimmt die Breite der Begrenzung.

Die Argumente *ymin* und *ymax* können leer gelassen werden, um einen zusätzlichen Scheitelpunkt im Streifen zu erhalten. Wenn beide leer sind, wird ein neuer Streifen begonnen. Für optimale Ergebnisse sollte die x-Koordinate in aufsteigender Reihenfolge festgelegt werden.

string *series* (in)
Name der Datenreihe, zu der der neue Min/Max-Streifen gehört.

float *xcrd* (in)
X-Koordinate des Streifens.

float *ymin* (in)
Die minimale y-Koordinate für den Streifen.

float *ymax* (in)
Die maximale y-Koordinate für den Streifen.

Note on histograms:

Bei Histogrammen wird die angegebene x-Koordinate als die x-Koordinate der *rechten* Seite des Balkens (bzw. des Liniensegments) interpretiert. Der erste Balken beginnt links an der y-Achse. Um den gesamten Bereich der x-Achse vollständig auszufüllen, sollten Sie einen Balken an der maximalen x-Koordinate zeichnen.

Für Histogramme können Sie auch den Befehl **plotcumulative** verwenden:

$histogram plotcumulative *series xcrd ycrd*
siehe unten ...

Die Argumente haben genau dieselbe Bedeutung wie beim Befehl **plot**, jedoch werden die Daten zu den bisherigen Werten addiert.

Für *xy-Diagramme*:

$xyplot trend *series xcrd ycrd*
Zeichnet anhand der bisher angegebenen Daten eine Trendlinie ein bzw. aktualisieren Sie diese.
string *series* (in)
Name der Datenreihe, zu der die Trendlinie gehört.
float *xcrd* (in)
X-Koordinate des neuen Datenpunkts
float *ycrd* (in)
Y-Koordinate des neuen Datenpunkts

$xyplot rchart *series xcrd ycrd*
Zeichnet die Daten auf dieselbe Weise wie bei der Methode "plot" ein, fügen Sie jedoch zwei Linien hinzu, die den erwarteten Bereich (+/- 3*Standardabweichung) der Daten anzeigen.
string *series* (in)
Name der Datenreihe, zu der der Datenpunkt gehört.
float *xcrd* (in)
X-Koordinate des neuen Datenpunkts
float *ycrd* (in)
Y-Koordinate des neuen Datenpunkts

$xyplot interval *series xcrd ymin ymax* ?ycentr?
Fügt dem Diagramm ein vertikales Fehlerintervall hinzu. Das Intervall wird von ymin bis ymax eingezeichnet. Wenn das Argument ycentr angegeben wird, wird an dieser Position ein Symbol eingezeichnet.
string *series* (in)
Name der Datenreihe, zu der das Intervall gehört.
float *xcrd* (in)
X-Koordinate des Intervalls
float *ymin* (in)
Minimale y-Koordinate des Intervalls.
float *ymax* (in)
Maximale y-Koordinate des Intervalls.
float *ycentr* (in)
Y-Koordinate, an der das Symbol gezeichnet werden soll (optional)

$xyplot box-and-whiskers *series xcrd ycrd*
Zeichnet ein Box-Whisker-Diagramm in die Grafik ein. Wenn das Argument *xcrd* eine Liste mehrerer Werte und das Argument *ycrd* einen einzelnen Wert enthält, wird ein horizontales Box-Whisker-Diagramm gezeichnet, wobei die Quartile anhand der in *xcrd* enthaltenen Werte ermittelt werden.

Wenn das Argument *ycrd* hingegen eine Liste mehrerer Werte enthält und das Argument *xcrd* einen einzelnen Wert enthält, wird ein vertikaler Kasten gezeichnet, wobei die Quartile anhand von *ycrd* bestimmt werden. (Es muss genau eine Liste mit mehreren Werten vorhanden sein. Andernfalls wird ein Fehler gemeldet.)

Die Option -boxwidth des Befehls "dataconfig" legt die Breite (bzw. Höhe) des Kastens fest (Standard: 10 Pixel).

Die Option -whiskers des Befehls "dataconfig" legt fest, ob die Whisker bis zu den Extremwerten (Wert: "extremes"), bis zum 1,5-fachen des Interquartilsabstands (Wert: "IQR" oder "iqr") oder gar nicht (Wert: "none") gezeichnet werden. Wenn der Wert bei "IQR" (Großbuchstaben) liegt, werden auch Extremwerte angezeigt (vom 1,5- bis zum 3-fachen des IQR als Punkte, oberhalb des 3-fachen IQR als Sterne). Wenn der Wert "iqr" (Kleinbuchstaben) vorhanden ist, werden keine Extremwerte angezeigt (Standardwert: IQR).

Die Option -whiskerwidth des Befehls "dataconfig" legt die Dicke der Linie fest, mit der die Whisker gezeichnet werden (Standard: 1 Pixel).

Die Option -mediancolour des Befehls "dataconfig" legt die Farbe der Linie fest, mit der der Median innerhalb des Kastens gezeichnet wird (Standard: wie bei -colour).

Die Option -medianwidth des Befehls "dataconfig" legt die Dicke der Linie fest, mit der der Median innerhalb des Kastens gezeichnet wird (Standard: 1 Pixel).

string *series* (in)
Name der Datenreihe, zu der das Box-and-Whiskers-Diagramm gehört.

float *xcrd* (in)
X-Koordinate des Rechtecks oder eine Liste von Werten.

float *ymin* (in)
Y-Koordinate des Rechtecks oder eine Liste von Werten.

Der Kasten endet am 1. und 3. Quartil, während die Whisker standardmäßig so dargestellt werden, dass sie sich über 1,5 IQR (Interquartilsabstand) vom 1. und 3. Quartil erstrecken.

$xyplot violin *series xcrd ycrd*

Zeichnet anhand des angegebenen Datensatzes ein Violin-Diagramm. Es handelt sich dabei um eine Alternative zur Box-and-Whiskers-Darstellung. Genau wie beim Unterbefehl "box-and-whiskers" wird eine horizontale "Violine" gezeichnet, wenn das Argument *xcrd* eine Liste mehrerer Werte und das Argument *ycrd* einen einzelnen Wert enthält.

Wenn das Argument *ycrd* hingegen eine Liste mehrerer Werte und das Argument *xcrd* einen einzelnen Wert enthält, wird eine vertikale "Violine" gezeichnet.

Die Option -violinwidth des Befehls "dataconfig" legt die Breite (oder Höhe) der Violine fest (Standard: 50 Pixel).

Die Option -colour (-color) legt die Farbe der Gliederung fest, und die Option -fillcolour (-fillcolor) bestimmt die Farbe, mit der die Violine ausgefüllt wird (Vorgaben: black und nicht gefüllt).

string *series* (in)
Name der zu verwendenden Datenreihe.

float *xcrd* (in)
X-Koordinate der Geige oder eine Liste von Werten.

float *ymin* (in)
Y-Koordinate der Geige oder eine Liste von Werten.

$xyplot vector *series xcrd ycrd ucmp vcmp*
Zeichnet einen Vektor in die Grafik ein. Der Vektor kann entweder als kartesische Koordinaten oder als Länge/Winkel angegeben werden, wobei der Winkel in Grad angegeben wird und entweder nach der mathematischen Konvention oder nach nautical. (Siehe den Unterbefehl "vectorconfig")
string *series* (in)
Name der Serie, zu der der Vektor gehört. Bestimmt das Erscheinungsbild und die Interpretation.
float *xcrd* (in)
X-Koordinate des Punktes, an dem der Pfeil erscheint
float *ycrd* (in)
Y-Koordinate des Punktes, an dem der Pfeil erscheint
float *ucmp* (in)
Die x-Komponente oder die Länge des Vektors
float *ycentr* (in)
Y-Komponente oder der Winkel des Vektors

$xyplot vectorconfig *series* -option *value* ...
Legt die Zeichenoptionen der Vektoren einer Reihe fest.
string *series* (in)
Name der Reihe, zu der der Vektor gehört.

Die Optionen können eine der folgenden sein:
colour
Die Farbe des Pfeils (Standard: schwarz; Synonym: Farbe)
scale value
Der Skalierungsfaktor, der zur Umrechnung der Pfeillänge in eine Pixelanzahl verwendet wird (Standardwert: 1,0)
centred onoff
Logischer Wert. Die xy-Koordinaten sind der Anfang des Pfeils oder sein Mittelpunkt (Voreinstellung 0; Synonym: centered).
type keyword
Deutung der Vektorkomponenten. "cartesian" (Voreinstellung) erwartet die x- und y-Komponenten. "polar": Winkel 0 liegt auf der positiven x-Achse, 90 auf der positiven y-Achse. "nautical": 0 ist "north" und 90 ist "east".

$xyplot dot *series xcrd ycrd value*
Zeichnet einen Punkt in das Diagramm ein. Größe und Farbe richten sich nach dem Wert und den für die zugehörige Reihe festgelegten Optionen. (Siehe den Unterbefehl "dotconfig")
string *series* (in)
Name der Serie, zu der der Punkt gehört. Legt Größe und Farbe fest
float *xcrd* (in)
X-Koordinate des Punktes, an dem der Pfeil erscheint

float *ycrd* (in)
Y-Koordinate des Punktes, an dem der Pfeil erscheint

float *value* (in)
Wert, der von Größe und Farbe abhängt

$xyplot dotconfig *series* -option *value* ...
Legt die Zeichenoptionen der Punkte einer Reihe fest.

string *series* (in)
Name der Serie, zu der der Punkt gehört.

Die Optionen können eine der folgenden sein:

colour
Die Farbe des Punktes, wenn keine Skalierung verwendet wird oder der Wert den oberen Grenzwert der Klassen überschreitet.

scale value
Der Skalierungsfaktor, der zur Umrechnung des Wertes in den Radius des Punktes in Pixeln verwendet wird (Standardwert: 1,0)

radius value
Der Standardradius der Punkte, der verwendet wird, wenn keine Skalierung anhand eines Wertes erfolgt (in Pixeln; Standardwert: 3)

scalebyvalue onoff
Legt fest, ob alle Punkte dieselbe Größe haben oder ob ihre Größe vom angegebenen Wert abhängt (Voreinstellung on).

outline onoff
Zeichnet einen Kreis in black um den Punkt oder nicht (Voreinstellung on).

classes list
Legt die Grenzwerte und die entsprechenden Farben fest. Zum Beispiel:

```
    $xyplot series1 -classes {0 blue 1 green} -colour red
```

Dies führt dazu, dass für Werte kleiner als 0 ein blauer Punkt, für Werte größer oder gleich 0, aber kleiner als 1 ein grüner Punkt und für Werte größer oder gleich 1 ein roter Punkt gezeichnet wird.

3deffect onoff
Zeigt eine Hervorhebung in den Punkten an, um einen 3D-Effekt zu imitieren (Voreinstellung off).

Wenn für die jeweilige Reihe keine Klassenliste vorliegt, werden die Punkte entsprechend dem Wert skaliert.

Sie können die Farbgebung nach Wert und die Skalierung nach Wert kombinieren, indem Sie eine Klassenliste festlegen und die Option *scalebyvalue* aktivieren.

$xyplot contourlines *xcrd ycrd values* ?classes?
Zeichnet Konturlinien für die im Grid angegebenen Werte. Das Grid wird durch die Argumente "xcrd" und "ycrd" definiert. Das Argument "xcrd" (bzw. "ycrd") soll eine Matrix sein, die als

Liste von Listen implementiert ist und die x-Koordinaten (bzw. y-Koordinaten) der Ecken der Grid-Zellen angibt. Die Werte werden an diesen Ecken angegeben. Die Anzahl der Zeilen in "xcrd" (bzw. "ycrd") beträgt "ny", und jede Zeile enthält "nx" Werte, sodass die Gesamtzahl der Werte in "xcrd" (bzw. "ycrd") "nx * ny" beträgt. Die Klassen legen fest, welche Konturlinien gezeichnet werden. Wenn ein Wert an einer der Ecken fehlt, werden die Konturlinien in dieser Zelle nicht gezeichnet.

Einträge in der Legende werden über den Unterbefehl *legendisolines* gezeichnet.

list *xcrd* (in)
Liste von Listen, wobei jeder Wert eine x-Koordinate für eine Ecke einer Grid-Zelle darstellt

list *ycrd* (in)
Liste von Listen, wobei jeder Wert eine y-Koordinate für eine Ecke einer Gitterzelle darstellt

list *values* (in)
Liste von Listen, wobei jeder Wert dem Wert an einer Gitterzellenecke entspricht

list *classes* (in)
Eine Liste von Klassenwerten oder eine Liste von Listen mit jeweils zwei Elementen (jede innere Liste enthält den Wert der Klasse und die zu verwendende Farbe). Ist diese leer oder fehlt sie, werden die Klassen automatisch ermittelt.

Hinweis: Die Werte der Klassen müssen den gesamten Wertebereich abdecken. *Hinweis:* Das Argument "xcrd" besteht in der Regel aus nypoints identischen Zeilen, während jede Zeile von "ycrd" aus einem einzigen Wert besteht.

$xyplot contourlinesfunctionvalues *xvec yvec valuesmat* ?classes?
Zeichnet Konturlinien für die im Grid angegebenen Werte. Das Grid wird durch die Argumente xvec und yvec definiert. Dabei ist xvec (bzw. yvec) eine Liste von x-Koordinaten (bzw. y-Koordinaten). Die Anzahl der Werte in "xvec" (bzw. "yvec") entspricht der Anzahl der Punkte an den x-Koordinaten (bzw. den y-Koordinaten). Die Funktionswerte sind an diesen Eckpunkten vorgegeben. Die Klassen legen fest, welche Konturlinien gezeichnet werden. Fehlt ein Wert an einem der Eckpunkte, werden die Konturlinien in dieser Zelle nicht gezeichnet.

Einträge in der Legende werden über den Unterbefehl *legendisolines* gezeichnet.

list *xvec* (in)
Liste der x-Koordinaten in aufsteigender Reihenfolge.

list *yvec* (in)
Listen Sie die y-Koordinaten in aufsteigender Reihenfolge auf.

list *valuesmat* (in)
Eine Liste aus ny Listen mit nx Werten, wobei jeder Wert den Wert an einer Ecke einer Grid-Zelle angibt. Die Gesamtzahl der Werte (valuesmat) beträgt nx × ny.

list *classes* (in)
Eine Liste von Klassenwerten oder eine Liste von Listen mit jeweils zwei Elementen (jede innere Liste enthält den Wert der Klasse und die zu verwendende Farbe). Wenn diese Liste leer oder fehlt, werden die Klassen automatisch festgelegt.

Hinweis: Die Klassenwerte müssen den gesamten Wertebereich abdecken.

$xyplot contourfill *xcrd ycrd values* ?classes?

Zeichnet ausgefüllte Konturen für die im Grid angegebenen Werte. (Die Verwendung dieser Methode entspricht der Methode "contourlines".)

Einträge in der Legende werden über den Unterbefehl *legendshades* gezeichnet.

$xyplot contourbox *xcrd ycrd values* ?classes?

Zeichnet die Zellen als ausgefüllte Vierecke. Die Farbe ergibt sich aus dem Durchschnitt der Werte an allen vier Ecken.

Die Einträge in der Legende werden über den Unterbefehl *legendshades* dargestellt.

$xyplot colorMap *colours*

Legt die Farben fest, die bei den Konturierungsmethoden verwendet werden sollen. Das Argument ist entweder eine vordefinierte Farbpalette (grey/gray, jet, hot oder cool) oder eine Liste von Farben. Bei der Auswahl der Farben für die eigentliche Darstellung der Konturen werden die angegebenen Farben interpoliert (basierend auf dem HLS-Schema).

list *colours* (in)

Eine Liste von Farbnamen oder Farbwerten oder eine der vordefinierten Zuordnungen:
- grey oder gray: graue Farben von dunkel nach hell
- jet: Regenbogenfarben
- hot: Farben von yellow über red nach darkred
- cool: Farben von cyan über blue nach magenta

$xyplot legendisolines *values classes*

Fügt die Konturklassen als farbige Linien in die Legende ein. Der Text gibt die Werte an.

list *values* (in)

Die Liste der Werte, die für die eigentliche Zeichnung verwendet werden. Dieses Argument wird nur verwendet, wenn die Liste der Klassen leer ist.

list *values* (in)

Die Liste der Klassen, die für die eigentliche Zeichnung verwendet wurden.

$xyplot legendshades *values classes*

Fügt die Konturklassen als farbige Rechtecke in die Legende ein. Der Text gibt die Werte an.

list *values* (in)

Die Liste der Werte, die für die eigentliche Zeichnung verwendet werden. Dieses Argument wird nur verwendet, wenn die Liste der Klassen leer ist.

list *values* (in)

Die Liste der Klassen, die für die eigentliche Zeichnung verwendet wurden.

$xyplot grid *xcrd ycrd*

Zeichnet die Grid-Zellen als Linien, die die (gültigen) Grid-Points miteinander verbinden.

list *xcrd* (in)

Liste von Listen, wobei jeder Wert eine x-Koordinate für eine Ecke einer Grid-Zelle darstellt

list *ycrd* (in)

Liste von Listen, wobei jeder Wert eine y-Koordinate für eine Ecke einer Gitterzelle darstellt

$xyplot xband *ymin ymax*

Zeichnet ein hellgraues Band über die ganze x-Achse. Es kann einen "typical"-Bereich der

Daten anzeigen.

float *ymin* (in)
Untergrenze für das Band

float *ymax* (in)
Obergrenze für das Band

$xyplot yband *xmin xmax*
Zeichnet ein hellgraues Band über die ganze y-Achse. Es kann einen "typical"-Bereich der Daten anzeigen.

float *xmin* (in)
Untergrenze für das Band

float *xmax* (in)
Obergrenze für das Band

$xyplot labeldot *x y text orient*
Zeichnet eine Beschriftung und ein Symbol in die Grafik ein. Die Beschriftung erscheint in der Nähe des Symbols. Die Beschriftung wird in Grau dargestellt, damit sie nicht zu sehr ins Auge fällt.

Sie können das Erscheinungsbild des Symbols mithilfe des Datenreihennamens "labeldot" konfigurieren: *$w dataconfig labeldot -colour red -type symbol -symbol dot*

float *x* (in)
X-Koordinate des zu zeichnenden Symbols

float *y* (in)
Y-Koordinate des zu zeichnenden Symbols

string *text* (in)
Text für das Etikett

string *orient* (in)
Optionale Ausrichtung (eine der Optionen "w", "e", "n" oder "s"), die die Position der Beschriftung relativ zum Symbol festlegt. Die Standardeinstellung ist "w" (sodass die Beschriftung links vom Symbol erscheint).

Für *Polardiagramme*:

$polarplot plot *series radius angle*
Fügt dem Polardiagramm einen Datenpunkt hinzu.

string *series* (in)
Name der Datenreihe, zu der der neue Datenpunkt gehört.

float *radius* (in)
Radialkoordinate des neuen Punktes.

float *angle* (in)
Koordinate des neuen Punktes (in Grad).

Für *Windrosendiagramme*:

$windrose plot *data colour*
Zeichnet die im Argument *data* enthaltenen Daten. Die Daten werden den vorhandenen Speichen in Richtung des äußeren Kreises hinzugefügt.

list *data* (in)
Datenliste (die Länge sollte der Anzahl der Sektoren entsprechen)

string *colour*
Farbe, in der die neuen Segmente gezeichnet werden sollen

Für *3D-Diagramme*:

$plot3d plotfunc *function*
Stellt eine Funktion dar, die über die beiden Variablen **x** und **y** definiert ist. Die Auflösung wird durch die festgelegten Grid-Größen bestimmt (weitere Informationen finden Sie in der Methode **gridsize**).

string *function* (in)
Name der Prozedur, die den z-Wert für die angegebenen x- und y-Koordinaten berechnet. Die Prozedur muss zwei Float-Argumente entgegennehmen (x ist das erste Argument, y das zweite) und einen Gleitkomma-Wert zurückgeben.

$plot3d plotfuncont *function contours*
Stellt eine über die beiden Variablen **x** und **y** definierte Funktion dar und verwenden Sie dabei die Konturwerte aus **contours**, um die Oberfläche in Farben einzufärben. Die Auflösung wird durch die festgelegten Grid-Größen bestimmt (weitere Informationen finden Sie in der Methode **gridsize**).

string *function* (in)
Name der Prozedur, die den z-Wert für die angegebenen x- und y-Koordinaten berechnet. Die Prozedur muss zwei Float-Argumente entgegennehmen (x ist das erste Argument, y das zweite) und einen Gleitkomma-Wert zurückgeben.

list *contours* (in)
Liste der Werte in aufsteigender Reihenfolge, die die Konturstufen darstellen (die Grenzen zwischen den Farben in der Konturkarte).

$plot3d gridsize *nxcells nycells*
Legt die Grid-Größe in beiden Richtungen fest. Zusammen bestimmen sie, wie viele Polygone für eine Funktionsgrafik gezeichnet werden.

int *nxcells* (in)
Anzahl der Grid-Zellen in x-Richtung. Es muss sich um eine ganze Zahl größer als Null handeln.

int *nycells* (in)
Anzahl der Grid-Zellen in y-Richtung. Muss eine ganze Zahl größer als Null sein.

$plot3d plotdata *data*
Stellt eine Datenmatrix grafisch dar.

list *data* (in)
Die zu darstellenden Daten. Die Daten müssen als verschachtelte Liste mit zwei Ebenen bereitgestellt werden. Die äußere Liste enthält Zeilen, die in y-Richtung dargestellt werden, und jede Zeile ist eine Liste, deren Elemente in x-Richtung dargestellt werden und die Spalten bilden. Beispiele:

```
set data {
{1.0 2.0 3.0}
{4.0 5.0 6.0}
}
```

$plot3d interpolatedata *data contours*
Stellt die Daten mittels bilinearer Interpolation dar und verwenden Sie dabei die Konturwerte aus **contours**, um die Oberfläche in Farben einzufärben. Die Auflösung wird durch die festgelegten Gittergrößen bestimmt (weitere Informationen finden Sie in der Methode **gridsize**).
list *data* (in)
Die zu darstellenden Daten, genau wie beim Unterbefehl "plotdata".
list *contours* (in)
Liste der Werte in aufsteigender Reihenfolge, die die Konturstufen darstellen (die Grenzen zwischen den Farben in der Konturkarte).

$plot3d colour *fill border*
Legt die Farbe fest, die für die Polygon-Rahmen und den Innenbereich verwendet werden soll. *Hinweis:* Der Unterbefehl "color" ist ein Synonym.
color *fill* (in)
Die Farbe, die zum Ausfüllen der Polygone verwendet werden soll.
color *border* (in)
Die Farbe, die für den Rahmen der Polygone verwendet werden soll.

$plot3d ribbon *yzpairs*
Zeichnet ein Band auf der Grundlage der yz-Koordinatenpaare. Die Farben für das Band selbst und den Rand werden aus der Option colours übernommen.
list *yzpairs* (in)
Liste der yz-Koordinatenpaare

Für 3D-Banddiagramme:

$plot3d plot *yzpairs*
Zeichnet ein Band auf der Grundlage der yz-Koordinatenpaare. Die Farben für das Band selbst und den Rand werden aus der Option colours übernommen.
list *yzpairs* (in)
Liste der yz-Koordinatenpaare

Für *XY-Diagramme*, *Streifendiagramme*, *Histogramme*, *Polardiagramme* und *ternäre Diagramme*:

$xyplot dataconfig *series* -option *value* ...
Legt den Wert für eine oder mehrere Optionen fest, die die Darstellung der Daten einer bestimmten Reihe betreffen.
string *series* (in)
Name der Datenreihe, deren Konfiguration wir ändern.

Folgende Optionen sind zulässig:
colour c

siehe unten ...

color c
Die Farbe, die beim Zeichnen der Datenreihe verwendet werden soll.

type enum
Der für die Serie gewählte Zeichenmodus. Dies kann entweder **line**, **symbol** oder **both** sein.

symbol enum
Welche Art von Symbol gezeichnet werden soll. Der Wert dieser Option wird ignoriert, wenn der Zeichenmodus **line** ausgewählt wurde. Mögliche Werte sind **plus**, **cross**, **circle**, **up** (nach oben zeigendes Dreieck), **down** (nach unten zeigendes Dreieck), **dot** (gefüllter Kreis), **upfilled** oder **downfilled** (gefüllte Dreiecke).

radius integer
Die Größe des Radius des Symbols. Die Gesamtbreite des Symbols beträgt das Zweifache des Radius. Der Standardradius beträgt 4.

width integer
Die Breite der Linie (wenn gezeichnet) oder die Breite der Polygon-Gliederung (im Fall von -filled).

filled enum
Ob der Bereich oberhalb oder unterhalb der Datenlinie ausgefüllt werden soll oder nicht. Mögliche Werte sind: **no**, **up** oder **down** (Spezialeffekte)

fillcolour colour
Farbe, die beim Ausfüllen des Bereichs verwendet werden soll, der der Datenlinie zugeordnet ist.

style enum
Der für Histogramme zu verwendende Stil:
- **filled**: Den Bereich unter den Datenpunkten mit Balken ausfüllen (Standard)
- **spike**: Zeichnen Sie vertikale Linien von der y-Achse (untere Begrenzung) zum Datenpunkt
- **symbol**: Zeichnen Sie ein Symbol am Datenpunkt
- **plateau**: Zeichnen Sie eine horizontale Linie auf Höhe des Datenpunkts
- **stair**: Zeichnen Sie eine durchgehende, treppenartige Linie als Verbindung der Datenpunkte

smooth boolean
(Wird bei ternären Diagrammen verwendet) Legt fest, ob die Linien und gefüllten Polygone mit der Option smooth on (abgerundete Ecken) gezeichnet werden sollen oder nicht.

Für *Kreisdiagramme* und *Spiralkreisdiagramme*:

$pie plot *data*
Füllen Sie ein Kreisdiagramm aus.

list *data* (in)
Eine Liste von Paaren (Bezeichnungen und Werte). Die Werte bestimmen die relative Größe der Kreissegmente. Die Bezeichnungen werden neben dem Kreis dargestellt.

$pie colours *colour1 colour2* ...
Legt die zu verwendenden Farben fest.

color *colour1* (in)
Die erste Farbe.
color *colour2* (in)
Die zweite Farbe und so weiter.

$pie explode *segment*
Ein Segment herausrücken (d. h.: ein Segment aus dem Kreis herausziehen). Wenn das Segment mit "auto" gekennzeichnet ist, können Sie auf ein Segment klicken. Dieses wird dann anstelle eines zuvor herausgerückten Segments herausgerückt.
int *segment*
Das Segment, das herausgerückt werden soll, oder "auto", wenn Sie dies interaktiv durchführen möchten.

Für *Radialdiagramme*:

$radial plot *data colour thickness*
Zeichnet eine neue Linie im Radialdiagramm ein
list *data* (in)
Eine Liste von Daten (jeweils eine für jede Speiche). Die Werte geben den Abstand vom Mittelpunkt der Verbindung zwischen den Speichen an.
color *colour* (in)
Die Farbe der Linie.
int *thickness* (in)
Ein optionales Argument für die Strichstärke.

$pie colours *colour1 colour2 ...*
Legt die zu verwendenden Farben fest.
color *colour1* (in)
Die erste Farbe.
color *colour2* (in)
Die zweite Farbe und so weiter.

Für *vertikale Balkendiagramme*:

$barchart plot *series ydata colour* ?dir? ?brightness?
Fügt einem Balkendiagramm eine Datenreihe hinzu. Die Balken sind zur Identifizierung mit dem Tag "data_\$series" versehen.
string *series* (in)
Name der Reihe, zu der die Werte gehören.
list *ydata* (in)
Eine Liste von Werten, jeweils einer pro Beschriftung auf der x-Achse.
color *colour* (in)
Die Farbe der Balken.
string *dir* (in)
Falls angegeben: "Top-down" oder "Bottom-up", um die Richtung anzugeben, in die sich die Farbe ändert. (Falls nicht angegeben, wird eine einheitliche Farbe verwendet.)
string *brightness* (in)

Falls angegeben, "bright" oder "dark" (Voreinstellung "bright"). Die Farbe geht je nach Richtung nach white oder nach black.

$barchart config -option *value* ...
Legt die Optionen für die Darstellung der Balken fest.
showvalues boolean
Ob die Werte (über den Balken) angezeigt werden sollen oder nicht
valuefont newfont
Name der Schriftart, die für die Werte verwendet werden soll
valuecolour colour
Farbe für die Werte
valueformat format
Formatzeichenfolge, die zur Formatierung der Werte verwendet werden soll

Für *horizontale Balkendiagramme*:

$barchart plot *series xdata colour* ?dir? ?brightness?
Fügt einem Balkendiagramm eine Datenreihe hinzu. Die Balken sind zur Identifizierung mit dem Tag "data_\$series" versehen.
string *series* (in)
Name der Reihe, zu der die Werte gehören.
list *xdata* (in)
Eine Liste von Werten, jeweils einer pro Beschriftung der y-Achse.
color *colour* (in)
Die Farbe der Balken.
string *dir* (in)
Falls angegeben, "left-right" oder "right-left", um die Richtung anzugeben, in der sich die Farbe ändert. (Wird dies nicht angegeben, wird eine einheitliche Farbe verwendet.)
string *brightness* (in)
Falls angegeben, "bright" oder "dark" (Voreinstellung "bright"). Die Farbe geht je nach Richtung nach white oder nach black.

$barchart config -option *value* ...
Legt die Optionen für die Darstellung der Balken fest.
showvalues boolean
Ob die Werte angezeigt werden sollen oder nicht (rechts neben den Balken)
valuefont newfont
Name der Schriftart, die für die Werte verwendet werden soll
valuecolour colour
Farbe für die Werte
valueformat format
Formatzeichenfolge, die zur Formatierung der Werte verwendet werden soll

Für *3D-Balkendiagramme*:

$barchart plot *label yvalue colour*
Fügt dem Balkendiagramm den nächsten Balken hinzu.

string *label* (in)
Die Beschriftung, die unterhalb der Spalte angezeigt werden soll.

float *yvalue* (in)
Der Wert, der die Höhe der Spalte bestimmt

color *colour* (in)
Die Farbe der Spalte.

$barchart config -option *value* ...
Legt einen oder mehrere Parameter der Konfiguration fest. Folgende Optionen werden unterstützt:

usebackground boolean
Ob links und hinter den Spalten Wände eingezeichnet werden sollen oder nicht

useticklines boolean
Ob Gitterlinien an den Wänden gezeichnet werden sollen oder nicht

showvalues boolean
Ob die Werte angezeigt werden sollen oder nicht

labelfont newfont
Name der Schriftart, die für Beschriftungen verwendet werden soll

labelcolour colour
Farbe für die Etiketten

valuefont newfont
Name der Schriftart, die für die Werte verwendet werden soll

valuecolour colour
Farbe für die Werte

Für *3D-Banddiagramme*:

$ribbon line *xypairs colour*
Stellt die angegebenen xy-Paare als Band im Diagramm dar

list *xypairs* (in)
Die zu zeichnenden x/y-Wertepaare (die Reihe wird als Ganzes gezeichnet)

color *colour* (in)
Die Farbe des Bandes.

$ribbon area *xypairs colour*
Stellt die angegebenen xy-Paare als Band mit einem ausgefüllten Bereich davor dar. Der Effekt entspricht dem einer Box, deren Oberseite aus den Daten besteht.

list *xypairs* (in)
Die zu zeichnenden x/y-Wertepaare (die Reihe wird als Ganzes gezeichnet)

color *colour* (in)
Die Farbe des Bandes/Bereichs.

Für *Boxplots*:

$boxplot plot *series label values*
Fügt dem Diagramm ein Box-and-Whisker-Diagramm hinzu. Mit dem Befehl "dataconfig" können Sie das Box-and-Whisker-Diagramm anpassen (Einzelheiten finden Sie im Befehl

"box-and-whiskers" für den Befehl "xyplot").

string *series* (in)
Name der Datenreihe, zu der die Box gehört

string *label* (in)
Die Beschriftung entlang der x- oder y-Achse, zu der die Daten gehören

list *values* (in)
Liste der Rohwerte; anhand dieser Liste werden die Ausdehnung des Kastens und die Länge der Whisker bestimmt.

$boxplot violin *series label values*
Fügt dem Diagramm eine "Violine" hinzu. Mit dem Befehl "dataconfig" können Sie das Erscheinungsbild anpassen (Einzelheiten finden Sie im Befehl "violin" für den xyplot).

string *series* (in)
Name der Datenreihe, die für die Einstellungen verwendet werden soll

string *label* (in)
Die Beschriftung entlang der x- oder y-Achse, zu der die Daten gehören

list *values* (in)
Liste der Rohwerte; anhand dieser Liste werden die Ausdehnung des Kastens und die Länge der Whisker bestimmt.

Für *Zeitdiagramme*:

$timechart period *text time_begin time_end colour*
Fügt dem Diagramm einen Zeitraum hinzu.

string *text* (in)
Der Text, der diese Epoche beschreibt.

string *time_begin* (in)
Startzeit des Zeitraums.

string *time_end* (in)
Endzeit des Zeitraums.

color *colour* (in)
Die Farbe des Balkens (standardmäßig schwarz).

$timechart addperiod *time_begin time_end colour*
Fügt der aktuellen Zeile im Diagramm einen neuen Zeitraum hinzu. Auf diese Weise können Sie mehrere Zeiträume in derselben Zeile hervorheben. Es wird kein neuer Text eingezeichnet.

string *time_begin* (in)
Startzeit des Zeitraums.

string *time_end* (in)
Endzeit des Zeitraums.

color *colour* (in)
Die Farbe des Balkens (standardmäßig schwarz).

$timechart milestone *text time colour*
Fügt dem Diagramm einen *Meilenstein* (dargestellt als nach unten zeigendes Dreieck) hinzu.

string *text* (in)
Der Text, der den Meilenstein beschreibt.

string *time* (in)
Zeitpunkt, zu dem der Meilenstein gesetzt werden muss.

color *colour* (in)
Die Farbe des Dreiecks (standardmäßig schwarz).

$timechart addmilestone *time colour*
Fügt der aktuellen Zeile im Diagramm einen weiteren *Meilenstein* hinzu.

string *time* (in)
Zeitpunkt, zu dem der Meilenstein gesetzt werden muss.

color *colour* (in)
Die Farbe des Dreiecks (standardmäßig schwarz).

$timechart vertline *text time colour*
Fügt dem Diagramm eine vertikale Linie (beispielsweise zur Kennzeichnung des Monatsanfangs) in der angegebenen Farbe hinzu.

string *text* (in)
Der Text, der oben angezeigt wird (beispielsweise eine Abkürzung für Datum und Uhrzeit).

string *time* (in)
Zeitpunkt, zu dem die Linie positioniert werden muss.

color *colour* (in)
Die Farbe der zu zeichnenden Linie (Vorgabe black)

$timechart hscroll *scrollbar*
Verbindet eine waagerechte Bildlaufleiste mit dem Diagramm. Siehe auch den Abschnitt zum Bildlauf.

widget *scrollbar* (in)
Die horizontale Bildlaufleiste, die eine Verbindung zum Diagramm hat

$timechart vscroll *scrollbar*
Verbindet eine senkrechte Bildlaufleiste mit dem Diagramm. Siehe auch den Abschnitt zum Bildlauf.

widget *scrollbar* (in)
Die senkrechte Bildlaufleiste, die mit dem Diagramm verbunden wird.

Für *Gantt-Diagramme*:

$ganttchart task *text time_begin time_end completed*
Fügt dem Diagramm eine Aufgabe mit ihrer Dauer und ihrem Fertigstellungsgrad hinzu. Gibt eine Liste von Canvas-Elementen zurück, die für weitere Bearbeitungen verwendet werden können, beispielsweise zur Erstellung einer Verbindung zwischen zwei Aufgaben.

string *text* (in)
Der Text, in dem die Aufgabe beschrieben wird.

string *time_begin* (in)
Startzeit der Aufgabe.

string *time_end* (in)
Endzeit der Aufgabe.

float *completed* (in)

Der Prozentsatz der bereits erledigten Aufgabe.

$ganttchart milestone *text time colour*

Fügt dem Diagramm einen *Meilenstein* (dargestellt als nach unten zeigendes Dreieck) hinzu.

string *text* (in)
Der Text, der den Meilenstein beschreibt.

string *time* (in)
Zeitpunkt, zu dem der Meilenstein gesetzt werden muss.

color *colour* (in)
Die Farbe des Dreiecks (standardmäßig schwarz).

$ganttchart vertline *text time*

Fügt dem Diagramm eine vertikale Linie hinzu (beispielsweise, um den Monatsanfang zu kennzeichnen).

string *text* (in)
Der Text, der oben angezeigt wird (beispielsweise eine Abkürzung für Datum und Uhrzeit).

string *time* (in)
Zeitpunkt, zu dem die Linie positioniert werden muss.

$ganttchart connect *from to*

Fügt einen Pfeil ein, der die Aufgabe *from* mit der Aufgabe *to* verbindet.

list *from* (in)
Die Liste der Elemente, die vom Befehl "task" gegeben wird und die die Aufgabe darstellt, von der aus der Pfeil ausgeht.

string *text* (in)
Der Text, der die Aufgaben zusammenfasst

list *args* (in)
Eine oder mehrere Aufgaben (die der Befehl "task" zurückgibt). Diese werden nach unten verschoben, um Platz für die Zusammenfassung zu schaffen.

list *to* (in)
Die Liste der Elemente, die vom Befehl "task" zurückgegeben wird und die die Aufgabe darstellt, an der der Pfeil endet.

$ganttchart summary *text args*

Fügt einen Zusammenfassungsbalken hinzu, der sich über alle aufgeführten Aufgaben erstreckt. Die grafische Darstellung ist ein breiter Balken, der von der Aufgabe ganz links bis zur Aufgabe ganz rechts verläuft.

Verwenden Sie diesen Befehl, bevor Sie die Aufgaben miteinander verbinden, da der Pfeil sonst nicht nach unten verschoben wird!

string *text* (in)
Der Text, der die Aufgaben zusammenfasst

list *args* (in)
Eine oder mehrere Aufgaben (die der Befehl "task" zurückgibt). Diese werden nach unten verschoben, um Platz für die Zusammenfassung zu schaffen.

$ganttchart color *keyword newcolor*

Legt die Farbe für einen Teil des Gantt-Diagramms fest. Diese Farben gelten für alle Elemente dieses Types.

string *keyword* (in)

Das Schlüsselwort gibt an, welcher Teil des Gantt-Diagramms geändert werden soll:
- description – Farbe des beschreibenden Textes
- completed – Farbe des gefüllten Balkens für den erledigten Teil einer Aufgabe
- left – Farbe für den noch nicht erledigten Teil
- odd – Hintergrundfarbe der ungeraden Einträge
- even – Hintergrundfarbe der geraden Einträge
- summary – Farbe des Zusammenfassungstextes
- summarybar – die Farbe des Balkens für eine Zusammenfassung

string *newcolor* (in)

Die neue Farbe für die gewählten Elemente.

$ganttchart font *keyword newfont*

Legt die Schriftart für einen Teil des Gantt-Diagramms fest. Diese Schriftarten gelten für alle Elemente dieses Types.

string *keyword* (in)

Das Schlüsselwort gibt an, welcher Teil des Gantt-Diagramms geändert werden soll:
- description – Schrift des beschreibenden Textes
- summary – Schrift der Zusammenfassungen
- scale – Schrift der Zeitskala

string *newfont* (in)

Die neue Schriftart für die ausgewählten Elemente.

$ganttchart hscroll *scrollbar*

Verbindet eine waagerechte Bildlaufleiste mit dem Diagramm. Siehe auch den Abschnitt zum Bildlauf.

widget *scrollbar* (in)

Die horizontale Bildlaufleiste, die eine Verbindung zum Diagramm hat

$ganttchart vscroll *scrollbar*

Verbindet eine senkrechte Bildlaufleiste mit dem Diagramm. Siehe auch den Abschnitt zum Bildlauf.

widget *scrollbar* (in)

Die senkrechte Bildlaufleiste, die mit dem Diagramm verbunden wird.

Für *isometrische Darstellungen* (wird noch erweitert):

$isoplot plot rectangle *x1 y1 x2 y2 colour*

Zeichnet die Gliederungen eines Rechtecks ein.

float *x1* (in)

Minimale x-Koordinate des zu zeichnenden Rechtecks.

float *y1* (in)

Minimale y-Koordinate des Rechtecks.

float *x2* (in)

Maximale x-Koordinate des zu zeichnenden Rechtecks.

float *y2* (in)
Maximale y-Koordinate des Rechtecks.

color *colour* (in)
Die Farbe des Rechtecks.

$isoplot plot filled-rectangle *x1 y1 x2 y2 colour*
Zeichnet ein Rechteck, das mit der angegebenen Farbe gefüllt ist.

float *x1* (in)
Minimale x-Koordinate des zu zeichnenden Rechtecks.

float *y1* (in)
Minimale y-Koordinate des Rechtecks.

float *x2* (in)
Maximale x-Koordinate des zu zeichnenden Rechtecks.

float *y2* (in)
Maximale y-Koordinate des Rechtecks.

color *colour* (in)
Die Farbe des Rechtecks.

$isoplot plot circle *xc yc radius colour*
Zeichnet die Gliederung eines Kreises ein.

float *xc* (in)
X-Koordinate des Kreismittelpunkts.

float *yc* (in)
Y-Koordinate des Kreismittelpunkts.

color *colour* (in)
Die Farbe des Kreises.

$isoplot plot filled-circle *xc yc radius colour*
Zeichnet einen Kreis, der mit der angegebenen Farbe ausgefüllt ist.

float *xc* (in)
X-Koordinate des Kreismittelpunkts.

float *yc* (in)
Y-Koordinate des Kreismittelpunkts.

color *colour* (in)
Die Farbe des Kreises.

Für *Tabellen* können Sie die folgenden Unterbefehle verwenden:

$table row *items*
Zeichnet eine einzelne Zeile mit Elementen. Das Erscheinungsbild der Elemente lässt sich explizit über den Befehl "format" steuern.

list *items* (in)
Liste der zu zeichnenden Textelemente, jeweils eines pro Spalte

$table separator
Zeichnet eine horizontale Linie, um zwei Zeilen voneinander zu trennen

$table formatcommand *procname*

Legt die Prozedur fest, die die Formatierung der Elemente steuert. Standardmäßig werden Elemente lediglich als zentrierter Text dargestellt.

string *procname* (in)

Name der zu verwendenden Prozedur. Ihre Signatur lautet:

```
proc procname {table widget row column value} {...}
```

Nutzt den Unterbefehl "cellconfigure", um die Attribute pro Zelle festzulegen.

$table cellconfigure *args*

Legt die Attribute für die nächste(n) zu zeichnende(n) Zelle(n) fest.

list *args* (in)

Schlüssel-Wert-Paare: -background legt die Hintergrundfarbe der Zellen fest, -cell legt die Vordergrundfarbe fest, -font legt die Schriftart fest, -anchor legt die Position des Textes innerhalb der Zelle fest und -justify steuert das Layout von mehrzeiligem Text.

Für *ternäre Diagramme* stehen Ihnen die folgenden Unterbefehle zur Verfügung:

$ternary plot *series xcrd ycrd zcrd text dir*

Zeichnet einen einzelnen Datenpunkt mit einer Beschriftung. Die drei Koordinaten werden so skaliert, dass ein eindeutiger Punkt im Dreieck entsteht. Daneben wird eine Beschriftung eingezeichnet.

string *series* (in)

Name der Datenreihe, zu der der Punkt gehört (dient zur Bestimmung von Farbe und Symbol)

float *xcrd* (in)

X-Koordinate des Datenpunkts (bezogen auf die linke untere Ecke).

float *ycrd* (in)

Y-Koordinate des Datenpunkts (bezogen auf die rechte untere Ecke).

float *zcrd* (in)

Z-Koordinate des Datenpunkts (bezogen auf die obere Ecke).

string *text* (in)

Bezeichnung, die den Datenpunkt beschreibt.

string *dir* (in)

Optionaler String, der die Richtung angibt, in der die Beschriftung dargestellt werden soll (e, n usw.)

$ternary line *series coords*

Zeichnet eine durchgehende Linie auf der Grundlage der angegebenen Koordinaten (Dreiergruppen).

string *series* (in)

Name der Datenreihe, zu der die Linie gehört (dient zur Bestimmung der Farbe und der Glättung)

list *coords* (in)

Die Koordinaten der Punkte, die die Gerade bestimmen (beachten Sie, dass ein Punkt durch drei Koordinaten definiert ist).

$ternary fill *series coords*

Zeichnet ein ausgefülltes Vieleck auf der Grundlage der angegebenen Koordinaten (Dreiergruppen).

string *series* (in)

Name der Datenreihe, zu der das Polygon gehört (dient zur Bestimmung von Farbe und Glättung)

list *coords* (in)

Die Koordinaten der Punkte, die das Polygon bestimmen (Hinweis: Ein Punkt wird durch drei Koordinaten definiert).

$ternary text *xtext ytext ztext*

Fügt an den drei Ecken des Diagramms Text ein, um die Komponenten zu kennzeichnen.

string *xtext* (in)

Text, der in der unteren linken Ecke dargestellt werden soll

string *ytext* (in)

Text, der in der rechten unteren Ecke dargestellt werden soll

string *ztext* (in)

Text, der in der oberen Ecke dargestellt werden soll

$ternary ticklines *colour*

Zeichnet Gitterlinien ein, um das Ablesen des Diagramms zu erleichtern.

string *colour* (in)

Optionales Argument, das als Farbe für die Gitterlinien verwendet wird. Die Standardfarbe ist Grau.

Für *Status-Zeitreihen-Diagramme* können Sie die folgenden Unterbefehle verwenden:

$timeline plot *series item start stop color*

Zeichnet für das Element *item* einen Balken in der angegebenen Farbe von *start* bis *stop*. Das Element dient lediglich als Bezeichner – es besteht kein Zusammenhang zu den Beschriftungen entlang der Achse. Die Elemente werden von unten nach oben gezeichnet.

string *item* (in)

Name zur Identifizierung der Leiste. Siehe Anmerkung oben.

float *start* (in)

X-Koordinate (bzw. Zeitpunkt), an dem der Balken beginnt

float *stop* (in)

X-Koordinate (bzw. Zeitpunkt), an der der Balken zum Stillstand kommt

string *color* (in)

Für die Leiste zu verwendende Farbe. Vorgabe ist black.

$timeline vertline *text time args*

Ziehen Sie eine vertikale Linie, um einen wichtigen Moment zu markieren.

string *text* (in)

Text zur Beschreibung des Augenblicks.

float *time* (in)

X-Koordinate (bzw. Zeitpunkt), an der die Linie gezeichnet werden soll

list *args* (in)

Einzelne optionale Argumente, die an den Unterbefehl *create line* des zugrunde liegenden Canvases übergeben werden. Auf diese Weise können Sie die Farbe oder die Linienbreite der vertikalen Linie festlegen.

Für *Dendrogramme* gibt es einen speziellen Unterbefehl:

$dendrogram plot *series data*

Zeichnet ein Dendrogramm gemäß den Optionen, die bei der Erstellung des Dendrogramm-Befehls angegeben oder impliziert wurden. Über den Namen "series" können Sie die Farbe der Zweige (-colour/-color) sowie die Farbe und Schriftart der Beschriftungen auswählen (-labelcolour/-labelcolor und -labelfont).

string *series* (in)

Name der Optionen, mit denen das Dendrogramm gezeichnet werden soll.

list *data* (in)

Verschachtelte Liste, die die Baumstruktur beschreibt. Siehe den Befehl *createDendrogram* .

Koordinatentransformationen

Neben den Befehlen, die sich direkt auf die Darstellungen und Diagramme beziehen, gibt es eine Reihe von Befehlen, mit denen sich Weltkoordinaten in Pixel umrechnen lassen und umgekehrt. Dazu gehören:

::Plotchart::viewPort *w pxmin pymin pxmax pymax*

Legt den Ansichtsbereich für das Fenster *w* fest. Sollte in Verbindung mit **::Plotchart::worldCoordinates** verwendet werden.

widget *w* (in)

Name des betreffenden Fensters (Canvas-Widget).

float *pxmin* (in)

Koordinate des Pixels ganz links.

float *pymin* (in)

Koordinate des obersten Pixels (die senkrechte Pixelkoordinate beginnt oben bei 0).

float *pxmax* (in)

Koordinate des Pixels ganz rechts.

float *pymax* (in)

Koordinate des untersten Pixels.

::Plotchart::worldCoordinates *w xmin ymin xmax ymax*

Legt die Extremwerte der Weltkoordinaten für das Fenster *w* fest. Die Weltkoordinaten müssen nicht in aufsteigender Reihenfolge vorliegen (d. h., xmin kann größer als xmax sein, sodass eine Umkehrung der x-Achse erreicht wird).

widget *w* (in)

Name des betreffenden Fensters (Canvas-Widget).

float *xmin* (in)

X-Koordinate, die auf die linke Seite des Ansichtsbereichs abgebildet werden soll.

float *ymin* (in)

Y-Koordinate, die auf den unteren Rand des Ansichtsfensters abgebildet werden soll.

float *xmax* (in)

X-Koordinate, die auf die rechte Seite des Ansichtsfensters abgebildet werden soll.

float *ymax* (in)
Y-Koordinate, die auf die Oberseite des Ansichtsfensters abgebildet werden soll.

::Plotchart::world3DCoordinates *w xmin ymin zmin xmax ymax zmax*
Legt die äußersten dreidimensionalen Weltkoordinaten für das Fenster *w* fest. Die Weltkoordinaten müssen nicht in aufsteigender Reihenfolge vorliegen (d. h., xmin kann größer als xmax sein, sodass eine Umkehrung der x-Achse erreicht wird).

widget *w* (in)
Name des betreffenden Fensters (Canvas-Widget).

float *xmin* (in)
X-Koordinate, die auf die Vorderseite des 3D-Ansichtsfensters abgebildet werden soll.

float *ymin* (in)
Y-Koordinaten, die auf die linke Seite des Ansichtsfensters abgebildet werden sollen.

float *zmin* (in)
Z-Koordinate, die auf den unteren Rand des Ansichtsfensters abgebildet werden soll.

float *xmax* (in)
X-Koordinate, die auf die Rückseite des Ansichtsfensters abgebildet werden soll.

float *ymax* (in)
Y-Koordinate, die auf die rechte Seite des Ansichtsfensters abgebildet werden soll.

float *zmax* (in)
Z-Koordinate, die auf die Oberseite des Ansichtsfensters abgebildet werden soll.

::Plotchart::coordsToPixel *w x y*
Gibt eine Liste von Pixelkoordinaten zurück, die für das angegebene Fenster gültig sind.

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben).

float *x* (in)
Zu abgebildende X-Koordinate.

float *y* (in)
Zu abgebildende Y-Koordinate.

::Plotchart::coords3DToPixel *w x y z*
Gibt eine Liste von Pixelkoordinaten zurück, die für das angegebene Fenster gültig sind.

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben).

float *x* (in)
Zu abgebildende X-Koordinate.

float *y* (in)
Zu abgebildende Y-Koordinate.

float *y* (in)
Zu abgebildende Z-Koordinate.

::Plotchart::polarCoordinates *w radmax*
Legt die äußersten Polarkoordinaten für das Fenster *w* fest. Der Winkel reicht stets von 0 bis

360 Grad, und der Radius beginnt bei 0. Daher müssen Sie lediglich den maximalen Radius angeben. *Hinweis:* Wenn der Darstellungsbereich nicht quadratisch ist, werden die Extremwerte bei dieser Prozedur nicht angepasst, was zu einer elliptischen Darstellung führen würde. Die Erstellungsroutine für eine polare Darstellung legt stets einen quadratischen Darstellungsbereich fest.

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben).

float *radmax* (in)
Maximaler Radius.

::Plotchart::polarToPixel *w rad phi*
Wrapper für einen Aufruf von **::Plotchart::coordsToPixel**. *Hinweis:* Diese Prozedur ist veraltet – bitte verwenden Sie stattdessen die Prozedur **::Plotchart::coordsToPixel**.

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben).

float *rad* (in)
Radius des Punktes.

float *phi* (in)
Winkel zur positiven x-Achse.

::Plotchart::pixelToCoords *w x y*
Gibt eine Liste von Weltkoordinaten zurück, die für das angegebene Fenster gültig sind.

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben).

float *x* (in)
X Pixel sollen abgebildet werden.

float *y* (in)
Zu abgebildendes Y-Pixel.

::Plotchart::pixelToIndex *w x y*
Gibt den Index des Kreissegments zurück, das die Pixelkoordinaten (x, y) enthält

widget *w* (in)
Name des betreffenden Canvas-Alias (wie von [\$anyplot canvas] gegeben), der ein Kreisdiagramm enthält.

float *x* (in)
X Pixel sollen abgebildet werden.

float *y* (in)
Zu abgebildendes Y-Pixel.

Darüber hinaus gibt es eine Routine zur Ermittlung "pretty"-Zahlen für die Verwendung in einer Achse:

::Plotchart::determineScale *xmin xmax inverted*
Ermittelt "pretty"-Zahlen aus dem angegebenen Bereich und gibt eine Liste zurück, die das Minimum, das Maximum und die Schrittweite enthält, die für eine (lineare) Achse verwendet werden können.

float *xmin* (in)

Ungefährer Wert für die Skalierung

float *xmax* (in)

Ungefährer Wert für die Skalierung.

boolean *inverted* (in)

Optionales Argument: Wenn der Wert 1 ist, gibt die zurückgegebene Liste eine umgekehrte Achse zurück. Der Standardwert ist 0 (die Achse verläuft vom Minimum zum Maximum).

::Plotchart::determineScaleFromList *values inverted*

Ermittelt aus der vorgegebenen Werteliste "pretty"-Zahlen und gibt eine Liste zurück, die das Minimum, das Maximum und die Schrittweite enthält, die für eine (lineare) Achse verwendet werden können.

float *values* (in)

Liste der Werte, die überprüft werden sollen. Kann fehlende Werte (leere Zeichenfolgen) enthalten.

boolean *inverted* (in)

Optionales Argument: Wenn der Wert 1 ist, gibt die zurückgegebene Liste eine umgekehrte Achse zurück. Der Standardwert ist 0 (die Achse verläuft vom Minimum zum Maximum).

Fehlende Werte

Häufig weisen Daten, die grafisch dargestellt werden sollen, Lücken auf – in einer Messdatenreihe können diese beispielsweise durch einen Geräteausfall, eine fehlerhafte Probenentnahme oder aus vielen anderen Gründen entstehen. Das *Plotchart* behandelt diese Lücken, indem es davon ausgeht, dass eine oder beide Koordinaten solcher Datenpunkte eine leere Zeichenkette sind:

```
#
# Create the plot with its x- and y-axes
#
set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0 20.0}]

foreach {x y} {0.0 32.0 10.0 {} 25.0 60.0 78.0 11.0 } {
    $s plot series1 $x $y
}
```

Die Wirkung hängt von der Art der Darstellung ab:

- Bei xy-Diagrammen, Radialdiagrammen und Streifendiagrammen führt der fehlende Datenpunkt zu einer Lücke in der Linie, die durch die Punkte verläuft.
- Bei Balkendiagrammen werden fehlende Werte so behandelt, als wäre der Wert Null angegeben worden.
- Bei Zeitdiagrammen und Gantt-Diagrammen führen fehlende Werte zu Fehlern – dort sind sie nutzlos.

Weitere Ausgabeformate

Neben der Ausgabe auf das Canvas ist das Modul in der Lage, über **canvas postscript** PostScript-Dateien zu erzeugen. Man mag sich fragen, ob es möglich ist, diese Auswahl an Ausgabeformaten zu erweitern, und die Antwort lautet "yes". In diesem Abschnitt wird versucht, die Aspekte der Nutzung dieses Moduls für eine andere Art der Ausgabe zusammenzufassen.

Eine Möglichkeit, Ausgabedateien in einem anderen Format zu erstellen, besteht darin, den Inhalt des Canvases zu überprüfen, nachdem alles gezeichnet wurde, und diesen Inhalt in der richtigen Form darzustellen. Dies ist wahrscheinlich der einfachste Weg, da er lediglich die Neuerstellung aller bereits vorhandenen Elemente in der Zeichnung erfordert.

Der Nachteil ist, dass ein Display nötig ist. Das fehlt oft bei einem CGI-Server oder Ähnlichem.

Eine Alternative ist, den Befehl canvas nachzubilden. Dafür muss klar sein, welche Unterbefehle von canvas benutzt werden und wozu. Offensichtlich wird der Unterbefehl *create* zum Erstellen von Linien, Texten und anderen Elementen verwendet. Aber auch die Unterbefehle *raise* und *lower* kommen zum Einsatz, da das Modul mit diesen die Zeichenreihenfolge beeinflussen kann – was wichtig ist, um ein Clipping-Rechteck um die Achsen herum zu simulieren. (Die Routine "DrawMask" ist hierfür zuständig – wenn das Ausgabeformat korrekte Bereiche für Clipping unterstützt, könnte eine Neudefinition dieser Routine das Problem möglicherweise lösen).

Darüber hinaus nutzt das Modul den Unterbefehl *cget*, um die Abmessungen des Canvases zu ermitteln. Ein eher praktischer Aspekt dabei ist, dass das Modul derzeit davon ausgeht, dass der Text 14 Pixel hoch ist und dass 80 Pixel Breite für die Achsenbeschriftungen ausreichen. Es wird kein "Hook" bereitgestellt, um diese Werte anzupassen.

Zusammenfassend lässt sich sagen:

- Den Unterbefehl *create* nachbilden, damit alle Elemente im richtigen Format entstehen
- Den Unterbefehl *cget* für die Optionen -width und -height nachbilden, damit Lage und Größe des Rechtecks stimmen
- Das Anheben und Absenken der Elemente so lösen, dass sie richtig beschnitten werden, etwa durch eine neue Definition der Routine DrawMask.
- Die derzeit festen Eigenschaften der Textgröße berücksichtigen

Spezialeffekte

Als Beispiel für einige Spezialeffekte, die Sie erzielen können, finden Sie hier den Code für eine Darstellung, bei der sich die Farbe des Bereichs unterhalb der Datenlinie ändert:

```
canvas .c  -background white -width 400 -height 200
pack .c -fill both

set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0 20.0}]

$s background gradient green top-down

$s dataconfig series1 -filled up -fillcolour white

$s plot series1  0.0 20.0
$s plot series1 10.0 20.0
$s plot series1 30.0 50.0
$s plot series1 35.0 45.0
$s plot series1 45.0 25.0
$s plot series1 75.0 55.0
$s plot series1 100.0 55.0

$s plaintext 30.0 60.0 "Peak" south
```

Der Trick besteht darin, den Hintergrund mit einer Farbe zu füllen, die sich von oben (grün) nach

unten (weiß) verändert. Anschließend wird der Bereich oberhalb der Datenlinie mit einem weißen Polygon ausgefüllt. Auf diese Weise variiert die grüne Schattierung je nach Höhe der Linie.

Verbesserungsbedarf

In dieser Version gibt es noch viele Dinge, die umgesetzt werden müssen:

- Robustere Behandlung fehlerhafter Aufrufe (derzeit können die Prozeduren bei fehlerhaftem Aufruf fehlschlagen):
  - Die Routinen zum Zeichnen von Achsen können derzeit keine umgekehrten Achsen verarbeiten.
  - Wenn der Benutzer eine ungültige Datums-/Zeitangabe eintippt, lösen die Routinen lediglich einen Fehler aus.

Größenanpassung

Plotchart ist nicht dafür ausgelegt, Diagramme zu erzeugen, die sich die eingegebenen Daten merken. Wenn eine Anwendung dem Benutzer also erlauben soll, die Größe des Fensters mit dem Diagramm zu ändern, muss sie selbst dafür sorgen, das gesamte Diagramm neu zu zeichnen.

Der folgende Code ist ein einfaches Beispiel dafür, wie dies umgesetzt werden kann:

```
package require Plotchart

grid [canvas .c -background white] -sticky news
grid columnconfigure . 0 -weight 1
grid rowconfigure . 0 -weight 1

bind .c <Configure> {doResize}

proc doPlot {} {
    #
    # Clean up the contents (see also the note below!)
    #
    .c delete all

    #
    # (Re)draw the bar chart
    #
    set p [::Plotchart::createBarchart .c {x y z} {0 100 10} 3]
    $p plot R {10 30 40} red
    $p plot G {30 40 60} green
}

proc doResize {} {
    global redo

    #
    # To avoid redrawing the plot many times during resizing,
    # cancel the callback, until the last one is left.
    #
    if { [info exists redo] } {
```

```
        after cancel $redo
    }

    set redo [after 50 doPlot]
}
```

Bitte beachten Sie: Der obige Code funktioniert einwandfrei für Balkendiagramme und viele andere Diagrammtypen; da **Plotchart** jedoch bei XY-Diagrammen bestimmte vertrauliche Informationen speichert, sind in diesen Fällen zusätzliche Maßnahmen erforderlich. Hierfür ist tatsächlich der Befehl "destroyPlot" erforderlich, um solche Details zu regeln. Eine nächste Version von **Plotchart** könnte diese Funktion enthalten.

Alternativ können Sie das Paket **xyplot** verwenden, das auf Plotchart aufbaut. Dieses Paket unterstützt das Vergrößern und Verkleinern sowie die Größenanpassung der gesamten Darstellung. Hier ist ein kleines Demonstrationsprogramm:

```
# xyplot_demo.tcl --
#     Demonstration of the xyplot package
#

package require xyplot

set xydata1 {}
set xydata2 {}
set xydata3 {}
set xydata4 {}
for { set i 0 } { $i < 1024 } { incr i } {
    lappend xydata1 [expr {$i-1000}] [expr {$i * sin($i/4096.0*3.1415*2) *
    (sin($i/256.0*3.1415*2))}]
    lappend xydata2 [expr {$i-1000}] [expr {$i * sin($i/4096.0*3.1415*2) *
    (sin($i/256.0*3.1415*2) + 0.25 * sin($i/256.0*3.1415*6))}]
    lappend xydata3 [expr {$i-1000}] [expr {$i * sin($i/4096.0*3.1415*2) *
    (sin($i/256.0*3.1415*2) + 0.25 * sin($i/256.0*3.1415*6) + 0.0625 *
    sin($i/256.0*3.1415*10))}]
    lappend xydata4 [expr {$i-1000}] [expr {$i * sin($i/4096.0*3.1415*2) *
    (sin($i/256.0*3.1415*2) + 0.25 * sin($i/256.0*3.1415*6) + 0.0625 *
    sin($i/256.0*3.1415*10) + 0.015625 * sin($i/256.0*3.1415*14))}]
}

set xyp [xyplot .xyp -xformat "%5.0f" -yformat "%5.0f" -title "XY plot testing"
    -background gray90]
pack $xyp -fill both -expand true

set s1 [$xyp add_data sf1 $xydata1 -legend "Serie 1 data" -color red]
set s2 [$xyp add_data sf2 $xydata2 -legend "Serie 2 data" -color green]
set s3 [$xyp add_data sf3 $xydata3 -legend "Serie 3 data" -color blue]
set s4 [$xyp add_data sf4 $xydata4 -legend "Serie 4 data" -color orange]

set xyp2 [xyplot .xyp2 -xticks 8 -yticks 4 -yformat %.2f -xformat %.0f]
pack $xyp2 -fill both -expand true

set s1 [$xyp2 add_data sf1 $xydata1]
set s2 [$xyp2 add_data sf2 $xydata2]
set s3 [$xyp2 add_data sf3 $xydata3]
set s4 [$xyp2 add_data sf4 $xydata4]
```

Zum Vergrößern wird bei gedrückter linker Maustaste ein Rechteck aufgezogen. Zum Verkleinern dient die rechte Maustaste. Ändert sich die Fenstergröße, ändern sich auch die Canvas-Widgets darin. Nach dem Vergrößern lässt sich die Darstellung über die automatisch angebrachten Bildlaufleisten verschieben.

Vergrößern

Da das "Plotchart"-Paket die Daten selbst nicht verwaltet, müsste eine Neuskalierung eines bestehenden Diagramms – beispielsweise beim Vergrößern – durch eine Neudefinition des Diagramms und ein erneutes Zeichnen der Daten erfolgen. Das "Canvas"-Widget bietet jedoch eine Lösung, indem es Elemente skaliert und verschiebt, sodass das Vergrößern etwas einfacher wird.

Ob das Zoomen tatsächlich sinnvoll ist, hängt von der Art der Darstellung ab. Derzeit ist diese Funktion nur für XY-Diagramme definiert. Die Methode heißt "rescale" und zeichnet die Achsen und Skalen einfach neu und verschiebt die Datenpunkte so, dass sie den neuen Achsen entsprechen. Der Nachteil besteht darin, dass alle Symbole um denselben Faktor skaliert werden. Die "rescale"-Methode eignet sich am besten für Diagramme, die nur Linien und keine Symbole enthalten.

Die Methode funktioniert ganz einfach:

```
$p rescale {newxmin newxmax newxstep} {newymin newymax newystep}
```

Konfigurationsoptionen und weitere Befehle

Mit den Befehlen **plotconfig** und **plotstyle** lassen sich verschiedene Optionen festlegen. Der Befehl **eraseplot** löscht eine Darstellung oder ein Diagramm vollständig. Die Syntax dieser Befehle lautet:

::Plotchart::plotconfig *charttype component property value*

Legt einen neuen Wert für die Eigenschaft einer Komponente in einem bestimmten Diagramm- oder Darstellungstyp fest oder fragen Sie deren aktuellen Wert ab. Geänderte Eigenschaften wirken sich nur auf die nachfolgenden Darstellungen aus, nicht auf bereits erstellte. Jedes Argument ist optional.

Hinweis: Der Befehl **plotstyle** bietet eine flexiblere Möglichkeit zur Steuerung der Konfigurationsoptionen.

string *charttype* (in)

Der Typ des Diagramms oder der Darstellung (siehe den Konfigurationstyp, der bei jedem create-Befehl angegeben ist). Wird dieser nicht angegeben oder ist er leer, gibt es eine Liste der Diagrammtypen zurück. Wird er angegeben, werden die Eigenschaften für diesen bestimmten Typ verwendet.

string *component* (in)

Die Komponente des Diagramms: leftaxis, rightaxis, background, margin und so weiter. Fehlt sie oder ist sie leer, kommt eine Liste der Komponenten zurück. Ist sie angegeben, gelten die Eigenschaften dieser Komponente für diesen Diagrammtyp.

string *property* (in)

Die Eigenschaft der Komponente: textcolor, die thickness der Achsenlinie und so weiter. Fehlt sie oder ist sie leer, kommt eine Liste der Eigenschaften zurück. Ist sie angegeben, wird diese Eigenschaft dieser Komponente für diesen Diagrammtyp gesetzt.

string *value* (in)

Der neue Wert der Eigenschaft. Ist er leer, kommt der aktuelle Wert zurück. Der Wert "default" stellt den Vorgabewert wieder her.

Ein leerer Wert ist manchmal sinnvoll. Dafür steht "none", etwa bei Farben und Formaten.

::Plotchart::plotstyle *subcmd style args*

Steuert den *style*, in dem die folgenden Darstellungen gezeichnet werden. Der Vorgabestil ist "default". Weitere Stile lassen sich definieren und laden.

string *subcmd* (in)

Der auszuführende Unterbefehl:
- *configure* – Mit diesem Unterbefehl können Sie die Optionen für die einzelnen

Diagrammtypen festlegen. Er akzeptiert dieselben Optionen wie der Befehl **plotconfig**.
- *current* – gibt den aktuellen Stil zurück
- *load* – Legt den angegebenen Style als aktiven Style für nachfolgende Diagramme und Grafiken fest
- *names* – gibt die Liste der derzeit definierten Styles zurück

string *style* (in)
Der Name des Diagrammstils, der bearbeitet werden soll

list *args* (in)
Die neuen Optionen für das Design. Jede Option wird wie folgt beschrieben: Diagrammtyp, Komponente des Diagramms, Eigenschaft der Komponente und der neue Wert für die Eigenschaft – Einzelheiten finden Sie im Befehl **plotconfig**.

Nachstehend finden Sie eine detaillierte Auflistung der Komponenten und Eigenschaften:

- Achsen gibt es in vielen Varianten:
  - "leftaxis", "rightaxis", "topaxis" und "bottomaxis" für die Diagramme mit rechteckiger Form.
  - xaxis, yaxis und zaxis gelten für die 3D-Diagramme
  - axis – die radiale und die tangentiale Achse eines Polardiagramms

- Alle Achsen weisen die folgenden Eigenschaften auf:
  - color – Farbe der Linie und der Skalenstriche
  - thickness – Breite der Achsenlinie selbst, nicht der Skalenstriche
  - ticklength – die Länge der Skalenstriche in Pixeln. Ein positiver Wert bedeutet nach außen, ein negativer Wert nach innen.
  - font – Schrift der Beschriftungen und des Textes an der Achse
  - format – Format der numerischen Beschriftungen. An der Zeitachse das Format für Datum und Uhrzeit.
  - textcolor – die Farbe für die Beschriftungen und den Text.
  - labeloffset – Abstand (in Pixeln) zwischen dem Skalenstrich und der eigentlichen Beschriftung
  - minorticks – Anzahl der kleinen Skalenstriche zwischen den großen
  - shownumbers – Legt fest, ob die Zahlen/Bezeichnungen angezeigt werden sollen oder nicht.
  - showaxle – Legt fest, ob die Achsenlinie angezeigt werden soll oder nicht.

- Die Komponente *margin* bestimmt das Layout. Derzeit lassen sich die Ränder nur bei rechteckigen Diagrammen setzen: left, right, top und bottom. Die Werte sind Pixel.
- Die Komponente *text* gilt für Text über den Unterbefehl plaintext. Die Eigenschaften sind textcolor, font und anchor (Lage des Textes zu den angegebenen Koordinaten).
- Die Komponente *background* hat zwei Eigenschaften: outercolor, die Farbe außerhalb der eigentlichen Darstellung, und innercolor, die Farbe innen. (Derzeit ist nur outercolor umgesetzt.)
- Die Komponente *mask* hat nur die Eigenschaft draw. Steht sie auf 1, der Vorgabe, werden weiße Rechtecke gezeichnet, die das Abschneiden nachahmen: überstehende Daten werden unsichtbar. Sonst entfallen die Rechtecke. Das hilft, das Layout enger zu steuern, etwa bei mehreren Darstellungen auf einem Canvas.
- Die *title*-Komponente verfügt über dieselben Eigenschaften wie die *text*-Komponente (ist jedoch von dieser unabhängig). Sie verfügt zudem über eine *background*-Eigenschaft: Ist diese

nicht festgelegt (oder auf die leere Zeichenkette gesetzt), entspricht sie der outercolor-Eigenschaft der *background*-Komponente; andernfalls handelt es sich um eine eigene Farbe.
- Die Komponente *legend* hat drei Eigenschaften: background, border und position. Die Bedeutung steht beim Unterbefehl legend.
- Die Komponente *bar* wird für alle Balkendiagramm-ähnlichen Darstellungen verwendet und verfügt über drei Eigenschaften: *barwidth* (relative Breite der Balken im Verhältnis zu den Elementen entlang der Achse), *innermargin* (relative Breite der Abstände zwischen Balken oder Balkengruppen) und die Farbe *outline*.
- Die Komponente *labels* dient dazu, das Erscheinungsbild der Beschriftungen von Kreisdiagrammen und "Spiral"-Kreisdiagrammen zu beschreiben. Die Eigenschaften lauten:
  - textcolor – Farbe des Beschriftungstextes
  - font – Schrift des Beschriftungstextes
  - placement – *out* außerhalb des Kreises oder *in* im Kreis
  - sorted – die Daten werden zuerst aufsteigend geordnet
  - shownumbers – Die Bezeichnungen werden gemäß dem Format mit den Zahlen kombiniert
  - format – das zu verwendende Format (Standardwert: "%s (%g)"), wenn die Zahlen angezeigt werden sollen. Der Befehl "format" gibt zunächst die Bezeichnung und anschließend die Zahl an.
  - formatright – Falls angegeben, das Format, das für Beschriftungen und Zahlen rechts neben dem Kreisdiagramm verwendet werden soll. Der Befehl "format" gibt zuerst die Zahl und dann die Beschriftung an. (Standardwert: "")

- Die Komponente *slice* verfügt über Eigenschaften, mit denen sich das Erscheinungsbild der Segmente im Kreisdiagramm steuern lässt:
  - outline – Farbe der Linie um die Segmente (Vorgabe: black)
  - outlinewidth – Breite der Linie um die Segmente herum (Standard: 1 Pixel)
  - startangle – Winkel gegen die positive x-Achse, an dem das erste Segment beginnt
  - direction – Richtung, in der die Segmente gezeichnet werden (Vorgabe: +, also im Uhrzeigersinn)

- Tabellendiagramme nutzen die allgemeinen Komponenten *title* und *margin* und dazu *header*, *oddrow*, *evenrow*, *cell* und *frame*:
  - *header*, *oddrow* und *evenrow* haben die Eigenschaften *background*, *font*, *color*, *height* und *anchor*.
  - Die Komponente *cell* definiert zusätzlich *leftspace*, *rightspace* und *topspace* zur feinen Steuerung der Abstände innerhalb der Zelle. Diese werden jedoch nicht über den Unterbefehl *cellconfigure* festgelegt.
  - Die Komponente *frame* nutzt *color*, *outerwidth* (Breite der Linie um die ganze Tabelle) und *innerwidth* (Breite der Linien zwischen Spalten und Zeilen).

- Das Zieldiagramm und das Taylor-Diagramm verwenden neben den verschiedenen allgemeinen Komponenten auch die *limits*-Komponente. Die *limits*-Komponente verfügt über eine Eigenschaft: die Farbe der Kreise und Kreissegmente.

  Beim Taylor-Diagramm können Sie die Farbe der Referenzkreise über die *reference*-Komponente festlegen.

::Plotchart::eraseplot *anyplot*

Löschen Sie die Kurve/das Diagramm samt aller Verbindungen zu den damit verbundenen Ressourcen.

string *anyplot* (in)

Der Befehl "plot/chart". Alle mit diesem Befehl verknüpften Canvas-Elemente sowie alle internen Ressourcen werden entfernt, einschließlich des Befehls "plot/chart" selbst.

Beispiele für die Verwendung finden Sie in der Datei "plotdemos7.tcl".

Scrollen bei Zeitdiagrammen und Gantt-Diagrammen

Für zwei Arten von Diagrammen wurde eine automatische Bildlaufsteuerung implementiert: Zeitdiagramme und Gantt-Diagramme. Die Unterbefehle *hscroll* und *vscroll* ordnen dem Diagramm (vorhandene) Bildlaufleisten zu, ähnlich wie dies bei Text- und Canvas-Widgets der Fall ist.

Sobald die Zuordnung hergestellt ist, werden die Bildlaufleisten automatisch aktualisiert, wenn:

- Ein Eintrag kommt hinzu, dessen Zeitraum länger ist als der aktuelle.
- Eine senkrechte Linie kommt für einen Zeitpunkt außerhalb der aktuellen Grenzen hinzu.
- Ein weiteres Element kommt hinzu, über die Zahl hinaus, mit der das Diagramm erzeugt wurde.

Zum Beispiel:

```
package require Plotchart

canvas .c -width 400 -height 200
scrollbar .y -orient vertical
scrollbar .x -orient horizontal

grid .c .y -sticky news
grid .x    -sticky news

source plotchart.tcl

set s [::Plotchart::createTimechart .c "1 january 2004"  "31 december 2004" 4]

$s period "Spring" "1 march 2004" "1 june 2004" green
$s period "Summer" "1 june 2004" "1 september 2004" yellow
$s vertline "1 jan" "1 january 2004"
$s vertline "1 apr" "1 april 2004"
$s vertline "1 jul" "1 july 2004"
$s vertline "1 oct" "1 october 2004"
$s vertline "1 jan" "1 january 2005"
$s vertline "1 apr" "1 april 2005"
$s vertline "1 jul" "1 july 2005"
$s milestone "Longest day" "21 july 2004"
$s milestone "Longest day 2" "21 july 2004"
$s milestone "Longest day 3" "21 july 2004"
$s milestone "Longest day 4" "21 july 2004"
$s milestone "Longest day 5" "21 july 2004"
$s milestone "Longest day 6" "21 july 2004"
$s title "Seasons (northern hemisphere)"

$s vscroll .y
$s hscroll .x
```

Der ursprüngliche Zeitraum des Diagramms erstreckt sich vom 1. Januar 2004 bis zum 31. Dezember 2004. Da jedoch im Jahr 2005 vertikale Linien hinzugefügt wurden und mehr Elemente vorhanden sind, als bei der Erstellung des Diagramms festgelegt waren, werden sowohl die horizontale als auch die vertikale Bildlaufleiste aktiviert.

Spezielle Diagramme

Die meisten der oben beschriebenen Diagramm- und Grafiktypen sind relativ allgemein einsetzbar, und Sie bereiten die zu darstellenden Daten einfach selbst vor. In diesem Abschnitt werden mehrere Diagrammtypen beschrieben, die insofern spezialisierter sind, als sie spezifische Zwecke erfüllen und Sie Rohdaten übergeben, die dann in den Darstellungsroutinen verarbeitet werden.

Derzeit gibt es folgende Typen:

- Zieldiagramme dienen dazu, die Fähigkeit numerischer Modelle zur Nachbildung von Messdaten zu bewerten. Sie werden ausführlich beschrieben in:

```
Jason K. Joliff et al.
    Summary diagrams for coupled hydrodynamic-ecosystem model skill assessment
    Journal of Marine Systems 76 (2009) 64-82
    DOI: 10.1016/j.jmarsys.2008.05.014
```

- Leistungsprofile dienen dazu, die Leistung numerischer Methoden oder deren Implementierungen miteinander zu vergleichen. Weitere Informationen finden Sie unter:

```
Desmond Higham and Nicholas Higham
    Matlab Guide
    SIAM, 2005, Philadephia
```

- Taylor-Diagramme sind eine weitere grafische Darstellung davon, wie numerische Modelle Messdaten nachbilden. Eine ausführliche Beschreibung findet sich in https://en.wikipedia.org/wiki/Taylor_diagram.

Die meisten allgemeinen Methoden für XY-Diagramme lassen sich auch auf diese Diagramme anwenden, doch ihre Erstellung und die Methoden zur Darstellung der Daten sind sehr spezifisch.

::Plotchart::createTargetDiagram *w limits scale*

Erzeugt ein neues Zieldiagramm mit Kreisen, die bestimmte Grenzwerte kennzeichnen. Die x-Achse stellt die unverzerrte "root-mean-square difference" dar (die typischerweise zwischen -1 und 1 schwankt), und die y-Achse stellt die normalisierte Verzerrung dar.

Datenpunkte, die näher am Ursprung liegen, stehen für bessere Ergebnisse als Datenpunkte, die weiter entfernt sind.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *limits* (in)

Liste der Radien der Kreise, die die Grenzwerte darstellen (beispielsweise: 0,5 und 0,7)

double *scale* (in)

Skalierung der Achsen – der Standardwert ist 1; wenn die Modellergebnisse jedoch nicht gut passen, ist dieser Wert möglicherweise zu klein. Beide Achsen werden auf dieselbe Weise skaliert.

$target plot *series xvalues yvalues*

Die Plot-Methode verwendet zwei Datensätze gleicher Länge, wobei der erste die Modellergebnisse und der zweite die Messwerte oder, allgemeiner gesagt, die Daten darstellt, die reproduziert werden sollen.

string *series* (in)

Name der Serie (wird als Symbol dargestellt, das über den Befehl *$target dataconfig* konfiguriert wird (siehe die entsprechende XY-Darstellung zur Erläuterung))

list *xvalues* (in)

Liste der Modellergebnisse (fehlende Werte werden als leere Zeichenfolgen dargestellt)

list *yvalues* (in)

Liste der Werte (fehlende Werte werden als leere Zeichenfolgen dargestellt; nur wenn sowohl der x- als auch der y-Wert angegeben sind, wird das Paar in den Berechnungen berücksichtigt)

::Plotchart::createPerformanceProfile *w max*

Erzeugt ein Diagramm, um die Leistungsfähigkeit verschiedener numerischer Methoden (oder Löser) darzustellen. Die Idee besteht darin, diese Methoden zunächst auf eine Reihe von Aufgaben anzuwenden und ihre Leistungsfähigkeit zu messen. Je kleiner der Wert, desto besser. Anschließend werden diese Methoden anhand eines sogenannten Leistungsprofils verglichen: Die Daten werden skaliert und geordnet, sodass die beste Methode an oberster Stelle steht.

Aufgrund der Art der Darstellung müssen alle Daten auf einmal angegeben werden.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

float *max* (in)

Maximaler Wert für die x-Achse (die x-Achse stellt den skalierten Wert der Reihe dar).

$performance plot *series_and_data_pairs*

Stellt die Daten für jede angegebene Methode grafisch dar. Die Daten werden anhand des Namens der Serie identifiziert, und die Darstellung wird über den zuvor eingegebenen "dataconfig"-Unterbefehl gesteuert.

list *series_and_data_pairs* (in)

Liste der Serientitel und Daten. Alle Daten müssen auf einmal angegeben werden.

::Plotchart::createTaylorDiagram *w radius_data args*

Erzeugt ein neues Taylor-Diagramm (ein Quadrant) mit Kreisen, die den Abstand zum Referenzpunkt angeben.

Die Datenpunkte sind als Standardabweichung und als Korrelation zum Referenzpunkt angegeben.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Grafik angezeigt werden soll.

list *radius_data* (in)

Eine Liste der maximalen Radien für die Standardabweichung und die Schrittweite. Es werden Viertelkreise mit den Achsenbeschriftungen eingezeichnet.

Derzeit wird eine Option unterstützt:

reference value

Wert, mit dem die Datenpunkte verglichen werden sollen. Er wird als Punkt auf der x-Achse dargestellt. Zusammen mit diesem Punkt werden Kreissegmente im gleichen Abstand wie die Achsenbeschriftungen eingezeichnet, um den Abstand der Datenpunkte zur Referenz anzuzeigen.

$taylor plot *series stdev corr*

Die Plot-Methode nimmt die Standardabweichung und die Korrelation zur Referenz als Eingabewerte entgegen und zeichnet ein Symbol als Darstellung. Die Standardabweichung dient als Abstand vom Ursprung, und die Korrelation bestimmt den Winkel.

string *series* (in)

Name der Serie (wird als Symbol dargestellt, das über den Befehl *$target dataconfig* konfiguriert wird (siehe die entsprechende XY-Darstellung zur Erläuterung))

float *stdev* (in)

Standardabweichung des Datenpunkts

list *corr* (in)

Korrelationskoeffizient mit dem Referenzdatensatz

::Plotchart::createHeatmap *w rowlabels columnlabels args*

Erzeugt eine Heatmap, d. h. eine Tabelle aus Rechtecken, deren Farben von bestimmten Daten abhängen. Die Zeilen- und Spalte-Bezeichnungen dienen beim Einfügen der Daten als Kennungen. Ihre Anzahl bestimmt die Größe der Tabelle.

Die Daten werden zeilenweise, spaltenweise oder zelleweise übergeben.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Heatmap angezeigt werden soll.

list *rowlabels* (in)

Liste der Bezeichnungen, die in jeder Zeile angezeigt werden sollen. Die Bezeichnungen dienen zudem als Kennungen.

list *columnlabels* (in)

Liste der Bezeichnungen, die in jeder Spalte angezeigt werden sollen. Die Bezeichnungen dienen zudem als Kennungen.

list *args* (in)

Null oder mehr Argumente zur Festlegung der Größe der Heatmap. Weitere Informationen finden Sie unter "XY-Diagramm".

$heatmap plot row *label data*

Nutzt die angegebenen Daten, um die Rechtecke der Zeile "label" auszufüllen.

string *label* (in)

Zeichenkette, die angibt, welche Zeile verwendet werden soll.

list *data* (in)

Liste der numerischen Daten, die für die Farbgebung der Rechtecke verwendet werden sollen.

$heatmap plot column *label data*

Füllt mit den angegebenen Daten die Rechtecke der Spalte "label".

string *label* (in)

Zeichenkette, die angibt, welche Spalte verwendet wird.

list *data* (in)

Liste der numerischen Daten, die für die Farbgebung der Rechtecke verwendet werden sollen.

$heatmap plot cell *rowlabel columnlabel value*

Nutzt den angegebenen Wert, um das Rechteck auszufüllen, das zu der Zelle mit den angegebenen Zeilen- und Spaltenbezeichnungen gehört.

string *rowlabel* (in)

Zeichenkette, die angibt, welche Zeile verwendet werden soll.

string *columnlabel* (in)

Zeichenkette, die angibt, welche Spalte verwendet wird.

float *value* (in)

Wert, der für die Farbgebung der Zelle verwendet werden soll.

$heatmap scale values *min max*

Legt den Bereich der Werte fest – die Werte werden mittels linearer Interpolation einer Farbe zugeordnet.

float *min* (in)

Zu verwendender Mindestwert.

float *max* (in)

Zu verwendender Höchstwert.

$heatmap scale colours *mincolour maxcolour*

Legt die Farben fest, die für den Minimal- und den Maximal-Wert verwendet werden sollen. Die tatsächliche Farbe wird durch lineare Interpolation der RGB-Werte ermittelt.

string *mincolour* (in)

Farbe beim Minimalwert.

string *maxcolour* (in)

Farbe beim Maximalwert.

::Plotchart::createCircleplot *w labels args*

Erzeugt einen circle plot, einen Kreis mit Beschriftungen, die sich durch farbige Bögen verbinden lassen. Typische Anwendung: Darstellung der Beziehungen zwischen den Elementen auf dem Kreis in grafischer Form.

Die Verbindungen können paarweise eingezeichnet werden.

widget *w* (in)

Name des *vorhandenen* Canvas-Widgets, in dem die Kreisgrafik angezeigt werden soll.

list *rowlabels* (in)

Liste der Beschriftungen, die entlang des Kreises angezeigt werden sollen. Jede Beschriftung wird von einem ausgefüllten Punkt begleitet.

list *args* (in)

Null oder mehr Argumente zur Festlegung der Größe des circle plot. Weitere Informationen finden Sie unter "XY-Diagramm".

$circleplot connect *label1 label2 colour width*

Schaffen Sie eine Verbindung zwischen den beiden Beschriftungen durch einen farbigen Bogen mit einer vorgegebenen Breite (der Bogen ist eigentlich eine Parabel).

string *label1* (in)

Zeichenkette, die die erste zu verbindende Beschriftung angibt.

string *label2* (in)

Zeichenkette, die die zweite Beschriftung angibt, mit der eine Verbindung hergestellt werden soll.

string *colour* (in)

Farbe, die für den verbundenen Bogen verwendet werden soll.

string *width* (in)

Breite der Verbindung des Bogens.

$circleplot modify *label args*

Passt das Aussehen der Beschriftung und des dazugehörigen Punkts an.

string *label* (in)

Zeichenkette, die angibt, welche Bezeichnung geändert werden soll.

list *args* (in)
Liste der Schlüssel-Wert-Paare:
- *-textcolour colour* – Farbe des Textes (Alternative: "-textcolor")
- *-font font* – Schrift des Textes
- *-dotcolour colour* – Farbe des Punktes (Alternative: "-dotcolor")

Eigene Diagrammmethoden hinzufügen

Mit dem Befehl *plotmethod* lassen sich neue Methoden für einen bestimmten Diagrammtyp hinzufügen. Er hilft, spezielle grafische Darstellungen zu entwickeln.

::Plotchart::plotmethod *charttype methodname plotproc*
Fügt eine neue Methode für den angegebenen Diagramm- oder Charttyp hinzu. Die Methode wird durch den im Argument "plotproc" angegebenen Befehl oder die dort angegebene Prozedur implementiert. Die Prozedur wird mit zwei zusätzlichen Argumenten aufgerufen: dem Namen des erstellten Diagramms und dem Canvas-Widget, das dieses enthält (siehe das Beispiel unten).

string *charttype* (in)
Die Art des Diagramms oder der Grafik, zu der die neue Methode hinzugefügt werden soll.

string *methodname* (in)
Name der zu verwendenden Methode.

string *plotproc* (in)
Name des Befehls oder der Prozedur, die die Methode implementiert.

Hier ist ein einfaches Beispiel für die Anwendung:

```
#
# The custom method "doodle" always adds the text "DOODLE"
# to the plot
#
proc doodle {p w x y} {
    $p plaintext $x $y "DOODLE"
}
::Plotchart::plotmethod xyplot doodle doodle

#
# Use it
pack [canvas .c]

set p [::Plotchart::createXYPlot .c {0 100 10} {0 20 5}]

$p doodle 40 10
```

Tabellendiagramme

Um zu veranschaulichen, welche Möglichkeiten Ihnen mit Tabellendiagrammen zur Verfügung stehen, finden Sie hier ein einfaches Beispiel, in dem eine Reihe von Zufallsdaten grafisch dargestellt wird. Die Farben richten sich nach dem Bereich, zu dem die Daten gehören. Hierfür wird die Prozedur *setColor* verwendet.

```
package require Plotchart

pack [canvas .c -bg white -height 300] -fill both -expand yes

::Plotchart::plotconfig table frame outerwidth 3
::Plotchart::plotconfig table frame color red

set t [::Plotchart::createTableChart .c {"Column 1" "Column 2" "Column 3"} 80]


proc setColor {table widget row col value} {
    $table cellconfigure -background white -color black
    if { $value < 2.0 } {
        $table cellconfigure -background red -color white
    }
    if { $value > 6.0 } {
        $table cellconfigure -background green
    }

    return [format "%6.3f" $value]
}

# Command must already exist ...
$t formatcommand setColor

$t title "Demonstration of table charts"
$t separator

for {set i 0} {$i < 9} {incr i} {
    set row {}

    for {set j 0} {$j < 3} {incr j} {
        lappend row [expr {10.0 * rand()}]
    }

    if { $i == 3 } {
        $t separator
    }

    $t row $row
}
```

Anzeigeinstrumente

TODO

Verwendung von Datums- und Zeitangaben

Die Optionen -timeformat und -gmt dienen dazu, die Anzeige von Datums- und Zeitangaben entlang der x-Achse für jene Diagrammtypen zu steuern, bei denen dies sinnvoll ist. Diese Optionen wurden implementiert, um Datums- und Zeitangaben für Streifendiagramme zu berücksichtigen, da sich bei einer Achse "static" auch eigene Beschriftungen (die Option -xlabels) setzen lassen. Da dies bei Streifendiagrammen nicht der Fall ist, war dies keine Option

(Tcllib/Tklib-Fehler 3613718). Sie können auch die Option -locale angeben, um eine lokalisierte Datums- und Zeitbeschriftung zu erhalten. Das folgende Beispiel veranschaulicht die Verwendung der Option -timeformat. Die Option -gmt unterdrückt lediglich die Berücksichtigung der Sommerzeit durch den Befehl [clock format].

```
package require Plotchart

pack [canvas .c -width 500 -bg white]

#
# Note that we need to present the x values as clock seconds
#
set start [clock scan  "0:00"]
set stop  [clock scan "10:00"]
set s [Plotchart::createStripchart .c [list $start $stop 7200] {0 10 1} -timeformat
    "%H:%M"]

# With a locale like "de":
#set s [Plotchart::createStripchart .c [list $start $stop 7200] {0 10 1}
    -timeformat "%H:%M" -locale de]

foreach {x y} {0 0 2 5 5 2 9 9 12 10} {
    set x [expr {$start + 3600 * $x}] ;# Convert hour to clock seconds
    $s plot a $x $y
}
```

Der Unterbefehl *plot* interpretiert die x- und y-Daten lediglich als reine Zahlen, die Umrechnung von Datum und Uhrzeit in "clock seconds" bleibt außen vor.

Anordnung mehrerer Diagramme auf einem Canvas

Mit dem Befehl *plotpack* können Sie den Inhalt einer Grafik in ein anderes Canvas-Widget kopieren. Dieses Canvas-Widget fungiert nicht als zusammengesetzte Grafik, kann jedoch beispielsweise als PostScript-Datei gespeichert werden: Hinweis: Der Befehl erstellt lediglich eine Momentaufnahme der Grafiken, wie sie zu diesem Zeitpunkt aussehen.

::Plotchart::plotpack *w dir args*

Kopieren Sie den Inhalt der Diagramme/Grafiken in ein anderes Widget, ähnlich wie beim Geometriemanager *pack*.

widget *w* (in)

Der Name des Canvas-Widgets, in das die Diagramme/Grafiken kopiert werden sollen

string *dir* (in)

Die Ausrichtung der Anordnung – oben, links, unten oder rechts

list *args* (in)

Liste der zu kopierenden Diagramme/Grafiken.

Beispiel:

```
    set p1 [createXYPlot ...]
    set p2 [createBarchart ...]

    ... fill the plots ...
```

```
    toplevel .t
    pack [canvas .t.c2 -width ...]

    #
    # Copy the two plots above each other in the new canvas
    #
    plotpack .t.c2 top $p1 $p2
```

Eine andere Methode besteht darin, bei der Erstellung der Grafik die Optionen *-box* und *-axesbox* zu verwenden. Diese steuern den Bereich auf dem Canvas, in dem die Grafik oder das Diagramm gezeichnet wird.

Die Option *-box* nimmt als Wert eine Liste aus vier Zahlen entgegen:

- X-Koordinate der oberen linken Ecke des Bereichs, in dem die Grafik oder das Diagramm angezeigt werden soll (einfach eine Canvas-Koordinate)
- Y-Koordinate der oberen linken Ecke
- Breite des Bereichs
- Höhe des Bereichs

Durch die Angabe von Breite und Höhe lässt sich der Bereich leichter im Verhältnis zu anderen Diagrammen neu positionieren.

Die Option *-axesbox* soll die Ausrichtung der Achsen eines Diagramms an denen anderer Diagramme erleichtern. Die Option erwartet eine Liste mit sechs Argumenten:

- Identifikation des Plots, in Bezug auf den er positioniert werden soll (der vom Befehl "create" zurückgegebene Befehl).
- Die zu verwendende Position des Ankers (n, nw, ...)
- X-Koordinate der oberen linken Ecke des Bereichs, in dem die Grafik oder das Diagramm angezeigt werden soll. Diese Koordinate wird relativ zur *Ankerposition* angegeben.
- Y-Koordinate der oberen linken Ecke
- Breite des Achsenbereichs
- Höhe des Achsenbereichs

Mit dieser Option wird zunächst die Fläche ermittelt, die die Achsen einnehmen, und die Gesamtfläche wird anhand der Außenabstände berechnet.

Beispiel:

```
    set p2 [::Plotchart::createXYPlot .c {0 10 1} {-5 5 2.5} -axesbox [list $p1 ne
    0 0 200 200]]
```

Erzeugt ein zweites Diagramm, dessen linke Achse mit der rechten Achse des Diagramms "\$p1" zusammenfällt und dessen Achsenoberkante sich ebenfalls auf derselben Höhe befindet – da die Achsen an einem Punkt positioniert sind, der 0 Pixel links und 0 Pixel unterhalb der nordöstlichen Ecke liegt.

Interaktive Nutzung

Plotchart verfügt über mehrere Funktionen für die interaktive Nutzung (vgl. Hinweise zu Tags):

- Sie können die Legende verschieben, indem Sie die linke Maustaste innerhalb des Legendenfeldes drücken und gedrückt halten.
- Mit den Befehlen *bindplot* und *bindlast* können Sie Aktionen definieren, die ausgeführt werden

sollen, wenn der Benutzer auf ein Element der Darstellung oder des Diagramms klickt. (siehe unten, siehe auch den Beispielcode in plotdemos12.tcl)
- *Piecharts* können ein Segment "exploded" anzeigen, das sich mit Maustaste 1 wählen lässt.

Andere Interaktion, die *Plotchart* selbst nicht bietet, lässt sich über die Tags der Canvas-Elemente mit weiteren Bindungen einrichten.

bindplot und *bindlast* sind so definiert:

$anyplot bindplot *event command args*

Registrieren Sie einen Befehl, der immer dann ausgeführt wird, wenn das angegebene Ereignis im Diagramm eintritt.

string *event*

Das Ereignis, an das Sie den Befehl binden möchten

string *command*

Name des Befehls bzw. der Prozedur, die Sie ausführen möchten. Den folgenden Argumenten werden die x- und y-Koordinaten des Punktes in der Darstellung (die Weltkoordinaten!) vorangestellt, sodass die Prozedur folgende Signatur aufweist:

```
    cmd $xworld $yworld $string1 $string2 $string3
```

Angenommen, das Argument "command" lautet: {cmd A B C}

$anyplot bindlast *series event command*

Registrieren Sie einen Befehl, der ausgeführt wird, wenn das Ereignis in der Nähe des zuletzt zur angegebenen Reihe hinzugefügten Punktes eintritt. (Sie können dies unmittelbar nach dem Einfügen eines Datenpunkts tun. Alle derartigen Befehle bleiben aktiv.)

string *event*

Das Ereignis, an das Sie den Befehl binden möchten

list *command*

Name des Befehls bzw. der Prozedur, die Sie ausführen möchten. Den folgenden Argumenten werden die x- und y-Koordinaten des Punktes in der Darstellung (die Weltkoordinaten!) vorangestellt, sodass die Prozedur folgende Signatur aufweist:

```
    cmd $xworld $yworld $string1 $string2 $string3
```

Angenommen, das Argument "command" lautet: {cmd A B C}

Hier ist ein Beispiel – zeigen Sie die Werte der Datenpunkte in einer Beschriftung an (aus dem Beispielcode in "plotdemos12.tcl"):

```
#
# Procedure for showing an annotation
#
proc showAnnotation {xcoord ycoord plot w} {

    $plot balloon $xcoord $ycoord "Data point: [format "%.3f, %.3f" $xcoord
    $ycoord]" north

    after 2000 [list removeAnnotation $w]
}
```

```
#
# Procedure for erase an annotation
#
proc removeAnnotation {w} {

    # Use the tags to remove all annotations
    $w delete BalloonText
    $w delete BalloonFrame
}

#
# Create a simple plot and a label
#
pack [canvas .c -bg white] [label .l -textvariable coords]

set p [::Plotchart::createXYPlot .c {0 1000 200} {0 10 1}]

$p dataconfig series1 -type both -symbol cross

foreach x {1 2 5 10 20 50 100 200 500 1000} {
    $p plot series1 $x [expr {log($x)}]

    #
    # Show the annotation for each data point
    #
    $p bindlast series1 <Enter> [list showAnnotation $p %W]
}
```

Hinweise zu Tags

Die Implementierung von *Plotchart* stützt sich in hohem Maße auf die Fähigkeit des Canvases, grafische Objekte anhand von Tags zu identifizieren und die Zeichenreihenfolge der Objekte zu ändern. In diesem Abschnitt werden die verwendeten Tags beschrieben.

(*Hinweis:* Die Tags werden nicht immer einheitlich verwendet – siehe die Hinweise zu den jeweiligen Tags. In diesem Abschnitt wird der aktuelle Stand beschrieben.)

General graphical objects:

- *mask* – Dient zur Bearbeitung der undurchsichtigen Rechtecke, die sicherstellen, dass Daten außerhalb des Ansichtsbereichs nicht angezeigt werden.
- *topmask, horizmask, vertmask* – spezielle Tags, die für scrollbare Diagramme verwendet werden.
- *title* – Tag für Titelzeichenketten.
- *BalloonText, BalloonFrame* – Dienen zur Bearbeitung von Sprechblasentexten.
- *PlainText* – Wird verwendet, um einfachen Text ohne jegliche Formatierung zu bearbeiten.
- *background* – Tag für Farbverlaufs- und Bildhintergründe und für Balken mit Farbverlauf.
- *xaxis*, *yaxis* – Tags für alle Objekte einer waagerechten oder senkrechten Achse, sowohl bei numerischen Achsen als auch bei Achsen mit Beschriftungen wie in Balkendiagrammen. Der *text* entlang der Achsen hat kein eigenes Tag.
- *raxis* – Tag, der für alle Objekte verwendet wird, die sich auf eine *rechte* Achse beziehen.
- *taxis* – Tag für alle Objekte einer Achse *time*.

- *axis3d* – Tag, das für 3D-Achsen verwendet wird
- *xtickline, ytickline* – Tags für die Gitterlinien.
- *legend, legengb, legendobj* – Tags, die für die Legende verwendet werden. Letzteres dient dazu, die Legende als Ganzes zu bearbeiten.
- *legend_series* – Tag zur Steuerung der Darstellung des Legenden-Eintrags ("series" sollte durch den Namen der Reihe ersetzt werden).
- *object* – Standard-Tag für alle Objekte, die **::Plotchart::drawobject** zeichnet. Tags von der Erzeugung kommen dazu.

XY-Diagramme (alle Achsenarten):

- *data* – allgemeines Tag für grafische Objekte, die zu Daten gehören. *data_seriesname* – Tag einer Datenreihe ("seriesname" wird ersetzt). *band* – das waagerechte oder senkrechte Band der Unterbefehle xband und yband trägt dieses Tag unter dem tatsächlichen Namen. *xtext* – Beschriftung der x-Achse. *ytext* – waagerechte Beschriftung der y-Achse. *vtext* – senkrechte Beschriftung der y-Achse.

Beschriftete Punkte tragen nur das Tag "data".

Kreisdiagramme und Spiralkreisdiagramme:

- *segment_segmentnumber* – Das Tag zur Identifizierung des Segments; die Zeichenfolge "segmentnumber" sollte durch die tatsächliche Nummer ersetzt werden. Dieses Tag zieht die Segmente heraus.

Balkendiagramme:

Balkendiagramme verwenden dieselben Tags wie XY-Diagramme (bei Balken mit Farbverlauf wird jedoch "data_seriesname" nicht verwendet).

Histogramme und isometrische Diagramme:

Derzeit wird ausschließlich das Tag "data" verwendet.

Zeitdiagramme:

Da diese Diagramme scrollbar sind, werden mehrere Tags verwendet, die spezifisch für das Scrollen sind: "vertscroll", "horizscroll", "below", "lowest", "above", "timeline", "tline". Jedes Element verfügt zudem über ein Tag der Form "item_number", wobei "number" durch die tatsächliche Sequenznummer des Elements ersetzt werden muss.

Gantt-Diagramme:

Zusätzlich zu den für Zeitdiagramme beschriebenen Tags werden folgende Tags verwendet: description, completed, summary und summarybar.

Radialdiagramme und Polardiagramme:

Derzeit haben die radialen Linien, die das Grid anzeigen, keine Tags. Die mit Daten verbundenen grafischen Objekte besitzen lediglich das Tag "data".

Windrosen:

Derzeit wird nur das Tag *data_number* verwendet ("number" sollte durch die fortlaufende Nummer der Daten ersetzt werden, beginnend bei 0.

Kontur- und Isoliniendiagramme:

Es werden keine Tags verwendet.

3D-Diagramme und 3D-Banddiagramme:

Für die Achsen und die Datenobjekte werden Tags verwendet:

- *data* – Das allgemeine Tag zur Kennzeichnung von grafischen Objekten, die mit Daten verknüpft sind. *line* – Das Tag für Linien, die mit dem Unterbefehl "plotline" erstellt wurden.

Diagramme mit 3D-Effekten:

Die folgenden Tags werden zur Kennzeichnung verschiedener Arten von grafischen Objekten verwendet: platform, background, d, u, ticklines.

Der mit den Balken verbundene Text hat keine Tags. Auch die Bandlinien und Flächen haben keine Tags.

Tabellen:

Verwendete Tags sind: "frame", "cellbg" und "celltext"

Spezielle Diagrammtypen (Zieldiagramme, Taylor-Diagramme):

Verwendete Tags sind: limits, limit_labels, reference *Zudem:* Um mehrere Plots und Diagramme auf einem einzigen Canvas darzustellen, erhalten alle Elemente zusätzlich als Tag den Plot bzw. das Diagramm, zu dem sie gehören. Dies ermöglicht es Plotchart, ausschließlich diese Elemente zu bearbeiten.

Noch zu tun – einige private Notizen

Ich habe folgende Wunschliste:

- Isometrische Darstellungen – ermöglichen die einfache Implementierung neuer Elemente.
- Ein allgemeiner 3D-Viewer – mit Schwerpunkt auf Geometrie, kein Raytracer.
- Mehrere Verbesserungen bei den Boxplots:
  - Die Höhe des Kastens skaliert mit dem Logarithmus der Punktanzahl
  - Markierungslinie zur Kennzeichnung eines Wertes "current"
  - Aus Quantilen erstellte Box

<

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *plotchart* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 2022 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


shtmlview

shtmlview-doctools

Stichwörter: shtmlview-doctools, doctools, html

Name

shtmlview_doctools – Erweiterung für shtmlview mit Unterstützung für doctools-Markup

Übersicht

package require dtplite
package require shtmlview::shtmlview

Beschreibung

Das Paket **shtmlview::doctools** erweitert die Widgets von **shtmlview::shtmlview** um die Darstellung von Dokumenten im Format *doctools*.

Durch das einfache Laden des Pakets wird die Unterstützung registriert.

Das Paket exportiert keinerlei Befehle, Klassen usw.

Änderungsprotokoll

2022-04-07 version 0.1

- Erste Umsetzung

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese dem Autor des Pakets. Bitte teilen Sie ihm auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation mit.

Copyright am Code

Art der BSD-Lizenz:

Sun Microsystems, Inc. Die folgenden Bedingungen gelten für alle mit der Software verbundenen Dateien, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren, vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten, deutlich angegeben sind.

In keinem Fall haften die Autoren oder Vertreiber gegenüber Dritten für direkte, indirekte, besondere, zufällige oder Folgeschäden, die sich aus der Nutzung dieser Software, ihrer Dokumentation oder daraus abgeleiteter Werke ergeben, selbst wenn die Autoren auf die Möglichkeit solcher Schäden hingewiesen wurden.

Die Autoren und Vertreiber schließen ausdrücklich jegliche Gewährleistung aus, einschließlich, aber nicht beschränkt auf die stillschweigenden Gewährleistungen der Marktgängigkeit, der Eignung für einen bestimmten Zweck und der Nichtverletzung von Rechten Dritter. Diese Software wird "wie besehen" bereitgestellt, und die Autoren und Vertreiber sind nicht verpflichtet, Wartung, Support, Updates, Erweiterungen oder Modifikationen bereitzustellen.

EINGESCHRÄNKTE RECHTE: Die Nutzung, Vervielfältigung oder Offenlegung durch die Regierung unterliegt den Einschränkungen, die in Unterabsatz (c) (1) (ii) der Klausel über Rechte an technischen Daten und Computersoftware gemäß DFARS 252.227-7013 und FAR 52.227-19 festgelegt sind.

Copyright

```
Copyright (c) 2022, Detlef Groth <detlef(at)dgroth(dot)de>
```

shtmlview-mkdoc

Stichwörter: shtmlview-mkdoc, html, mkdoc

Name

shtmlview_mkdoc – Erweiterung für shtmlview, die Unterstützung für das mkdoc-Markup hinzufügt

Übersicht

package require mkdoc::mkdoc
package require shtmlview::shtmlview

Beschreibung

Das Paket **shtmlview::mkdoc** erweitert die Widgets von **shtmlview::shtmlview** um die Darstellung von Dokumenten im Format *mkdoc*.

Durch das einfache Laden des Pakets wird die Unterstützung registriert.

Das Paket exportiert keinerlei Befehle, Klassen usw.

Änderungsprotokoll

2022-04-07 version 0.1

- Erste Umsetzung

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese dem Autor des Pakets. Bitte teilen Sie ihm auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation mit.

Copyright am Code

Art der BSD-Lizenz:

Sun Microsystems, Inc. Die folgenden Bedingungen gelten für alle mit der Software verbundenen Dateien, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren, vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten, deutlich angegeben sind.

In keinem Fall haften die Autoren oder Vertreiber gegenüber Dritten für direkte, indirekte, besondere, zufällige oder Folgeschäden, die sich aus der Nutzung dieser Software, ihrer Dokumentation oder daraus abgeleiteter Werke ergeben, selbst wenn die Autoren auf die

Möglichkeit solcher Schäden hingewiesen wurden.

Die Autoren und Vertreiber schließen ausdrücklich jegliche Gewährleistung aus, einschließlich, aber nicht beschränkt auf die stillschweigenden Gewährleistungen der Marktgängigkeit, der Eignung für einen bestimmten Zweck und der Nichtverletzung von Rechten Dritter. Diese Software wird "wie besehen" bereitgestellt, und die Autoren und Vertreiber sind nicht verpflichtet, Wartung, Support, Updates, Erweiterungen oder Modifikationen bereitzustellen.

EINGESCHRÄNKTE RECHTE: Die Nutzung, Vervielfältigung oder Offenlegung durch die Regierung unterliegt den Einschränkungen, die in Unterabsatz (c) (1) (ii) der Klausel über Rechte an technischen Daten und Computersoftware gemäß DFARS 252.227-7013 und FAR 52.227-19 festgelegt sind.

Copyright

```
Copyright (c) 2022, Detlef Groth <detlef(at)dgroth(dot)de>
```

shtmlview

Stichwörter: shtmlview, html, text, widget

Name

shtmlview – Erweitertes Tcl/Tk-Text-Widget mit grundlegender Unterstützung für die Darstellung von HTML und Markdown

Übersicht

package require Tk
package require snit
package require Markdown ;# optional Markdown support
package require img::jpeg ;# optional jpeg image support
package require shtmlview::shtmlview ?**1.1.2**?
::shtmlview::converter *extension description cmdprefix*
::shtmlview::shtmlview *pathName* ?*options*?
pathName **back**
pathName **browse** *filename* ?*args*?
pathName **dosearch** *string direction*
pathName **edittext** *cmd* ?*options*?
pathName **editView**
pathName **forward**
pathName **getFiles**
pathName **getHistory**
pathName **getEditWidget**
pathName **getTextWidget**
pathName **helptext** *cmd* ?*options*?
pathName **home**
pathName **open**
pathName **reload**
pathName **render** *text* ?*ext*?
pathName **sourceView**
pathName **url**

Beschreibung

Das Paket **shtmlview::shtmlview** stellt ein reines Tcl/Tk-Widget mit dem gleichen Namen bereit, um einfache HTML- und Markdown-Dateien sowie Zeichenfolgen zu rendern und anzuzeigen.

Ein kurzer historischer Rückblick: Das Widget basiert auf der Bibliothek **htmllib**, die in den 90er Jahren von Stephen Uhler und Clif Flynt entwickelt wurde. Im Jahr 2009 integrierte Robert Heller diese Bibliothek in den hervorragenden Megawidget-Rahmen **snit**. Das daraus resultierende Widget war jedoch direkt in ein Hilfesystem eingebunden. Der Verfasser dieses Dokuments hat zunächst lediglich den Anzeigeteil isoliert und anschließend einige Funktionen hinzugefügt, wie beispielsweise die Änderung der Schriftgröße und einige Schaltflächen in der Symbolleiste. Später wurde eine rudimentäre Darstellung von Datentabellen hinzugefügt. Noch später wurden Unterstützung für Inline-Bilder, erweiterte Tastenkombinationen sowie Markdown-Unterstützung integriert.

Das Widget ist *kein* Webbrowser. Es unterstützt ausschließlich relative Links im lokalen Dateisystem. Es unterstützt *keine* Stylesheets. Es unterstützt weder HTTP(S)-Links noch Bilder. Es ist als letzte Ausweichmöglichkeit gedacht, die in Fällen genutzt werden soll, in denen keine anderen Möglichkeiten bestehen, HTML- oder Markdown-Markup aus einer Tk-Anwendung heraus anzuzeigen.

Anwendungsfälle sind der Hilfe-Viewer sowie alle Fälle, in denen der Entwickler die Kontrolle über die verwendeten HTML-Tags hat.

Kommentare und Rückmeldungen sind willkommen.

Das Widget **shtmlview::shtmlview** überlädt das Text-Widget und bietet neue Befehle und Optionen. Diese werden in den Abschnitten Widget-Befehle und Widget-Optionen erläutert.

Hinweis: Die Datei "*shtmlview.tcl*" ist nicht nur ein Paket, sondern auch eine eigenständige Anwendung zur direkten Anzeige von Markdown- und HTML-Dateien. Aufruf:

```
tclsh shtmlview.tcl filename.html
```

in einem Terminal.

Die in diesem Dokument beschriebene API stellt nicht die gesamte API dar, die vom snit-Objekt **::shtmlview::shtmlview** bereitgestellt wird. Vielmehr handelt es sich um eine Teilmenge dieser API, bei der davon ausgegangen wird, dass sie sich in zukünftigen Versionen nicht ändern wird.

Befehle

::shtmlview::converter *extension description cmdprefix*

Registriert ein Konvertierungspräfix (*cmdprefix*) für Dateien mit der *extension*. Die *description* ist eine kurze Zusammenfassung der Dateien, die diese Erweiterung tragen sollen.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Stößt ein shtmlview-Widget auf eine Datei mit dieser *extension*, ruft es das registrierte Präfix mit einem Argument auf, dem Pfad der zu konvertierenden Datei. Der Aufruf soll das HTML liefern, das dargestellt wird.

Ein Beispiel steht im Abschnitt Erweiterungen.

::shtmlview::shtmlview *pathName* ?*options*?
Erzeugt und konfiguriert das shtmlview-Widget *pathName*.

Das Ergebnis des Befehls ist der *pathName*.

Ein Fehler entsteht, wenn für *pathName* schon ein Widget besteht.

Ein Fehler entsteht auch, wenn das Eltern-Widget von *pathName* nicht besteht.

Die erkannten Optionen stehen im Abschnitt Widget-Optionen.

Die Methoden des neuen Widgets stehen im Abschnitt Widget-Befehle.

Widget-Optionen

Die Methode **helptext** konfiguriert das interne Text-Widget.

-browsecmd cmd
Jedes Mal, wenn eine HTML- oder Markdown-Seite gerendert wird, wird der angegebene Befehl *cmd* aufgerufen. Die aktuelle URL wird als erstes Argument an den Befehl angehängt.

-closebutton boolean
Wenn **true** (Standard) eingestellt ist, wird in der Symbolleiste eine Schaltfläche zum Schließen angezeigt. Dies ist nützlich, wenn das Widget das einzige untergeordnete Element eines Toplevel-Fensters ist.

-historycombo boolean
Ist der Wert **true** und ist das Paket **tile** vorhanden, kommt eine **ttk::combobox** für den Seitenverlauf in die Symbolleiste. Voreinstellung ist **false**. Die Option lässt sich nur beim Erzeugen des Widgets setzen.

-home string
Legt den Dateinamen der Startseite des Widgets fest. Falls dieser nicht festgelegt wird, wird die erste aufgerufene Seite automatisch als Startseite festgelegt.

-tablesupport boolean
Wenn **true** (Standard) eingestellt ist, wird eine gewisse Unterstützung für Tabellen bereitgestellt. Dies führt zu unerwünschten Ergebnissen, wenn Tabellen-Tags als Markup-Werkzeug missbraucht werden. Einfache HTML-Tabellen mit den Tags **th** und **td** sollten jedoch korrekt dargestellt werden, sofern innerhalb dieser Tags kein internes Markup verwendet wird.

-toolbar boolean
Wenn **true** (Standard) eingestellt ist, wird oben eine Symbolleiste angezeigt. Die Symbolleiste enthält Standardschaltflächen für die Methoden **back**, **forward** und **home** sowie Suchfunktionen. Standardmäßig ist **true** eingestellt.

Widget-Befehle

Jedes mit dem oben genannten Befehl erstellte shtmlview-Widget unterstützt die folgenden Befehle und Optionen:

***pathName* back**
Zeigt die vorherige HTML- und Markdown-Seite im Browserverlauf an, sofern vorhanden.

***pathName* browse *filename* ?args?**
Zeigt den HTML- oder Markdown-Text der genannten Datei. Weitere Argumente sind ebenfalls Dateinamen und kommen nur auf den Verlaufsstapel. Die Verlaufstasten **f** und **b** gehen sie durch.

pathName* dosearch *string direction
Sucht die angegebene Zeichenkette ab dem aktuellen Index in der angegebenen Richtung und hebt sie hervor. Die Richtung ist **forward** (Voreinstellung) oder **backwards**.

***pathName* edittext *cmd* ?*options*?**
Legt das interne Texteditor-Widget zur Konfiguration offen. Beispiel:

```
::shtmlview::shtmlview .help
.help browse index.md
.help editView
.help edittext configure -background salmon
```

***pathName* editView**
Wechselt das Widget von der Anzeige zur Bearbeitung. In diesem Modus lässt sich die geladene Datei ändern. Die nicht bearbeitbare Quellansicht ist **sourceView** weiter unten.

```
::shtmlview::shtmlview .help
.help browse index.md
.help editView
```

***pathName* forward**
Zeigt die nächste HTML- oder Markdown-Seite im Browserverlauf an, sofern vorhanden.

***pathName* getFiles**
Gibt die Liste aller besuchten Dateien zurück. Doppelte Einträge und Ankerlinks entfallen aus den Rohdaten.

***pathName* getHistory**
Gibt die aktuelle Verlaufsliste der besuchten Dateien und Anker zurück.

***pathName* getEditWidget**
Gibt den internen Pfadnamen des Texteditor-Widgets zurück, mit dem das aktuelle Dokument bearbeitet wird. Damit lässt sich direkt darauf zugreifen. Das ist gefährlich. Siehe auch **edittext** weiter oben.

***pathName* getTextWidget**
Gibt den internen Pfadnamen des internen Text-Widgets der Anzeige zurück. Damit lässt sich direkt darauf zugreifen. Das ist gefährlich. Siehe auch **helptext** weiter unten.

***pathName* helptext *cmd* ?*options*?**
Legt das interne Text-Widget der Anzeige zur Konfiguration offen. Beispiel:

```
::shtmlview::shtmlview .help
.help browse index.html
.help helptext configure -background yellow
```

***pathName* home**
Zeigt entweder die durch die Option **-home** festgelegte Seite oder die erste Seite an, für die **browse** aufgerufen wurde.

***pathName* open**

Verwendet einen Standard-Datei-Öffnen-Dialog, um ein Dokument in einem der unterstützten Formate zur Anzeige auszuwählen, und rendert diese Datei anschließend, wenn vorhanden.

***pathName* reload**
Lädt die aktuell angezeigte Seite neu und zeigt sie erneut an.

***pathName* render *text* ?ext?**
Stellt den angegebenen *text* in der Anzeige dar. Ist eine Erweiterung *ext* angegeben, gilt die Zeichenkette als in dem zugehörigen Format, und der zugehörige Konverter wird benutzt. Sonst gilt die Zeichenkette als HTML oder Markdown. Als HTML gilt *text* nur, wenn er mit einem erkannten HTML-Tag beginnt. Sonst gilt er als Markdown.

***pathName* sourceView**
Dieser Befehl wechselt die Ansicht des Widgets vom aktuellen Dokument selbst zur Quelltextansicht dieses Dokuments. Um zur bearbeitbaren Quelltextansicht zu wechseln, siehe **editView** oben.

```
    ::shtmlview::shtmlview .help
    .help browse index.md
    .help sourceView
```

***pathName* url**
Gibt die aktuell angezeigte URL zurück.

Bindungen für das Widget

Dem Widget sind die folgenden Tasten für die Navigation und andere Aktionen zugewiesen:

```
<cursor>
: Standardmäßige Cursorbewegung in der Ansicht
```

b
Zurück. Zeigt die vorherige Seite im Verlauf.

f
Vorwärts. Zeigt die nächste Seite im Verlauf.

p
Vorheriger Treffer. Bewegt den Cursor zum vorherigen Suchtreffer.

n
Weiter. Bewegt den Cursor zum nächsten Suchtreffer.

q
Entfernt die aktuelle Seite aus dem Verlauf. Das setzt **f** voraus.

r
Suche rückwärts starten

s
Suche vorwärts starten

Ctrl-E
Bearbeitungsmodus umschalten

Ctrl-s
Im Bearbeitungsmodus: Seite speichern

Ctrl-r
Aktuelle Seite neu laden

Ctrl-u
Zwischen Quell- und gerenderter Ansicht umschalten

Return
Folgt dem Link unter dem Cursor.

TAB
Cursor zum nächsten Link auf der aktuellen Seite bewegen

Beispiel

```
package require shtmlview::shtmlview
proc browsed {url} {
    puts "You browsed $url"
}
::shtmlview::shtmlview .help -toolbar true -browsecmd browsed
.help browse index.html
pack .help -fill both -expand true -side left
package require Markdown
.help browser test.md
```

Weitere Beispiele stehen in den Quelldateien des Pakets.

Erweiterungen

Zwar unterstützt das Paket standardmäßig nur HTML-Dokumente sowie Markdown-Dokumente, wenn das entsprechende Paket "**Markdown**" verfügbar ist, doch lässt sich der Bereich der unterstützten Formate über einen Erweiterungsmechanismus vergrößern.

Der wichtigste Einstiegspunkt in dieses System ist der Befehl **shtmlview::converter**. Mit ihm ist es möglich, ein Dokumentformat und einen zugehörigen Konvertierungsbefehl zu registrieren. Das Format wird anhand seiner Dateierweiterung identifiziert, wie beispielsweise "*.md*", "*.man*" usw. Der Konvertierungsbefehl soll den Inhalt der ihm übergebenen Datei zur Anzeige in HTML konvertieren.

Die Pakete **shtmlview::doctools** und **shtmlview::mkdoc** sind Beispiele für solche Erweiterungen. Das erste unterstützt das Format *doctools*, mit dem *Tcllib* und *Tklib* ihre Handbuchseiten schreiben. Das zweite unterstützt Tcl-Quelldateien mit *mkdoc*, also Tcl-Dateien mit eingebetteter Dokumentation in der Syntax *mkdoc*.

Nachstehend der reine Tcl-Code des Pakets **shtmlview::mkdoc**:

```
package require shtmlview::shtmlview
package require mkdoc::mkdoc

::shtmlview::converter .tcl {Tcl+mkdoc files}   ::shtmlview::mkdoc
::shtmlview::converter .tm  {Tcl+mkdoc modules} ::shtmlview::mkdoc

proc ::shtmlview::mkdoc {url} {
close [file tempfile htmltemp .html]

mkdoc::mkdoc $url $htmltemp -html

if {[catch {
    open $htmltemp r
} result]} {
```

```
        return -code error "Cannot open $url: $result"
    }

    set html [read $result]
    close $result
    file delete $htmltemp

    return $html
    }

    package provide shtmlview::mkdoc 0.1
```

Erweiterungen können statt eines Tcl-Pakets eine externe Anwendung wie **pandoc** benutzen, die das darzustellende HTML erzeugt.

Und natürlich ist es mit diesem Paket auch möglich, Konvertierungsbefehle direkt aus der Anwendung heraus zu registrieren, anstatt über ein separates Paket zu gehen.

Änderungsprotokoll

2022-02-25 version 0.9.2

- Korrektur zu **tk_textCopy** und Aktualisierung der Dokumentation

2022-03-06 version 0.9.3

- Unterstützung für Mausrad-Bindungen
- Korrektur von Hyperlinks auf http(s)-Links
- Unterstützung für Datei-Anker-Links wie **file.html#anchor**
- Unterstützung für **#** als Link zum Seitenanfang
- Vielen Dank an aplsimple für die Vorschläge und Fehlermeldungen

2022-03-26 version 1.0.0

- HTML 3.2-Tags: div, sub, sup, small, big
- Erste Unterstützung für Markdown-Dateien
- Erste Unterstützung für Base64-kodierte Inline-Bilddateien
- Unterstützung für JPEG-Bilder, wenn das Paket "**img::jpeg**" verfügbar ist
- Unterstützung für SVG-Bilder, wenn entweder **critcl** und *librsvg-dev(el)* oder die Terminal-Anwendung **rsvg-convert** bzw. **cairosvg** verfügbar sind
- Zurück und Vorwärts auch für Anker
- Schaltflächen für den ersten und letzten Eintrag des Verlaufs
- Verlauf mit vollständigem Dateipfad, damit sich das Verzeichnis wechseln lässt
- Verbesserte Aufrufzeile und Installationsoption
- Bindungen für die nächste und die vorherige Suche
- Return und Tab für Links
- Option -historycombo
- Korrektur der Symbolleiste
- Korrektur von browse für nicht vorhandene Dateien
- Nicht verwendeter CSS-/Stylesheet- und WebForms-Code wurde entfernt
- Vielen Dank an pepdiz für die Fehlermeldungen und Vorschläge

2022-04-XX version 1.1.0

- Tastenbindung Ctrl-u für die Quellansicht von HTML- und Markdown-Dateien

- Tastenbindung Ctrl-Shift-e für einen einfachen Texteditor zur Dateibearbeitung
- Der Dialog zum Öffnen von Dateien speichert nun das zuletzt verwendete Verzeichnis und die zuletzt verwendete Erweiterung.
- Es wurde eine Schaltfläche zum Schließen der Datei mit der Option -closebutton hinzugefügt, die für Toplevel-Fenster nützlich ist
- Einige Korrekturen bezüglich des Backslashs bei Markdown-Links
- Hinzufügen eines "span"-Tags zur Formatierung
- Hinzufügen einer Render-Methode zum direkten Einlesen von HTML ohne Dateinamen
- Erweiterungsstruktur für weitere Dateitypen wie die Doctools von Tcllib oder mkdoc
- Die Unterstützung für Tcl doctools ist in einem eigenen Paket enthalten
- Die Unterstützung für Tcl-mkdoc liegt in einem eigenen Paket.

Noch zu tun

- Markdown-Rendering mithilfe des tcllib-Pakets **Markdown**, falls eine URL mit "*.md*" endet (erledigt)
- Unterstützung für SVG-Bilder, beispielsweise mithilfe von svgconvert, zumindest unter Linux/Windows
- Weitere Tags, siehe die Geschichte der HTML-Tags, HTML-3.2-Tags ergänzen: http://www.martinrinehart.com/frontend-engineering/engineers/html/html-tag-history.html (erledigt)
- Quellansicht mit Ctrl-u (erledigt)
- Bearbeitungsansicht mit Ctrl-Shift-e
- Behebung von Problemen mit dem Mausrad

Dank

Stephen Uhler, Clif Flynt und Robert Heller haben den Großteil des Codes für dieses Widget beigesteuert.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese dem Autor des Pakets. Bitte teilen Sie ihm auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation mit.

Copyright am Code

Art der BSD-Lizenz:

Sun Microsystems, Inc. Die folgenden Bedingungen gelten für alle Dateien, die mit der Software in Zusammenhang stehen, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren, vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten,

deutlich angegeben sind.

In keinem Fall haften die Autoren oder Vertreiber gegenüber Dritten für direkte, indirekte, besondere, zufällige oder Folgeschäden, die sich aus der Nutzung dieser Software, ihrer Dokumentation oder daraus abgeleiteter Werke ergeben, selbst wenn die Autoren auf die Möglichkeit solcher Schäden hingewiesen wurden.

Die Autoren und Vertreiber schließen ausdrücklich jegliche Gewährleistung aus, einschließlich, aber nicht beschränkt auf die stillschweigenden Gewährleistungen der Marktgängigkeit, der Eignung für einen bestimmten Zweck und der Nichtverletzung von Rechten Dritter. Diese Software wird "wie besehen" bereitgestellt, und die Autoren und Vertreiber sind nicht verpflichtet, Wartung, Support, Updates, Erweiterungen oder Modifikationen bereitzustellen.

EINGESCHRÄNKTE RECHTE: Die Nutzung, Vervielfältigung oder Offenlegung durch die Regierung unterliegt den Einschränkungen, die in Unterabsatz (c) (1) (ii) der Klausel über Rechte an technischen Daten und Computersoftware gemäß DFARS 252.227-7013 und FAR 52.227-19 festgelegt sind.

Copyright

```
Copyright (c) 2018-2022, Detlef Groth <detlef(at)dgroth(dot)de>
Copyright (c) 2009, Robert Heller
Copyright (c) 2000, Clif Flynt
Copyright (c) 1995-1999, Stephen Uhler
```

swaplist

swaplist

Stichwörter: swaplist, dialog, disjointlistbox, listbox

Name

swaplist – Ein Dialog, mit dem sich Einträge zwischen zwei Listen verschieben lassen

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require swaplist ?**0.1**?
::swaplist::swaplist *pathName variable completeList selectedList* ?*options*?

Beschreibung

Dieses Paket stellt einen Dialog aus zwei Listboxen bereit, mit Schaltflächen, die Einträge zwischen ihnen verschieben und die rechte Liste umordnen.

::swaplist::swaplist *pathName variable completeList selectedList* ?options?

Erzeugt einen Dialog mit einem Paar Listboxen. Einträge werden ausgewählt, indem die Schaltflächen sie in die rechte Liste verschieben. Beim Schließen des Dialogs wird der Inhalt der rechten Liste in die *variable* geschrieben. Der Befehl gibt einen booleschen Wert zurück, der angibt, ob der Benutzer OK gedrückt hat. Ist -geometry nicht angegeben, wird der Dialog im übergeordneten Toplevel zentriert. Ist das übergeordnete Fenster., wird der Dialog auf dem Bildschirm zentriert.

Optionen:

-embed
Ist dieses Flag angegeben, erzeugt die Prozedur ein Swaplist-Widget namens *pathName* und setzt *variable* als listvariable der rechten Listbox. Dieses Flag bewirkt außerdem, dass -title und -geometry ignoriert werden.

-reorder
Boolescher Wert. Legt fest, ob Schaltflächen erscheinen, mit denen sich die Reihenfolge der rechten Listbox ändern lässt. Voreingestellt ist true.

-title
Setzt den Titel des Dialogfensters. Voreingestellt ist "Configuration".

-llabel
Setzt die Überschrift über der linken Liste. Voreingestellt ist "Available:".

-rlabel
Setzt die Überschrift über der rechten Liste. Voreingestellt ist "Selected:".

-lbuttontext
Setzt den Text der Schaltfläche "nach links". Voreingestellt ist "<<".

-rbuttontext
Setzt den Text der Schaltfläche "nach rechts". Voreingestellt ist ">>".

-ubuttontext
Setzt den Text der Schaltfläche "nach oben". Voreingestellt ist "Move Up".

-dbuttontext
Setzt den Text der Schaltfläche "nach unten". Voreingestellt ist "Move Down".

-geometry
Setzt die Geometrie des Dialogfensters.

Beispiel

```
package require swaplist
namespace import swaplist::*

if {[swaplist .slist opts "1 2 3 4 5 6 7 8 9" "1 3 5"]} {
    puts "user chose numbers: $opts"
}
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *swaplist* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

tkpiechart

canvaslabel

Stichwörter: canvaslabel, canvas, labeler, pie, slice

Name

canvasLabel – tkpiechart-Klasse für Canvas-Beschriftungen

Übersicht

package require stooop 4.1
package require switched 2.2
package require tkpiechart 6.6
stooop::new canvasLabel *canvas* ?*options*?
switched::configure *canvasLabelObject* ?*options*?
switched::cget *canvasLabelObject option*
stooop::delete *canvasLabelObject*

Beschreibung

Die Klasse canvasLabel gibt dem Text-Element des Canvas einige Fähigkeiten des Tk-Label-Widgets, etwa einen Hintergrund und einen Rand.

Ein canvasLabel besteht aus einem Bullet-Rechteck links vom Text. Das Relief folgt dem Auswahlzustand, bei Auswahl das übliche Relief sunken.

Die Beschriftung hat ein eigenes Tag. Damit lassen sich über die Canvas-Funktionen die Koordinaten des Objekts holen oder es verschieben.

stooop::new canvasLabel *canvas* ?options?
Erstellt ein "canvasLabel"-Objekt in der angegebenen Tk-Canvas. Die Objekt-ID des "canvasLabel"-Objekts wird zurückgegeben (in diesem Dokument als *canvasLabelObject* bezeichnet).

switched::configure *canvasLabelObject* ?options?
Konfiguriert ein "canvasLabel"-Objekt oder gibt alle Optionen mit ihren aktuellen Werten zurück, wenn keine Optionen als Parameter übergeben werden.

switched::cget *canvasLabelObject option*
Gibt einen Wert für das angegebene "canvasLabel"-Objekt zurück.

stooop::delete *canvasLabelObject*
Löscht das angegebene "canvasLabel"-Objekt.

Optionen

-anchor value
Ankerposition von Rechteck und Text, relativ zum Positionierungspunkt. Das Verhalten entspricht der Option **-anchor** des *text*-Elements von **canvas**, nur dass das Rechteck mitzählt. Voreingestellt ist *center*.

-background color
Hintergrundfarbe des Bullet-Rechtecks, wie die Option **-fill** des *rectangle*-Elements von **canvas**. Voreingestellt ist durchsichtig, die leere Zeichenkette.

-bordercolor color
Randfarbe des Rechtecks, wie die Option **-outline** des *rectangle*-Elements von **canvas**. Voreingestellt ist schwarz.

-borderwidth value
Randbreite des Rechtecks, wie die Option **-width** des *rectangle*-Elements von **canvas**. Voreingestellt ist die Breite 1 Pixel, das ist die kleinste Breite.

-bulletwidth value
Breite des Rechtecks links vom Text. Voreingestellt ist *10*.

-font value
Schrift des Textes, wie die Option **-font** des *text*-Elements von **canvas**. Voreingestellt ist die systemabhängige Schrift.

-foreground color
Farbe des Textes, wie die Option **-fill** des *text*-Elements von **canvas**. Voreingestellt ist schwarz.

-justify value
Ausrichtung des Textes, wie die Option **-justify** des *text*-Elements von **canvas**. Voreingestellt ist *left*.

-minimumwidth value
Die Gesamtbreite der Beschriftung unterschreitet den angegebenen Wert nicht, kann aber größer sein, wenn der Text es verlangt.

-padding value
Abstand zwischen dem Text und der nächsten Rechteckkante. Die Einheiten sind dieselben wie im Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

-scale list
Liste von 2 Gleitkommazahlen, der Skalierungsfaktor auf der x- und der y-Achse. Die Skalierung gilt sofort. Voreingestellt ist 1.

-select boolean
Setzt den Zustand der Beschriftung.

-selectrelief value
Entweder *flat*, *raised* oder *sunken*. Der gewünschte 3D-Effekt des Textbereichs, wenn die Beschriftung ausgewählt ist.

-stipple bitmap
Punktmuster, das das Rechteck füllt, wie die Option **-stipple** des *rectangle*-Elements von **canvas**. Voreingestellt gibt es keine Bitmap.

-text text
Die Zeichenkette im Textbereich, wie die Option **-text** des *text*-Elements von **canvas**. Voreingestellt ist die leere Zeichenkette.

-textbackground color
Farbe des Hintergrunds des Textbereichs.

-width value
Maximale Zeilenlänge des Textes, wie die Option **-width** des *text*-Elements von **canvas**. Voreingestellt ist *0*.

Tags

Der Labeler hat das folgende besondere Tag. Einzelheiten stehen auf der Handbuchseite **canvas** im Abschnitt *ITEM-IDS UND TAGS*:

- canvasLabel(canvasLabelObject)

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere

Probleme. Bitte melden Sie diese in der Kategorie *tkpiechart* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


pie

Stichwörter: pie, canvas, labeler, slice

Name

pie – 2D- oder 3D-Kreisdiagramm-Objekt in einem Canvas

Übersicht

package require stooop 4.1
package require switched 2.2
package require tkpiechart 6.6
stooop::new pie *canvas x y ?options?*
switched::configure *pieObject ?options?*
switched::cget *pieObject option*
stooop::delete *pieObject*

pie::newSlice *pieObject* ?labelText?

pie::deleteSlice *pieObject sliceObject*

pie::sizeSlice *pieObject sliceObject unitShare* ?displayedValue?

pie::labelSlice *pieObject sliceObject string*

pie::selectedSlices *pieObject*

Beschreibung

Ein pie-Objekt stellt eine Menge von Werten dar, gewöhnlich als Anteile an einer Summe. Jeder Wert ist ein farbiges Segment, mit zwei- oder dreidimensionalem Aussehen. Zu jedem Segment gehört eine Beschriftung mit dem Datennamen und ein Zahlenfeld mit dem prozentualen Anteil des Segments. Die Beschriftungen setzt das gewählte Labeler-Objekt, Option **-labeler**. Die Farbe jeder Beschriftung entspricht dem zugehörigen Segment.

Ein Kreisdiagramm besteht aus Tk-Canvas-Elementen in den Objekten **pieBoxLabeler**, **piePeripheralLabeler** und **canvasLabel**, die das pie-Objekt bilden. Der Konstruktor erzeugt das Kreisdiagramm selbst und sein Hintergrundsegment im übergeordneten Canvas. Sobald das pie-Objekt existiert, lassen sich Segmente erzeugen und in der Größe ändern. Beim Erzeugen muss das übergeordnete Tk-Widget **canvas** schon existieren.

Die Segmentfarben werden automatisch erzeugt, mit einer voreingestellten Farbliste für alle Kreisdiagramme, sofern nicht eine andere Liste gilt, Option **-colors**. Bei dreidimensionalem Aussehen ist die Kante eines Segments dunkler als die Oberseite, im selben Farbton.

stooop::new pie *canvas x y* ?options?
: Erzeugt ein pie-Objekt im angegebenen Tk-Canvas. Die linke obere Ecke des Kreisdiagramms liegt an den angegebenen Koordinaten im Canvas. Zurückgegeben wird der Bezeichner des pie-Objekts, in diesem Dokument *pieObject*.

switched::configure *pieObject* ?options?
Konfiguriert ein "pie"-Objekt oder gibt alle Optionen mit ihren aktuellen Werten zurück, wenn keine Optionen als Parameter übergeben werden.

switched::cget *pieObject option*
Gibt einen Wert für die Option für das angegebene Pie-Objekt zurück.

stooop::delete *pieObject*
Löscht das angegebene Kreisdiagramm-Objekt.

Überblick

Die Klasse "pie" ist Teil der Erweiterung "tkpiechart", die es dem Programmierer ermöglicht, 2D- oder 3D-Kreisdiagramme in einer Tcl/Tk-Anwendung zu erstellen und dynamisch zu aktualisieren. Das Paket "tkpiechart" ist ausschließlich in Tcl geschrieben und nutzt dank des in "tcllib" enthaltenen Pakets "stooop" objektorientierte Techniken.

Optionen

-autoupdate boolean
Boolescher Wert. Legt fest, ob alle Segmente und ihre Beschriftungen neu gezeichnet werden, wenn sich eine Segmentgröße ändert. Voreingestellt eingeschaltet. Ausschalten und **pie::update** aufrufen, wenn viele Segmente auf einmal geändert werden und das schneller gehen soll.

-background color
Die Segmente füllen das Kreisdiagramm zu 100 % oder auch nicht. Der unbelegte Teil ist ein Segment, das 100 % des Kreisdiagramms einnimmt. Voreingestellt ist es durchsichtig mit einem schwarzen Rand. Die Farbe dieses Hintergrundsegments lässt sich mit Farbnamen setzen wie bei der Standardoption **-background**. Einzelheiten stehen auf der Tk-Handbuchseite **options**. Hat das Kreisdiagramm ein 3D-Aussehen, ist der Hintergrund einer Segmentkante dunkler als die Oberseite und nutzt denselben Farbton.

-colors list
Liste von Farben für die Segmente. Die Segmentfarben werden der Reihe nach aus der Liste genommen und wieder von vorn, wenn es mehr Segmente als Farben gibt. Das Format ist dasselbe wie bei **-background**.

-height value
Gesamthöhe des Kreisdiagramms, einschließlich des Platzes für die Beschriftungen des Labelers. Die Segmente werden in der Größe angepasst, wenn Beschriftungen hinzukommen oder wegfallen, also beim Erzeugen oder Löschen von Segmenten, sodass die Gesamthöhe gleich bleibt. Der Wert hat eine der Formen aus dem Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

-labeler object
Ein Platzierungsobjekt für die Segmentbeschriftungen, sodass zum Beispiel die Segmentwerte daneben stehen können. Fehlt die Angabe, wird *pieBoxLabeler* verwendet, siehe die zugehörige Handbuchseite. Die andere Möglichkeit ist die Klasse *piePeripheralLabeler*. Jeder Labeler hat ein eigenes Verhalten, das seine Optionen setzen. Das Labeler-Objekt wird mit dem Kreisdiagramm-Objekt gelöscht. Nach dem Erzeugen des Kreisdiagramms lässt sich der Labeler nicht mehr wechseln.

-selectable boolean

Boolescher Wert. Legt fest, ob Segmente auswählbar sind. Zulässig sind die booleschen Werte von Tcl. Auswählbare Segmente lassen sich mit der ersten Maustaste wählen, durch einen Klick auf das Segment oder seine Beschriftung. Die Auswahl lässt sich mit den üblichen Klicks *control* oder *shift* erweitern. Die Liste der gerade ausgewählten Segmente liefert jederzeit die Elementprozedur **selectedSlices** der Klasse pie.

-title text
Titeltext über dem Kreisdiagramm.

-titlefont value
Schrift des Titeltextes.

-titleoffset value
Abstand zwischen der Unterkante des Titeltextes und der Oberkante der Segmente. Der Wert hat eine der Formen aus dem Abschnitt **GRÖßEN** unten.

-thickness value
Die Dicke ist voreingestellt 0. Das Kreisdiagramm ist dann eine einfache 2D-Form und lässt sich deutlich schneller zeichnen. Ein positiver Wert gibt ihm ein 3D-Aussehen mit dazu passenden dunkleren Farben an den Segmentkanten. Die Werte haben eine der Formen aus dem Abschnitt *GRÖßEN* unten.

-width value
Gesamtbreite des Kreisdiagramms, einschließlich des Platzes für die Beschriftungen des Labelers. Die Segmente werden in der Größe angepasst, wenn Beschriftungen hinzukommen oder wegfallen, also beim Erzeugen oder Löschen von Segmenten, sodass die Gesamtbreite gleich bleibt. Der Wert hat eine der Formen aus dem Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

Prozeduren für Mitglieder

pie::newSlice *pieObject* ?labelText?
Erzeugt ein Segment. Zurückgegeben wird ein eindeutiger Objektbezeichner, in diesem Dokument *sliceObject*. Die Segmentfarbe wird automatisch vergeben, die Beschriftung setzt der angegebene Labeler, Option **-labeler**. Das Segment selbst liegt im Uhrzeigersinn hinter den vorhandenen Segmenten. Der Objektbezeichner dient zum Setzen und Ändern der Größe.

Fehlt der Beschriftungstext, wird er "*slice n*", wobei *n* die Nummer des Segments in der Erzeugungsreihenfolge ist. Das erste Segment hat die Nummer 1.

pie::deleteSlice *pieObject sliceObject*
Löscht ein Segment. Die folgenden Segmente im Uhrzeigersinn rücken nach, sodass die Lücke verschwindet.

pie::sizeSlice *pieObject sliceObject unitShare* ?displayedValue?
Setzt oder ändert die Größe eines Segments. Das Segment wird neu berechnet, sodass es den passenden Anteil am ganzen Kreisdiagramm einnimmt. *unitShare* ist eine Gleitkommazahl als Anteil am ganzen Kreisdiagramm, zwischen 0 und 1. Die folgenden Segmente im Uhrzeigersinn rücken auf die neue Größe. Der Größenwert neben der Segmentbeschriftung wird der neue Anteil oder, wenn angegeben, *displayedValue*.

pie::labelSlice *pieObject sliceObject string*
Aktualisiert eine Segmentbeschriftung. Der Aufruf ist jederzeit möglich.

pie::selectedSlices *pieObject*

Gibt die Liste der gerade ausgewählten Segmentobjekte zurück.

Tags
Das gesamte Kreisdiagramm, die Grafiken des Kreisdiagramms (alle Segmente) sowie jedes einzelne Segment sind mit den folgenden spezifischen Tags versehen:

- **pie(pieObject)**
- **pieSlices(pieObject)**
- **slice(sliceObject)**

Zum Beispiel lässt sich das ganze Kreisdiagramm mit dem Befehl **move** von **canvas** am pie-Tag verschieben, und Bindungen an Segmenten lassen sich über die Segment-Tags setzen. Einzelheiten stehen auf der Handbuchseite **canvas** im Abschnitt *ITEM-IDS UND TAGS*.

Größen
Alle Größen zu Kreisdiagrammen sind Gleitkommazahlen. Koordinaten und Größen sind Bildschirmeinheiten, Gleitkommazahlen, optional gefolgt von einem der Buchstaben aus dem Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

Einschränkungen
Ist die Zahl der Segmente zu groß, bekommen einige Segmente dieselbe Farbe. Dann lassen sich eigene Farben setzen.

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tkpiechart* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


pieboxlabeler
Stichwörter: pieboxlabeler, canvas, labeler, pie, slice

Name
pieBoxLabeler – Labeler-Klasse von tkpiechart im Kastenstil

Übersicht
stooop::new pieBoxLabeler *canvas* ?*options*?
switched::configure *pieBoxLabelerObject* ?*options*?
switched::cget *pieBoxLabelerObject option*

Beschreibung
Das Labeler-Objekt im Kastenstil setzt die Segmentbeschriftungen eines Objekts **pie** und wird dem Konstruktor von pie über die Option **-labeler** übergeben. Siehe die Handbuchseite der Klasse **pie**.

Die Beschriftungen stehen in 2 Spalten unter der Kreisdiagrammgrafik. Jeder Beschriftungstext steht rechts von einem Rechteck, dessen Hintergrundfarbe zum zugehörigen Segment passt.

Der Anteilswert des Segments steht rechts vom Beschriftungstext, getrennt durch ein Semikolon. Jede Beschriftung ist ein canvasLabel-Objekt. Weitere Angaben stehen auf der Handbuchseite der Klasse **canvasLabel**.

Ein Objekt **pieBoxLabeler** muss nicht gelöscht werden. Die Klasse pie erledigt das.

stooop::new pieBoxLabeler *canvas* ?options?
Erzeugt ein pieBoxLabeler-Objekt im angegebenen Tk-Canvas. Zurückgegeben wird der Objektbezeichner, in diesem Dokument *pieBoxLabelerObject*.

switched::configure *pieBoxLabelerObject* ?options?
Konfiguriert ein pieBoxLabeler-Objekt oder gibt alle Optionen mit ihren aktuellen Werten zurück, wenn keine Optionen übergeben werden.

switched::cget *pieBoxLabelerObject option*
Gibt einen Optionswert des angegebenen pieBoxLabeler-Objekts zurück.

Optionen

-font value
Eine Schrift für die Segmentbeschriftungen. Fehlt die Angabe, ist die Schrift systemabhängig.

-justify value
Ausrichtung der Beschriftungen in der eigenen Spalte. Einer von *left*, *center* oder *right*. Voreingestellt ist *left*. Ist die Ausrichtung *right*, fluchten die rechten Kanten aller Spaltenbeschriftungen.

-offset value
Abstand zwischen der Kreisdiagrammgrafik und der nächsten Segmentbeschriftung. Der Wert hat eine der Formen aus dem Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

Tags

Der Labeler hat das folgende besondere Tag. Einzelheiten stehen auf der Handbuchseite **canvas** im Abschnitt *ITEM-IDS UND TAGS*:

- pieBoxLabeler(pieBoxLabelerObject)

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tkpiechart* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```

pieperipherallabeler

Stichwörter: pieperipherallabeler, canvas, labeler, pie, slice

Name

piePeripheralLabeler – Labeler-Klasse von tkpiechart mit Beschriftung am Rand

Übersicht

stooop::new piePeripheralLabeler *canvas* ?*options*?

switched::configure *piePeripheralLabelerObject* ?*options*?
switched::cget *piePeripheralLabelerObject option*

Beschreibung

Das Labeler-Objekt mit Beschriftung am Rand setzt die Segmentbeschriftungen eines Objekts **pie** und wird dem Konstruktor von pie über die Option **-labeler** übergeben. Siehe die Handbuchseite der Klasse **pie**.

Die Beschreibungstexte der Segmente stehen in 2 Spalten unter der Kreisdiagrammgrafik. Die Segmentwerte stehen neben dem Segment und folgen ihm, wenn das Kreisdiagramm aktualisiert wird. Jeder Beschreibungstext steht rechts von einem Rechteck, dessen Hintergrundfarbe zum zugehörigen Segment passt. Jede Beschreibungsbeschriftung ist ein canvasLabel-Objekt.

Ein Objekt **piePeripheralLabeler** muss nicht gelöscht werden. Die Klasse pie erledigt das.

stooop::new piePeripheralLabeler *canvas* ?options?
: Erzeugt ein piePeripheralLabeler-Objekt im angegebenen Tk-Canvas. Zurückgegeben wird der Objektbezeichner, in diesem Dokument *piePeripheralLabelerObject*.

switched::configure *piePeripheralLabelerObject* ?options?
: Konfiguriert ein piePeripheralLabeler-Objekt oder gibt alle Optionen mit ihren aktuellen Werten zurück, wenn keine Optionen übergeben werden.

switched::cget *piePeripheralLabelerObject option*
: Gibt einen Optionswert des angegebenen piePeripheralLabeler-Objekts zurück.

Optionen

-font value
: Eine Schrift für die Segmentbeschriftungen. Fehlt die Angabe, ist die Schrift systemabhängig.

-justify value
: Ausrichtung der Beschriftungen in der eigenen Spalte. Einer von *left*, *center* oder *right*. Voreingestellt ist *left*. Ist die Ausrichtung *right*, fluchten die rechten Kanten aller Spaltenbeschriftungen.

-offset value
: Abstand zwischen der Kreisdiagrammgrafik und der nächsten Segmentbeschriftung. Der Wert hat eine der Formen aus dem Abschnitt *KOORDINATEN* der Handbuchseite **canvas**.

-smallfont
: Eine Schrift für die Segmentwerte. Gewöhnlich eine kleine Schrift, damit sich die Werte nicht überdecken, wenn zwei Segmente sehr nah beieinander liegen. Fehlt die Angabe, gilt die Schrift der Beschreibungsbeschriftung, Option **-font**.

-widestvaluetext
: Eine Zeichenkette maximaler Breite für die Segmentwerte, die um das Kreisdiagramm neben den Segmenten stehen, damit beim Setzen von Breite und Höhe genug Platz für diese Wertbeschriftungen bleibt. Voreingestellt ist 00.0. Zum Beispiel kann der Wert "00.00 %" sein.

Tags

Der Labeler hat das folgende besondere Tag. Einzelheiten stehen auf der Handbuchseite **canvas** im Abschnitt *ITEM-IDS UND TAGS*:

- piePeripheralLabeler(piePeripheralLabelerObject)

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tkpiechart* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


tooltip

tipstack

Stichwörter: tipstack, balloon, help, hover

Name

tipstack – Gestapelte Tooltips

Übersicht

package require Tcl 8.5
package require msgcat 1.3
package require tooltip ?**1.7**?
package require tipstack ?**1.0.1**?
::tipstack::push *widget* ?*-index index*? *text*
::tipstack::pop *widget* ?*-index index*?
::tipstack::clear *widget* ?*-index index*?
::tipstack::def *widget text ...*
::tipstack::defsub *base widget text ...*
::tipstack::clearsub *base*

Beschreibung

Dieses Paket erweitert die Funktionalität des Pakets **tooltip**. Es stellt einen dynamischen Stapel von Hinweistexten pro Widget bereit. Dies ermöglicht dynamische, vorübergehende Änderungen an den Hinweisen, beispielsweise um eine Standarderklärung zu einem Feld vorübergehend durch eine Fehlermeldung zu ersetzen.

Befehle

::tipstack::push *widget* ?-index *index*? *text*

Legt einen neuen *text* auf den Tooltip des *widget*. Bei einem Listen-Widget gilt

```
-index ...
```

um den betreffenden Eintrag anzusprechen, für den der

Tooltip geändert werden soll.

Das Ergebnis des Befehls ist die leere Zeichenfolge

::tipstack::pop *widget* ?-index *index*?

Nimmt den aktuellen Tooltip des *widget* vom Stapel und stellt den vorherigen Text wieder her. Das ist eine leere Operation, wenn der Stapel dadurch leer würde. Der Basistext des Tooltips kann nicht entnommen werden. Bei einem Listen-Widget gilt

```
-index ...
```

um den betreffenden Eintrag anzusprechen, für den der
Tooltip geändert werden soll.

Das Ergebnis des Befehls ist die leere Zeichenfolge

::tipstack::clear *widget* ?-index *index*?
Leert den Stapel des *widget* und stellt den Basiszustand wieder her. Bei einem Listen-Widget gilt

```
-index ...
```

um den betreffenden Eintrag anzusprechen, für den der
Tooltip geändert werden soll.

Das Ergebnis des Befehls ist die leere Zeichenfolge

::tipstack::def *widget text* ...
Legt in einem Aufruf Texte auf mehrere unabhängige *widget*s. Für Listen-Widgets geht das nicht, weil die nötige Indexangabe fehlt. Auch mit Menüs geht es nicht.

Das Ergebnis ist die leere Zeichenkette.

::tipstack::defsub *base widget text* ...
Variante von **::tipstack::def**, bei der alle Widgets, auf die ein Text gelegt wird, Unterwidgets von *base* sind. Das passt zu Megawidgets.

Entweder beginnt jedes *widget* mit einem Punkt (**.**), oder *base* endet mit einem Punkt.

Das Ergebnis ist die leere Zeichenkette.

::tipstack::clearsub *base*
Variante von **::tipstack::clear**. Sie leert alle Kindwidgets von *base*, auf die Text gelegt wurde. Mit Menüs geht das nicht.

Das Ergebnis ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tooltip* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright

```
Copyright (c) 2003 ActiveState Corp
```

tooltip

Stichwörter: tooltip, balloon, help, hover

Name
tooltip – Verwaltung von Tooltips

Übersicht
package require Tcl 8.5
package require tooltip ?**2.0.4**?
::tooltip::tooltip *command* ?*options*?
::tooltip::tooltip *pathName* ?*option value...*? ?**--**? *message*

Beschreibung
Dieses Paket bietet Tooltips, d. h. kurze Textmeldungen, die angezeigt werden können, wenn der Mauszeiger über ein Widget, einen Menüpunkt, ein Canvas-Element, einen Listbox-Eintrag, einen Eintrag oder eine Spaltenüberschrift von ttk::treeview, ein Register von ttk::notebook oder ein Text-Widget-Tag bewegt wird.

Befehle
::tooltip::tooltip *command* ?*options*?
Verwaltet das Paket tooltip über die folgenden Unterbefehle.

clear ?*pattern*?
Verhindert, dass die festgelegten Widgets Tooltips anzeigen. *pattern* ist ein Glob-Muster und passt standardmäßig auf alle Widgets.

configure ?*option* ?*value option value ...*??
Fragt die Konfigurationsoptionen des Tooltips ab oder ändert sie. Unterstützt werden **-backgroud**, **-foreground** und **-font**. Ohne *option* gibt der Befehl ein Dictionary der Optionswerte zurück. Mit einer *option* ohne Wert gibt er den Wert dieser Option zurück. Sonst setzt er die angegebenen *option*s auf die zugehörigen *value*s.

delay ?*millisecs*?
Fragt die Hover-Verzögerung ab oder setzt sie. Das ist die Zeit, die der Zeiger über dem Widget bleiben muss, bevor der Tooltip erscheint. Die Verzögerung wird in Millisekunden angegeben und muss mindestens 50 ms betragen. Ohne Argument wird die aktuelle Verzögerung zurückgegeben.

fade ?*boolean*?
Schaltet das Ausblenden des Tooltips ein oder aus. Unter Win32 und Aqua ist das Ausblenden vorgegeben. Bei Leave-Ereignissen blendet der Tooltip aus, anstatt zu verschwinden.

disable
siehe unten ...

off
Alle Tooltips deaktivieren

enable
siehe unten ...

on
Aktiviert Tooltips für definierte Widgets.

::tooltip::tooltip *pathName* ?*option value*...? ?--? *message*

Sorgt dafür, dass das Widget *pathName* einen Tooltip mit *message* anzeigt.

Ist das Widget ein **menu**, **canvas**, **listbox**, **ttk::treeview**, **ttk::notebook** oder **text**, binden zusätzliche Optionen den Tooltip an bestimmte Menü-, Canvas- oder Listbox-Einträge, ttk::treeview-Einträge oder Spaltenüberschriften, ttk::notebook-Register oder Text-Tags.

-heading *columnId*

Setzt einen Tooltip für eine Spaltenüberschrift eines ttk::treeview. Die Spalte muss noch nicht vorhanden sein. Dieselben Bezeichner sollten nicht zugleich für Spalten und Einträge eines Widgets verwendet werden, sonst vermischen sich die Tooltips. Das Widget muss ein ttk::treeview sein.

-image *image*

Das angegebene Bild (photo) erscheint links neben der primären Tooltip-*message*.

-index *index*

Mit dieser Option können Sie einen Tooltip für einen Menüpunkt festlegen. Der Index kann entweder der Eintragsindex oder die Eintragsbezeichnung sein. Das Widget muss ein Menü-Widget sein, die Einträge müssen jedoch zum Zeitpunkt der Festlegung des Tooltips nicht vorhanden sein.

-info *info*

Der angegebene Text *info* erscheint als zusätzliche Angabe unter der primären Tooltip-*message*.

-items *items*

Diese Option dient dazu, einen Tooltip für Elemente im Canvas, in der listbox oder ttk::treview festzulegen. Beim Canvas-Widget muss das Element bereits im Canvas vorhanden sein und wird über eine **find withtag**-Abfrage ermittelt. Bei Listbox- und ttk::treview-Widgets können die Elemente zu einem späteren Zeitpunkt erstellt werden; der Programmierer ist jedoch dafür verantwortlich, die Verknüpfung zwischen dem Index des Listbox- oder ttk::treview-Elements und dem entsprechenden Tooltip zu verwalten. Wenn die Elemente der Listbox oder von ttk::treview neu angeordnet werden, müssen die Tooltips angepasst werden.

Handelt es sich bei dem Widget nicht um ein Canvas-, Listbox- oder ttk::treview-Widget, wird ein Fehler ausgelöst.

-tab *tabId*

Setzt einen Tooltip für ein Register eines ttk::notebook. Das Register muss beim Aufruf schon vorhanden sein, sonst wird ein Fehler zurückgegeben. Das Widget muss ein ttk::notebook sein.

-tag *name*

Setzt einen Tooltip für ein Tag eines Text-Widgets. Das Tag muss beim Aufruf schon vorhanden sein, sonst wird ein Fehler zurückgegeben. Das Widget muss ein Text-Widget sein.

--

Die Option **--** markiert das Ende der Optionen. Das darauf folgende Argument wird als *message* behandelt, selbst wenn es mit **-** beginnt.

Beispiel

```
# Demonstrate widget tooltip
package require tooltip
pack [label .l -text "label"]
tooltip::tooltip .l "This is a label widget"
```

```
# Demonstrate menu tooltip
package require tooltip
. configure -menu [menu .menu]
.menu add cascade -label Test -menu [menu .menu.test -tearoff 0]
.menu.test add command -label Tooltip
tooltip::tooltip .menu.test -index 0 "This is a menu tooltip"
```

```
# Demonstrate canvas item tooltip
package require tooltip
pack [canvas .c]
set item [.c create rectangle 10 10 80 80 -fill red]
tooltip::tooltip .c -item $item "Canvas item tooltip"
```

```
# Demonstrate listbox item tooltip
package require tooltip
pack [listbox .lb]
.lb insert 0 "item one"
tooltip::tooltip .lb -item 0 "Listbox item tooltip"
```

```
# Demonstrate ttk::notebook tab tooltip
package require tooltip
pack [ttk::notebook .nb]
.nb add [frame .nb.f1 -height 50] -text "First tab"
.nb add [frame .nb.f2 -height 50] -text "Second tab"
tooltip::tooltip .nb -tab 0 "Tooltip for the 1st notebook tab"
tooltip::tooltip .nb -tab 1 "Tooltip for the 2nd notebook tab"
```

```
# Demonstrate text tag tooltip
package require tooltip
pack [text .txt]
.txt tag configure TIP-1 -underline 1
tooltip::tooltip .txt -tag TIP-1 "tooltip one text"
.txt insert end "An example of a " {} "tooltip" TIP-1 " tag.\n" {}
```

Migration von Version 1

Die Versionen 1.3 bis 1.7 riefen vor der Anzeige eines Tooltips die Funktion **msgcat::mc** auf und verwendeten dabei den Tooltip-Namespace. **msgcat::mc** benötigt die Umgebung des Aufrufers. Aus diesem Grund wurde in Version 1.8 der Namespace des Aufrufers erfasst und bei dem Aufruf verwendet. In Version 2.0 wurde jegliche Unterstützung für **msgcat::mc** entfernt. Die Optionen **-namespace**, **-msgargs** und **-infoargs** wurden entfernt. Ab TCL 8.7 unterstützt **msgcat::mc** OO-Klassen und OO-Methoden. Allerdings ist die OO-Aufruferumgebung nicht vorhanden, wenn **msgcat::mc** bei der Anzeige von Tooltips aufgerufen wurde, was zu Fehlern in der Laufzeit führte. Es wurde als schlechtes Design bewertet, **msgcat::mc** zu spät aufzurufen. Der Aufrufer sollte die Tooltips bei einer eventuellen Änderung der Meldung neu installieren.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *tooltip* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Copyright


```
Copyright (c) 1996-2008, Jeffrey Hobbs
Copyright (c) 2024 Emmanuel Frecon
```


widget

widget

Stichwörter: widget, bundle, calendar, dateentry, dialog, megawidget, menu, panelframe, ruler, screenruler, scrolledwindow, snit, statusbar, superframe, toolbar

Name

widget – Megawidget-Bündel

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.2**?
package require snit
widget::validate *as* ?*options*?
widget::calendar *pathname* ?*options*?
widget::dateentry *pathname* ?*options*?
widget::dialog *pathname* ?*options*?
widget::menuentry *pathname* ?*options*?
widget::panelframe *pathname* ?*options*?
widget::ruler *pathname* ?*options*?
widget::screenruler *pathname* ?*options*?
widget::scrolledwindow *pathname* ?*options*?
widget::statusbar *pathname* ?*options*?
widget::superframe *pathname* ?*options*?
widget::toolbar *pathname* ?*options*?

Beschreibung

Dieses Paket stellt Megawidgets bereit, die auf dem snit-OO-System (snidgets) basieren. Es nutzt den thematisierten Widget-Satz von Tile/Ttk.

widget::validate *as* ?options?
Befehle:

Widgets

widget::calendar *pathname* ?options?
Optionen:

widget::dateentry *pathname* ?options?

Optionen:

widget::dialog ***pathname*** **?options?**
Optionen:

widget::menuentry ***pathname*** **?options?**
Optionen:

widget::panelframe ***pathname*** **?options?**
Optionen:

widget::ruler ***pathname*** **?options?**
Optionen:

widget::screenruler ***pathname*** **?options?**
Optionen:

widget::scrolledwindow ***pathname*** **?options?**
Optionen:

widget::statusbar ***pathname*** **?options?**
Optionen:

widget::superframe ***pathname*** **?options?**
Optionen:

widget::toolbar ***pathname*** **?options?**
Optionen:

Beispiel

```
package require widget::superframe ; # or widget::all
pack [widget::superframe .f -type separator -text "SuperFrame:"]
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

widget_calendar

Stichwörter: widget_calendar, calendar, date, megawidget, snit, widget

Name

widget_calendar – Kalender-Megawidget

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::calendar ?**1.0.2**?
widget::calendar *pathname* ?*options*?

Beschreibung

Dieses Paket enthält ein Kalender-Megawidget (snidget).

- **widget::calendar** *pathname* ?options?

Widget-Optionen

-command
Ein Skript, das ausgewertet wird, wenn ein Datum ausgewählt wurde.

-dateformat
Das Format des zurückgegebenen Datums. Standard: %m/%d/%Y.

-firstday
Setzt den ersten Wochentag, entweder sunday oder monday. Die Vorgabe ist monday.

-font
Wählt die im Widget verwendete Schriftart. Die Vorgabe ist Helvetica 9.

-highlightcolor
Wählt die Hintergrundfarbe des ausgewählten Tages. Vorgabe: #FFCC00

-language
Legt die Sprache des Kalenderinhalts fest. Die Sprache wird durch Abkürzungen angegeben, zum Beispiel en – englisch, de – deutsch. Die Vorgabe ist en.

Unterstützte Sprachen: de en es fr gr he it ja sv pl pt zh fi tr nl ru crk crx-nak crx-lhe

-shadecolor
Wählt die Farbe der Teile mit schattiertem Hintergrund. Vorgabe: #888888

-showpast
Legt fest, ob die Vergangenheit angezeigt wird. Ein boolescher Wert, Vorgabe 1.

-textvariable
Legt den Namen einer Variablen fest, deren Wert mit dem Inhalt des Entry-Widgets verknüpft ist. Ändert sich der Wert der Variablen, wird der Inhalt des Widgets aktualisiert, und umgekehrt.

Widget-Befehl

pathname **get** ?*what*?

Gibt einen Teil des ausgewählten Datums oder all zurück. Das Argument *what* wählt den Teil. Zulässige Werte für *what* sind day, month, year und all. all ist die Vorgabe und gibt das vollständige Datum in dem mit -dateformat angegebenen Format zurück.

Standard-Bindungen

Beim Erstellen des Kalender-Widgets werden die folgenden Bindungen eingerichtet. Beim Drücken einer Taste wird der Befehl aufgerufen und die Textvariable aktualisiert. aktualisiert.

- Home – Zum aktuellen Datum wechseln
- Up – Zur Woche vor dem aktuellen Datum wechseln
- Down – Zur Woche nach dem aktuellen Datum wechseln
- Left – Zum Tag vor dem aktuellen Datum wechseln
- Right – Zum Tag nach dem aktuellen Datum wechseln
- Control-Left – Zum Monat vor dem aktuellen Datum wechseln

- Control-Right – Zum Monat nach dem aktuellen Datum wechseln
- Control-Up – Zum Jahr vor dem aktuellen Datum wechseln
- Control-Down – Zum Jahr nach dem aktuellen Datum wechseln

Beispiel

```
    package require widget::calendar ; # or widget::all
    set t [widget::calendar .t]
    pack $t -fill x -expand 1
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::calendar* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

widget_dateentry

Stichwörter: widget_dateentry, date, dateentry, megawidget, snit, widget

Name

widget_dateentry – Dateentry-Megawidget

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::dateentry ?**0.99**?
widget::dateentry *pathname* ?*options*?
pathname **get**
pathname **set** *date*

Beschreibung

Dieses Paket stellt ein Dateentry-Megawidget (snidget) bereit. Es basiert auf einem ttk::entry. Alle Widget-Befehle des ttk::entry stehen für das Dateentry zur Verfügung.

- **widget::dateentry** *pathname* ?options?

Widget-Optionen

-command
: Ein Befehlspräfix, das ausgewertet wird, wenn ein Datum ausgewählt wurde. Das Befehlspräfix wird im globalen Namespace ausgeführt und erhält zwei Argumente: das Rohdatum in Sekunden sowie das formatierte Datum gemäß der Option **-dateformat**.

-dateformat
: Das Format des Datums, das eingegeben oder zurückgegeben wird. Standard: %m/%d/%Y.

-firstday
: Siehe die Handbuchseite widget::calendar.

-font
Wählt die im Widget verwendete Schriftart. Die Vorgabe ist Helvetica 9.

-highlightcolor
Siehe die Handbuchseite widget::calendar. Siehe die Handbuchseite calendar.

-language
Siehe die Handbuchseite widget::calendar.

-shadecolor
Siehe die Handbuchseite widget::calendar.

-showpast
Siehe die Handbuchseite widget::calendar.

-textvariable
Legt den Namen einer Variablen fest, deren Wert mit dem Inhalt des Entry-Widgets verknüpft ist. Ändert sich der Wert der Variablen, wird der Inhalt des Widgets aktualisiert, und umgekehrt.

Widget-Befehl

***pathname* get**
Gibt das ausgewählte Datum zurück.

pathname* set *date
Legt programmgesteuert ein neues Datum fest. Es wird davon ausgegangen, dass das Datum dasselbe Format aufweist, wie es durch die Option **-dateformat** konfiguriert wurde.

Der gleiche Effekt lässt sich auch durch die Einstellung einer verknüpften Option **-textvariable** erzielen.

Standard-Bindungen

Beim Erstellen des Widgets dateentry werden die folgenden Bindungen eingerichtet. Zur Navigation im Kalender siehe dessen Handbuchseite.

- Button-1 – Auswahl übernehmen, das Datum auswählen und das Kalenderfenster schließen.
- Return, space – Das ausgewählte Datum übernehmen und das Kalenderfenster schließen
- Escape – Kalenderfenster schließen
- Control-space – Kalenderfenster anzeigen

Beispiel

```
package require widget::dateentry; # or widget::all
set t [widget::dateentry .de]
pack $t -fill x -expand 1
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::dateentry* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

widget_toolbar

Stichwörter: widget_toolbar, megawidget, snit, toolbar, widget

Name

widget_toolbar – Toolbar-Megawidget

Übersicht

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::toolbar ?**1.0**?
widget::toolbar *pathname* ?*options*?

getframe

add ?item? ?args?

delete item1 ?item2? ?...?

itemcget symbol option

itemconfigure symbol ?args?

items ?pattern?

remove ?-destroy? item1 ?item2? ?...?

Beschreibung

Dieses Paket stellt ein Toolbar-Megawidget (snidget) bereit. Es nutzt den thematisierten Widget-Satz von Tile/Ttk.

- **widget::toolbar** *pathname* ?options?
- getframe
- add ?item? ?args?
- delete item1 ?item2? ?...?
- itemcget symbol option
- itemconfigure symbol ?args?
- items ?pattern?
- remove ?-destroy? item1 ?item2? ?...?

Widget-Optionen

- **-ipad**
- **-pad**
- **-separator**

Element-Optionen

- **-pad**
- **-separator**
- **-sticky**
- **-weight**

Beispiel

```
package require widget::toolbar ; # or widget::all
set t [widget::toolbar .t]
pack $t -fill x -expand 1
$t add button [button .b -text foo]
$t add separator -pad {2 4}
$t add button [button .c -text bar]
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::toolbar* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Kategorie

Widget

widgetPlus

widgetPlus

Stichwörter: widgetPlus

Name

widgetPlus – Erweiterte Widgets Entry, Spinbox und Combobox mit Undo/Redo und weiteren Funktionen

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require snit
package require widgetPlus ?**1.0**?
widgetPlus::entryPlus ?*create*? *pathName*
widgetPlus::spinboxPlus ?*create*? *pathName*
widgetPlus::ttkEntryPlus ?*create*? *pathName*
widgetPlus::ttkSpinboxPlus ?*create*? *pathName*
widgetPlus::ttkComboboxPlus ?*create*? *pathName*
widgetPlus::EnableBWidget
pathName **edit** ?*arg arg ...*?
pathName **edit canredo**
pathName **edit canundo**
pathName **edit modified** ?*boolean*?
pathName **edit redo**
pathName **edit reset**
pathName **edit separator**
pathName **edit undo**

Beschreibung

Das Paket "**widgetPlus**" erweitert die Widgets "**entry**", **spinbox**, **ttk::entry**, **ttk::spinbox** und **ttk::combobox**.

Neu sind Undo und Redo, ein Ereignis **<>** und für die nicht thematisierten **Tk**-Widgets die Option **-inactiveselectbackground**, dazu kleinere Korrekturen an den Bindungen. Die neuen Funktionen orientieren sich eng an denen des **text**-Widgets.

Die überarbeiteten Widgets **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox** sowie **ttk::combobox** werden durch die Befehle **widgetPlus::entryPlus**, **widgetPlus::spinboxPlus**, **widgetPlus::ttkEntryPlus**, **widgetPlus::ttkSpinboxPlus** bzw. **widgetPlus::ttkComboboxPlus** erstellt.

Zum Beispiel ist **widgetPlus::entryPlus** ein *Snit*-**snit::widgetadaptor**, der ein **Tk**-Widget **entry** umhüllt. Seine **winfo class** ist **Entry**, und der Bindtag entryPlus ersetzt den Bindtag **Entry**. Die anderen Widgets sind auf ähnliche Weise implementiert.

Das Paket verwendet den Namespace **::widgetPlus**, der (ausschließlich) die fünf unten definierten Befehle exportiert, die mit einem Kleinbuchstaben beginnen (**EnableBWidget** wird nicht exportiert). Diese Befehle können in den globalen Namespace importiert werden; der Einfachheit halber wird in der Beschreibung dieses Handbuchs fortan auf die Angabe des Namespace-Qualifizierers verzichtet.

```
namespace import ::widgetPlus::*
```

Widget-Optionen

Zusätzlich zu den üblichen Optionen der Widgets **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox** und **ttk::combobox** bieten die Wrapper **entryPlus**, **spinboxPlus**, **ttkEntryPlus**, **ttkSpinboxPlus** und **ttkComboboxPlus** die folgenden Optionen.

-inactiveselectbackground (inactiveSelectBackground/Foreground)
: Farbe der Auswahl, wenn das Fenster nicht den Eingabe-**focus** hat. Ist der Wert leer, **{}**, wird ohne **focus** keine Auswahl gezeigt. Die Option **-inactiveselectbackground** wirkt nicht auf die thematisierten Widgets **ttkEntryPlus**, **ttkSpinboxPlus** und **ttkComboboxPlus**.

-maxundo (maxUndo/MaxUndo)
: Maximale Zahl zusammengesetzter Undo-Aktionen auf dem *Undo-Stapel*. Null oder ein negativer Wert bedeuten einen unbegrenzten *Undo-Stapel*.

-undo (undo/Undo)
: Ein *boolean*. Legt fest, ob der Undo-Mechanismus aktiv ist.

Befehle

widgetPlus::entryPlus ?*create*? *pathName*
: Erzeugt ein Widget **entryPlus** oder wandelt ein vorhandenes Widget **entry** in ein **entryPlus** um.

widgetPlus::spinboxPlus ?*create*? *pathName*
: Erzeugt ein Widget **spinboxPlus** oder wandelt ein vorhandenes Widget **spinbox** in ein **spinboxPlus** um.

widgetPlus::ttkEntryPlus ?*create*? *pathName*
: Erzeugt ein Widget **ttkEntryPlus** oder wandelt ein vorhandenes Widget **ttk::entry** in ein **ttkEntryPlus** um.

widgetPlus::ttkSpinboxPlus ?*create*? *pathName*
: Erzeugt ein Widget **ttkSpinboxPlus** oder wandelt ein vorhandenes Widget **ttk::spinbox** in ein **ttkSpinboxPlus** um.

widgetPlus::ttkComboboxPlus ?*create*? *pathName*
Erzeugt ein Widget **ttkComboboxPlus** oder wandelt ein vorhandenes Widget **ttk::combobox** in ein **ttkComboboxPlus** um.

widgetPlus::EnableBWidget
Erlaubt **widgetPlus**, **BWidget**-Widgets **Entry** und ihre Zusammensetzungen zu ändern, etwa die **BWidget**-**ComboBox**. Der Befehl legt den Bindtag **BwEntryPlus** an, der den voreingestellten Bindtag **BwEntry** ersetzt. Das geschieht beim Laden des Pakets nicht, weil **BWidget** vorher geladen sein muss. Der Befehl wird aus dem Namespace **::widgetPlus** nicht exportiert.

Widget-Befehle

Zusätzlich zu den üblichen Befehlen der Widgets **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox** und **ttk::combobox** bieten die Wrapper **entryPlus**, **ttkEntryPlus**, **spinboxPlus**, **ttkSpinboxPlus** und **ttkComboboxPlus**:

***pathName* edit ?*arg arg ...*?**
Dieser Befehl steuert den Undo-Mechanismus und das *Änderungsflag*. Das genaue Verhalten hängt vom Optionsargument nach **edit** ab. Derzeit gibt es diese Formen:

***pathName* edit canredo**
Gibt ein *boolean* **true** zurück, wenn Redo möglich ist, also wenn der *Redo-Stapel* nicht leer ist. Sonst **false**.

***pathName* edit canundo**
Gibt ein *boolean* **true** zurück, wenn Undo möglich ist, also wenn der *Undo-Stapel* nicht leer ist. Sonst **false**.

***pathName* edit modified ?*boolean*?**
Ist *boolean* nicht angegeben, gibt der Befehl das *Änderungsflag* des Widgets zurück. Die Befehle **insert**, **delete**, **edit undo** und **edit redo**, oder über diesen Befehl der Aufrufer, können das *Änderungsflag* setzen oder löschen. Ist *boolean* angegeben, setzt der Befehl das *Änderungsflag* des Widgets auf *boolean*.

***pathName* edit redo**
Ist **-undo true**, wendet der Befehl die zuletzt rückgängig gemachten Änderungen wieder an, sofern seither keine anderen Änderungen erfolgten. Ist der *Redo-Stapel* leer, entsteht ein Fehler. Ist **-undo false**, geschieht nichts.

***pathName* edit reset**
Leert den *Undo-Stapel* und den *Redo-Stapel*.

***pathName* edit separator**
Wird der Vollständigkeit halber aufgenommen, hat jedoch keine Auswirkung.

(Anmerkung: Bei jedem Aufruf eines Widget-Befehls (z. B. **insert** oder **delete**), der den Inhalt des Widgets verändert, wird dem *Undo-Stapel* ein implizites Trennzeichen hinzugefügt. Sie trennen damit das Eintippen oder Löschen einzelner Zeichen, außer dass *cut* und *paste* jeweils ein eigener Schritt sind.)

***pathName* edit undo**
Macht die letzte Änderung rückgängig, wenn **-undo true** ist. Eine Änderung ist ein einzelner Befehl **insert** oder **delete** auf dem *Undo-Stapel*. Ist der *Undo-Stapel* leer, entsteht ein Fehler. Ist **-undo false**, geschieht nichts.

Bindungen

Klassenbindungen

Das Klassen-Bindungs-Tag für verschiedene Widgets weist folgende Standardwerte auf:

- **entry** verwendet **Entry**
- **spinbox** verwendet **Spinbox**
- **ttk::entry** verwendet **TEntry**
- **ttk::spinbox** verwendet **TSpinbox**
- **ttk::combobox** verwendet **TCombobox**
- entryPlus verwendet **EntryPlus**
- spinboxPlus verwendet **SpinboxPlus**
- **ttkEntryPlus** verwendet TEntryPlus
- **ttkSpinboxPlus** verwendet TSpinboxPlus
- **ttkComboboxPlus** verwendet TComboboxPlus

Rückgängig und Wiederherstellen

Die virtuellen Ereignisse "**<>**" und "**<>**" werden diesen Tastaturereignissen zugeordnet:

- Auf X11-Systemen (Linux®, BSD®, Solaris® usw.):
  - **<>** ist zugeordnet
- **<>** ist zugeordnet
- Auf Windows®-Systemen:
  - **<>** ist zugeordnet
- **<>** ist zugeordnet
- Auf Aqua®-Systemen (d. h. Apple® macOS® oder OS X®):
  - **<>** ist zugeordnet
- **<>** ist zugeordnet

entryPlus

Die entryPlus-Bindungen basieren auf denen des Bindtags **Entry**, wobei folgende Änderungen vorgenommen wurden:

- Neue Bindungen zu **<>** und **<>**.
- Eine Anpassung der Bindung **<>**, damit diese sich unter X11 genauso verhält wie unter anderen Fenstersystemen (d. h., sie löscht die vorhandene Auswahl, sofern eine solche vorhanden ist).
- Die Entfernung der veralteten Bindung "".
- Neue Bindungen zu und, um die Implementierung der Option **-inactiveselectbackground** zu erleichtern.

spinboxPlus

Die spinboxPlus-Bindungen basieren auf denen des Bindtags **Spinbox**, wobei dieselben Änderungen vorgenommen wurden wie bei der Konvertierung von **Entry** in entryPlus.

TEntryPlus

Die TEntryPlus-Bindungen basieren auf denen des Bindtags **TEntry**, wobei folgende Änderungen vorgenommen wurden:

- Neue Bindungen zu **<>** und **<>**.

TSpinboxPlus

Die TSpinboxPlus-Bindungen basieren auf denen des Bindtags **TSpinbox**, wobei folgende Änderungen vorgenommen wurden:

- Neue Bindungen zu **<>** und **<>**.

TComboboxPlus

Die TComboboxPlus-Bindungen basieren auf denen des Bindtags **TCombobox**, wobei folgende Änderungen vorgenommen wurden:

- Neue Bindungen zu **<>** und **<>**.

Virtuelles Ereignis <>

Jedes **widgetPlus**-Widget generiert ein virtuelles Ereignis **<>**, sobald sich der Wert seiner Auswahl ändert. Anderer **Tcl**-Code, zum Beispiel das Paket **persistentSelection**, kann die Auswahl in diesen Widgets verfolgen.

Verwendung mit Megawidgets

Wenn ein **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox** oder **ttk::combobox** Komponente eines Megawidgets ist, ist es häufig möglich, **widgetPlus** auf diese Komponente anzuwenden, ohne den Code des Megawidgets zu ändern.

Combobox (von Bryan Oakley)

Ein **combobox** mit dem Pfad **$w** enthält ein Widget **entry $w.entry**, das den Bindtag **Entry** benutzt. **widgetPlus** lässt sich nach dem Erzeugen auf das **combobox** anwenden, mit diesem Befehl (siehe Beispiele)

```
entryPlus $w.entry -undo 1 -maxundo 0
```

Combobox (BWidget)

Eine **BWidget**-**ComboBox** mit dem Pfad **$w** enthält ein Widget **entry $w.e**, das statt des voreingestellten Tags **Entry** den Bindtag **BwEntry** benutzt.

Das Paket **widgetPlus** stellt ein Ersatz-Bindtag **BwEntryPlus** bereit; berechnet wird er beim Laden des Pakets nicht, weil **BWidget** vorher geladen sein muss. **::widgetPlus::EnableBWidget** muss aufgerufen werden, damit **BwEntryPlus** existiert. Danach lässt sich die **BWidget**-**ComboBox** ändern (siehe Beispiele)

```
::widgetPlus::EnableBWidget
entryPlus $w.e -undo 1 -maxundo 0
```

Noch zu tun

- Undo und Redo gleichen das *Änderungsflag* nicht ab. Sie setzen den Wert immer auf *boolean* **true**.
- Undo und Redo gleichen die Position des Einfügecursors nicht ab.
- Das Ereignis "**<>**" wird nicht generiert.
- Trennzeichen für Undo und Redo und die Option **-autoseparators**, wie sie im Widget **text** zu finden sind, sind nicht implementiert. Bei jedem Aufruf eines Widget-Befehls (z. B. **insert** oder **delete**), der den Inhalt des Widgets verändert, wird ein implizites Trennzeichen hinzugefügt.

Der Befehl **edit separator** hat keine Wirkung. Die Trennzeichen-Funktionalität wäre für Widgets wie **entryPlus** weniger nützlich als für ein **text**-Widget.
- Ein Modus *overwrite*, umgeschaltet mit auf Systemen, die nicht macOS® sind.

Bugs

Diese Version von **widgetPlus** bietet Kompatibilität mit allen Releases von **Tk** 8.5 und 8.6 sowie mit den Zweigen *core-8-5-branch*, *core-8-6-branch* sowie *trunk* im Quellcode-Repository für **Tk**. Jede Inkompatibilität mit einer dieser Versionen, unabhängig vom verwendeten **Tk** -Fenstersystem, sollte als Fehler gemeldet werden. Bitte melden Sie solche Fehler in der Kategorie *widgetPlus* der Tklib-Tracker.

Beispiele

Ein Widget **entryPlus** .ep mit unbegrenztem Undo/Redo-Stapel erzeugen und zuordnen:

```
package require widgetPlus
widgetPlus::entryPlus .ep -undo 1 -maxundo 0
pack .ep
```

Das Gleiche, jedoch unter Verwendung von **namespace import**:

```
package require widgetPlus
namespace import widgetPlus::*
entryPlus .ep -undo 1 -maxundo 0
pack .ep
```

Ein vorhandenes Widget **entry** .ee in ein **entryPlus** mit unbegrenztem Undo/Redo-Stapel umwandeln:

```
entry .ee

    .
    .
    .

package require widgetPlus
widgetPlus::entryPlus .ee -undo 1 -maxundo 0
```

Das Gleiche, jedoch unter Verwendung von **namespace import**:

```
entry .ee

    .
    .
    .

package require widgetPlus
namespace import widgetPlus::*
entryPlus .ee -undo 1 -maxundo 0
```

widgetPlus auf eine **combobox** von Bryan Oakley anwenden:

```
package require combobox
combobox::combobox .cb

    .
    .
    .

package require widgetPlus
namespace import widgetPlus::*
entryPlus .cb.entry -undo 1 -maxundo 0
```

widgetPlus auf eine **BWidget-ComboBox** anwenden. Zuerst **::widgetPlus::EnableBWidget** aufrufen.

```
package require BWidget
ComboBox .cb

    .
    .
    .

package require widgetPlus
::widgetPlus::EnableBWidget
namespace import widgetPlus::*

entryPlus .cb.e -undo 1 -maxundo 0
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widgetPlus* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

widgetl

widget_listentry

Stichwörter: widget_listentry, data entry lists, data entry ordered list, data entry set of strings, data entry unordered list, list entry panel, set entry panel, widget

Name

widget_listentry – widget::listentry-Widget

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require widget::listentry ?**0.1.2**?
package require widget::validator ?**0.1**?
package require widget::scrolledwindow
package require snit
package require tooltip
package require img::png

package require msgcat
widget::listentry *pathname* ?*options*?
widgetCmd destroy
widgetCmd configure
widgetCmd configure *option value ...*
widgetCmd configure *option*
widgetCmd cget *option*
{}cmdprefix* **get**
{}cmdprefix* **set** *values*
{}cmdprefix* **text** *errvar*
{}cmdprefix* **text**
{}cmdprefix* **cookievar**

Beschreibung

Dieses Paket stellt ein Megawidget zur interaktiven Eingabe geordneter und ungeordneter Listen bereit. Ein einfacheres, enger gefasstes Megawidget ist das Paket widget::listsimple.

API für Klassen

Die Widget-Klasse unterstützt einen einzigen Befehl zur Erstellung von Widgets.

widget::listentry *pathname* ?options?
Dieser Befehl erstellt und konfiguriert neue Instanzen des Widgets.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

Das Ergebnis des Befehls ist der Pfadname des neuen Widgets.

Instanz-API

Alle Widget-Instanzen unterstützen die folgenden Methoden.

widgetCmd destroy
Diese Methode löscht das Widget. Jeder weitere Zugriff auf das Widget führt zu Fehlern.

Das Ergebnis des Befehls ist die leere Zeichenkette.

widgetCmd configure
Diese Methode gibt es in drei Varianten. Diese Variante gibt eine Liste zurück, die die aktuelle Konfiguration des Widgets enthält, d. h. die Werte für alle Optionen.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

widgetCmd configure *option value ...*
Diese Methode gibt es in drei Varianten. Diese Variante konfiguriert das Widget neu und setzt die angegebenen Optionen auf die angegebenen Werte.

Hinweis: Es ist nicht möglich, Optionen zu ändern, die ausschließlich zur Erstellungszeit gelten.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Allgemeine Optionen für Widgets.

Das Ergebnis des Befehls ist die leere Zeichenkette.

widgetCmd configure *option*

Diese Methode gibt es in drei Varianten. Die vorliegende Variante ist ein Alias für die unten aufgeführte Methode **cget** und mit dieser identisch.

widgetCmd cget *option*

Diese Methode gibt den aktuellen Wert der angegebenen *option* zurück.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

Widget-Optionen

In diesem Abschnitt werden alle Optionen erläutert, die für Instanzen von **widget::listentry** verfügbar sind. Beachten Sie, dass einige der Optionen nur bei der Erstellung der Instanz festgelegt werden können. Die meisten Optionen können jedoch sowohl bei der Erstellung als auch zur Laufzeit festgelegt werden.

Optionen, die ausschließlich zur Erstellungszeit des Widgets gelten

-ordered boolean

Diese Option sagt der neuen Instanz, ob die Liste geordnet ist. Ist sie geordnet, zeigt das Widget Schaltflächen, mit denen sich die ausgewählten Einträge verschieben lassen.

Voreingestellt ist **false**, eine ungeordnete Liste.

-allow-duplicates boolean

Diese Option legt fest, ob die neue Widget-Instanz eine Zeichenfolge mehrfach zur Liste hinzufügen darf oder nicht.

Die Standardeinstellung ist **false**, was bedeutet, dass Duplikate nicht zulässig sind.

-values cmdprefix

Diese Option legt eine Rückruffunktion für die Verwaltung einer vordefinierten Liste von Zeichenfolgen fest, die ein Benutzer eingeben kann.

Wenn diese Option angegeben wird, verwendet das Widget anstelle eines einfachen Eingabefeldes eine Combobox und füllt deren Liste während der Erstellung des Widgets mit den von dieser Rückruffunktion gelieferten Daten. Die Rückruffunktion wird zudem jedes Mal aufgerufen, wenn ein neuer Wert in die Hauptliste eingegeben wird, um die erweiterte Liste der vordefinierten Werte zu speichern.

Die Signatur dieser Rückruffunktion lautet

***{*}cmdprefix* get**

In dieser Form soll die Rückruffunktion eine Liste von Zeichenfolgen geben. Die Zeichenfolgen werden in die Liste der internen Combobox geladen, um dem Benutzer einen schnellen Zugriff zu ermöglichen.

Die Funktion wird einmalig während der Erstellung des Widgets aufgerufen, um die Liste der vordefinierten Zeichenfolgen zu laden, die ein Benutzer eingeben kann.

{*}cmdprefix* set *values

In dieser Form erhält die Rückruffunktion eine Liste neuer Zeichenfolgen und soll diese für

die spätere Verwendung an einem Ort speichern.

Der Rückgabewert der Rückruffunktion wird ignoriert.

Diese Form wird immer dann aufgerufen, wenn der Benutzer eine neue Zeichenfolge in die Hauptliste eingegeben hat. Die Reihenfolge der Zeichenfolgen in *values* entspricht der Reihenfolge, in der sie in der Liste der Combobox angezeigt werden.

-validate cmdprefix

Diese Option legt eine Rückruffunktion fest, die nach jeder Änderung des Inhalts des internen Eingabefelds aufgerufen wird. Die Signatur dieser Rückruffunktion lautet

{*}cmdprefix text errvar

wobei *text* die zu validierende Zeichenkette und *errvar* der Name einer Variablen ist, in der die Rückruffunktion eine Fehlermeldung speichern kann, wenn sie ungültige Daten feststellt.

Das Widget erwartet, dass die Rückruffunktion einen booleschen Wert gibt, wobei **true** angibt, dass *text* gültig ist.

Die Standardvalidierung behandelt – sofern keine Rückruffunktion festgelegt wurde – eine leere Zeichenkette als ungültig und alle anderen Eingaben als gültig.

Bitte beachten Sie, dass das Widget die Eingabe doppelter Werte in die Hauptliste verhindert, unabhängig von der ansonsten bestehenden Gültigkeit der Eingabe. Dies steht im Einklang mit der Tatsache, dass dieses Widget für die Eingabe ungeordneter Listen vorgesehen ist, also im Wesentlichen für eine Reihe von Zeichenfolgen.

-transform cmdprefix

Diese Option legt eine Rückruffunktion fest, die immer dann aufgerufen wird, wenn ein neu eingegebenes Element aus dem Eingabefeld in die Hauptliste verschoben wird oder wenn das Eingabefeld im Rahmen einer Duplikatsprüfung validiert wird, sofern Duplikate nicht zulässig sind. Die Rückruffunktion erhält den Text aus dem Eingabefeld und muss den Text geben, der tatsächlich zur Liste hinzugefügt oder mit dieser abgeglichen wird.

Der Anwendungsfall für diese Rückruffunktion ist im Wesentlichen die *Eingabenormalisierung*. Der einfachste Fall hierfür wäre beispielsweise die Verwendung von

```
string tolower
```

um eine einheitliche Schreibung

der Daten durchzusetzen. Weitere Beispiele sind denkbar, etwa mehrere Eingabeformen auf eine kanonische Form zu bringen. Die Signatur dieser Rückruffunktion lautet

{*}cmdprefix text

wobei *text* die zu transformierende Zeichenkette ist.

Das Widget erwartet, dass die Rückruffunktion die transformierte Zeichenkette zurückgibt.

Standardmäßig wird keine Transformation der eingegebenen Zeichenfolgen durchgeführt.

-browse cmdprefix

Ist diese Option angegeben, zeigt das Widget eine Schaltfläche "Search" und ruft die

Rückruffunktion auf, wenn diese Schaltfläche geklickt wird. Die Signatur lautet
{*}cmdprefix cookievar
wobei die Cookie-Variable Zugriff auf Kontextinformationen bietet, die die Rückruffunktion möglicherweise zwischen den Aufrufen beibehalten möchte. Der in der Variablen gespeicherte Anfangswert ist die leere Zeichenkette. Danach enthält sie alles, was die Rückruffunktion speichern möchte.

Das Widget erwartet, dass die Rückruffunktion eine Liste von Zeichenketten gibt, die dann – vorbehaltlich Validierung und Duplikatsprüfungen – der Liste hinzugefügt werden.

Voreingestellt gibt es keine Browse-Rückruffunktion und keine Schaltfläche.

Allgemeine Optionen für Widgets

-listvariable varname
Diese Option legt die Variable fest, die die Liste enthält, die vom Widget bearbeitet werden soll. Alle Änderungen an der Liste, die außerhalb des Widgets vorgenommen werden, werden automatisch in das Widget übernommen. Ebenso werden alle Änderungen, die das Widget an der Liste vornimmt, automatisch in diese Variable exportiert.

-state normal|disabled
Zustand des Widgets, **normal**, also aktiv, oder **disabled**. Ein deaktiviertes Widget lässt sich nicht bearbeiten.

Voreingestellt ist **normal**.

-height integer
Höhe der internen Listbox, in Zeilen.

Voreingestellt ist **20**.

-skin-add string
siehe unten ...

-skin-remove string
siehe unten ...

-skin-up string
siehe unten ...

-skin-down string
siehe unten ...

-skin-browse string
siehe unten ...

-skin-tip-add string
siehe unten ...

-skin-tip-remove string
siehe unten ...

-skin-tip-up string
siehe unten ...

-skin-tip-down string
siehe unten ...

-skin-tip-browse string
siehe unten ...
-skin-tip-main string
siehe unten ...
-skin-tip-entry string
siehe unten ...
-skin-tip-list string
siehe unten ...
-skin-tip-empty string
siehe unten ...
-skin-tip-duplicate string
siehe unten ...
-skin-tip-add-none string
siehe unten ...
-skin-tip-remove-none string
siehe unten ...
-skin-img-add image
siehe unten ...
-skin-img-remove image
siehe unten ...
-skin-img-up string
siehe unten ...
-skin-img-down string
siehe unten ...
-skin-img-browse string
siehe unten ...
-skin-invalid-color color
Diese Optionen ändern alle das Erscheinungsbild des Widgets, d. h. dessen Skin.

Alle Optionen, die ein Zeichenfolgenargument entgegennehmen, beeinflussen die verschiedenen angezeigten Beschriftungen, wobei insbesondere die Optionen vom Typ **-skin-tip-*** die Tooltips beeinflussen, die beim Bewegen des Mauszeigers über verschiedene Bereiche angezeigt werden.

Alle Zeichenfolgen werden zunächst durch **msgcat** geleitet, wodurch die Lokalisierung von Text über Meldungskataloge ermöglicht wird. Die Standardwerte sind Schlüssel in den Meldungskatalogen, die Teil des Pakets selbst sind.

Die Optionen, die Bilder als Argument annehmen, ändern die Bilder, die auf den Schaltflächen zum Hinzufügen und Entfernen von Elementen der Liste angezeigt werden. Standardmäßig werden die mit dem Paket selbst ausgelieferten PNG-Bilder verwendet.

Die einzige Option, die einen Farbwert akzeptiert, ändert die Farbe, mit der ungültige Daten hervorgehoben werden, die in das interne Eingabefeld des Widgets eingegeben wurden. Diese

Option ist standardmäßig auf **yellow** gesetzt.

Beispiel

Fehler, Ideen, Rückmeldungen
Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::listentry* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

widget_listsimple
Stichwörter: widget_listsimple, data entry lists, data entry set of strings, data entry unordered list, list entry panel, set entry panel, widget

Name
widget_listsimple – widget::listsimple-Widget

Übersicht
package require Tcl 8.5
package require Tk 8.5
package require widget::listsimple ?**0.1.2**?
package require widget::validator ?**0.1**?
package require widget::scrolledwindow
package require snit
package require tooltip
package require img::png
package require msgcat
widget::listsimple *pathname* ?*options*?
widgetCmd destroy
widgetCmd configure
widgetCmd configure *option value ...*
widgetCmd configure *option*
widgetCmd cget *option*
{}cmdprefix* **get**
{}cmdprefix* **set** *values*
{}cmdprefix* **text** *errvar*

Beschreibung
Dieses Paket enthält ein Megawidget für die interaktive Eingabe von ungeordneten Listen. Für geordnete Listen und mehr gibt es das Paket widget::listentry.

API für Klassen
Die Widget-Klasse unterstützt einen einzigen Befehl zur Erstellung von Widgets.

widget::listsimple *pathname* ?options?
Dieser Befehl erstellt und konfiguriert neue Instanzen des Widgets.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

Das Ergebnis des Befehls ist der Pfadname des neuen Widgets.

Instanz-API

Alle Widget-Instanzen unterstützen die folgenden Methoden.

widgetCmd destroy

Diese Methode löscht das Widget. Jeder weitere Zugriff auf das Widget führt zu Fehlern.

Das Ergebnis des Befehls ist die leere Zeichenkette.

widgetCmd configure

Diese Methode gibt es in drei Varianten. Diese Variante gibt eine Liste zurück, die die aktuelle Konfiguration des Widgets enthält, d. h. die Werte für alle Optionen.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

widgetCmd configure *option value ...*

Diese Methode gibt es in drei Varianten. Diese Variante konfiguriert das Widget neu und setzt die angegebenen Optionen auf die angegebenen Werte.

Hinweis: Es ist nicht möglich, Optionen zu ändern, die ausschließlich zur Erstellungszeit gelten.

Einzelheiten zu den verfügbaren Optionen stehen im Abschnitt Allgemeine Optionen für Widgets.

Das Ergebnis des Befehls ist die leere Zeichenkette.

widgetCmd configure *option*

Diese Methode gibt es in drei Varianten. Die vorliegende Variante ist ein Alias für die unten aufgeführte Methode **cget** und mit dieser identisch.

widgetCmd cget *option*

Diese Methode gibt den aktuellen Wert der angegebenen *option* zurück.

Einzelheiten zu den verfügbaren Optionen finden Sie im Abschnitt Widget-Optionen.

Widget-Optionen

In diesem Abschnitt werden alle Optionen erläutert, die für Instanzen von **widget::listsimple** verfügbar sind. Einige Optionen lassen sich nur beim Erzeugen der Instanz setzen. Die meisten Optionen können jedoch sowohl bei der Erstellung als auch zur Laufzeit festgelegt werden.

Optionen, die ausschließlich zur Erstellungszeit des Widgets gelten

-values cmdprefix

Diese Option legt eine Rückruffunktion für die Verwaltung einer vordefinierten Liste von Zeichenfolgen fest, die ein Benutzer eingeben kann.

Wenn diese Option angegeben wird, verwendet das Widget anstelle eines einfachen Eingabefeldes eine Combobox und füllt deren Liste während der Erstellung des Widgets mit den von dieser Rückruffunktion gelieferten Daten. Die Rückruffunktion wird zudem jedes Mal aufgerufen, wenn ein neuer Wert in die Hauptliste eingegeben wird, um die erweiterte Liste

der vordefinierten Werte zu speichern.

Die Signatur dieser Rückruffunktion lautet

***{*}cmdprefix* get**

In dieser Form soll die Rückruffunktion eine Liste von Zeichenfolgen geben. Die Zeichenfolgen werden in die Liste der internen Combobox geladen, um dem Benutzer einen schnellen Zugriff zu ermöglichen.

Die Funktion wird einmalig während der Erstellung des Widgets aufgerufen, um die Liste der vordefinierten Zeichenfolgen zu laden, die ein Benutzer eingeben kann.

{*}cmdprefix* set *values

In dieser Form erhält die Rückruffunktion eine Liste neuer Zeichenfolgen und soll diese für die spätere Verwendung an einem Ort speichern.

Der Rückgabewert der Rückruffunktion wird ignoriert.

Diese Form wird immer dann aufgerufen, wenn der Benutzer eine neue Zeichenfolge in die Hauptliste eingegeben hat. Die Reihenfolge der Zeichenfolgen in *values* entspricht der Reihenfolge, in der sie in der Liste der Combobox angezeigt werden.

-validate cmdprefix

Diese Option legt eine Rückruffunktion fest, die nach jeder Änderung des Inhalts des internen Eingabefelds aufgerufen wird. Die Signatur dieser Rückruffunktion lautet

{*}cmdprefix text errvar

wobei *text* die zu validierende Zeichenkette und *errvar* der Name einer Variablen ist, in der die Rückruffunktion eine Fehlermeldung speichern kann, wenn sie ungültige Daten feststellt.

Das System erwartet, dass die Rückruffunktion einen booleschen Wert gibt, wobei **true** angibt, dass *text* gültig ist.

Die Standardvalidierung behandelt – sofern keine Rückruffunktion festgelegt wurde – eine leere Zeichenkette als ungültig und alle anderen Eingaben als gültig.

Bitte beachten Sie, dass das Widget die Eingabe doppelter Werte in die Hauptliste verhindert, unabhängig von der ansonsten bestehenden Gültigkeit der Eingabe. Dies steht im Einklang mit der Tatsache, dass dieses Widget für die Eingabe ungeordneter Listen vorgesehen ist, also im Wesentlichen für eine Reihe von Zeichenfolgen.

Allgemeine Optionen für Widgets

-listvariable varname

Diese Option legt die Variable fest, die die Liste enthält, die vom Widget bearbeitet werden soll. Alle Änderungen an der Liste, die außerhalb des Widgets vorgenommen werden, werden automatisch in das Widget übernommen. Ebenso werden alle Änderungen, die das Widget an der Liste vornimmt, automatisch in diese Variable exportiert.

-skin-add string

siehe unten ...

-skin-remove string

siehe unten ...

-skin-tip-add string
siehe unten ...

-skin-tip-remove string
siehe unten ...

-skin-tip-main string
siehe unten ...

-skin-tip-entry string
siehe unten ...

-skin-tip-list string
siehe unten ...

-skin-tip-empty string
siehe unten ...

-skin-tip-duplicate string
siehe unten ...

-skin-tip-add-none string
siehe unten ...

-skin-tip-remove-none string
siehe unten ...

-skin-img-add image
siehe unten ...

-skin-img-remove image
siehe unten ...

-skin-invalid-color color
Diese Optionen ändern alle das Erscheinungsbild des Widgets, d. h. dessen Skin.

Alle Optionen, die ein Zeichenfolgenargument entgegennehmen, beeinflussen die verschiedenen angezeigten Beschriftungen, wobei insbesondere die Optionen vom Typ **-skin-tip-*** die Tooltips beeinflussen, die beim Bewegen des Mauszeigers über verschiedene Bereiche angezeigt werden.

Alle Zeichenfolgen werden zunächst durch **msgcat** geleitet, wodurch die Lokalisierung von Text über Meldungskataloge ermöglicht wird. Die Standardwerte sind Schlüssel in den Meldungskatalogen, die Teil des Pakets selbst sind.

Die Optionen, die Bilder als Argument annehmen, ändern die Bilder, die auf den Schaltflächen zum Hinzufügen und Entfernen von Elementen der Liste angezeigt werden. Standardmäßig werden die mit dem Paket selbst ausgelieferten PNG-Bilder verwendet.

Die einzige Option, die einen Farbwert akzeptiert, ändert die Farbe, mit der ungültige Daten hervorgehoben werden, die in das interne Eingabefeld des Widgets eingegeben wurden. Diese Option ist standardmäßig auf **yellow** gesetzt.

Beispiel

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::listsimple* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

widgetv

widget_validator

Stichwörter: widget_validator, invalid, state management, ttk::combobox, ttk::entry, validation, widget validation

Name

widget_validator – Verhalten von widget::validator

Übersicht

package require Tcl 8.5
package require Tk 8.5
package require widget::validator ?**0.1**?
widget::validator attach *w color cmdprefix*
widget::validator detach *w*
widget::validator validate *w*

Beschreibung

Dieses Paket stellt eine einheitliche Validierungs-API für die Widgets Entry und Combobox von **ttk** bereit.

Das hier festgelegte Validierungsverhalten weist ungültige Bearbeitungen nicht zurück. Es hebt nur das Entry mit den ungültigen Daten hervor und setzt das passende Zustandsflag.

Es liegt in der Verantwortung des verwendenden Pakets oder der Anwendung, zu entscheiden, wie und wann solche ungültigen Inhalte tatsächlich zurückgewiesen werden sollen.

widget::validator attach *w color cmdprefix*

Diese Methode fügt dem Widget *w* ein Verhalten zur Validierung hinzu.

Immer wenn sich der Inhalt des Eingabefelds des Widgets ändert, wird das angegebene *cmdprefix* aufgerufen und muss einen booleschen Wert geben, wobei **true** bedeutet, dass der Inhalt in Ordnung ist, und **false**, dass der Inhalt ungültig ist. Zum Befehlspräfix siehe den Abschnitt Validierung. Im letzteren Fall wird die Hintergrundfarbe des Eingabefeldes in *color* geändert, um die Ungültigkeit anzuzeigen.

Das System unterstützt weder die Verschachtelung von Validatoren auf einem Widget noch das Anhängen mehrerer Validatoren. Zum Ändern der Bedingungen zuerst den aktuellen Validator mit **detach** lösen und dann den neuen anhängen.

Es wird ein Fehler ausgelöst, wenn dem Widget bereits ein Validierungsverhalten zugewiesen ist.

Das Ergebnis der Methode ist die leere Zeichenkette.

Um seine Ziele zu erreichen, überschreibt das Paket verschiedene Konfigurationsoptionen des Widgets, an das das Verhalten angehängt wird. Diese Optionen werden bei **detach** auf ihre vorherigen Werte zurückgesetzt.

Wenn andere Verhaltensweisen zugeordnet sind, kann der Validator vorübergehend (teilweise) funktionsunfähig werden. Ebenso kann bei einer Trennung des Validators, während gleichzeitig eine andere Verhaltensweise zugeordnet ist, die Wiederherstellung der Konfiguration dazu führen, dass die andere Verhaltensweise funktionsunfähig wird.

widget::validator detach *w*

Diese Methode entfernt das Validierungsverhalten aus dem Widget *w* und versetzt es in seinen ursprünglichen Zustand zurück.

Es wird ein Fehler ausgelöst, wenn dem Widget kein Validierungsverhalten zugeordnet ist.

Das Ergebnis der Methode ist die leere Zeichenkette.

widget::validator validate *w*

Der Aufruf dieser Methode erzwingt eine Validierung des Widgets *w*, vorausgesetzt, diesem ist ein Validator-Verhalten zugeordnet.

Das Ergebnis der Methode ist die leere Zeichenkette.

Validierung

Der für die Validierung verwendete Befehls-Präfix muss die folgende Signatur aufweisen:

{*}cmdprefix text

Das Argument ist der zu prüfende Text.

Das Ergebnis des Rückrufs muss ein boolescher Wert sein. **true** bedeutet, dass *text* in Ordnung ist, **false**, dass *text* ungültig ist.

Beispiel

```
package require Tk 8.5
package require widget::validator

set TE {}
set TC {}

ttk::entry    .e -textvariable TE
ttk::combobox .c -textvariable TC -values {fruit vegetable corn}
ttk::combobox .n -values {fruit vegetable corn}
ttk::button   .x -command ::exit -text Exit

pack .e -expand 1 -fill both -side top
pack .c -expand 1 -fill both -side top
pack .n -expand 1 -fill both -side top
pack .x -expand 1 -fill both -side top
```

```
widget::validator attach .e lightblue {apply {text {
    expr {$text ne {}}
}}}

widget::validator attach .c yellow {apply {text {
    expr {$text ni {{} hello world}}
}}}

widget::validator attach .n pink {apply {text {
    expr {$text ni {{} blub}}
}}}
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese in der Kategorie *widget::validator* der Tklib-Tracker. Bitte melden Sie dort auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation.

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung entstand mit Unterstützung von DeepL und KI-Werkzeugen und wurde von Hand durchgesehen und überarbeitet.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Lizenzbedingungen license

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND

THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Stichwortverzeichnis

2

2D geometry 68

A

addon 124, 127, 128, 134, 139, 140
alert 153
append tag 45
arc 68
area display 122, 124, 126
area-display 122
area-file 123
area-map-display 124
area-store-fs 124
area-store-mem 125
area-table-display 126
arrow 68
autoscroll 9

B

background 38
balloon 262, 264
bindtags 154, 158, 161, 165
bitmap 5
bitmap-editor 5
box 68
box display 126, 128, 130
box entry 127
box-display 126
box-entry 127
box-file 128
box-map-display 128
box-store-fs 129
box-store-mem 129
box-table-display 130
bundle 267

C

calendar 267, 268
canvas 5, 10, 12, 17, 23, 29, 33, 38, 39, 41, 42, 43, 45, 46, 59, 68, 253, 256, 259, 260
canvas_drag 10
canvas_ecircle 12
canvas_epoints 17
canvas_epolyline 23
canvas_equad 29

canvas_erectangle 33
canvas_gradient 38
canvas_highlight 39
canvas_mvg 41
canvas_pdf 41
canvas_snap 42
canvas_sqmap 43
canvas_tags 45
canvas_trlines 46
canvas_zoom 48
canvaslabel 253
cell 43
center 125, 129, 136, 141
character 120
chat 49
chatwidget 49
circle 12, 68
clock 66
color gradient 38
composite widget 49
concave 29
controlling 51
controlwidget 51
conversion 5
convex 29
cross-hairs 59
crosshair 46, 59
ctext 61
cursor 65

D

data entry lists 279, 285
data entry ordered list 279
data entry set of strings 279, 285
data entry unordered list 279, 285
date 66, 268, 270
dateentry 66, 267, 270
datefield 66
diagram 5, 68
diagram-viewer 5
dialog 112, 252, 267
diameter 125, 129, 136, 141
diamond 68
disjointlistbox 252
display 131
displaying 51
dll 114

doctools 7, 241
dragging 10
drawing 68
drum 68

E

editing 12, 17, 23, 29, 33
editor 5
ellipse 68
enter callback 39
entry 66, 112, 113, 114, 117
exe 114

F

file 123, 128, 134, 139
filesystem 124, 129, 135, 140
filesystem store 124, 129, 135, 140

G

geo/area 123, 124, 125
geo/area pixels 125
geo/box 128, 129
geo/box pixels 129
geo/point 134, 135, 136
geo/point pixels 136
geo/track 139, 140, 141
geo/track pixels 141
geobox 129
gradients 38
graphics 41
grid 43

H

help 262, 264
highlighting 39
history 113
hover 262, 264
html 7, 241, 243, 244

I

i18n 120
ico 114
icon 114
image 5, 38, 42, 43, 68
imagemagick 41
input 120
insert tag 45
international 120
interpolation 68

intersection 68
invalid 289
ip address 117
ipentry 117
irc 49

K

khim 120

L

labeler 253, 256, 259, 260
leave callback 39
length 125, 129, 136, 141
line 68
list entry panel 279, 285
listbox 252
location 59
location marking 133

M

magick vector graphics 41
map 43, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142
map display 124, 127, 128, 131, 134, 139, 140
map_overview 132
mapnik 137
mark 133
mark location 133
markdown 7
mega-widget 49
megawidget 267, 268, 270, 272
memory 125, 129, 136, 141
memory store 125, 129, 136, 141
menu 267
menubar 143
method 120
mkdoc 7, 243
move 68
mvg 41

N

network 117
non-convex 29
notification 153
notifywindow 153
ntext 154
ntextBindings 158
ntextIndent 161
ntextWordBreak 165

numerical values 51

O

openstreetmap 137

P

panelframe 267
pdf 41
perimeter 125, 129, 136, 141
perimeter length 125, 129, 136, 141
persistentSelection 168
photo 42
picture 68
pie 253, 256, 259, 260
pieboxlabeler 259
pieperipherallabeler 260
pixel 5
pixels 125, 129, 136, 141
plane geometry 68
plotchart 172
plotting 68
point 68
point cloud 17
point display 134, 136
point-file 134
point-map-display 134
point-store-fs 135
point-store-mem 136
point-table-display 136
points 17
polyline 23
portable document format 41
print screen 41, 42
provider 137
provider-osm 137

Q

quadrilateral 29

R

raster image 68
re_syntax 154, 161, 165
rectangle 33
regexp 154, 161, 165
remove tag 45
rubberband 46
ruler 267

S

scale widget 51
screenruler 267
scroll 9
scrollbar 9
scrolledwindow 267
serialization 41
set entry panel 279, 285
shtmlview 7, 244
shtmlview-doctools 241
shtmlview-mkdoc 243
slice 253, 256, 259, 260
snapshot 42
snit 267, 268, 270, 272
spline 68
square map 43
state management 289
statusbar 267
store 124, 125, 129, 135, 136, 140, 141
string 112
superframe 267
swaplist 252
syntax highlighting 61

T

tabular 122, 126, 130, 136, 138, 142
tags 45
text 61, 68, 154, 158, 161, 165, 244
tile 43
tile provider 137
tiles 137
tipstack 262
tk 153
tk_getString 112
tklib_history 113
toolbar 267, 272
tooltip 264
track display 138, 140, 142
track entry 139
track-display 138
track-entry 139
track-file 139
track-map-display 140
track-store-fs 140
track-store-mem 141
track-table-display 142
tracking 46, 59

ttk::combobox 289
ttk::entry 289

V

validation 289
vector 5, 68
vector graphics 41
viewer 7
viewport 59

W

widget 49, 61, 66, 244, 267, 268, 270, 272, 279, 285
widget background 38
widget validation 289
widget_calendar 268
widget_dateentry 270
widget_listentry 279
widget_listsimple 285
widget_toolbar 272
widget_validator 289
widgetPlus 273

X

xbm 5

Z

zoom 48, 125, 129, 136, 141